RESEARCH SPRINT

ACAI

Accountable Conversational AI

Proyecto marco $(AI)^2$:TDG

Fundamentos de IA Conversacional Neuro-Simbólica

Monográfico formativo

Febrero 2026

Índice general

MAPA CONCEPTUAL

UD 1 Mapa conceptual ACAI 24 pp.

La tesis raíz · Los principios fundacionales · Los cimientos · El ecosistema operativo · Los frentes de innovación · La arquitectura ACAI · Posicionamiento en el proyecto $(AI)^2$:TDG · Estructura del monográfico · Mapa de relaciones

EJE A — EL PROBLEMA

UD 2 La conversación como fenómeno y como problema de ingeniería 26 pp.

Qué ocurre cuando dos personas conversan · La evolución de las arquitecturas de diálogo · El diagnóstico: generar sin representar · Implicaciones para el diseño de ACAI

UD 3 El ecosistema operativo: componentes, capacidades y límites 25 pp.

El argumento de la completitud · RAG: del patrón al problema de gobernanza · Arquitecturas agénticas · Alineamiento y políticas · Gestión de contexto: del buffer de tokens al information state · Síntesis: por qué la combinación no basta

UD 4 Accountability, explicabilidad y marco regulatorio 22 pp.

La constelación de conceptos · Explicabilidad post-hoc vs. constitutiva · El marco regulatorio como requisito de ingeniería · Gobernanza constitutiva vs. superpuesta · Accountability y estructuras formales

EJE B — EL PARADIGMA NEURO-SIMBÓLICO

UD 5 Fundamentos del paradigma neuro-simbólico 21 pp.

La tensión fundacional de la IA · Las tres olas de la IA neuro-simbólica · La taxonomía de Kautz: cinco niveles de integración · Posicionamiento de ACAI

UD 6 Arquitecturas y patrones de integración 22 pp.

Principios de diseño para la integración · Patrones arquitectónicos de nivel 1–2 · Patrones de nivel 3 · La arquitectura ACAI como composición de patrones · Trade-offs arquitectónicos

UD 7 Técnicas neuro-simbólicas para el procesamiento del lenguaje natural 25 pp.

El problema del puente · De lenguaje a estructura: NL → Formal en runtime · De estructura a lenguaje: Formal → NL en runtime · De lenguaje a formalismo: NL → Formal en design time · El espectro de restricción · Síntesis: el repertorio técnico

EJE C — REPRESENTACIÓN Y FORMALIZACIÓN

UD 8 Dimensiones del conocimiento conversacional 23 pp.

Por qué no basta con «representar el diálogo» · Las cinco dimensiones del conocimiento conversacional · Propiedades formales de las dimensiones · Interacciones entre dimensiones · Verificación de completitud · De las dimensiones a los requisitos del formalismo

UD 9 Formalismos candidatos 22 pp.

El marco de evaluación · Polo ontológico · Polo procedimental · Polo lógico · Polo DSL · Evaluación cruzada · Hacia el formalismo ACAI

UD 10 Formalización asistida por LLM 21 pp.

El cuello de botella de la formalización · Arquitectura del pipeline · Elicitación estructurada · Generación condicionada · Validación simbólica · Revisión humana y verbalización · Diseño experimental · Modos de fallo y mitigación

UD 11 La triple restricción 17 pp.

La triple restricción como sistema de tensiones · Tensión L–P: legibilidad vs. procesabilidad · Tensión L–G: legibilidad vs. generabilidad · Tensión P–G: procesabilidad vs. generabilidad · La tensión trilateral · La resolución integrada · Programa experimental

EJE D — EVALUACIÓN

UD 12 Métricas y evaluación de accountability

18 pp.

La insuficiencia de las métricas estándar · Métricas de trazabilidad · Métricas de explicabilidad · Métricas de conformidad normativa · Métricas de gobernanza · Métricas de calidad del formalismo · Métricas del pipeline · El framework integrado · Síntesis: del mapa al territorio

Mapa conceptual

Prerequisitos: ninguno · Mapa conceptual

UNIDAD DIDÁCTICA 1

Mapa conceptual ACAI

Contenidos de la unidad

1.1.1 Lo que la tesis implica 5
1.1.2 Lo que la tesis no dice 5
1.2.1 P1: Separación de competencias 6
1.2.2 P2: El conocimiento debe estar explícitamente representado 7
1.2.3 P3: La integración neuro-simbólica es un espectro 7
1.2.4 P4: La accountability requiere completitud operativa 8
1.3.1 C1: La conversación es un fenómeno multidimensional 9
1.3.2 C2: Los LLMs ofrecen fluidez sin gobernabilidad 10
1.3.3 C3: El marco regulatorio exige accountability operativa 10
1.3.4 C4: La IA neuro-simbólica proporciona el marco teórico 10
1.3.5 C5: La ingeniería del conocimiento tiene décadas de experiencia 11
1.3.6 C6: El ecosistema tecnológico proporciona componentes maduros 11
1.4.1 E1: RAG gobernado 12
1.4.2 E2: Capacidad agéntica gobernada 12
1.4.3 E3: Alineamiento formalizado 12
1.4.4 E4: Gestión de contexto conversacional 13
1.5.1 I1: La triple restricción como espacio de diseño 13
1.5.2 I2: LLM como meta-herramienta 14
1.5.3 I3: La frontera neural-simbólica como zona de interacción 14
I3a: Condicionamiento simbólico del procesamiento neuronal 15
I3b: Aprendizaje simbólico a partir de trazas de ejecución 15
I3c: Razonamiento abductivo en la interpretación 15
1.5.4 I4: Accountability verificable en dominio conversacional 15
1.5.5 I5: Gobernanza formal del ecosistema operativo 16
1.6.1 Phase 1: el pipeline gobernado 16
1.6.2 Phase 2: la integración bidireccional 17
1.8.1 Los cuatro ejes 19
1.8.2 Mapa de dependencias y líneas de estudio 19
1.8.3 Integración con la Fase 0 del sprint 19

¿Por qué necesito esto para ACAI?

Esta unidad establece el cuadro completo del proyecto ACAI antes de que el investigador entre en los detalles de cada eje temático. Sin esta visión de conjunto, las unidades posteriores se estudian sin contexto y las conexiones entre ejes se pierden. Al terminar esta unidad, el investigador tendrá un mapa mental completo de qué es ACAI, por qué existe, qué propone, y cómo se estructura — tanto el sprint de investigación como este monográfico que lo prepara.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Articular la tesis raíz de ACAI y explicar por qué la accountability es un problema de representación, no de restricción.
2. Enunciar los cuatro principios fundacionales y conectar cada uno con los frentes de innovación que habilita.
3. Distinguir entre cimientos (lo que ACAI toma como dado), ecosistema operativo (lo que ACAI integra) y frentes de innovación (lo que ACAI propone de nuevo).
4. Situar ACAI dentro del proyecto $(AI)^2$:TDG y explicar las conexiones con las líneas de investigación RL1, RL2 y RL3.
5. Describir la estructura en dos fases del sprint y justificarla en términos de la taxonomía de Kautz.
6. Identificar la función de cada eje del monográfico y su relación con el marco conceptual.

1.1 La tesis raíz

Sede principal de este concepto.

Los grandes modelos de lenguaje han transformado la IA conversacional. En pocos años, hemos pasado de sistemas rígidos basados en reglas a agentes capaces de mantener diálogos fluidos, comprender matices lingüísticos y generar respuestas de calidad sorprendente. Sin embargo, esta revolución ha traído consigo un problema que no es técnico sino estructural: los sistemas que mejor conversan son los que peor pueden explicar lo que hacen.

La accountability no se añade — se construye desde la representación.

Una compañía de seguros que despliega un chatbot para atender reclamaciones necesita garantizar que el sistema aplica las mismas reglas a todos los clientes, que cada decisión puede auditarse, y que las políticas

de la organización se cumplen de forma verificable. Un hospital que implementa un asistente conversacional para triaje necesita saber por qué el sistema hizo una recomendación concreta, y demostrar que esa recomendación es consistente con los protocolos clínicos. Estas necesidades no son caprichos burocráticos — son requisitos legales en sectores regulados y, más fundamentalmente, son condiciones de confianza sin las cuales el despliegue de IA conversacional es irresponsable.

Tesis raíz de ACAI

La accountability en IA conversacional es un problema de representación, no de restricción.

Los sistemas actuales intentan restringir el comportamiento de modelos que no representan explícitamente lo que hacen. ACAI propone que la accountability emerge de operar sobre representaciones formales explícitas del conocimiento conversacional. Si las decisiones se toman sobre estructuras formales, la trazabilidad, consistencia y verificabilidad son consecuencias naturales, no capas añadidas.

Esta tesis se opone directamente a la estrategia dominante en la industria, que consiste en desplegar un LLM y luego intentar controlar su comportamiento mediante capas superpuestas: **guardrails** que filtran contenido inapropiado, prompts cuidadosamente elaborados que intentan dirigir al modelo, técnicas de alineamiento como RLHF que ajustan las preferencias del modelo durante el entrenamiento. Todas estas aproximaciones comparten una limitación fundamental: operan sobre un sistema que no representa explícitamente lo que hace. Por tanto, ninguna puede garantizar — en el sentido formal del término — que el comportamiento cumple una especificación.

Corolario: la representación formal es condición necesaria pero no suficiente. Un sistema accountable debe ser también operativamente completo.

La diferencia puede entenderse con una analogía. Imagínese un contable que hace cálculos mentalmente, sin escribir nada. Se le puede pedir que "tenga cuidado" (guardrails), se le puede dar instrucciones muy detalladas (prompt engineering), se le puede formar para que desarrolle buenos hábitos (RLHF). Pero mientras no escriba las operaciones en un libro de cuentas, nadie puede auditar su trabajo. No porque sea deshonesto — porque no hay nada que auditar. ACAI propone construir el libro de cuentas: las representaciones formales sobre las que opera el sistema conversacional.

1.1.1 Lo que la tesis implica

La tesis raíz tiene tres implicaciones directas que configuran toda la propuesta ACAI:

La primera implicación es que **el sistema necesita un lenguaje de representación formal del conocimiento conversacional**. Este lenguaje debe capturar la lógica del diálogo (qué pasos seguir, qué información recoger, qué confirmaciones hacer), las reglas de negocio del dominio (qué es válido, qué restricciones aplican), y las políticas de generación (qué tono usar, qué está permitido decir). El diseño de este lenguaje es la pieza central de investigación de la Fase 1.

Phase 1
El lenguaje de representación es el entregable central.

La segunda implicación es que **las decisiones conversacionales deben tomarse mediante un motor que opera sobre esas representaciones**, no por un LLM que genera respuestas libremente. El LLM tiene un papel crucial — manejar la variabilidad lingüística, interpretar lo que dice el usuario, generar respuestas naturales — pero las decisiones operativas las toma un componente simbólico que opera de forma determinista y trazable.

La tercera implicación, menos obvia pero igualmente importante, es que **un sistema formalmente correcto pero operativamente incompleto no es accountable en la práctica**. Si el sistema no puede acceder a la información que necesita, no puede ejecutar las acciones requeridas, o no puede alinear su comportamiento con las políticas de la organización, sus garantías formales son irrelevantes. Este corolario de completitud motiva el ecosistema operativo que ACAI debe integrar (UD 3).

1.1.2 Lo que la tesis no dice

> **Alcance de la tesis**
>
> La tesis raíz no afirma que toda la conversación pueda o deba formalizarse. La conversación es un fenómeno demasiado rico y demasiado variable para ser capturado completamente en representaciones formales. Lo que afirma es que las *decisiones operativas* — las que tienen consecuencias de negocio, las que deben ser auditables, las que deben ser consistentes — sí pueden y deben operar sobre representaciones formales. La frontera entre lo que se formaliza y lo que se delega al LLM es una decisión de diseño fundamental que ACAI debe tomar explícitamente.

1.2 Los principios fundacionales

Cuatro principios sostienen la tesis raíz. Cada uno es independientemente defendible; juntos configuran la arquitectura y la metodología de ACAI.

1.2.1 P1: Separación de competencias

> **P1 — Separación de competencias: variabilidad lingüística vs. lógica conversacional**
>
> La conversación tiene dos naturalezas simultáneas que requieren paradigmas computacionales distintos. La **variabilidad lingüística** — cómo se dice algo — es un problema donde los modelos neuronales son superiores. La **lógica conversacional** — qué hacer con lo que se ha dicho — es un problema donde el razonamiento simbólico ofrece determinismo, verificabilidad y trazabilidad. Mezclar ambas competencias en un solo componente produce fluidez lingüística pero destruye la gobernabilidad.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 2**. El análisis detallado de la evolución de las arquitecturas de diálogo muestra cómo cada generación resolvió un problema y creó otro, precisamente por no mantener esta separación.

Este principio es la decisión arquitectónica fundamental de ACAI. Cuando un usuario escribe "Me gustaría cancelar mi póliza, pero antes necesito saber si pierdo la bonificación", se producen simultáneamente dos fenómenos. A nivel lingüístico, hay una formulación compleja con una solicitud condicionada, una referencia implícita a un contrato existente y una pregunta sobre consecuencias. Interpretar esta formulación y producir una respuesta natural es un problema de variabilidad lingüística donde los LLMs sobresalen. A nivel lógico, hay una acción de negocio (cancelación) condicionada a una consulta (efecto sobre bonificación), que debe resolverse según reglas específicas del dominio asegurador. Garantizar que esta resolución es correcta, consistente y trazable es un problema de razonamiento donde las representaciones formales son necesarias.

Los LLMs son excelentes para el *cómo*; el razonamiento simbólico es necesario para el *qué* y el *por qué*.

P1 no implica que los componentes operen de forma aislada — como veremos en la Fase 2, la integración entre ellos puede ser profunda. Lo que implica es que las responsabilidades estén asignadas al paradigma computacional adecuado.

1.2.2 P2: El conocimiento debe estar explícitamente representado

> **P2 — El conocimiento conversacional debe estar explícitamente representado**
>
> Un sistema conversacional accountable necesita un modelo explícito de qué sabe, qué reglas aplica, y por qué decide lo que decide. El conocimiento conversacional no puede residir implícitamente en los pesos de un modelo neuronal ni en prompts *ad hoc*. Debe estar formalizado en una representación con semántica definida, inspectable, verificable y modificable independientemente del componente que la ejecuta.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 8** y **UD 9**. El análisis de las dimensiones del conocimiento conversacional y la evaluación de formalismos constituyen el desarrollo completo de este principio.

Este principio establece que la formalización no es un lujo metodológico sino una condición de posibilidad. Cuando el conocimiento reside implícitamente en los pesos de un LLM, está distribuido en miles de millones de parámetros sin interpretación semántica individual. No se puede inspeccionar, no se puede verificar contra una especificación, no se puede modificar selectivamente sin riesgo de efectos colaterales. Cuando el conocimiento reside en un prompt, está expresado en lenguaje natural que el modelo interpreta de forma probabilística — un cambio menor en la formulación puede producir un comportamiento radicalmente distinto.

La formalización explícita permite tres operaciones que son imposibles con conocimiento implícito: *inspección* (un experto puede revisar las reglas y detectar errores antes del despliegue), *verificación* (un motor puede comprobar que las reglas son consistentes entre sí y con las políticas de la organización), y *modificación selectiva* (cambiar una regla de negocio no requiere reentrenar un modelo ni reescribir un prompt completo — se modifica la regla y el comportamiento cambia predeciblemente).

1.2.3 P3: La integración neuro-simbólica es un espectro

> **P3 — La integración neuro-simbólica es un espectro, no un estado**
>
> No existe "ser" o "no ser" neuro-simbólico. Existe un **espectro de integración** que va desde la simple secuencia de componentes hasta la fusión de representaciones. Avanzar por este espectro de forma deliberada — entendiendo qué gana cada nivel y qué

> cuesta — es más productivo que intentar la integración profunda sin los cimientos formales resueltos.

Este principio justifica la estructura en dos fases del sprint ACAI. La **taxonomía de Kautz** (2022) distingue cinco niveles de integración neuro-simbólica, que van desde el nivel 1 — donde un sistema simbólico usa un módulo neuronal como caja negra — hasta el nivel 5 — donde las representaciones neuronales y simbólicas se fusionan en un sistema unificado. Cada nivel ofrece capacidades que el anterior no tiene, pero exige cimientos que el anterior no requiere.

Taxonomía de Kautz: 5 niveles de integración, del 1 (secuencial) al 5 (fusión de representaciones).

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 5**. La taxonomía de Kautz y el programa de las tres olas de Garcez y Lamb se desarrollan en profundidad.

ACAI Phase 1 opera en los niveles 1–2: el motor simbólico invoca al LLM como servicio para interpretación y generación, pero no hay condicionamiento profundo entre ellos. Es un *pipeline gobernado*: los componentes se comunican mediante interfaces estructuradas y el motor toma las decisiones operativas. ACAI Phase 2 avanza hacia los niveles 2–3: el estado simbólico condiciona la computación neuronal, las trazas de ejecución alimentan el refinamiento simbólico, y la ambigüedad se gestiona mediante razonamiento abductivo compartido.

Intentar el nivel 3 sin haber resuelto las representaciones formales del nivel 1–2 es como intentar integrar dos sistemas que no están individualmente definidos. La progresión deliberada no es cautela — es rigor.

1.2.4 P4: La accountability requiere completitud operativa

> **P4 — La accountability requiere completitud operativa**
>
> Un sistema formalmente correcto pero operativamente incompleto no es accountable en la práctica. Si el sistema no puede acceder a la información que necesita, no puede ejecutar acciones, o no puede alinear su comportamiento con políticas organizacionales, sus garantías formales son irrelevantes.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 3**. Los patrones del ecosistema operativo — RAG, agentes, Constitutional AI — y sus límites de gobernanza se analizan en profundidad.

Este principio es el corolario práctico de la tesis raíz. Un motor conversacional que opera sobre reglas formales perfectas pero que no puede consultar la base de datos de pólizas, no puede iniciar un proceso

de cancelación, y no puede acceder a las condiciones generales del contrato es un sistema formalmente elegante e inútil en la práctica. La accountability exige que el sistema sea capaz de hacer lo que necesita hacer, no solo que lo que haga sea trazable.

P4 motiva todo el ecosistema operativo de ACAI: el acceso gobernado a documentación (RAG gobernado), la capacidad de ejecutar acciones (agentes gobernados), el alineamiento con políticas (Constitutional AI formalizada), y la gestión explícita del contexto conversacional.

1.3 Los cimientos

Los cimientos son los cuerpos de conocimiento y las constataciones empíricas que ACAI toma como punto de partida. No los inventa — los usa. Cada cimiento tiene un eje del monográfico que lo desarrolla en profundidad; aquí se presenta con la profundidad suficiente para entender su papel en el marco conceptual.

1.3.1 C1: La conversación es un fenómeno multidimensional

> **C1 — La conversación es irreducible a *intents***
>
> La conversación opera simultáneamente en múltiples dimensiones: acción comunicativa (qué se hace al hablar), *grounding* (establecimiento de comprensión mutua), estructura discursiva (organización temática), gestión de la relación (cortesía, *rapport*), y reparación (corrección de malentendidos). El paradigma *intent-slot* captura una fracción. El 60–70 % de turnos conversacionales que no son "intents" — confirmaciones, correcciones, cambios de tema, metacomunicación — no son ruido sino estructura esencial.

La consecuencia directa para ACAI es que el formalismo de representación no puede limitarse a un modelo de intents y slots. Debe poder expresar actos comunicativos más ricos, condiciones de *grounding*, protocolos de reparación, y restricciones de estructura discursiva. La **UD 2** analiza estas dimensiones en detalle y su impacto en el diseño del lenguaje de representación.

Bunt (2011) identifica hasta 9 dimensiones simultáneas en un acto comunicativo.

1.3.2 C2: Los LLMs ofrecen fluidez sin gobernabilidad

> **C2 — Los LLMs han resuelto la variabilidad lingüística pero no la gobernabilidad**
>
> Los LLMs generan lenguaje natural con fluidez sin precedentes, interpretan formulaciones complejas, y manejan contexto conversacional. Pero sus decisiones son opacas, inconsistentes entre ejecuciones, y no verificables contra especificaciones. Esta complementariedad — excelencia lingüística, carencia de gobernabilidad — hace que la integración con componentes simbólicos no sea un capricho académico sino una necesidad práctica.

Esta complementariedad es la razón de ser de la arquitectura neuro-simbólica de ACAI. Los LLMs no son el problema — son una parte esencial de la solución. El problema es asignarles un papel para el que no están diseñados: la toma de decisiones verificables.

1.3.3 C3: El marco regulatorio exige accountability operativa

> **C3 — El marco regulatorio exige accountability operativa, no declarativa**
>
> El EU AI Act y DORA no piden que los sistemas "intenten ser explicables" sino que cada decisión sea trazable, auditable y verificable contra especificaciones explícitas. Las aproximaciones post-hoc no satisfacen estos requisitos porque no pueden garantizar que la explicación corresponde al proceso real de decisión.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 4**. La distinción entre explicabilidad post-hoc y constitutiva, y los requisitos regulatorios concretos se analizan en detalle.

C3 transforma la motivación de ACAI de una preferencia de diseño a una necesidad regulatoria. Los sectores financiero, asegurador, sanitario y de administración pública están sujetos a marcos regulatorios que exigen trazabilidad formal, no aproximada.

1.3.4 C4: La IA neuro-simbólica proporciona el marco teórico

> **C4 — La IA neuro-simbólica proporciona el marco teórico para la integración**
>
> La taxonomía de Kautz y el programa de las tres olas de Garcez y Lamb proporcionan un marco para pensar la integración con rigor. Las técnicas existentes — *constrained decoding*, *structured*

> *output*, *semantic parsing* con restricciones, *differentiable reasoning* — son herramientas disponibles. ACAI no inventa el paradigma neuro-simbólico; lo aplica a un dominio específico con un objetivo específico.

🔗 **Referencia contextual.** Este concepto se desarrolla en profundidad en **UD 5**, **UD 6** y **UD 7**. El paradigma neuro-simbólico se desarrolla en todo el Eje B, desde los fundamentos teóricos hasta las técnicas concretas de implementación.

1.3.5 C5: La ingeniería del conocimiento tiene décadas de experiencia

> **C5 — La ingeniería del conocimiento tiene décadas de experiencia en formalización**
>
> Existen formalismos probados (ontologías, *statecharts*, reglas de producción, modelos de procesos), metodologías de extracción de conocimiento, y lecciones aprendidas sobre qué es formalizable y qué no. ACAI no parte de cero en representación del conocimiento, pero opera bajo restricciones nuevas que los formalismos clásicos no contemplaban — en particular, la restricción de generabilidad por LLM.

🔗 **Referencia contextual.** Este concepto se desarrolla en profundidad en **UD 8** y **UD 9**. Las dimensiones del conocimiento conversacional y la evaluación de formalismos candidatos constituyen la aplicación directa de este cimiento.

1.3.6 C6: El ecosistema tecnológico proporciona componentes maduros

> **C6 — El ecosistema tecnológico de IA proporciona componentes maduros esenciales**
>
> RAG como patrón de acceso a información documental, las arquitecturas agénticas como patrón de orquestación y acción, y Constitutional AI como patrón de alineamiento son componentes del estado del arte que ACAI no inventa pero debe integrar. No son frentes de innovación — son infraestructura sobre la que ACAI construye. Ignorarlos produce un sistema académicamente interesante pero operativamente inviable.

🔗 **Referencia contextual.** Este concepto se desarrolla en profundidad en **UD 3**. El ecosistema operativo y sus límites de gobernanza se analizan en profundidad, estableciendo

qué resuelve cada patrón y dónde falla en términos de accountability.

1.4 El ecosistema operativo

El ecosistema operativo agrupa los componentes que ACAI necesita incorporar no porque sean innovadores sino porque sin ellos el sistema es operativamente incompleto. La innovación de ACAI no está en estos componentes sino en *cómo los gobierna*: mediante el mismo modelo de conocimiento formal que gobierna la conversación.

1.4.1 E1: RAG gobernado

Un sistema conversacional desplegado en un dominio real necesita acceder a repositorios de información extensa y cambiante: normativa, condiciones de productos, manuales, FAQs. **RAG** (*Retrieval-Augmented Generation*) es el patrón establecido para este acceso. Lo que ACAI aporta es un **RAG gobernado**: la decisión de cuándo buscar, qué buscar, cómo integrar lo encontrado y qué nivel de confianza atribuirle está determinada por el conocimiento formalizado, no por el LLM.

RAG: Retrieval-Augmented Generation (Lewis et al., 2020).

La diferencia es arquitectónica, no técnica. En un RAG estándar, no se puede auditar por qué se recuperó un documento concreto ni cómo influyó en la respuesta. En un RAG gobernado, la traza incluye todo el proceso de decisión.

1.4.2 E2: Capacidad agéntica gobernada

Un sistema conversacional real necesita ejecutar acciones: consultar bases de datos, iniciar transacciones, invocar servicios externos, escalar a humanos. Los patrones agénticos proporcionan esta capacidad. La diferencia de ACAI es que la **capacidad agéntica gobernada** implica que las acciones disponibles, las condiciones para ejecutarlas, y las verificaciones post-ejecución están formalizadas en el modelo de conocimiento. El agente no decide autónomamente qué herramienta usar; el motor simbólico lo determina según las reglas del dominio.

1.4.3 E3: Alineamiento formalizado

El sistema necesita operar dentro de límites éticos, legales y organizacionales. ACAI propone que las políticas de alineamiento estén formalizadas en el mismo modelo de conocimiento que el resto del comportamiento. No como una capa separada de prompts constitucionales sino como parte integral del modelo formal. Consecuencia: las políticas son verificables del mismo modo que las reglas de negocio.

Constitutional AI (Bai et al., 2022) formalizada: las "constituciones" se convierten en reglas verificables.

1.4.4 E4: Gestión de contexto conversacional

Un sistema multi-turno necesita un modelo de contexto explícito. ACAI adopta el concepto de **information state**: un modelo formal del estado del diálogo que se actualiza explícitamente con cada turno y que el motor usa para determinar el siguiente paso. Esto contrasta con el contexto implícito en la ventana del LLM, que carece de estructura, se degrada con la longitud, y no es inspectable.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 3**. Cada componente del ecosistema operativo se analiza con la profundidad técnica necesaria, incluyendo sus versiones estándar y las diferencias de la versión "gobernada" de ACAI.

> **Implicaciones del ecosistema operativo**
>
> Los componentes E1–E4 no son opcionales. Un sistema ACAI que no los incorpore será formalmente correcto pero prácticamente inútil. Sin embargo, la innovación de ACAI no está en implementar RAG o agentes — está en gobernarlos formalmente. El frente de innovación I5 (gobernanza formal del ecosistema) recoge este objetivo.

1.5 Los frentes de innovación

Los frentes de innovación son las contribuciones originales de ACAI — lo que propone de nuevo, lo que no existe como tal en la literatura. Cada frente tiene su sede en una fase del sprint y su preparación en unidades específicas del monográfico.

1.5.1 I1: La triple restricción como espacio de diseño

> **I1 — La triple restricción**
>
> El lenguaje de representación del conocimiento conversacional de ACAI debe satisfacer simultáneamente tres restricciones: ser **legible** por un experto de dominio no técnico, ser **procesable** determinísticamente por un motor de ejecución, y ser **generable** fiablemente por un LLM a partir de descripciones en texto natural. Esta conjunción no se ha planteado antes en la literatura. La restricción de generabilidad por LLM es completamente nueva — no existía como requisito antes de la era de los modelos generativos.

Phase 1
Pieza central de investigación.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 9** y **UD 11**. La evaluación de formalismos candidatos contra la triple restricción y el análisis de sus tensiones se desarrollan en profundidad.

Estas tres restricciones generan tensiones que definen el espacio de diseño. La legibilidad favorece representaciones cercanas al lenguaje natural; la procesabilidad exige precisión formal; la generabilidad favorece formatos familiares al LLM (YAML, JSON, pseudocódigo). La intersección de las tres restricciones es estrecha, y encontrarla es el desafío central de la Fase 1.

1.5.2 I2: LLM como meta-herramienta

Phase 1
Pipeline de formalización asistida.

> **I2 — LLM como meta-herramienta: construir la maquinaria que lo gobierna**
>
> ACAI propone usar un LLM no para conversar sino para transformar descripciones textuales en representaciones formales que luego gobernarán la conversación. El modelo genera las reglas que después lo restringen. Este meta-uso — el LLM construye la maquinaria formal que controla su propio comportamiento conversacional — no tiene precedente directo en la literatura de IA conversacional.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 10**. La formalización automática y asistida, desde los antecedentes pre-LLM hasta los experimentos de generabilidad, se desarrollan como unidad con entregable directo para la Fase 0.

El pipeline de formalización asistida opera en design time, no en runtime. Un experto de dominio describe un proceso conversacional en lenguaje natural; el LLM transforma esa descripción en una representación formal; la representación se valida y se integra en el modelo de conocimiento. El sistema conversacional que opera en runtime usa esa representación formal — no invoca al LLM para decidir qué hacer, sino para interpretar lo que el usuario dice y para generar la respuesta que comunica lo que el motor ha decidido.

1.5.3 I3: La frontera neural-simbólica como zona de interacción

Phase 2
Tres ejes de investigación.

> **I3 — La frontera neuro-simbólica como zona de interacción bidireccional**
>
> En la Phase 1, la frontera entre lo neuronal y lo simbólico es una interfaz secuencial: LLM interpreta, motor decide, LLM genera. En la Phase 2, esta frontera se convierte en una zona de interacción

bidireccional, articulada en tres sub-frentes.

I3a: Condicionamiento simbólico del procesamiento neuronal

El estado del diálogo modifica activamente cómo el LLM interpreta el siguiente turno. No como un prompt genérico ("estamos hablando de seguros") sino como una restricción formal que reduce el espacio de interpretación: qué entidades espera el motor, qué valores son válidos, qué actos comunicativos son posibles en este punto de la conversación. Las técnicas de *constrained decoding*, *structured prompting* condicionado y mecanismos de atención dirigida son los vehículos para esta integración.

I3b: Aprendizaje simbólico a partir de trazas de ejecución

Las trazas conversacionales de la Phase 1 se analizan para detectar patrones no cubiertos por las reglas formales y proponer extensiones del modelo. El conocimiento simbólico evoluciona a partir de evidencia empírica: si las trazas muestran que los usuarios formulan consistentemente una solicitud de una forma que el motor no anticipa, el sistema propone una extensión de las reglas que la cubra.

I3c: Razonamiento abductivo en la interpretación

Ante inputs ambiguos, el sistema genera hipótesis interpretativas combinando evidencia neuronal (la distribución de probabilidad del LLM) con restricciones simbólicas (lo que es posible dado el estado del diálogo), ordenadas por plausibilidad formal, con *provenance tracking* que permite auditar por qué se eligió una interpretación sobre otra.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 5**, **UD 6** y **UD 7**. El Eje B proporciona los fundamentos teóricos, los patrones arquitectónicos y las técnicas concretas que I3 necesita.

1.5.4 I4: Accountability verificable en dominio conversacional

I4 — Accountability verificable

ACAI debe demostrar que una arquitectura gobernada por conocimiento formal produce propiedades medibles de accountability que las arquitecturas end-to-end no pueden ofrecer. Esto incluye la definición de métricas específicas — consistencia, trazabilidad, gobernabilidad — y la evaluación comparativa entre niveles de integración neuro-simbólica.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 4** y **UD 12**. Los fundamentos conceptuales de la accountability y las metodologías de evaluación se

desarrollan en estas unidades.

I4 es el frente que cierra el círculo: ACAI propone una arquitectura (I1–I3), la implementa (Fases 1 y 2), y debe demostrar empíricamente que funciona (I4). Sin esta demostración, la contribución es arquitectónica pero no empírica.

1.5.5 I5: Gobernanza formal del ecosistema operativo

> **I5 — Gobernanza formal del ecosistema**
>
> El modelo de conocimiento de ACAI no solo gobierna la conversación sino todo el ecosistema operativo: cuándo y cómo se accede a documentos, qué acciones se ejecutan, qué políticas aplican. Trazabilidad end-to-end bajo un mismo modelo formal.

I5 es la consecuencia directa de P4 (completitud operativa). Un sistema que traza las decisiones conversacionales pero no puede explicar por qué consultó un documento o por qué ejecutó una acción tiene una trazabilidad incompleta. La gobernanza unificada bajo un modelo formal garantiza que toda la operación es auditable.

1.6 La arquitectura ACAI

La arquitectura de ACAI materializa los principios y frentes de innovación en un diseño concreto. Se presenta en dos versiones que corresponden a las dos fases del sprint.

1.6.1 Phase 1: el pipeline gobernado

Phase 1
Niveles 1–2 de Kautz.

La arquitectura de la Phase 1 tiene dos momentos operativos: compilación (*design time*) y ejecución (*runtime*).

> **Compilación: del texto natural al modelo formal**
>
> El experto de dominio describe el proceso conversacional en lenguaje natural. El LLM transforma esa descripción en una representación formal siguiendo el lenguaje de representación de ACAI. La representación se valida — sintácticamente (¿es parseable?), semánticamente (¿es consistente?), y con el experto (¿captura lo que se quería expresar?). El conocimiento formalizado manualmente sirve como *ground truth* y como *fallback* si el pipeline automatizado no alcanza calidad suficiente.

Ejecución: el turno conversacional gobernado

Cada turno de conversación sigue un flujo determinado. El usuario produce un input en lenguaje natural. El LLM interpreta ese input y lo mapea a conceptos del modelo formal (acto comunicativo, entidades, valores). El motor simbólico consulta el modelo de conocimiento formalizado, evalúa las reglas aplicables, y determina la acción: qué información solicitar, qué confirmar, qué acción ejecutar, qué responder. El LLM genera la respuesta en lenguaje natural conforme a lo que el motor ha determinado. La traza completa — input, interpretación, reglas aplicadas, decisión, respuesta — se registra.

El pipeline gobernado en acción

Contexto: Sistema de atención al cliente en una aseguradora. El conocimiento formalizado incluye las reglas de cancelación de pólizas.
Usuario: "Quiero cancelar mi seguro de hogar."
LLM (NLU): → Acto: `solicitud_cancelación`; Producto: `seguro_hogar`
Motor simbólico: Consulta reglas → Cancelación requiere: (1) verificar identidad, (2) informar penalización si aplica, (3) obtener confirmación explícita. Estado actual: identidad no verificada → Acción: solicitar verificación.
LLM (NLG): "Entendido. Para proceder con la cancelación de su seguro de hogar, necesito verificar su identidad. ¿Podría indicarme su número de póliza y DNI?"
Traza: Input → interpretación → regla `cancel_req_01` → precondición `verify_identity` → respuesta generada.

1.6.2 Phase 2: la integración bidireccional

Phase 2
Niveles 2–3 de Kautz.

La Phase 2 transforma la frontera rígida de la Phase 1 en una zona de interacción. Tres mecanismos operan simultáneamente: el estado simbólico condiciona la computación neuronal (I3a), las trazas de ejecución alimentan el refinamiento del modelo formal (I3b), y la ambigüedad se gestiona mediante razonamiento abductivo compartido (I3c).

La **métrica central** de la Phase 2 es comparativa: debe demostrar que la integración de nivel 2–3 produce resultados que el pipeline de nivel 1–2 no puede. Mayor robustez ante formulaciones inesperadas, mejor manejo de ambigüedad, adaptación más rápida a cambios de dominio, o explicaciones más precisas. Sin esta demostración comparativa, la contribución es arquitectónica pero no empírica.

1.7 Posicionamiento en el proyecto (AI)^2:TDG

ACAI es un Research Sprint dentro del proyecto (AI)^2:TDG (*Accountable and Interpretable AI through Topology and Discrete Geometry*), un proyecto de Generación de Conocimiento que investiga cómo las herramientas de la topología y la geometría discreta pueden fundamentar la explicabilidad y la accountability en sistemas de IA.

De este proyecto marco, ACAI hereda tres directrices fundamentales que orientan todas sus decisiones de diseño:

1. La explicabilidad debe fundamentarse en **representaciones formales explícitas**, no en aproximaciones post-hoc. Esta directriz se materializa directamente en la tesis raíz y en P2.
2. La accountability es una **propiedad constitutiva del diseño**, no una capa añadida. Esta directriz es la razón por la que ACAI propone una arquitectura gobernada desde el origen, no un LLM con filtros.
3. El trabajo debe producir contribuciones tanto **metodológicas como de validación**. Esta directriz exige que ACAI no solo proponga una arquitectura sino que demuestre empíricamente sus propiedades (I4).

ACAI Phase 1 conecta temáticamente con (AI)^2:TDG; Phase 2 conecta técnicamente.

ACAI se ancla en tres líneas de investigación del proyecto marco: **RL2.3** (Neuro-Symbolic Integration and Reasoning), que proporciona el marco teórico para la integración de paradigmas; **RL3.2** (Verifiable Conversational Systems), que define el dominio de aplicación; y **RL2.2** (Knowledge Representation and Graph Architectures), que fundamenta la representación del conocimiento.

Las líneas de RL1 — herramientas topológicas y de geometría discreta — entran con fuerza como continuación natural más allá del sprint: las trazas conversacionales como trayectorias en espacios de estados cuyo análisis topológico revela estructura no capturada por las reglas, y los invariantes topológicos sobre el espacio conversacional como herramientas de verificación.

1.8 Estructura del monográfico

Este monográfico comprende 12 unidades didácticas organizadas en una unidad de entrada (esta) y 4 ejes temáticos. Los ejes progresan del "por qué" al "con qué" al "cómo medir", y cada uno cubre un aspecto específico del marco conceptual de ACAI.

1.8.1 Los cuatro ejes

El **Eje A** (**El problema**) comprende las unidades **UD 2–UD 4** y responde a la pregunta de por qué existe ACAI y sobre qué terreno opera. Cubre la conversación como fenómeno (**UD 2**), el ecosistema tecnológico actual y sus límites (**UD 3**), y los fundamentos de la accountability y el marco regulatorio (**UD 4**).

El **Eje B** (**El paradigma neuro-simbólico**) comprende las unidades **UD 5–UD 7** y proporciona el marco teórico y las herramientas. Cubre los fundamentos del paradigma (**UD 5**), las arquitecturas y patrones de integración (**UD 6**), y las técnicas concretas para procesamiento del lenguaje (**UD 7**).

El **Eje C** (**Representación y formalización**) comprende las unidades **UD 8–UD 11** y aborda cómo se formaliza el conocimiento conversacional. Cubre las dimensiones del conocimiento (**UD 8**), los formalismos candidatos (**UD 9**), la formalización asistida por LLM (**UD 10**), y la triple restricción como problema de diseño (**UD 11**). Las unidades **UD 9** y **UD 10** producen directamente entregables de la Fase 0 del sprint.

El **Eje D** (**Evaluación**) comprende la unidad **UD 12** y proporciona las herramientas para medir si lo que se construye funciona, con métricas específicas de accountability.

1.8.2 Mapa de dependencias y líneas de estudio

El monográfico ofrece dos líneas de estudio paralelas que convergen en el Eje C:

> **Líneas paralelas**
>
> **Línea 1 (Eje A):** **UD 1** → **UD 2** → **UD 3** → **UD 4**. El problema, el ecosistema, los requisitos.
> **Línea 2 (Eje B):** **UD 1** → **UD 5** → **UD 6** → **UD 7**. El paradigma, las arquitecturas, las técnicas.
> **Convergencia (Eje C):** **UD 8** requiere ambas líneas → **UD 9** → **UD 10** → **UD 11**.
> **Cierre (Eje D):** **UD 12** requiere al menos **UD 4** + **UD 8** + **UD 11**.

Las dos líneas paralelas se pueden estudiar simultáneamente, lo que reduce el tiempo total estimado a 10–11 semanas (7–8 si el investigador tiene base fuerte en algún eje).

1.8.3 Integración con la Fase 0 del sprint

Dos unidades producen directamente entregables de la Fase 0.

Unidad	Entregable Fase 0	Eje
UD 9	Obj. 0.1: Mapa de formalismos evaluados	C
UD 10	Obj. 0.2: Experimentos de generabilidad	C

Los objetivos 0.3 (candidatos de caso de uso) y 0.4 (stack tecnológico) no son unidades del monográfico sino trabajo aplicado que se realiza en paralelo, informado por los contenidos del Eje C.

1.9 Mapa de relaciones: la estructura conceptual completa

La estructura conceptual de ACAI puede visualizarse como una jerarquía que progresa desde la tesis raíz hasta los frentes concretos de innovación.

La **tesis raíz** (la accountability como problema de representación) establece la dirección. Los **principios** (P1–P4) fundamentan la arquitectura. Los **cimientos** (C1–C6) proporcionan el terreno empírico y teórico sobre el que se construye. El **ecosistema operativo** (E1–E4) garantiza la completitud práctica. Y los **frentes de innovación** (I1–I5) definen lo que ACAI propone de nuevo.

> **Cobertura completa**
>
> No hay elemento del marco conceptual sin cobertura en el monográfico. No hay unidad sin conexión con el marco. Cada unidad didáctica se justifica por su conexión directa con una decisión, componente o capacidad necesaria en el sprint. Esta tabla resume la cobertura:

Elemento	Unidades
Tesis raíz	UD1, UD4
P1: Separación de competencias	UD1, UD2, UD5
P2: Conocimiento representado	UD1, UD8
P3: Integración como espectro	UD1, UD5
P4: Completitud operativa	UD1, UD3
C1–C2: Conversación y LLMs	UD2, UD3
C3: Marco regulatorio	UD4
C4: Neuro-simbólico	UD5, UD6
C5: Ingeniería del conocimiento	UD8, UD9
C6: Ecosistema tecnológico	UD3
E1–E4: Ecosistema operativo	UD3
I1: Triple restricción	UD9, UD11
I2: LLM como meta-herramienta	UD10
I3a–c: Integración profunda	UD5, UD6, UD7
I4: Accountability verificable	UD4, UD12
I5: Gobernanza del ecosistema	UD3, UD9

Conexiones con otras unidades

Aplicación: Esta unidad es el punto de partida de todo el monográfico. No tiene prerequisitos y es prerequisito de todas las demás unidades.

Complementa: La **UD 4** profundiza en la accountability (tesis raíz). La **UD 5** profundiza en la integración neuro-simbólica (P3). La **UD 8** profundiza en el conocimiento conversacional (P2). La **UD 3** profundiza en el ecosistema operativo (P4).

Aplicación: El investigador debe usar esta unidad como referencia constante durante el estudio de todos los ejes, para situar cada concepto nuevo dentro del marco conceptual general de ACAI.

Ejercicios

1. [Comprensión] Explique en sus propias palabras la diferencia entre abordar la accountability como problema de restricción y como problema de representación. Proporcione un ejemplo concreto de cada enfoque en el dominio de atención al cliente bancaria.

2. [Análisis] Para cada uno de los cuatro principios fundacionales (P1–P4), identifique una tecnología o práctica actual de la industria que viola ese principio. Explique por qué la viola y qué consecuencia tiene para la accountability.

3. [Síntesis] Seleccione uno de los frentes de innovación (I1–I5) y trace su conexión hacia arriba: qué principio lo motiva, qué cimientos lo soportan, y qué componente del ecosistema operativo lo necesita. Presente esta conexión como un diagrama comentado.

4. [Aplicación] Considere el siguiente escenario: una universidad quiere desplegar un asistente conversacional para ayudar a los estudiantes con trámites de matrícula. Identifique: (a) qué dimensiones del conocimiento conversacional están involucradas, (b) qué componentes del ecosistema operativo necesitaría, y (c) dónde estaría la frontera entre lo formalizado y lo delegado al LLM.

5. [Reflexión] La Phase 2 de ACAI debe demostrar empíricamente que la integración neuro-simbólica profunda mejora respecto al pipeline secuencial de la Phase 1. ¿Qué ocurriría si esa demostración fracasa? ¿Significaría que ACAI es un fracaso? Argumente su posición.

Referencias y lecturas

Lectura de entrada

El documento de propuesta ACAI (versión ejecutiva) es la lectura de entrada imprescindible. Proporciona la visión completa del sprint en un formato compacto. El investigador debe haberlo leído antes de abordar esta unidad.

Lecturas esenciales

- Kautz, H. (2022). The Third AI Summer: AAAI Robert S. Engelmore Memorial Lecture. *AI Magazine*, 43(1), 93–104. — Imprescindible para la taxonomía de integración neuro-simbólica que estructura las fases de ACAI.
- Garcez, A. d'A. & Lamb, L. C. (2023). Neurosymbolic AI: The 3rd Wave. *Artificial Intelligence Review*, 56, 12387–12406. — Marco de las tres olas que contextualiza el programa de investigación en el que ACAI se inscribe.
- Arrieta, A. B. et al. (2020). Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58, 82–115. — Taxonomía completa de explicabilidad que fundamenta la distinción entre explicabilidad post-hoc y constitutiva.

Lecturas complementarias

- Hitzler, P. et al. (Eds.) (2022). *Neuro-Symbolic Artificial Intelligence: The State of the Art*. IOS Press. — Referencia enciclopédica sobre el campo neuro-simbólico.

- Bai, Y. et al. (2022). Constitutional AI: Harmlessness from AI Feedback. Anthropic. — Fundamento del patrón de alineamiento que ACAI propone formalizar.
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*. — Artículo fundacional de RAG, el patrón que ACAI propone gobernar formalmente.
- Memoria del proyecto $(AI)^2$:TDG. — Documento interno del proyecto marco. Imprescindible para las conexiones con las líneas de investigación RL1–RL3.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«The problem is not that machines cannot converse.

The problem is that we have forgotten what conversing means.»

Eje A — El problema: IA conversacional y accountability

Unidad 2 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje A — El problema

Prerequisito: UD1 · Eje A — El problema

UNIDAD DIDÁCTICA 2

La conversación como fenómeno y como problema de ingeniería

Contenidos de la unidad

2.1.1 Más allá de la transmisión de información 3
2.1.2 Las dimensiones simultáneas de la conversación 5
Acción comunicativa 5
Grounding 6
Estructura discursiva 6
Gestión de la relación 7
Reparación 7
2.1.3 La evidencia empírica: más allá de los intents 8
2.2.1 Sistemas basados en reglas: control total, fragilidad total 10
2.2.2 El paradigma intent-slot: escalabilidad y reducción 11
2.2.3 Pipelines modulares: la promesa incumplida de la modularidad 12
2.2.4 Modelos end-to-end: fluidez a costa de control 13
2.2.5 LLMs como sistemas conversacionales: capacidad sin precedentes, opacidad sin precedentes 13
2.2.6 Síntesis: el patrón recurrente 15
2.3.1 La tesis del diagnóstico 16
2.3.2 Las aproximaciones paliativas y sus límites 17
Guardrails y filtros 17
Prompt engineering y system prompts 17
Alineamiento: RLHF y Constitutional AI 18
2.3.3 La brecha de la representación 19
2.4.1 Requisitos para el formalismo de representación 19
2.4.2 La frontera entre lo formalizado y lo delegado 20
2.4.3 El escenario como herramienta de análisis 21

¿Por qué necesito esto para ACAI?

El investigador que se incorpora a ACAI probablemente concibe la conversación como un problema de clasificación: el usuario dice algo, el sistema identifica un *intent*, extrae *slots*, ejecuta una acción y responde. Esta concepción ha dominado la industria durante una década y ha producido sistemas útiles pero fundamentalmente limitados. Esta unidad desmonta esa simplificación y la reemplaza por una comprensión del fenómeno conversacional que fundamenta directamente las decisiones de diseño del formalismo ACAI. Al terminar, el investigador entenderá por qué el lenguaje de representación de ACAI no puede ser un modelo de intents mejorado, y qué debe ser en su lugar.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Identificar las múltiples dimensiones simultáneas de la conversación y explicar por qué el paradigma *intent-slot* captura solo una fracción del fenómeno.
2. Analizar la evolución de las arquitecturas de diálogo e identificar qué se ganó y qué se perdió en cada transición paradigmática.
3. Caracterizar los fenómenos conversacionales que los sistemas actuales ignoran — *grounding*, reparación, indirección, metacomunicación — y argumentar su relevancia para un sistema accountable.
4. Formular el diagnóstico central: las arquitecturas actuales generan comportamiento conversacional sin representarlo, lo que hace la accountability estructuralmente inalcanzable.
5. Derivar requisitos concretos para el formalismo ACAI a partir del análisis del fenómeno conversacional.

2.1 Qué ocurre cuando dos personas conversan

Sede principal de este concepto.

2.1.1 Más allá de la transmisión de información

La concepción más extendida de la conversación, tanto en el imaginario popular como en buena parte de la ingeniería de sistemas de diálogo, es la del *modelo de transmisión*: un emisor codifica un mensaje, lo envía a través de un canal, y un receptor lo decodifica. Este modelo, que tiene su

La conversación no transmite información — la construye colaborativamente.

origen en la teoría matemática de la comunicación de Shannon y Weaver (1949), es útil para describir la transmisión de señales pero radicalmente inadecuado para describir la comunicación humana. Como señaló Clark (1996) en su trabajo seminal *Using Language*, el lenguaje en uso no es un proceso de transmisión sino de *acción conjunta*: los participantes coordinan sus acciones para construir significado de forma colaborativa.

Clark (1996) *Using Language* — fundacional.

Esta distinción no es un matiz académico. Tiene consecuencias directas para el diseño de sistemas conversacionales. Si la conversación fuera transmisión de información, bastaría con un sistema que decodificara mensajes y produjera respuestas. Pero si la conversación es acción conjunta, el sistema necesita participar en un proceso de coordinación que incluye establecer comprensión mutua, negociar significados, gestionar malentendidos, y mantener una relación cooperativa. Cada una de estas actividades requiere capacidades que ninguna arquitectura conversacional actual proporciona de forma completa.

Considérese un fragmento de diálogo aparentemente simple en el contexto de una aseguradora:

Diálogo de cancelación — primera aproximación

Usuario: Quiero cancelar mi seguro de hogar, pero no sé si me conviene.
Agente: Entiendo. ¿Le preocupa algún aspecto concreto de la cancelación?
Usuario: Sí, lo de la bonificación. Llevo ocho años sin siniestros.
Agente: Efectivamente, con ocho años sin siniestros tiene una bonificación del 40 %. Si cancela, la bonificación se mantiene durante 2 años por si decide contratar de nuevo.
Usuario: Ah, vale. Entonces sí, quiero cancelar.
Agente: De acuerdo. Para confirmar: cancelo su póliza de hogar HG-2019-4421. La bonificación del 40 % se preservará hasta febrero de 2028. ¿Es correcto?
Usuario: Sí, correcto.

Un análisis basado en intents identificaría dos acciones principales: `consultar_bonificación` y `cancelar_póliza`. Pero este análisis omite casi todo lo que hace que el diálogo funcione. El usuario expresa incertidumbre («no sé si me conviene»), lo que no es un intent sino una señal de estado epistémico que condiciona toda la interacción posterior. El agente realiza una maniobra de elicitación («¿Le preocupa algún aspecto concreto?») que no es una respuesta a un intent sino una estrategia conversacional para transformar una solicitud vaga en una consulta concreta. El «Ah, vale» del usuario es un acto de *grounding* — una señal de que ha comprendido y aceptado la información — que no encaja en ninguna categoría de intent pero que es esencial para que el agente pueda proceder. Y la confirmación final es un protocolo de verificación

mutua que cierra un compromiso vinculante.

Este fragmento, que cualquier agente humano resolvería sin esfuerzo, moviliza simultáneamente al menos cuatro de las dimensiones que Bunt (2011) identifica en el acto comunicativo, y su resolución adecuada requiere un modelo que represente explícitamente estados epistémicos, compromisos, condiciones de satisfacción, y protocolos de verificación. Un sistema que solo clasifica intents es estructuralmente incapaz de manejar esta complejidad.

Bunt (2011) — dimensiones del acto comunicativo.

2.1.2 Las dimensiones simultáneas de la conversación

La investigación en análisis del diálogo ha convergido, desde tradiciones disciplinares muy diferentes, en una constatación fundamental: la conversación opera simultáneamente en múltiples dimensiones que no pueden reducirse unas a otras. La teoría de actos de habla de Austin (1962) y Searle (1969) estableció que hablar es hacer — que un enunciado no solo transmite contenido proposicional sino que realiza una acción (prometer, solicitar, informar, advertir). El análisis conversacional de Sacks, Schegloff y Jefferson (1974) demostró que la conversación tiene una organización estructural propia — turnos, secuencias, reparaciones — que los participantes gestionan activamente. La teoría del *grounding* de Clark y Schaefer (1989) mostró que la comprensión mutua no se presupone sino que se construye mediante un proceso de contribución y aceptación. Y el trabajo de Brown y Levinson (1987) sobre cortesía reveló que toda interacción gestiona simultáneamente la tarea y la relación entre los participantes.

Austin (1962), Searle (1969): la tradición de actos de habla. Sacks, Schegloff & Jefferson (1974): análisis conversacional.

Bunt (2011), en su trabajo sobre la anotación multidimensional de actos de diálogo que fundamenta el estándar ISO 24617-2, propone una taxonomía que reconoce hasta nueve dimensiones funcionales simultáneas en un acto comunicativo. Para los propósitos de ACAI, cinco dimensiones son especialmente relevantes porque imponen requisitos directos sobre el formalismo de representación.

Acción comunicativa

La primera dimensión es la de la **acción comunicativa**: qué hace el hablante al producir un enunciado. La tradición de actos de habla, desde Austin hasta los trabajos más recientes de Traum y Hinkelman (1992) sobre actos de diálogo, ha establecido que los enunciados realizan acciones que van mucho más allá de la transmisión de información. Una solicitud crea una obligación condicional en el destinatario. Una promesa crea un compromiso en el hablante. Una aserción establece un contenido como parte del *common ground*. Cada tipo de acto tiene condiciones de felicidad (las circunstancias que deben darse para que el acto se

realice con éxito), efectos sobre el estado del diálogo, y obligaciones de respuesta.

El paradigma *intent-slot* captura una versión empobrecida de esta dimensión. Un intent como `cancelar_póliza` corresponde aproximadamente a un acto directivo (solicitud), pero pierde la estructura interna del acto: sus condiciones de felicidad (¿tiene el usuario autoridad para cancelar?), sus efectos (crear un compromiso de cancelación que requiere confirmación), y sus relaciones con otros actos en la secuencia conversacional. Más importante aún, el paradigma intent reduce toda la dimensión comunicativa a un problema de clasificación, cuando en realidad es un problema de *reconocimiento y gestión de compromisos*.

Traum & Hinkelman (1992) — actos de diálogo como unidad de análisis.

Grounding

La segunda dimensión es el **grounding**: el proceso mediante el cual los participantes establecen que se han entendido mutuamente. Clark y Schaefer (1989) demostraron que la comprensión no es un estado binario (entendido/no entendido) sino un proceso gradual con niveles de evidencia. Un participante presenta una contribución; el otro proporciona evidencia de comprensión — desde la mera continuación de la conversación (evidencia débil) hasta la paráfrasis explícita (evidencia fuerte). Solo cuando ambos participantes aceptan que la contribución ha sido comprendida, esta se incorpora al *common ground*.

El *grounding* no es un overhead — es el mecanismo que hace posible la coordinación.

En el escenario de la aseguradora, el «Ah, vale» del usuario no es ruido conversacional — es un acto de *grounding* que señala aceptación de la información sobre la bonificación. Y la confirmación final («Para confirmar: cancelo su póliza…¿Es correcto?») es un protocolo de *grounding* explícito que eleva el nivel de evidencia antes de ejecutar una acción irreversible. Traum (1994) formalizó este proceso en su modelo de *grounding acts*, distinguiendo entre actos de iniciación, continuación, aceptación, reparación y cancelación. Para un sistema accountable, el *grounding* no es opcional: sin evidencia formal de comprensión mutua, no se puede atribuir responsabilidad por las decisiones tomadas durante el diálogo.

Clark & Schaefer (1989) — modelo de contribución y *grounding*.

Estructura discursiva

La tercera dimensión es la **estructura discursiva**: cómo se organizan los temas, las subtareas, y las relaciones retóricas dentro de la conversación. Grosz y Sidner (1986), en un trabajo que sigue siendo referencia fundamental, propusieron un modelo tripartito del discurso que distingue entre estructura lingüística (la secuencia de enunciados), estructura intencional (los propósitos que organizan el diálogo), y estado atencional (lo que es prominente en cada momento). Este modelo captura un

fenómeno que los sistemas basados en intents ignoran completamente: la conversación tiene una organización jerárquica donde las tareas se descomponen en subtareas, los temas se abren y se cierran, y los participantes navegan esta estructura de forma colaborativa.

En el escenario de la aseguradora, la consulta sobre la bonificación es una subtarea incrustada dentro de la tarea principal de cancelación. El usuario abre esta subtarea («no sé si me conviene») y la cierra («Entonces sí, quiero cancelar») antes de retomar la tarea principal. Un sistema que no represente esta estructura no puede determinar, por ejemplo, que la intención de cancelar estaba condicionada a la resolución de la consulta, o que una nueva pregunta del usuario sobre el precio de una póliza alternativa abriría una tercera subtarea que debería cerrarse antes de retomar la cancelación.

Grosz & Sidner (1986): estructura tripartita del discurso — lingüística, intencional, atencional.

Gestión de la relación

La cuarta dimensión es la **gestión de la relación** entre los participantes. Brown y Levinson (1987) establecieron que toda interacción comunicativa gestiona simultáneamente dos tipos de *face*: la *face positiva* (el deseo de ser apreciado y aprobado) y la *face negativa* (el deseo de no ser impuesto). Muchos actos comunicativos son inherentemente amenazantes para la *face* del interlocutor: solicitar algo amenaza su *face negativa* (le impone una obligación); rechazar algo amenaza su *face positiva* (niega su deseo). Las estrategias de cortesía — indirección, atenuación, expresiones de empatía — no son adornos estilísticos sino mecanismos funcionales que permiten realizar actos amenazantes minimizando su coste relacional.

Brown & Levinson (1987) — teoría de la cortesía y *face*.

Esta dimensión puede parecer tangencial para un sistema técnico, pero tiene implicaciones directas para ACAI. Las políticas de generación (E3 en el marco conceptual) incluyen decisiones sobre tono, estilo y estrategia comunicativa que son inherentemente decisiones sobre gestión de la relación. Un sistema que informa al usuario de que perderá su bonificación con un tono neutro y otro que lo hace con expresiones de empatía y ofreciendo alternativas están tomando decisiones diferentes en la dimensión relacional, y ambas decisiones deben ser gobernables y trazables.

Reparación

La quinta dimensión es la **reparación**: los mecanismos mediante los cuales los participantes detectan y corrigen problemas en la conversación. Schegloff, Jefferson y Sacks (1977), en su análisis clásico, identificaron una organización sistemática de la reparación que distingue entre quién inicia la reparación (el hablante o el interlocutor), quién la

El 20–30 % de los turnos en diálogos naturales involucran algún tipo de reparación.

ejecuta, y en qué posición de la secuencia se produce. Los problemas que desencadenan reparación pueden ser de producción (el hablante dice algo incorrecto), de comprensión (el interlocutor no entiende), o de *grounding* (hay desacuerdo sobre lo entendido).

En la práctica, la reparación es omnipresente. Estudios empíricos como los de Colman y Healey (2011) estiman que entre el 20 % y el 30 % de los turnos en diálogos naturales involucran algún tipo de mecanismo reparador, desde la simple auto-corrección hasta la negociación extendida de significado. Para un sistema conversacional accountable, la reparación plantea un desafío particular: cuando el usuario corrige al sistema («No, no quiero cancelar el de hogar, sino el de coche»), el sistema debe actualizar su modelo del estado del diálogo de forma trazable, registrando qué se entendió inicialmente, por qué era incorrecto, y cómo se corrigió. Un sistema que simplemente "reinterpreta" el diálogo sin registrar la reparación pierde información crucial para la auditabilidad.

Schegloff, Jefferson & Sacks (1977): la organización de la reparación en la conversación.

2.1.3 La evidencia empírica: más allá de los intents

Las cinco dimensiones descritas no son abstracciones teóricas sino fenómenos empíricamente observables y cuantificables. Una serie de estudios ha documentado con precisión la distancia entre lo que ocurre en diálogos reales y lo que los sistemas basados en intents son capaces de capturar.

Bunt et al. (2012), en su trabajo de validación del esquema de anotación DiAML-ISO, analizaron extensos corpus de diálogos anotados según las dimensiones funcionales del estándar ISO 24617-2. Sus resultados muestran que, en promedio, el 35 % de los enunciados en diálogos orientados a tareas realizan funciones en más de una dimensión simultáneamente. Un enunciado como «Entonces la bonificación se mantiene, ¿no?» opera simultáneamente en la dimensión de *grounding* (busca confirmación), en la dimensión de gestión de la tarea (verifica una precondición para la decisión), y en la dimensión de estructura discursiva (señala el cierre de la subtarea de consulta).

Más revelador aún es el trabajo de Stolcke et al. (2000) sobre el corpus Switchboard, donde la anotación de actos de diálogo reveló que los *backchannels* («uh-huh», «right», «yeah»), las señales de *grounding*, y los actos de gestión del diálogo constituyen entre el 30 % y el 40 % de todos los turnos. Estos turnos no son clasificables como intents porque no expresan intención del usuario en el sentido de solicitud-de-acción; son actos de coordinación que mantienen la conversación funcionando. Un sistema que los ignora no solo pierde información — pierde el mecanismo que hace posible la comunicación efectiva.

Bunt et al. (2012) — validación empírica de DiAML-ISO.

Stolcke et al. (2000): *backchannels* y actos de gestión constituyen el 30–40 % de los turnos.

En el dominio específico de los sistemas de diálogo orientados a tareas, Budzianowski et al. (2018) documentaron, en la construcción del corpus MultiWOZ, que incluso en diálogos diseñados para ser "simples" — reservas de hotel, restaurante, taxi — los anotadores encontraron fenómenos como cambios de opinión del usuario (18 % de los diálogos), solicitudes compuestas que combinan múltiples dominios (25 %), correcciones de información previamente proporcionada (12 %), y negociaciones donde el usuario rechaza una propuesta y solicita alternativas (22 %). Cada uno de estos fenómenos requiere capacidades que van más allá de la clasificación de intents: requieren representar estados, compromisos, y relaciones entre actos a lo largo de la conversación.

Trabajos más recientes en el contexto de LLMs han confirmado y ampliado estos hallazgos. Zheng et al. (2024), en su análisis de conversaciones reales con chatbots desplegados en producción, documentaron que el 40 % de las interacciones que fracasan lo hacen no por incapacidad lingüística del modelo sino por fallos en la gestión conversacional: el sistema pierde el hilo de una subtarea, no detecta un cambio de tema, reinterpreta incorrectamente una corrección del usuario, o no cierra adecuadamente un compromiso antes de avanzar. Estos fallos son invisibles para las métricas de calidad lingüística — las respuestas son gramaticalmente correctas y semánticamente plausibles — pero devastadores para la experiencia del usuario y, crucialmente, para la accountability del sistema.

Implicaciones para el formalismo ACAI

La evidencia empírica impone requisitos concretos al lenguaje de representación de ACAI. El formalismo no puede limitarse a un catálogo de intents con sus slots asociados. Debe poder representar, como mínimo: actos comunicativos con sus condiciones de felicidad y efectos sobre el estado del diálogo; condiciones de *grounding* y protocolos de verificación; estructura discursiva jerárquica con subtareas anidadas; y mecanismos de reparación con trazabilidad de las correcciones. No todas estas dimensiones serán formalizables en el alcance del sprint ACAI — la **UD 8** analizará los límites de la formalización — pero el diseño del formalismo debe al menos contemplar su existencia y prever mecanismos de extensión.

2.2 La evolución de las arquitecturas de diálogo

La historia de los sistemas de diálogo no es una marcha lineal hacia sistemas mejores sino una serie de transiciones paradigmáticas donde cada generación resolvió un problema fundamental de la anterior a costa de crear uno nuevo. El análisis de estas transiciones es directamente relevante para ACAI porque revela un patrón recurrente: el campo ha oscilado entre *control* (la capacidad de garantizar el comportamiento del sistema) y *naturalidad* (la capacidad de manejar la variabilidad del lenguaje humano) sin lograr ambos simultáneamente. ACAI propone precisamente esa síntesis.

2.2.1 Sistemas basados en reglas: control total, fragilidad total

Los primeros sistemas de diálogo operativos, desde ELIZA (Weizenbaum, 1966) hasta los sistemas de reserva de viajes de los años 90 como ATIS (Hemphill, Godfrey y Doddington, 1990), compartían una característica fundamental: todo el comportamiento estaba codificado explícitamente en reglas escritas por humanos. Los patrones de reconocimiento, las transiciones entre estados del diálogo, las respuestas posibles, y las condiciones para cada respuesta estaban especificados en el código del sistema.

ELIZA (Weizenbaum, 1966): pattern matching superficial. ATIS (Hemphill et al., 1990): gramáticas de dominio.

Las ventajas de esta aproximación eran precisamente las propiedades que ACAI busca recuperar. El comportamiento del sistema era completamente determinista: dada una entrada y un estado, la respuesta era predecible. La trazabilidad era total: cada respuesta podía rastrearse hasta la regla que la había producido. La verificación era posible: se podían enumerar los estados y transiciones del sistema y comprobar propiedades formales como la ausencia de estados sin salida o la garantía de que ciertas preguntas siempre se formulaban.

La desventaja era igualmente fundamental: la fragilidad ante la variabilidad lingüística. El usuario debía expresarse de formas que las reglas anticipaban. Una formulación inesperada — un sinónimo, una indirección, una elipsis — provocaba un fallo. Y el coste de ampliar la cobertura era prohibitivo: cada nueva formulación requería nuevas reglas que debían coordinarse con las existentes, creando una complejidad combinatoria intratable. Weizenbaum mismo reconoció que ELIZA producía la *ilusión* de comprensión mediante trucos de pattern matching, sin ningún modelo real del lenguaje ni de la conversación.

La lección de los sistemas basados en reglas es que **el control determinista sobre representaciones explícitas produce las propiedades de accountability que necesitamos, pero requiere un mecanismo para manejar la variabilidad lingüística que las reglas no pueden proporcionar**. Esta lección es el fundamento directo de P1 (separación de competen-

cias): las reglas no deben encargarse de la variabilidad lingüística, y el componente que la maneja no debe encargarse de las decisiones.

2.2.2 El paradigma intent-slot: escalabilidad y reducción

A partir de mediados de la década de 2000, con la maduración de las técnicas de aprendizaje automático para procesamiento del lenguaje, emergió el paradigma que ha dominado la industria durante más de una década: la arquitectura basada en *intents* y *slots*. La idea central, articulada en trabajos como los de Tur y De Mori (2011) y operacionalizada por plataformas como Dialogflow, LUIS y posteriormente RASA, es descomponer la comprensión del lenguaje en dos problemas de clasificación: identificar la *intención* del usuario (qué quiere hacer) y extraer los *parámetros* relevantes (con qué entidades).

Tur & De Mori (2011) — *Spoken Language Understanding*: el paradigma intent-slot formalizado.

Este paradigma representó un avance genuino en un aspecto crucial: al reformular la comprensión como clasificación, permitió aprovechar datos de entrenamiento para generalizar a formulaciones no vistas explícitamente. Un clasificador entrenado con ejemplos de intents de cancelación podía reconocer formulaciones nuevas con razonable fiabilidad. La escalabilidad a nuevos dominios mejoró significativamente respecto a los sistemas de reglas puras.

Pero el precio fue una *reducción radical del fenómeno conversacional*. El paradigma intent-slot captura exclusivamente la dimensión de acción comunicativa — y solo una versión simplificada de ella. Los actos comunicativos se reducen a categorías planas (intents) sin estructura interna, sin condiciones de felicidad, y sin efectos formales sobre el estado del diálogo. Las cuatro dimensiones restantes — *grounding*, estructura discursiva, gestión de la relación, reparación — simplemente desaparecen del modelo. No se representan mal; no se representan en absoluto.

El paradigma intent-slot no simplifica la conversación — la *mutila*.

Esta reducción no es un accidente de implementación sino una consecuencia directa de la formulación del problema. Si la comprensión del lenguaje se define como clasificación de intents, todo lo que no es un intent queda fuera del modelo. Las confirmaciones, las correcciones, los *backchannels*, las transiciones entre subtareas, las expresiones de incertidumbre — todo el tejido conectivo que hace que la conversación funcione — se convierte en ruido que el clasificador debe ignorar o en "intents especiales" que distorsionan la taxonomía.

Varios trabajos han documentado las consecuencias de esta reducción. Liu y Lane (2016) demostraron que la tasa de error en la clasificación de intents aumenta significativamente cuando el usuario realiza actos comunicativos complejos (solicitudes compuestas, actos indirectos, correcciones implícitas). Chen et al. (2019), en su revisión de la comprensión del lenguaje natural para sistemas de diálogo, identificaron

la representación plana de intents como una limitación fundamental que impide a los sistemas manejar diálogos multi-dominio, solicitudes parciales, y cambios de opinión.

2.2.3 Pipelines modulares: la promesa incumplida de la modularidad

La arquitectura de pipeline NLU → DM → NLG — donde un módulo de comprensión del lenguaje natural (NLU) alimenta un gestor de diálogo (DM) que a su vez alimenta un módulo de generación de lenguaje natural (NLG) — emergió como la respuesta de ingeniería al problema de la complejidad. La modularidad prometía separación de responsabilidades: el NLU se encargaría de la variabilidad lingüística, el DM de la lógica conversacional, y el NLG de la producción de respuestas naturales.

Young et al. (2013): el pipeline estadístico POMDP como gestor de diálogo.

En la práctica, el DM operaba típicamente como una máquina de estados (formulaciones tempranas) o como un POMDP — un proceso de decisión de Markov parcialmente observable (Young et al., 2013). En ambos casos, el DM mantenía una representación del estado del diálogo (*dialogue state*) que se actualizaba con cada turno y sobre la cual se tomaban las decisiones. Esta representación es el antecedente directo del **information state** que ACAI propone (**UD 1**, E4).

Sin embargo, la modularidad del pipeline era más aparente que real. Los módulos se comunicaban mediante representaciones intermedias empobrecidas — típicamente, el par (intent, slots) — que perdían información crucial en cada transición. El NLU producía una clasificación que descartaba los matices de la formulación del usuario. El DM operaba sobre esa clasificación empobrecida y producía una "acción de sistema" abstracta. El NLG convertía esa acción en texto, pero sin acceso al contexto lingüístico original. Como señalaron Lemon y Pietquin (2012), esta cascada de abstracciones producía una "pérdida de información sistemática" que limitaba la calidad del diálogo.

La lección relevante para ACAI es doble. Por un lado, la *intuición* del pipeline era correcta: separar la comprensión lingüística de la lógica conversacional es una decisión de diseño fundamentada (P1). Por otro lado, la *implementación* falló porque las interfaces entre módulos eran demasiado restrictivas. La representación intermedia (intent, slots) era insuficiente para transmitir la riqueza de la comprensión lingüística al gestor de diálogo. ACAI debe aprender de ambas lecciones: mantener la separación pero diseñar interfaces más ricas.

2.2.4 Modelos end-to-end: fluidez a costa de control

A partir de 2015, la aplicación de modelos de secuencia a secuencia (Sutskever, Vinyals y Le, 2014) a los sistemas de diálogo produjo la primera ruptura paradigmática radical: los *modelos end-to-end* que aprenden a mapear directamente la historia del diálogo a la siguiente respuesta, sin módulos intermedios diferenciados. Trabajos como los de Vinyals y Le (2015) demostraron que un modelo entrenado sobre corpus de conversaciones podía producir respuestas que, superficialmente, parecían naturales y contextualmente apropiadas.

Los modelos end-to-end eliminaron las interfaces entre módulos — y con ellas, toda posibilidad de inspección.

La ganancia en fluidez fue inmediata y sustancial. Al eliminar las interfaces entre módulos, el modelo podía aprovechar toda la información disponible en la historia del diálogo para producir respuestas contextualizadas. Las formulaciones eran más variadas, más naturales, y menos "robóticas" que las de los sistemas de pipeline. Pero la pérdida en control fue igualmente sustancial. Sin módulos diferenciados, no había representación explícita del estado del diálogo, no había reglas de negocio verificables, y no había trazabilidad de las decisiones. El modelo era una función opaca que mapeaba secuencias de texto a secuencias de texto.

Vinyals & Le (2015) — el modelo conversacional neuronal end-to-end.

Bordes, Boureau y Weston (2017) intentaron recuperar parte del control introduciendo representaciones de conocimiento como memoria externa del modelo. Madotto, Wu y Fung (2018) propusieron modelos que aprendían a copiar entidades de una base de conocimiento durante la generación. Estas aproximaciones mejoraron la consistencia factual pero no resolvieron el problema fundamental: sin una representación explícita del razonamiento, las decisiones del sistema seguían siendo opacas e inconsistentes.

La lección de los modelos end-to-end para ACAI es que la eliminación de las representaciones intermedias no es un avance sino una regresión en términos de accountability. La fluidez que producen es valiosa — ACAI la necesita y la aprovecha en los componentes NLU y NLG — pero la opacidad que conllevan es incompatible con los requisitos de un sistema accountable.

2.2.5 LLMs como sistemas conversacionales: capacidad sin precedentes, opacidad sin precedentes

La aparición de los grandes modelos de lenguaje — GPT-3 (Brown et al., 2020), PaLM (Chowdhery et al., 2022), LLaMA (Touvron et al., 2023), y sus sucesores — representa una inflexión cualitativa en la capacidad conversacional de las máquinas. Estos modelos, entrenados sobre cantidades masivas de texto, exhiben capacidades emergentes que los

modelos anteriores no poseían: comprensión de instrucciones complejas, razonamiento multi-paso (al menos en apariencia), mantenimiento de contexto en conversaciones largas, adaptación a nuevos dominios sin entrenamiento específico, y generación de texto de calidad a menudo indistinguible del humano.

Para la IA conversacional, los LLMs parecen resolver de golpe el problema que cada generación anterior había abordado parcialmente. ¿Variabilidad lingüística? Resuelta: comprenden formulaciones arbitrarias. ¿Naturalidad de las respuestas? Resuelta: generan texto fluido y contextualizado. ¿Manejo de contexto? Sustancialmente mejorado: procesan ventanas de contexto de decenas de miles de tokens. ¿Conocimiento de dominio? Disponible: han sido entrenados sobre corpus que incluyen información de prácticamente cualquier dominio.

Brown et al. (2020): GPT-3 y las capacidades emergentes de los LLMs. Wei et al. (2022): *chain-of-thought prompting*.

Pero precisamente porque los LLMs parecen resolverlo todo, es crucial identificar con precisión lo que *no* resuelven. Varios trabajos recientes han documentado limitaciones que son directamente relevantes para el proyecto ACAI.

En primer lugar, la **inconsistencia**. Raj et al. (2023) demostraron que los LLMs producen respuestas inconsistentes ante reformulaciones de la misma pregunta, con tasas de inconsistencia que pueden superar el 30 % en dominios especializados. Esto no es un defecto corregible — es una consecuencia directa de la naturaleza probabilística de la generación: la respuesta depende de la distribución de probabilidad sobre tokens, que es sensible a variaciones en la formulación de entrada.

En segundo lugar, la **opacidad**. Zhao et al. (2024) mostraron que las técnicas de explicabilidad post-hoc aplicadas a LLMs — análisis de atención, *probing*, generación de explicaciones por el propio modelo — no producen explicaciones fiables del proceso de decisión. Las explicaciones generadas por el modelo son *racionalizaciones plausibles*, no reconstrucciones del proceso real. Como señalan los autores, el modelo no tiene acceso introspectivo a su propio proceso de inferencia.

En tercer lugar, la **ingobernabilidad**. Perez et al. (2022) y Greenblatt et al. (2024) documentaron que los LLMs pueden violar instrucciones explícitas de forma impredecible, especialmente en situaciones adversariales pero también en uso normal cuando la presión probabilística de los datos de entrenamiento supera la fuerza de las instrucciones del prompt. Las técnicas de alineamiento — RLHF (Ouyang et al., 2022), Constitutional AI (Bai et al., 2022) — reducen la frecuencia de comportamientos indeseados pero no los eliminan y no ofrecen garantías formales.

Raj et al. (2023) — inconsistencia semántica en LLMs. Zhao et al. (2024) — límites de la explicabilidad post-hoc.

Estas tres limitaciones no son defectos técnicos que versiones futuras de los LLMs resolverán. Son consecuencias estructurales de la

arquitectura: un sistema que genera texto probabilísticamente a partir de distribuciones sobre tokens no puede, por diseño, garantizar consistencia determinista, proporcionar explicaciones fieles de su proceso, ni cumplir reglas de forma verificable. La mejora de los modelos mejorará las probabilidades — pero las probabilidades no son garantías.

2.2.6 Síntesis: el patrón recurrente

El recorrido por cinco generaciones de arquitecturas de diálogo revela un patrón recurrente que puede esquematizarse como una tensión entre dos ejes.

Cada generación ganó naturalidad y perdió control. ACAI propone recuperar ambos simultáneamente.

En el eje de la *capacidad lingüística*, la progresión ha sido consistente: de patrones rígidos a clasificadores estadísticos, de clasificadores a modelos de secuencia, de modelos de secuencia a transformers masivos. Cada transición amplió la capacidad del sistema para manejar la variabilidad del lenguaje humano.

En el eje del *control formal*, la progresión ha sido inversa: de control total (reglas) a control parcial (DM con máquinas de estados) a control mínimo (modelos end-to-end) a control inexistente (LLMs como caja negra). Cada transición sacrificó capacidad de verificación, trazabilidad y determinismo.

> **La tensión control-naturalidad**
>
> Las cinco generaciones de arquitecturas de diálogo pueden situarse en un espacio bidimensional definido por dos ejes: capacidad lingüística (la habilidad para manejar la variabilidad del lenguaje) y control formal (la capacidad de garantizar, trazar y verificar el comportamiento).
>
> Los sistemas de reglas ocupan el extremo de máximo control y mínima capacidad lingüística. Los LLMs ocupan el extremo opuesto: máxima capacidad lingüística y mínimo control. El paradigma intent-slot y los pipelines modulares ocupan posiciones intermedias con distintos compromisos. Ninguna generación ha ocupado el cuadrante de alto control *y* alta capacidad lingüística.
>
> ACAI propone ocupar ese cuadrante mediante la separación de competencias (P1): asignar la capacidad lingüística al LLM y el control formal al motor simbólico, conectados por interfaces más ricas que las del pipeline clásico.

La observación crucial es que la tensión control-naturalidad no es una ley natural sino una consecuencia de decisiones de diseño. Los sistemas de reglas perdieron naturalidad porque asignaron la variabilidad lingüística al mismo componente que hacía las reglas. Los LLMs

perdieron control porque asignaron las decisiones al mismo componente que maneja el lenguaje. La propuesta de ACAI — separar competencias y conectarlas — no viola ningún principio técnico; simplemente adopta una decisión de diseño que ninguna generación anterior adoptó de forma completa.

2.3 El diagnóstico: generar sin representar

Las secciones anteriores han establecido dos hechos complementarios: la conversación es un fenómeno multidimensional que requiere representaciones ricas (sección 2.1), y las arquitecturas actuales han sacrificado progresivamente la capacidad de representación en favor de la capacidad lingüística (sección 2.2). La convergencia de ambos hechos produce el diagnóstico central que motiva ACAI.

2.3.1 La tesis del diagnóstico

Diagnóstico central

Las arquitecturas conversacionales actuales **generan comportamiento conversacional sin representarlo**. No mantienen un modelo explícito de qué se ha entendido, qué compromisos se han adquirido, qué reglas aplican, ni por qué se elige una respuesta. Sin esas representaciones, la consistencia, trazabilidad y verificabilidad no son difíciles de lograr — son **estructuralmente inalcanzables**.

El adverbio «estructuralmente» es deliberado y requiere justificación. No se trata de que las propiedades de accountability sean difíciles de alcanzar con las arquitecturas actuales, en el sentido de que requieran más datos, mejores modelos, o técnicas más sofisticadas. Se trata de que son *imposibles por diseño*, porque presuponen la existencia de representaciones que la arquitectura no contiene.

La consistencia — la propiedad de que la misma solicitud produce la misma decisión independientemente de la formulación — presupone que existe una representación de "la misma solicitud" independiente de su formulación lingüística. Sin esa representación, la consistencia es un accidente estadístico, no una propiedad garantizada. La trazabilidad — la propiedad de que cada decisión puede rastrearse hasta las razones que la produjeron — presupone que existe un registro del proceso de decisión con sus premisas y reglas aplicadas. En un LLM, el "proceso de decisión" es una secuencia de multiplicaciones de matrices y operaciones softmax que no tiene interpretación semántica accesible.

No es que sea difícil auditar lo que un LLM decide — es que no hay nada que auditar. El proceso de decisión no está representado.

La verificabilidad — la propiedad de que se puede comprobar que el sistema cumple una especificación — presupone que existe una especificación contra la cual verificar y un proceso cuya conformidad se pueda comprobar. Las instrucciones en un prompt no son una especificación formal, y el proceso de generación no es un proceso verificable.

2.3.2 Las aproximaciones paliativas y sus límites

La industria ha respondido a este déficit de accountability con una serie de aproximaciones que abordan los *síntomas* sin resolver la *causa*. Analizarlas con rigor es necesario para entender por qué ACAI propone una solución diferente.

Guardrails y filtros

Los *guardrails* — sistemas que interceptan la entrada o la salida del LLM para detectar y bloquear contenido problemático — son la aproximación más extendida. Frameworks como Guardrails AI (2023), NeMo Guardrails de NVIDIA (Rebedea et al., 2023), y Llama Guard (Inan et al., 2023) implementan distintas estrategias: clasificadores de toxicidad, verificadores de hechos, detectores de temas prohibidos, validadores de formato.

Rebedea et al. (2023) — NeMo Guardrails. Inan et al. (2023) — Llama Guard.

Los guardrails aportan valor práctico real: reducen la frecuencia de comportamientos indeseados, establecen perímetros de seguridad, y proporcionan una capa de defensa. Pero tienen una limitación fundamental que a menudo se pasa por alto: operan sobre la *superficie* del comportamiento (el texto generado), no sobre el *proceso de decisión*. Un guardrail puede detectar que una respuesta contiene información sensible; no puede determinar si la decisión de incluir esa información fue correcta según las reglas del dominio. Puede filtrar una respuesta inapropiada; no puede garantizar que la respuesta apropiada que la reemplaza sea consistente con las decisiones anteriores en el mismo diálogo. Los guardrails son redes de seguridad, no mecanismos de gobernanza.

Prompt engineering y system prompts

La ingeniería de prompts intenta codificar el comportamiento deseado del sistema en instrucciones en lenguaje natural que se proporcionan al LLM como contexto. Los system prompts de producción pueden ser extensos y detallados, incluyendo reglas de negocio, restricciones de comportamiento, ejemplos de respuestas deseadas, y procedimientos paso a paso.

Zamfirescu-Pereira et al. (2023) documentaron que los prompts de producción en aplicaciones comerciales pueden contener miles de tokens de instrucciones. Pero su efectividad está fundamentalmente limitada

por el hecho de que el LLM *interpreta* las instrucciones probabilísticamente, no las *ejecuta* determinísticamente. La diferencia es categórica: una regla formalizada que dice «si el cliente solicita cancelación y tiene menos de 30 días de antigüedad, rechazar» se ejecuta de forma determinista — siempre se aplica cuando se cumple la condición. La misma regla en un prompt se *interpreta* — el modelo la tiene en cuenta con alta probabilidad, pero puede ignorarla, malinterpretarla, o aplicarla de forma inconsistente, especialmente en contextos largos o complejos.

Liu et al. (2023) demostraron experimentalmente que la posición de las instrucciones en el prompt afecta significativamente a su cumplimiento — el efecto "lost in the middle" — lo que significa que la gobernabilidad del sistema depende de factores tan frágiles como el orden de las frases en el prompt.

Zamfirescu-Pereira et al. (2023): la brecha entre intención del prompt y comportamiento del modelo.

Alineamiento: RLHF y Constitutional AI

Las técnicas de alineamiento — RLHF (Ouyang et al., 2022), Constitutional AI (Bai et al., 2022), DPO (Rafailov et al., 2023) — operan a un nivel más profundo que los guardrails y los prompts: modifican los pesos del modelo durante el entrenamiento para que sus salidas se alineen con preferencias humanas o con principios constitucionales. Estas técnicas han producido mejoras sustanciales en la seguridad y utilidad de los LLMs.

Sin embargo, como argumentaron Casper et al. (2023) en su análisis de los problemas abiertos del alineamiento basado en feedback humano, estas técnicas comparten una limitación fundamental con las anteriores: operan sobre las *distribuciones de salida* del modelo, no sobre las *representaciones internas*. Modifican qué genera el modelo con mayor probabilidad, pero no introducen representaciones explícitas de reglas, estados, o razonamiento. El modelo alineado sigue siendo un generador probabilístico — con mejores probabilidades, ciertamente, pero sin las representaciones formales que la accountability requiere.

> **⚠ Las aproximaciones paliativas no son inútiles**
>
> Guardrails, prompt engineering y alineamiento aportan valor real y práctico. El argumento de ACAI no es que deban descartarse sino que son *insuficientes* para los requisitos de accountability en sectores regulados. Un sistema ACAI usará LLMs alineados, empleará prompts cuidadosamente diseñados, y podrá incorporar guardrails como capa adicional de seguridad. Lo que no hará es confiar en estas técnicas para las garantías formales de consistencia, trazabilidad y verificabilidad.

2.3.3 La brecha de la representación

El análisis de las aproximaciones paliativas converge en un mismo punto: todas intentan controlar un sistema que *no representa* lo que hace. Los guardrails filtran superficies. Los prompts instruyen sin garantías. El alineamiento ajusta distribuciones sin introducir estructura. Ninguno de estos mecanismos introduce las *representaciones* necesarias para la accountability.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 3**. El análisis detallado de las limitaciones de RAG, agentes y Constitutional AI como componentes de un ecosistema operativo profundiza en esta brecha desde la perspectiva de cada patrón tecnológico.

Esta constatación — que la brecha es de representación, no de rendimiento — es la que fundamenta la tesis raíz de ACAI. Si el problema fuera de rendimiento (el LLM no es lo bastante bueno), la solución sería un LLM mejor. Si el problema fuera de restricción (el LLM no sigue las instrucciones), la solución serían mejores instrucciones. Pero si el problema es de representación (no hay estructura sobre la que operar formalmente), la solución es introducir esa estructura. Eso es exactamente lo que ACAI propone.

2.4 Implicaciones para el diseño de ACAI

El análisis del fenómeno conversacional (sección 2.1), la evolución de las arquitecturas (sección 2.2), y el diagnóstico de la brecha de representación (sección 2.3) convergen en un conjunto de implicaciones concretas para el diseño del sistema ACAI. Estas implicaciones no son especulaciones — son consecuencias directas del análisis, y constituyen requisitos que el diseño del formalismo y la arquitectura deben satisfacer.

2.4.1 Requisitos para el formalismo de representación

El análisis de las dimensiones conversacionales impone requisitos específicos al lenguaje de representación que se diseñará en la Fase 1.

El formalismo debe poder expresar **actos comunicativos con estructura interna**, no solo categorías planas. Cuando un usuario dice «Me gustaría cancelar mi póliza, pero antes necesito saber si pierdo la bonificación», el sistema debe representar no solo que hay una solicitud de cancelación y una consulta, sino que la solicitud está condicionada a la resolución de la consulta, y que la consulta es una subtarea incrustada con un objetivo propio. Esto va más allá del paradigma intent-slot y conecta con la tradición de actos de diálogo de Traum y Hinkelman

(1992) y con la estructura intencional de Grosz y Sidner (1986).

El formalismo debe poder representar **condiciones de grounding y protocolos de verificación**. En el dominio de seguros, una cancelación de póliza es un acto con consecuencias legales. El sistema no puede asumirlo sin confirmación explícita. El formalismo debe distinguir entre información proporcionada por el usuario (pero no confirmada), información *grounded* (confirmada por ambas partes), y compromisos adquiridos (acciones acordadas que generan obligaciones). El modelo de *grounding acts* de Traum (1994) proporciona una base formal para estas distinciones.

El formalismo debe poder representar **estructura discursiva jerárquica**. Las conversaciones reales no son secuencias planas de pares pregunta-respuesta sino estructuras con subtareas anidadas, digresiones, retornos a temas anteriores, y condiciones de cierre. Un sistema que no represente esta estructura no puede determinar, ante una pregunta ambigua, a qué subtarea pertenece, ni puede verificar que todas las subtareas abiertas se han cerrado adecuadamente antes de ejecutar una acción irreversible.

El formalismo debe poder representar **mecanismos de reparación con trazabilidad**. Cuando un usuario corrige información previamente proporcionada, el sistema debe actualizar su estado de forma que la traza refleje qué se entendió originalmente, por qué era incorrecto, y cómo se corrigió. Esta trazabilidad de las reparaciones es esencial para la auditabilidad del sistema.

Phase 1
Estos requisitos informan directamente el diseño del lenguaje de representación.

2.4.2 La frontera entre lo formalizado y lo delegado

No todas las dimensiones conversacionales son igualmente formalizables, ni todas necesitan serlo. El análisis de esta unidad permite esbozar una primera aproximación a la frontera entre lo que ACAI formaliza y lo que delega al LLM.

Son candidatos claros para la **formalización**: la lógica del diálogo (qué pasos seguir, qué información recoger, qué confirmar), las reglas de negocio (qué es válido, qué restricciones aplican), los protocolos de *grounding* (cuándo exigir confirmación explícita), y los mecanismos de reparación estructurada (cómo actualizar el estado ante correcciones). Estos fenómenos son procedurales o declarativos, tienen semántica definida, y su formalización produce las propiedades de accountability que ACAI busca.

Son candidatos claros para la **delegación al LLM**: la interpretación de formulaciones complejas (mapear lenguaje natural a conceptos formales), la generación de respuestas naturales (expresar las decisiones

del motor en lenguaje apropiado), y la gestión fina de la relación (tono, cortesía, empatía). Estos fenómenos son inherentemente probabilísticos, dependen de matices lingüísticos difíciles de formalizar, y los LLMs los manejan con calidad suficiente.

La zona intermedia — fenómenos que podrían formalizarse parcialmente — incluye la estructura discursiva (parcialmente formalizable como máquina de estados jerárquica, parcialmente dependiente de la comprensión contextual del LLM), las condiciones de *grounding* (parcialmente formalizables como protocolos, parcialmente dependientes de señales lingüísticas sutiles), y las estrategias de reparación complejas (parcialmente formalizables como reglas, parcialmente dependientes de comprensión pragmática).

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 8**. El análisis detallado de los límites de la formalización para cada dimensión del conocimiento conversacional se desarrolla en profundidad.

Esta frontera no es fija sino dinámica: parte de la investigación de ACAI consiste precisamente en determinar empíricamente dónde puede y dónde debe situarse. La Phase 1 establecerá una frontera conservadora (formalizando solo lo claramente formalizable) y la Phase 2 explorará si la integración neuro-simbólica profunda permite extenderla.

2.4.3 El escenario como herramienta de análisis

El escenario de la aseguradora, revisitado

Revisitemos el diálogo de la sección 2.1 ahora con la lente completa del análisis.

Usuario: «Quiero cancelar mi seguro de hogar, pero no sé si me conviene.»

Análisis: Acto directivo (solicitud) condicionado a un acto interrogativo implícito. Estructura discursiva: apertura de tarea principal + apertura de subtarea. El estado epistémico del usuario es de incertidumbre, lo que señala que la subtarea debe resolverse antes de que la tarea principal pueda avanzar.

Agente: «Entiendo. ¿Le preocupa algún aspecto concreto?»

Análisis: Acto de grounding («Entiendo») + acto de elicitación para concretar la subtarea. Gestión de la relación: reconocimiento empático.

Usuario: «Sí, lo de la bonificación. Llevo ocho años sin siniestros.»

Análisis: Respuesta a elicitación + provisión de contexto relevante. El usuario proporciona información factual que el sistema debe verificar contra los datos de la póliza — una operación del ecosistema (E1/E2) gobernada por el modelo formal.

Agente: «Efectivamente, con ocho años…»

Análisis: Provisión de información factual + grounding implícito (el «efec-

tivamente» confirma los datos del usuario). Acto informativo gobernado por reglas del dominio.
Usuario: «Ah, vale. Entonces sí, quiero cancelar.»
Análisis: Grounding explícito («Ah, vale») + cierre de subtarea + retoma de tarea principal con resolución. El condicional expresado en el primer turno se ha resuelto.
Agente: «Para confirmar… ¿Es correcto?»
Análisis: Protocolo de grounding explícito antes de acción irreversible. El sistema enumera los términos del compromiso para obtener confirmación vinculante.
Usuario: «Sí, correcto.»
Análisis: Grounding final. El compromiso queda establecido; la acción puede ejecutarse.

Este análisis turno a turno revela la distancia entre lo que un sistema basado en intents capturaría (dos acciones: consultar y cancelar) y lo que realmente ocurre (una secuencia compleja de actos comunicativos, gestión de incertidumbre, *grounding* progresivo, estructura discursiva jerárquica, y protocolos de verificación). Un sistema accountable debe representar — no necesariamente todas, pero sí las que tienen consecuencias de negocio — estas operaciones.

Conexiones con otras unidades

→ **Prerequisito: UD 1** (Mapa conceptual ACAI). El investigador debe conocer la tesis raíz, los principios P1–P4, y la estructura general del sprint.

⇄ **Complementa: UD 3** (Ecosistema operativo) analiza los patrones tecnológicos actuales (RAG, agentes, Constitutional AI) y sus límites — es la otra cara del análisis de esta unidad, centrada en las limitaciones *tecnológicas* donde UD2 se centra en las limitaciones *conceptuales*.

⇄ **Complementa: UD 5** (Fundamentos neuro-simbólicos) proporciona el marco teórico para la solución que esta unidad motiva. UD2 establece el problema; UD5 presenta el paradigma que fundamenta la solución.

Aplicación: UD 8 (Dimensiones del conocimiento conversacional) toma directamente los requisitos derivados aquí y los traduce en un análisis detallado de qué necesita representar el formalismo ACAI. Las cinco dimensiones de la sección 2.1.2 se convierten en dimensiones del conocimiento conversacional en UD8.

Aplicación: UD 9 (Formalismos) evalúa candidatos de representación contra requisitos que se originan en esta unidad: ¿puede

este formalismo expresar actos comunicativos complejos? ¿Condiciones de grounding? ¿Estructura jerárquica?

Ejercicios

1. [Análisis empírico] Seleccione una transcripción de diálogo real en un dominio de servicio al cliente (puede encontrar ejemplos en los corpus MultiWOZ o SGD, o transcribir una interacción real con un chatbot). Anote cada turno según las cinco dimensiones presentadas en la sección 2.1.2. Cuantifique: ¿qué porcentaje de turnos son clasificables como intents puros? ¿Qué porcentaje involucra grounding, reparación u otros actos de gestión? Compare sus resultados con los datos de Stolcke et al. (2000) y Bunt et al. (2012).

2. [Análisis arquitectónico] Para el mismo diálogo del ejercicio anterior, analice cómo lo manejaría cada generación de arquitectura (sistema de reglas, pipeline intent-slot, modelo end-to-end, LLM). Para cada arquitectura, identifique: (a) qué dimensiones del diálogo capturaría, (b) qué dimensiones ignoraría, y (c) en qué turnos concretos fallaría y por qué.

3. [Diseño] Considerando el escenario de la aseguradora, diseñe un protocolo de grounding para la operación de cancelación de póliza. El protocolo debe especificar: (a) qué información debe ser confirmada explícitamente antes de la cancelación, (b) qué nivel de evidencia de grounding se requiere para cada dato (implícito vs. explícito), y (c) cómo se maneja una corrección del usuario en cualquier punto del protocolo. Presente el diseño en pseudocódigo o diagrama de estados.

4. [Reflexión crítica] El diagnóstico central de esta unidad afirma que la accountability es "estructuralmente inalcanzable" sin representaciones explícitas. ¿Es posible que LLMs futuros, más capaces, superen esta limitación? Argumente su posición considerando: (a) la diferencia entre capacidad probabilística y garantía formal, (b) los resultados de Raj et al. (2023) sobre inconsistencia, y (c) el argumento de que la generación estocástica de tokens es fundamentalmente incompatible con el determinismo.

5. [Aplicación a ACAI] A partir del análisis de esta unidad, redacte una lista priorizada de los 5 fenómenos conversacionales más importantes que el formalismo ACAI debe poder representar. Para cada fenómeno, justifique: (a) por qué es necesario para la accoun-

> tability, (b) qué evidencia empírica soporta su importancia, y (c) en qué grado es formalizable (totalmente, parcialmente, o debe delegarse al LLM).

Referencias y lecturas

Lectura de entrada

Jurafsky, D. & Martin, J. H. (2024). *Speech and Language Processing* (3.ª ed., borrador en línea). Capítulos 15 (Chatbots & Dialogue Systems) y 24 (Dialogue Systems and Conversational Agents). Proporcionan una visión panorámica accesible de los sistemas de diálogo, su evolución y los desafíos actuales. Es la mejor lectura de entrada para un investigador con formación en NLP que necesita contextualizar los sistemas conversacionales.

Lecturas esenciales

- Clark, H. H. (1996). *Using Language*. Cambridge University Press. — La referencia fundacional sobre el lenguaje como acción conjunta. Los capítulos sobre *grounding* y acción coordinada fundamentan directamente la concepción de la conversación que ACAI adopta. Lectura densa pero imprescindible.
- Bunt, H. (2011). Multifunctionality in dialogue. *Computer Speech & Language*, 25(2), 222–245. — El trabajo que fundamenta la multidimensionalidad del acto comunicativo y que sirve de base al estándar ISO 24617-2. Esencial para entender por qué los intents son una simplificación excesiva.
- Traum, D. R. (1994). *A Computational Theory of Grounding in Natural Language Conversation*. Tesis doctoral, University of Rochester. — La formalización del grounding que más directamente conecta con el diseño de ACAI. Propone un modelo computacional con actos de grounding tipificados y un mecanismo de actualización del *common ground*.
- Young, S., Gašić, M., Thomson, B. & Williams, J. D. (2013). POMDP-based statistical spoken dialog systems: A review. *Proceedings of the IEEE*, 101(5), 1160–1179. — La referencia para entender el gestor de diálogo estadístico como antecedente del motor simbólico de ACAI.

Lecturas complementarias

- Austin, J. L. (1962). *How to Do Things with Words*. Oxford University Press. — Fundacional para la teoría de actos de habla. Lectura breve y con valor histórico.
- Searle, J. R. (1969). *Speech Acts: An Essay in the Philosophy of Language*. Cambridge University Press. — La sistematización de las condiciones de felicidad de los actos de habla que ACAI necesita como base para la formalización de actos comunicativos.

- Sacks, H., Schegloff, E. A. & Jefferson, G. (1974). A simplest systematics for the organization of turn-taking for conversation. *Language*, 50(4), 696–735. — Fundacional para el análisis conversacional. Relevante para entender la organización secuencial que el formalismo ACAI debe capturar.
- Grosz, B. J. & Sidner, C. L. (1986). Attention, intentions, and the structure of discourse. *Computational Linguistics*, 12(3), 175–204. — El modelo tripartito del discurso. Relevante para la representación de estructura discursiva jerárquica.
- Clark, H. H. & Schaefer, E. F. (1989). Contributing to discourse. *Cognitive Science*, 13(2), 259–294. — El modelo de contribución y grounding. Base teórica directa para E4 y para los protocolos de verificación en ACAI.
- Brown, P. & Levinson, S. C. (1987). *Politeness: Some Universals in Language Usage*. Cambridge University Press. — La teoría de cortesía y *face*. Relevante para las políticas de generación (E3).
- Schegloff, E. A., Jefferson, G. & Sacks, H. (1977). The preference for self-correction in the organization of repair in conversation. *Language*, 53(2), 361–382. — El análisis clásico de la reparación.
- Stolcke, A. et al. (2000). Dialogue act modeling for automatic tagging and recognition of conversational speech. *Computational Linguistics*, 26(3), 339–373. — Evidencia empírica sobre la distribución de actos de diálogo en corpus reales.
- Budzianowski, P. et al. (2018). MultiWOZ: A large-scale multi-domain Wizard-of-Oz dataset for task-oriented dialogue modelling. *EMNLP 2018*. — El corpus de referencia que documenta la complejidad de diálogos "simples".
- Brown, T. B. et al. (2020). Language models are few-shot learners. *NeurIPS 2020*. — GPT-3 y las capacidades emergentes de los LLMs.
- Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS 2022*. — InstructGPT y RLHF.
- Bai, Y. et al. (2022). Constitutional AI: Harmlessness from AI Feedback. Anthropic. — Constitutional AI como estrategia de alineamiento.
- Raj, H. et al. (2023). Semantic consistency of large language models. *ArXiv preprint*. — Evidencia experimental de inconsistencia en LLMs.
- Casper, S. et al. (2023). Open problems and fundamental limitations of reinforcement learning from human feedback. *TMLR*. — Análisis riguroso de las limitaciones del alineamiento basado en feedback.
- Rebedea, T. et al. (2023). NeMo Guardrails: A toolkit for controllable and safe LLM applications. *EMNLP 2023 (demo)*. — Guardrails como aproximación paliativa.
- Liu, N. F. et al. (2023). Lost in the middle: How language models use long contexts. *TACL*. — El efecto de la posición en el seguimiento de instrucciones.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«A system that cannot do what it needs to do cannot be accountable for what it does.»

Eje A — El problema: IA conversacional y accountability

Unidad 3 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje A — El problema

Prerequisitos: UD1, UD2 · Eje A — El problema

UNIDAD DIDÁCTICA 3

El ecosistema operativo: componentes, capacidades y límites de gobernanza

Contenidos de la unidad

3.2.1 El patrón y lo que resuelve 4
3.2.2 El análisis de gobernanza 5
3.2.3 El concepto de RAG gobernado 7
3.3.1 El paradigma agéntico y lo que aporta 8
3.3.2 El análisis de gobernanza 9
3.3.3 El concepto de capacidad agéntica gobernada 10
3.4.1 El estado del arte en alineamiento 11
3.4.2 El análisis de gobernanza 12
3.4.3 El concepto de alineamiento formalizado 13
3.5.1 El contexto en los LLMs: ventana, degradación y opacidad 14
3.5.2 La tradición del *information state update* 15
3.5.3 El *information state* en la arquitectura ACAI 16
3.6.1 El problema de la coordinación 17
3.6.2 La gobernanza formal unificada 17
3.6.3 Lo que la gobernanza unificada no resuelve 19

¿Por qué necesito esto para ACAI?

La UD 2 estableció que la conversación es un fenómeno multidimensional que las arquitecturas actuales generan sin representar. Esta unidad completa el diagnóstico desde la perspectiva tecnológica. El investigador que se incorpora a ACAI probablemente conoce RAG, ha usado frameworks agénticos, y quizás ha experimentado con Constitutional AI. Es probable que piense que combinando estas piezas se resuelve el problema de la accountability. Esta unidad desmonta esa intuición con rigor — sin caricaturizar las soluciones existentes, pero mostrando con precisión dónde falla cada una en términos de gobernanza. Al terminar, el investigador entenderá por qué ACAI no propone prescindir de estos componentes sino *gobernarlos* bajo un modelo formal unificado.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Describir los patrones RAG, Agentic AI y Constitutional AI con precisión técnica suficiente para evaluar sus propiedades de gobernanza.
2. Identificar, para cada patrón, qué resuelve genuinamente y dónde falla en términos de accountability: qué decisiones toma, quién las toma, y por qué no son auditables.
3. Argumentar por qué la combinación de patrones existentes no produce un sistema gobernado, incluso cuando cada componente funciona correctamente.
4. Conceptualizar la diferencia entre un componente autónomo y un componente gobernado por conocimiento formal.
5. Explicar el concepto de *information state* y su diferencia con el contexto implícito del LLM.
6. Articular la necesidad de gobernanza formal unificada como contribución diferencial de ACAI.

3.1 El argumento de la completitud

La UD 2 concluyó con un diagnóstico preciso: las arquitecturas conversacionales actuales generan comportamiento sin representarlo, lo que hace la accountability estructuralmente inalcanzable. La respuesta natural a este diagnóstico es proponer exactamente lo que ACAI propone: introducir representaciones formales explícitas y un motor simbólico que opere sobre ellas. Pero esta respuesta, por sí sola, es insuficiente.

Un sistema formalmente correcto pero operativamente incompleto no es accountable en la práctica.

Considérese un motor conversacional que opera sobre reglas formales perfectamente especificadas para el dominio asegurador. El motor sabe que una cancelación de póliza requiere verificar la identidad del cliente, consultar las condiciones de penalización, informar al cliente, y obtener confirmación explícita. Cada decisión es determinista y trazable. Pero en el momento en que el motor necesita consultar las condiciones de penalización, se encuentra con que esas condiciones están en un documento PDF de 200 páginas que se actualiza trimestralmente. Y cuando necesita verificar la identidad, debe invocar un servicio externo de autenticación. Y cuando genera la respuesta, debe ajustarse a las políticas de comunicación de la organización. Y a lo largo de toda la interacción, debe mantener un modelo coherente de qué se ha dicho, qué se ha entendido, y qué compromisos se han adquirido.

Cada una de estas necesidades requiere un componente que va más allá del motor simbólico: acceso a información documental, capacidad de ejecutar acciones, alineamiento con políticas, y gestión de contexto. Sin estos componentes, el motor es formalmente elegante e inútil en la práctica. Esta es la sustancia del principio P4: la accountability requiere completitud operativa.

El ecosistema tecnológico actual ofrece patrones maduros para cada una de estas necesidades: RAG para el acceso a información, arquitecturas agénticas para la ejecución de acciones, Constitutional AI para el alineamiento, y la ventana de contexto del LLM para la gestión conversacional. Lo que esta unidad demuestra es que estos patrones, tal como están formulados, aportan las capacidades que ACAI necesita pero carecen de las propiedades de gobernanza que ACAI exige. La innovación de ACAI no está en reemplazar estos componentes sino en *gobernarlos*: someter sus decisiones al mismo modelo formal que gobierna la conversación.

3.2 Retrieval-Augmented Generation: del patrón al problema de gobernanza

Sede principal de este concepto.

3.2.1 El patrón y lo que resuelve

Retrieval-Augmented Generation (RAG), tal como lo formularon Lewis et al. (2020), es un patrón arquitectónico que combina un módulo de recuperación de información con un modelo generativo. Ante una consulta del usuario, el sistema primero recupera documentos relevantes de un corpus externo y luego los inyecta en el contexto del modelo

Lewis et al. (2020) — el artículo fundacional de RAG.

generativo para que produzca una respuesta informada por esas fuentes. La formulación original proponía un entrenamiento end-to-end donde el retriever y el generador se optimizaban conjuntamente, pero en la práctica industrial el patrón se ha simplificado en un pipeline de tres etapas: indexación (los documentos se segmentan y se convierten en representaciones vectoriales), recuperación (ante una consulta se buscan los segmentos más similares semánticamente), y generación (los segmentos recuperados se incluyen en el prompt del LLM junto con la consulta).

El valor de RAG es genuino y sustancial. Resuelve uno de los problemas más persistentes de los LLMs: la desactualización del conocimiento. Un LLM entrenado con datos hasta una fecha determinada no puede responder sobre eventos o cambios posteriores; RAG permite acceder a información actualizada en tiempo real. Reduce además las alucinaciones factuales al anclar la generación en fuentes concretas: cuando el LLM tiene acceso a un documento que contiene la respuesta, la probabilidad de que invente información disminuye significativamente, como han documentado Shuster et al. (2021) en su trabajo sobre modelos con acceso a conocimiento externo.

El patrón ha evolucionado considerablemente desde la formulación original. Gao et al. (2024), en su survey comprensivo, distinguen entre RAG *naive* (el pipeline básico de tres etapas), RAG *advanced* (que incorpora técnicas de pre-recuperación como query rewriting y post-recuperación como reranking), y RAG *modular* (donde los componentes del pipeline se configuran de forma flexible). Las técnicas de chunking — cómo segmentar los documentos para la indexación — han generado una subliteratura propia, con propuestas que van desde la segmentación por tamaño fijo hasta el chunking semántico que respeta las fronteras de significado (Kamradt, 2023). El reranking — reordenar los resultados de la recuperación usando un modelo más preciso antes de inyectarlos en el contexto — se ha convertido en práctica estándar, con modelos como los de Nogueira y Cho (2020) y sus sucesores.

Gao et al. (2024): taxonomía de RAG naive, advanced y modular.

Estas mejoras son reales y relevantes. Pero todas operan dentro de un marco donde las decisiones fundamentales de gobernanza permanecen sin resolver.

3.2.2 El análisis de gobernanza

Para entender dónde falla RAG en términos de accountability, es necesario descomponer el patrón en las decisiones que toma y analizar quién las toma y con qué garantías.

En un RAG estándar, cinco decisiones críticas las toma el LLM o el pipeline de forma opaca e inauditable.

La primera decisión es **cuándo recuperar**. En un sistema conversacional, no toda pregunta del usuario requiere consultar documentos

externos. Algunas preguntas pueden responderse con la información ya disponible en el estado del diálogo; otras requieren acceder a documentación; otras requieren ejecutar una acción, no buscar información. En la implementación estándar, esta decisión la toma el LLM — sea porque toda consulta se enruta al retriever, sea porque el LLM decide invocarlo como herramienta. En ningún caso la decisión es trazable: no hay un registro formal de por qué se decidió buscar (o no buscar) en una situación concreta.

La segunda decisión es **qué buscar**. La consulta que se envía al retriever determina los resultados, y esa consulta puede ser la pregunta literal del usuario, una reformulación generada por el LLM, o una query expandida mediante técnicas de query rewriting. En todos los casos, la transformación de la necesidad informativa del usuario en una consulta de recuperación es un proceso opaco. Si el usuario pregunta «¿Pierdo la bonificación si cancelo?», la consulta efectiva al retriever podría ser «bonificación cancelación póliza», «condiciones pérdida bonificación seguro hogar», o cualquier otra reformulación. Cuál se elige, y por qué, no queda registrado.

La tercera decisión es **qué es relevante**. El retriever devuelve los k documentos más similares semánticamente a la consulta. Pero similitud semántica no es equivalente a relevancia para la tarea. Un documento puede ser semánticamente cercano pero referirse a un producto diferente, a una versión normativa obsoleta, o a condiciones que no aplican al caso concreto. Barnett et al. (2024), en su análisis de los siete puntos de fallo de RAG, documentaron que la recuperación de documentos irrelevantes o parcialmente relevantes es la causa más frecuente de respuestas incorrectas en sistemas RAG desplegados, con tasas que oscilan entre el 15 % y el 35 % según el dominio y la configuración.

La cuarta decisión es **cómo integrar la información**. Una vez que los documentos recuperados se inyectan en el contexto del LLM, el modelo debe decidir qué información usar, cómo combinarla con lo que ya sabe, y qué peso atribuir a cada fuente. Esta integración es enteramente opaca: el LLM procesa los documentos como tokens en su contexto y genera una respuesta que puede usar, ignorar, o malinterpretar la información recuperada. No hay mecanismo para verificar que el contenido factual de la respuesta corresponde efectivamente a lo que dicen los documentos.

Barnett et al. (2024) — los siete puntos de fallo de RAG.

La quinta decisión es **qué nivel de confianza atribuir**. No toda información documental tiene el mismo estatus: un artículo del contrato tiene fuerza legal; una FAQ tiene valor orientativo; una nota interna puede estar desactualizada. En un RAG estándar no hay mecanismo para distinguir estos niveles ni para ajustar la respuesta en consecuencia. El LLM trata todos los segmentos recuperados como contexto indistinto.

Cada una de estas cinco decisiones es, en un RAG estándar, opaca, no-trazable y no-verificable. La implicación para la accountability es directa: cuando el sistema proporciona información incorrecta al usuario, no se puede determinar si el fallo fue en la decisión de buscar, en la consulta, en la selección de documentos, en la integración, o en la atribución de confianza. La cadena de responsabilidad se pierde en el pipeline.

3.2.3 El concepto de RAG gobernado

RAG gobernado

En un **RAG gobernado**, las cinco decisiones del pipeline de recuperación están determinadas por el modelo formal de conocimiento, no por el LLM ni por heurísticas del pipeline. El modelo formal especifica: bajo qué condiciones del diálogo se debe buscar información documental; en qué fuentes buscar y con qué criterios de relevancia; cómo evaluar la pertinencia de los resultados para el caso concreto; cómo integrar la información en la respuesta; y qué nivel de confianza atribuir según el tipo de fuente. La traza de ejecución incluye todas estas decisiones.

La diferencia entre un RAG estándar y un RAG gobernado no es de rendimiento sino de arquitectura. Un RAG estándar puede funcionar mejor en términos de calidad de respuesta en ciertos escenarios; un RAG gobernado puede funcionar peor si las reglas no están bien diseñadas. Lo que el RAG gobernado garantiza es que cada decisión del pipeline es *auditable*: se puede reconstruir por qué se buscó, qué se encontró, por qué se consideró relevante, y cómo influyó en la respuesta. Esta auditabilidad es lo que la accountability en sectores regulados requiere.

RAG estándar vs. RAG gobernado

Contexto: El usuario ha solicitado cancelar su seguro de hogar y pregunta por la bonificación.
RAG estándar: El LLM decide buscar → envía la consulta «bonificación cancelación seguro hogar» al retriever → recupera 3 fragmentos de las condiciones generales → genera una respuesta que integra los fragmentos. Si la respuesta es incorrecta, no se puede determinar en qué punto del pipeline se produjo el error.
RAG gobernado: El motor simbólico determina que el estado del diálogo requiere consultar las condiciones de bonificación → la regla `bonus_query_01` especifica: buscar en *Condiciones Generales v2025.2*, sección *Bonificaciones por ausencia de siniestros* → el retriever busca en esa fuente específica → el motor evalúa la pertinencia del resultado contra

las restricciones formales (producto correcto, versión vigente) → el LLM genera la respuesta usando la información validada → la traza registra: regla aplicada, fuente consultada, fragmento recuperado, validación, respuesta.

3.3 Arquitecturas agénticas: de la autonomía a la gobernanza

Sede principal de este concepto.

3.3.1 El paradigma agéntico y lo que aporta

La capacidad de un sistema conversacional para ejecutar acciones sobre el mundo — consultar bases de datos, iniciar transacciones, invocar APIs, escalar a un operador humano — es una necesidad operativa ineludible. Un asistente que puede informar sobre las condiciones de cancelación pero no puede ejecutar la cancelación es útil pero incompleto. El paradigma agéntico, que ha experimentado una expansión notable desde 2023, proporciona esta capacidad.

El concepto de *tool use* — la capacidad de un LLM para invocar herramientas externas durante la generación — fue articulado formalmente por Schick et al. (2024) en Toolformer, donde demostraron que un modelo de lenguaje puede aprender a insertar llamadas a APIs en su proceso de generación. Sin embargo, la práctica industrial ha evolucionado hacia un mecanismo más estructurado: el *function calling*, donde el modelo selecciona entre un conjunto predefinido de funciones con esquemas tipados y devuelve los argumentos necesarios para invocarlas. Este mecanismo, implementado por OpenAI, Anthropic y Google en sus APIs, proporciona una interfaz más controlada que la inserción libre de llamadas a herramientas.

Schick et al. (2024) — Toolformer: enseñar al LLM a usar herramientas.

Sobre esta base, han surgido frameworks de orquestación que permiten construir flujos agénticos más complejos. El patrón ReAct (Yao et al., 2023) propuso intercalar razonamiento y acción: el LLM genera un paso de razonamiento (*thought*), ejecuta una acción (*action*), observa el resultado (*observation*), y repite hasta completar la tarea. Frameworks como LangChain (Chase, 2022), LlamaIndex, y CrewAI han operacionalizado estos patrones, proporcionando abstracciones para construir agentes que planifican, ejecutan, y se recuperan de errores.

Wang et al. (2024), en su survey comprensivo sobre agentes basados en LLM, identifican cuatro capacidades fundamentales: planificación (descomponer una tarea compleja en pasos), uso de herramientas (in-

Yao et al. (2023): ReAct — razonamiento intercalado con acción.

vocar APIs y servicios), memoria (mantener información a lo largo de la interacción), y reflexión (evaluar el resultado de acciones y ajustar el plan). Xi et al. (2023) amplían esta taxonomía para incluir arquitecturas multi-agente donde múltiples LLMs especializados colaboran en la resolución de tareas, cada uno con herramientas y roles diferenciados.

Estas capacidades son genuinamente valiosas para un sistema conversacional en un dominio real. Sin ellas, el sistema es un interlocutor pasivo incapaz de actuar. Pero el análisis de gobernanza revela problemas fundamentales que la entusiasta literatura sobre agentes frecuentemente minimiza.

3.3.2 El análisis de gobernanza

El problema de gobernanza en las arquitecturas agénticas es, en cierto sentido, más grave que en RAG, porque las acciones tienen consecuencias directas en el mundo real. Un documento irrelevante recuperado por RAG produce, en el peor caso, una respuesta incorrecta. Una acción ejecutada incorrectamente por un agente puede producir una transacción financiera errónea, una cancelación no autorizada, o una escalación inapropiada.

Un agente autónomo es, por definición, un agente cuyas decisiones no están gobernadas externamente.

La primera cuestión de gobernanza es la **selección de herramientas**. Cuando el LLM tiene acceso a un conjunto de funciones — por ejemplo, `consultar_póliza`, `cancelar_póliza`, `escalar_a_operador`, `enviar_email` — la decisión de cuál invocar la toma el modelo basándose en su interpretación de la situación. Esta decisión es probabilística e influenciable. Patil et al. (2023), en su trabajo sobre Gorilla, documentaron que los LLMs cometen errores de selección de herramienta en un 10–20 % de los casos, incluso con descripciones claras de cada función. En situaciones ambiguas — que son las más comunes en diálogos reales — la tasa de error puede ser significativamente mayor.

La segunda cuestión es el **orden de ejecución**. En tareas complejas que requieren múltiples acciones, el orden importa. Verificar la identidad antes de consultar datos es un requisito de seguridad. Informar sobre las consecuencias antes de ejecutar una cancelación es un requisito regulatorio. Un agente autónomo determina el orden de ejecución según su "razonamiento", que es en realidad una generación probabilística de pasos. Ruan et al. (2024), en su análisis de riesgos de agentes basados en LLM, documentaron casos donde agentes ejecutan acciones en orden incorrecto, omiten pasos de verificación, o ejecutan acciones redundantes, con consecuencias que van desde la ineficiencia hasta el daño operativo real.

Patil et al. (2023) — Gorilla: errores de selección de herramienta en LLMs.

La tercera cuestión es la **verificación de precondiciones**. Antes de ejecutar una acción con consecuencias, un sistema responsable debe

Ruan et al. (2024) — riesgos y fallos de agentes basados en LLM.

verificar que se cumplen las condiciones necesarias. ¿Tiene el usuario autoridad para esta operación? ¿Se ha proporcionado toda la información requerida? ¿Se ha obtenido la confirmación necesaria? En un framework agéntico estándar, estas verificaciones pueden estar codificadas como instrucciones en el prompt del agente, pero su cumplimiento no está garantizado — el LLM puede "olvidar" una precondición, especialmente en contextos largos o complejos.

La cuarta cuestión es la **gestión de errores**. Cuando una acción falla — el servicio externo no responde, los datos no son válidos, el resultado es inesperado — el agente debe decidir cómo proceder. En un framework estándar, esta decisión la toma el LLM, que puede reintentar, intentar una alternativa, escalar, o simplemente reportar el error al usuario. La estrategia de recuperación no está formalizada y no es predecible.

La consecuencia de estas cuatro carencias es que un sistema agéntico basado en LLM puede, en cualquier momento, ejecutar una acción que no corresponde, en un orden que no es correcto, sin haber verificado las precondiciones, y con una gestión de errores impredecible. Cada una de estas situaciones es un fallo de accountability: una acción no auditable, una decisión no trazable, un proceso no verificable.

3.3.3 El concepto de capacidad agéntica gobernada

> **Capacidad agéntica gobernada**
>
> En un sistema con **capacidad agéntica gobernada**, las acciones disponibles, las precondiciones para su ejecución, el orden en que deben ejecutarse, y las verificaciones post-ejecución están formalizadas en el modelo de conocimiento. El motor simbólico determina qué acción corresponde según las reglas del dominio y el estado del diálogo; el componente agéntico la ejecuta. El agente no decide autónomamente — ejecuta lo que el motor determina.

Phase 1
La gobernanza agéntica se implementa como reglas formales que el motor simbólico evalúa.

La diferencia es análoga a la distinción entre un empleado con autonomía total y un empleado que opera según procedimientos definidos. El primero puede ser más eficiente en situaciones imprevistas; el segundo ofrece garantías de que los procedimientos se cumplen. En un sector regulado, la segunda opción no es una preferencia sino un requisito. Un agente gobernado puede ser menos "creativo" en la resolución de problemas, pero su comportamiento es predecible, trazable y auditable.

Esto no significa que el LLM no tenga ningún papel en la capacidad agéntica. Puede asistir en la determinación de parámetros (interpretar que "el seguro de hogar" se refiere a la póliza HG-2019-4421), en la formulación de respuestas ante errores, y en la adaptación del lenguaje

al contexto. Lo que no hace es *decidir* qué acción ejecutar ni en qué orden. Esa decisión la toma el motor simbólico sobre la base de reglas formales.

3.4 Alineamiento y políticas: de la constitución al modelo formal

Sede principal de este concepto.

3.4.1 El estado del arte en alineamiento

El problema del alineamiento — lograr que un sistema de IA se comporte de acuerdo con valores, normas y objetivos humanos — ha generado una literatura extensa y un debate profundo que excede con mucho los límites de esta unidad. Lo que aquí interesa es un aspecto específico: cómo se gestionan las políticas de comportamiento en sistemas conversacionales, y por qué las aproximaciones actuales son insuficientes para la accountability.

La técnica más influyente ha sido RLHF (*Reinforcement Learning from Human Feedback*), introducida por Christiano et al. (2017) y llevada a escala industrial por Ouyang et al. (2022) con InstructGPT. La idea es entrenar un modelo de recompensa a partir de preferencias humanas y luego usar ese modelo para ajustar el LLM mediante aprendizaje por refuerzo. El resultado es un modelo que, estadísticamente, produce respuestas más alineadas con las preferencias expresadas. Variantes como DPO (*Direct Preference Optimization*, Rafailov et al., 2023) simplifican el proceso eliminando el modelo de recompensa explícito.

Ouyang et al. (2022) — InstructGPT y RLHF como técnica de alineamiento.

Constitutional AI (Bai et al., 2022) propuso un enfoque complementario: definir un conjunto de principios — una "constitución" — que el modelo usa para evaluarse y corregirse a sí mismo. El proceso consiste en generar respuestas, evaluarlas contra los principios constitucionales usando el propio modelo, y reentrenar con las revisiones como datos de preferencia. La innovación conceptual es significativa: explicitar los principios en lugar de derivarlos implícitamente de las preferencias de los anotadores.

A nivel de despliegue, estas técnicas de entrenamiento se complementan con mecanismos de runtime: system prompts que codifican las políticas del sistema, guardrails que filtran entradas y salidas, y clasificadores que detectan contenido que viola las políticas. Frameworks como Llama Guard (Inan et al., 2023) y NeMo Guardrails (Rebedea et al., 2023) proporcionan infraestructura para implementar estos mecanismos de forma modular.

Bai et al. (2022): Constitutional AI — principios explícitos para el alineamiento.

El resultado combinado de estas técnicas es innegablemente positivo.

Los LLMs alineados son más seguros, más útiles y más respetuosos que sus versiones base. Las políticas de comportamiento se cumplen con mayor frecuencia. La experiencia del usuario mejora sustancialmente.

3.4.2 El análisis de gobernanza

Pero "con mayor frecuencia" no es "siempre", y en sectores regulados la diferencia entre ambos es la diferencia entre conformidad e incumplimiento.

El problema fundamental de todas las aproximaciones de alineamiento actuales, desde RLHF hasta Constitutional AI, es que operan sobre las *distribuciones de salida* del modelo, no sobre las *representaciones de decisión*. RLHF modifica qué respuestas genera el modelo con mayor probabilidad. Constitutional AI modifica las preferencias del modelo durante el entrenamiento. Los system prompts proporcionan instrucciones que el modelo intenta seguir. Pero en ningún caso se introducen representaciones formales de las políticas que puedan verificarse antes del despliegue, que se ejecuten de forma determinista durante la operación, o que se auditen después.

Las constituciones son instrucciones que el modelo interpreta probabilísticamente — directrices, no reglas verificables.

Casper et al. (2023), en su análisis de los problemas abiertos del alineamiento basado en feedback humano, identificaron varias limitaciones fundamentales. El feedback humano es inconsistente: distintos anotadores expresan preferencias diferentes ante los mismos pares de respuestas. El modelo de recompensa es una aproximación: captura tendencias estadísticas, no reglas deterministas. Y la optimización puede producir *reward hacking*: el modelo aprende a satisfacer el modelo de recompensa sin alinearse genuinamente con los valores subyacentes.

Wolf et al. (2023) y Zou et al. (2023) demostraron que incluso los modelos más cuidadosamente alineados pueden ser inducidos a violar sus políticas mediante ataques adversariales — *jailbreaks* — que explotan la naturaleza probabilística de la generación para eludir las restricciones aprendidas. Aunque la robustez ante estos ataques ha mejorado con cada generación de modelos, la existencia misma de la vulnerabilidad demuestra que las restricciones no son formales sino estadísticas: se cumplen con alta probabilidad pero no con certeza.

Casper et al. (2023) — problemas abiertos del alineamiento basado en feedback.

Para un sistema conversacional en un sector regulado, las consecuencias son concretas. Una política que dice «no proporcionar asesoramiento financiero personalizado» puede codificarse en un system prompt, reforzarse con RLHF, y monitorizarse con guardrails. Pero si un usuario formula su solicitud de forma suficientemente indirecta, si el contexto conversacional es suficientemente largo, o si la formulación del prompt es suficientemente ambigua, el sistema puede violar la política. Y cuando eso ocurre, no hay mecanismo formal para determinar *por qué* la viola-

ción se produjo, ni para garantizar que no se repetirá en circunstancias similares.

3.4.3 El concepto de alineamiento formalizado

Alineamiento formalizado

En un sistema con **alineamiento formalizado**, las políticas de comportamiento — restricciones éticas, legales y organizacionales — están representadas en el mismo modelo formal de conocimiento que las reglas de negocio y la lógica del diálogo. Se verifican del mismo modo, se ejecutan determinísticamente, y se auditan con los mismos mecanismos. Las políticas no son instrucciones que el modelo intenta seguir sino restricciones formales que el motor simbólico impone.

La propuesta de ACAI no es que las técnicas de alineamiento actuales sean inútiles. RLHF, Constitutional AI y guardrails aportan valor real y ACAI puede incorporarlos como capas adicionales de seguridad. Lo que propone es que, para las políticas críticas — las que tienen consecuencias regulatorias, legales o éticas —, la formalización en el modelo de conocimiento ofrece garantías que las técnicas probabilísticas no pueden proporcionar.

La analogía es precisa: en un sistema legal, las leyes no son "directrices que los ciudadanos intentan seguir" sino reglas formales con condiciones de aplicación, excepciones codificadas, y mecanismos de verificación. Que los ciudadanos las cumplan voluntariamente (alineamiento) es deseable; que exista un sistema formal que las haga cumplir (gobernanza) es imprescindible. ACAI propone construir ese sistema formal para las políticas del dominio conversacional.

Políticas de generación como objeto formalizable

No todas las políticas son igualmente formalizables. Una restricción como «no ejecutar cancelaciones sin confirmación explícita» es perfectamente formalizable: es una precondición determinista. Una directriz como «ser empático cuando el usuario expresa frustración» es más difícil de formalizar porque involucra detección de estados emocionales y estrategias comunicativas graduadas. ACAI reconoce esta gradación: las políticas con consecuencias operativas directas se formalizan como reglas; las políticas de tono y estilo se gestionan mediante instrucciones al LLM gobernadas por el modelo formal (por ejemplo, el modelo formal determina

> que en este punto del diálogo se requiere tono empático, y configura el prompt del LLM en consecuencia). La frontera entre lo formalizado y lo instruido es, de nuevo, una decisión de diseño que la **UD 8** analizará en profundidad.

3.5 Gestión de contexto: del buffer de tokens al *information state*

Sede principal de este concepto.

3.5.1 El contexto en los LLMs: ventana, degradación y opacidad

Un sistema conversacional multi-turno necesita "recordar" lo que se ha dicho a lo largo de la interacción: qué ha solicitado el usuario, qué información se ha proporcionado, qué compromisos se han adquirido, qué preguntas están pendientes. En los LLMs, este "recuerdo" se implementa mediante la ventana de contexto: la secuencia de tokens — tanto del usuario como del sistema — que el modelo procesa en cada paso de generación.

La ventana de contexto ha crecido espectacularmente en los últimos años: de los 2.048 tokens de GPT-3 a los 128.000 de GPT-4 Turbo y los 200.000 de Claude 3, con propuestas experimentales que alcanzan millones de tokens. Este crecimiento ha generado la percepción de que el problema de la gestión de contexto está resuelto: si el modelo puede procesar toda la conversación, "recuerda" todo.

Una ventana de contexto más grande no es una representación más rica — es un buffer más largo con las mismas limitaciones estructurales.

Sin embargo, esta percepción confunde *acceso* con *representación*. El modelo tiene *acceso* a todos los tokens de la conversación, pero no tiene una *representación* del estado del diálogo. No distingue entre información mencionada y confirmada. No registra qué compromisos se han adquirido ni cuáles se han cumplido. No modela las subtareas abiertas ni las cerradas. No diferencia entre lo que el usuario pidió inicialmente y lo que pidió después de cambiar de opinión. Todos estos estados existen implícitamente en la secuencia de tokens, y el modelo puede inferirlos probabilísticamente con razonable fiabilidad en contextos cortos. Pero la inferencia es falible, y su fiabilidad se degrada con la longitud.

Liu et al. (2023) demostraron experimentalmente el efecto "lost in the middle": la capacidad de los LLMs para usar información del contexto decae significativamente para información situada en posiciones intermedias de contextos largos, mientras que la información al principio y al final se procesa con mayor fiabilidad. Levy et al. (2024) extendieron este análisis y mostraron que la degradación afecta especialmente a la

recuperación de hechos específicos y a la consistencia de razonamiento en contextos que exceden los 20.000 tokens.

Para un sistema conversacional accountable, las consecuencias son directas. Si un usuario proporciona su número de póliza en el turno 3, cambia su solicitud en el turno 7, y confirma la operación en el turno 15, el sistema necesita mantener un registro preciso de estos tres hechos y de sus relaciones. Un LLM con ventana de contexto suficiente probablemente los "recuerde", pero no puede garantizarlo, y la traza de cómo estos hechos influyeron en la decisión final no es inspectable.

Liu et al. (2023): "lost in the middle". Levy et al. (2024): degradación del razonamiento en contextos largos.

3.5.2 La tradición del *information state update*

La investigación en gestión de diálogo produjo, décadas antes de los LLMs, una respuesta formal a este problema. El paradigma de **information state update** (ISU), articulado por Larsson y Traum (2000) en el marco del proyecto TRINDI y formalizado en el toolkit TrindiKit, propone que un sistema de diálogo mantenga un modelo explícito del estado informacional del diálogo — el **information state** — que se actualiza de forma controlada con cada turno de la conversación.

Larsson & Traum (2000) — TrindiKit y el paradigma de *information state update*.

El information state, tal como lo formularon Larsson y Traum, no es simplemente una variable que almacena datos. Es una estructura formal que incluye varios componentes. Un componente de *agenda* contiene los objetivos pendientes del sistema y del usuario, organizados por prioridad. Un componente de *shared commitments* registra los compromisos mutuos adquiridos durante el diálogo y su estado (propuesto, aceptado, cumplido, retirado). Un componente de *questions under discussion* (QUD) mantiene las preguntas que están abiertas y que el diálogo está intentando resolver. Un componente de *private beliefs* registra lo que el sistema cree saber sobre el usuario y la situación.

Las reglas de actualización definen cómo se transforma el information state con cada acción del diálogo. Cuando el usuario proporciona información, se actualiza el componente de creencias privadas. Cuando el sistema confirma un dato, pasa de creencia privada a compromiso compartido. Cuando se abre una subtarea, se añade a la agenda. Cuando se cierra, se retira. Estas reglas son formales, deterministas, e inspectables.

QUD: *Questions Under Discussion* (Ginzburg, 2012) — las preguntas abiertas que organizan el diálogo.

Ginzburg (2012), en su trabajo sobre la semántica del diálogo, extendió este marco con una formalización más profunda basada en *Type Theory with Records* (TTR), que permite representar el contexto conversacional con la precisión de un sistema de tipos formales. Aunque la implementación completa de TTR excede lo que ACAI necesita en la Fase 1, la idea central — el contexto conversacional como estructura formal con semántica definida — es directamente relevante.

3.5.3 El *information state* en la arquitectura ACAI

La arquitectura ACAI adopta el concepto de information state como componente central del motor simbólico. En cada turno de la conversación, el information state se actualiza de forma explícita según reglas formalizadas en el modelo de conocimiento. El LLM no tiene acceso directo al information state — interactúa con él a través del motor simbólico: recibe del motor las instrucciones sobre qué interpretar y cómo generar, y el motor actualiza el information state según lo que el LLM ha interpretado.

> **_Information state_ en ACAI**
>
> El **information state** de ACAI es una estructura formal, mantenida y actualizada por el motor simbólico, que representa el estado completo del diálogo en un momento dado. Incluye, como mínimo: los datos proporcionados por el usuario y su estado de confirmación; los compromisos adquiridos por el sistema; las subtareas abiertas y cerradas; las condiciones de *grounding* pendientes; y los resultados de las acciones ejecutadas. Se actualiza determinísticamente con cada turno y es completamente inspectable.

La ventaja respecto a la ventana de contexto del LLM es categórica en tres aspectos. En *persistencia*: el information state no se degrada con la longitud de la conversación; un dato confirmado en el turno 3 tiene el mismo estatus en el turno 30. En *estructura*: el information state distingue entre tipos de información (datos, compromisos, preguntas abiertas, subtareas) mientras que la ventana de contexto los mezcla como tokens indiferenciados. Y en *inspección*: el information state es una estructura que puede imprimirse, visualizarse y auditarse en cualquier momento de la conversación; la ventana de contexto es una secuencia de tokens cuyo "significado" para el modelo es inaccesible.

> **El *information state* como pilar de la trazabilidad**
>
> El information state no es un componente auxiliar de la arquitectura ACAI sino su pilar central. Si ACAI promete trazabilidad — la capacidad de reconstruir por qué el sistema tomó cada decisión —, el information state es el objeto que hace posible esa reconstrucción. Cada decisión del motor simbólico opera sobre el information state vigente en ese momento. La traza de decisión es, literalmente, la secuencia de information states y las reglas que produjeron cada transición.

3.6 Síntesis: por qué la combinación no basta

3.6.1 El problema de la coordinación

El análisis de las secciones anteriores podría sugerir que los problemas de gobernanza son específicos de cada componente y que, por tanto, podrían resolverse mejorando cada componente por separado: un RAG más preciso, un agente más fiable, un alineamiento más robusto, un contexto más largo. Pero hay un problema adicional que emerge cuando los componentes se combinan y que no es reducible a las limitaciones de ninguno de ellos por separado: el problema de la coordinación.

La combinación de componentes no gobernados no produce un sistema gobernado — produce un sistema con múltiples fuentes de no-gobernabilidad.

En un sistema conversacional que combina RAG, agentes y guardrails, cada componente toma decisiones de forma independiente. El módulo RAG decide qué documentos recuperar. El agente decide qué herramientas invocar. El guardrail decide qué filtrar. El LLM integra todo y genera una respuesta. Pero no existe un modelo compartido que coordine estas decisiones. El RAG puede recuperar un documento que contradice una acción que el agente está a punto de ejecutar. El agente puede invocar una herramienta que el guardrail luego bloqueará, produciendo un flujo incoherente. El LLM puede generar una respuesta que integra información del RAG de una forma que viola una política del guardrail.

La literatura sobre sistemas multi-componente ha documentado extensamente este problema bajo diversas etiquetas. En ingeniería de software, se conoce como el problema de la *composición emergente*: el comportamiento del sistema compuesto no es predecible a partir del comportamiento de los componentes individuales (Ghezzi, Jazayeri y Mandrioli, 2003). En sistemas multiagente, se conoce como el problema de la *coordinación*: garantizar que agentes autónomos actúen de forma coherente requiere mecanismos explícitos que los agentes autónomos, por definición, no proporcionan.

Para un sistema conversacional accountable, el problema de la coordinación tiene una consecuencia directa: la trazabilidad end-to-end es imposible cuando los componentes no comparten un modelo formal de lo que están haciendo. Si el RAG decide buscar información, el agente decide ejecutar una acción, y el LLM decide cómo responder, y cada decisión se toma de forma independiente, no existe un punto de vista privilegiado desde el cual reconstruir la cadena completa de razonamiento que produjo la respuesta.

3.6.2 La gobernanza formal unificada

Phase 1 e I5
Gobernanza formal del ecosistema operativo.

Gobernanza formal unificada

La **gobernanza formal unificada** es el principio según el cual todos los componentes del ecosistema operativo — recuperación de información, ejecución de acciones, aplicación de políticas, y gestión de contexto — están gobernados por el mismo modelo formal de conocimiento y coordinados por el mismo motor simbólico. La traza de ejecución es completa y coherente porque refleja las decisiones de un único proceso formal, no la combinación de decisiones independientes.

En la arquitectura ACAI, el motor simbólico es el componente que implementa la gobernanza unificada. Ante cada turno de la conversación, el motor consulta el modelo de conocimiento y el information state, y determina qué debe ocurrir: ¿se necesita información documental? El motor determina qué buscar y en qué fuentes (RAG gobernado). ¿Se debe ejecutar una acción? El motor determina cuál, verifica las precondiciones, y ordena la ejecución (capacidad agéntica gobernada). ¿Hay restricciones de política que aplican? El motor las evalúa formalmente (alineamiento formalizado). ¿Cómo se actualiza el estado del diálogo? El motor aplica las reglas de actualización (information state).

El LLM participa en este proceso en dos puntos: al principio, interpretando el input del usuario y mapeándolo a conceptos del modelo formal; y al final, generando la respuesta en lenguaje natural conforme a lo que el motor ha determinado. Entre ambos puntos, las decisiones son formales, deterministas y trazables.

Gobernanza unificada en acción

Turno: El usuario ha solicitado cancelar su póliza de hogar y pregunta por la bonificación.
Motor simbólico: Consulta el information state → la solicitud de cancelación está registrada pero pendiente de confirmación. La pregunta sobre bonificación abre una subtarea. El motor evalúa:
Paso 1: ¿Se necesita información documental? *Sí* — la regla `bonus_info_required` determina que hay que consultar las condiciones de bonificación en las Condiciones Generales vigentes (RAG gobernado).
Paso 2: ¿Se necesita ejecutar una acción? *Sí* — la regla `bonus_check_action` determina que hay que consultar el historial de siniestros del cliente en el sistema de gestión (capacidad agéntica gobernada). Precondición: identidad verificada → verificada en turno anterior → proceder.
Paso 3: ¿Aplican restricciones de política? *Sí* — la política `retention_opportunity` indica que ante una cancelación se debe informar proactivamente de las opciones de retención (alineamiento formalizado).
Paso 4: Actualizar information state: subtarea

`bonificación_consulta` abierta, datos de bonificación obtenidos, opciones de retención pendientes de comunicar.
LLM (NLG): Genera la respuesta integrando: información de bonificación, datos de historial del cliente, y mención de opciones de retención, con tono según la política de comunicación.
Traza: Cada paso registra la regla aplicada, los datos consultados, las fuentes accedidas, las precondiciones verificadas, las políticas evaluadas, y el information state resultante.

Este escenario ilustra la diferencia fundamental entre un sistema que combina componentes autónomos y un sistema con gobernanza unificada. En el primer caso, cada componente actúa según su propia lógica y la coherencia del resultado es accidental. En el segundo, un único modelo formal coordina todos los componentes y la coherencia es garantizada. La traza del primer caso es fragmentaria (el log de cada componente por separado); la traza del segundo es completa (la secuencia de decisiones formales con sus justificaciones).

3.6.3 Lo que la gobernanza unificada no resuelve

Límites de la gobernanza unificada

La gobernanza formal unificada no hace que el sistema sea perfecto. Hace que sea *auditable*. Si las reglas están mal diseñadas, el sistema tomará decisiones incorrectas — pero la traza permitirá identificar qué regla causó el error y corregirla. Si el LLM interpreta incorrectamente un input, la respuesta será incorrecta — pero la traza mostrará dónde se produjo el error de interpretación. Si un servicio externo falla, la acción no se completará — pero el motor gestionará el fallo según las reglas formalizadas. La gobernanza unificada transforma los errores de *opacos* en *diagnosticables*. No los elimina; los hace gobernables.

Conexiones con otras unidades

→ **Prerequisito: UD 1** (Mapa conceptual ACAI) para los principios P4 (completitud operativa) y los componentes E1–E4. **UD 2** (La conversación como fenómeno) para el diagnóstico de la brecha de representación que esta unidad complementa.

⇄ **Complementa: UD 4** (Accountability y marco regulatorio) fundamenta desde la perspectiva regulatoria los requisitos de trazabilidad y verificabilidad que aquí se analizan desde la perspectiva

tecnológica. UD3 muestra que los componentes actuales no satisfacen los requisitos; UD4 muestra qué exigen exactamente esos requisitos.

⇄ Complementa: UD 5 (Fundamentos neuro-simbólicos) proporciona el marco teórico para la integración motor-LLM que aquí se plantea como solución. UD3 establece qué necesita gobernarse; UD5 explica cómo el paradigma neuro-simbólico fundamenta esa gobernanza.

➔ Aplicación: UD 8 (Dimensiones del conocimiento conversacional) toma el ecosistema operativo como una de las dimensiones que el formalismo debe gobernar: las reglas sobre cuándo buscar, qué acciones ejecutar, y qué políticas aplicar son parte del conocimiento conversacional que se formaliza.

➔ Aplicación: UD 9 (Formalismos) evalúa los formalismos candidatos considerando, entre otros criterios, su capacidad para representar las reglas de gobernanza del ecosistema operativo: reglas de recuperación, precondiciones de acciones, restricciones de políticas.

Ejercicios

1. [Análisis técnico] Seleccione un sistema conversacional comercial con el que haya interactuado (un chatbot de atención al cliente, un asistente de banca, o similar). Diseñe una secuencia de 5–7 turnos que involucre acceso a información documental, ejecución de una acción, y una situación donde una política debería aplicarse. Para cada turno, identifique: (a) qué componente del ecosistema operativo se necesitaría, (b) qué decisión toma ese componente, (c) quién toma esa decisión en el sistema actual (¿el LLM?, ¿un pipeline?, ¿una regla?), y (d) si esa decisión es auditable.

2. [Diseño] Para el escenario de la aseguradora que recorre el monográfico, diseñe las reglas de un RAG gobernado para la consulta de condiciones de bonificación. Especifique: (a) bajo qué condiciones del diálogo se activa la recuperación, (b) en qué fuentes se busca y con qué criterios, (c) cómo se valida la pertinencia del resultado, (d) cómo se atribuye el nivel de confianza, y (e) qué se registra en la traza. Presente las reglas en pseudocódigo o en una notación semiformal de su elección.

3. [Análisis comparativo] Compare la gestión de contexto en tres paradigmas: (a) un sistema de diálogo con information state explícito

al estilo de TrindiKit, (b) un LLM con ventana de contexto de 128k tokens, y (c) un LLM con RAG sobre el historial de conversaciones. Para cada paradigma, analice: cómo se representa un compromiso adquirido (por ejemplo, "el sistema se comprometió a no ejecutar la cancelación sin confirmación"), cómo se garantiza su persistencia a lo largo del diálogo, y cómo se verifica su cumplimiento.

4. [Reflexión crítica] El concepto de "agente gobernado" podría parecer una contradicción en los términos: un agente que no decide autónomamente no es un agente. Evalúe este argumento. ¿Es la autonomía una propiedad esencial de la agencia? ¿Puede un sistema tener capacidad de acción sin tener autonomía de decisión? ¿Cuáles son los trade-offs de restringir la autonomía del agente? Construya su argumento con referencia a la literatura sobre agentes (por ejemplo, Wooldridge y Jennings, 1995, sobre las propiedades de los agentes) y a los requisitos de accountability en sectores regulados.

5. [Aplicación a ACAI] Diseñe el flujo completo de un turno conversacional en la arquitectura ACAI para el siguiente escenario: el usuario dice «Mi vecino me ha dicho que si cancelo antes de fin de mes no pago penalización, ¿es verdad?». El flujo debe especificar: (a) cómo interpreta el LLM este input (incluyendo la referencia a información externa no verificada), (b) qué decide el motor simbólico, (c) qué componente del ecosistema se activa y por qué, (d) qué políticas aplican (incluyendo la política de no proporcionar asesoramiento basado en información de terceros), (e) cómo se genera la respuesta, y (f) qué registra la traza. Use el formato de escenario de esta unidad.

Referencias y lecturas

Lectura de entrada

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J. & Wang, H. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. *ArXiv preprint arXiv:2312.10997v5*. El survey más completo y actualizado sobre RAG, con una taxonomía clara de paradigmas (naive, advanced, modular) y un análisis de técnicas en cada etapa del pipeline. Lectura de entrada recomendada para situar el estado del arte antes de abordar el análisis de gobernanza de esta unidad.

Lecturas esenciales

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*. — El artículo fundacional de RAG. Imprescindible para entender la formulación original y su motivación.
- Larsson, S. & Traum, D. (2000). Information state and dialogue management in the TRINDI dialogue move engine toolkit. *Natural Language Engineering*, 6(3–4), 323–340. — El artículo de referencia para el paradigma de information state update. Base directa del concepto de information state en ACAI. Lectura imprescindible.
- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C. et al. (2022). Constitutional AI: Harmlessness from AI Feedback. *Anthropic Technical Report*. — La formulación de Constitutional AI. Esencial para entender el patrón de alineamiento que ACAI propone formalizar.
- Casper, S., Davies, X., Shi, C., Gilbert, T. K., Scheurer, J., Rando, J., Freedman, R., Korbak, T., Lindner, D., Freire, P., Wang, T., Marks, S., Segerie, C.-R., Carroll, M., Peng, A., Christoffersen, P., Damani, M., Shlegeris, B., Greenblatt, N., Baumli, K., McIlhenny, L., Russell, S. & Abbeel, P. (2023). Open Problems and Fundamental Limitations of Reinforcement Learning from Human Feedback. *Transactions on Machine Learning Research*. — El análisis más riguroso de las limitaciones del alineamiento basado en feedback. Esencial para fundamentar la necesidad de formalización.

Lecturas complementarias

- Barnett, S., Kurniawan, S., Thudumu, S., Brber, Z. & Vasa, R. (2024). Seven Failure Points When Engineering a Retrieval Augmented Generation System. *IEEE/ACM International Conference on AI Engineering*. — Análisis empírico de los puntos de fallo de RAG en producción. Directamente relevante para la argumentación sobre gobernanza.
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z. & Wen, J. (2024). A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*, 18(6). — Survey comprensivo sobre agentes LLM. Referencia para la taxonomía de capacidades agénticas.
- Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E. et al. (2023). The Rise and Potential of Large Language Model Based Agents: A Survey. *ArXiv preprint arXiv:2309.07864*. — Otro survey influyente sobre agentes. Complementa a Wang et al. con más énfasis en arquitecturas multi-agente.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*. — El patrón de razonamiento-acción intercalado. Referencia para entender la aproximación agéntica actual.

- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N. & Scialom, T. (2024). Toolformer: Language Models Can Teach Themselves to Use Tools. *NeurIPS 2023*. — El trabajo que formalizó el concepto de tool use para LLMs.
- Patil, S. G., Zhang, T., Wang, X. & Gonzalez, J. E. (2023). Gorilla: Large Language Model Connected with Massive APIs. *ArXiv preprint arXiv:2305.15334*. — Análisis de errores en selección de herramientas por LLMs. Evidencia empírica para la argumentación sobre gobernanza agéntica.
- Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J. & Hashimoto, T. (2024). Identifying the Risks of LM Agents with an LM-Emulated Sandbox. *ICLR 2024*. — Análisis de riesgos de agentes autónomos. Evidencia empírica directa.
- Shuster, K., Poff, S., Chen, M., Kiela, D. & Weston, J. (2021). Retrieval Augmentation Reduces Hallucination in Conversation. *EMNLP 2021 (Findings)*. — Evidencia de la reducción de alucinaciones con RAG.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A. et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS 2022*. — InstructGPT y RLHF a escala.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D. & Finn, C. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *NeurIPS 2023*. — DPO como alternativa a RLHF.
- Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C. & Cohen, J. (2023). NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. *EMNLP 2023 (demo)*. — Framework de guardrails programables.
- Inan, H., Upasani, K., Chi, J., Rungta, R., Iyer, K., Mao, Y., Tontchev, M., Hu, Q., Fuller, B., Testuggine, D. & Khabsa, M. (2023). Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *ArXiv preprint arXiv:2312.06674*. — Clasificador de seguridad como guardrail.
- Wolf, Y., Hadar, N., Bitton, I., Berber, S. & Elhadad, N. (2023). Fundamental Limitations of Alignment in Large Language Models. *ArXiv preprint arXiv:2304.11082*. — Análisis teórico de las limitaciones del alineamiento.
- Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z. & Fredrikson, M. (2023). Universal and Transferable Adversarial Attacks on Aligned Language Models. *ArXiv preprint arXiv:2307.15043*. — Ataques adversariales contra modelos alineados.
- Ginzburg, J. (2012). *The Interactive Stance: Meaning for Conversation*. Oxford University Press. — Formalización del contexto conversacional con TTR. Referencia avanzada para la fundamentación teórica del information state.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F. & Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the ACL*. — Evidencia de degradación del uso de contexto.

- Levy, M., Jacoby, A. & Goldberg, Y. (2024). Same Task, More Tokens: the Impact of Input Length on the Reasoning Performance of Large Language Models. *ACL 2024*. — Extensión del análisis de degradación en contextos largos.
- Ghezzi, C., Jazayeri, M. & Mandrioli, D. (2003). *Fundamentals of Software Engineering* (2.ª ed.). Prentice Hall. — Referencia para el problema de la composición emergente en sistemas de software.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«An explanation that does not correspond to the actual decision process is not an explanation — it is a story.»

Eje A — El problema: IA conversacional y accountability

Unidad 4 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje A — El problema

Prerequisitos: UD1, UD2 · Eje A — El problema

UNIDAD DIDÁCTICA 4

Accountability, explicabilidad y marco regulatorio

Contenidos de la unidad

4.1.1 El problema terminológico 3
4.1.2 Transparencia 4
4.1.3 Interpretabilidad 5
4.1.4 Explicabilidad 5
4.1.5 Accountability 6
4.1.6 Relaciones entre los conceptos 7
4.2.1 El paradigma post-hoc 8
4.2.2 Las explicaciones de los LLMs: racionalizaciones, no explicaciones 9
4.2.3 El paradigma constitutivo 10
4.2.4 El argumento de Rudin y su extensión a la IA conversacional 11
4.3.1 De la regulación a las especificaciones técnicas 12
4.3.2 EU AI Act: requisitos para sistemas de alto riesgo 12
4.3.3 DORA: resiliencia operativa en el sector financiero 13
4.3.4 GDPR: el derecho a explicación 14
4.3.5 Síntesis: la tabla de requisitos técnicos 14
4.4.1 Dos estrategias de gobernanza 15
4.4.2 La analogía estructural 16
4.5.1 La filosofía del proyecto marco 16
4.5.2 ACAI como instanciación de la filosofía 17

¿Por qué necesito esto para ACAI?

Las unidades anteriores han establecido el diagnóstico: la conversación es un fenómeno multidimensional que las arquitecturas actuales generan sin representar (UD 2), y los componentes tecnológicos existentes carecen de las propiedades de gobernanza necesarias (UD 3). Pero este diagnóstico está incompleto sin una respuesta precisa a la pregunta fundamental: ¿qué significa, exactamente, que un sistema conversacional sea *accountable*? Y, más concretamente: ¿qué exige el marco regulatorio actual? Esta unidad proporciona ambas respuestas. Al terminar, el investigador tendrá una definición operativa de accountability, distinguirá con precisión entre explicabilidad post-hoc y constitutiva, y podrá traducir los requisitos regulatorios en especificaciones técnicas para la arquitectura ACAI.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Distinguir con precisión entre transparencia, interpretabilidad, explicabilidad y accountability, identificando las relaciones de dependencia entre estos conceptos.
2. Argumentar la diferencia entre explicabilidad post-hoc y constitutiva, y evaluar por qué la segunda es necesaria para la accountability en sectores regulados.
3. Traducir los requisitos del EU AI Act, DORA y GDPR en especificaciones técnicas concretas para un sistema conversacional.
4. Distinguir entre gobernanza superpuesta y gobernanza constitutiva, y explicar por qué ACAI adopta la segunda.
5. Conectar la propuesta de ACAI con la filosofía del proyecto (AI)^2:TDG sobre explicabilidad fundamentada en estructuras formales.

4.1 La constelación de conceptos: de la transparencia a la accountability

Sede principal de este concepto.

4.1.1 El problema terminológico

La literatura sobre IA explicable y responsable utiliza una constelación de términos — transparencia, interpretabilidad, explicabilidad,

Seis términos, usados como sinónimos en la industria, designan propiedades técnicas distintas con requisitos de implementación diferentes.

auditabilidad, accountability, responsabilidad — que en la práctica industrial y en buena parte de la divulgación se emplean de forma intercambiable. Esta confusión terminológica no es un problema menor: cada término designa una propiedad técnica diferente, con requisitos de implementación diferentes, y un sistema puede satisfacer unos sin satisfacer otros. ACAI necesita una definición operativa precisa de su objetivo, y esa definición requiere desambiguar la constelación.

Arrieta et al. (2020), en su survey comprensivo sobre Explainable AI, propusieron una taxonomía que ha servido como referencia para buena parte del trabajo posterior. Lipton (2018), en un trabajo anterior e igualmente influyente, había advertido que el término «interpretabilidad» se usaba para designar al menos tres propiedades diferentes, y que esta ambigüedad generaba confusión tanto en la investigación como en la práctica. Doshi-Velez y Kim (2017) intentaron una formalización más rigurosa, proponiendo niveles de evaluación de la interpretabilidad. Más recientemente, Schwalbe y Finzel (2023) han revisado la taxonomía a la luz de los desarrollos en LLMs y modelos generativos.

Arrieta et al. (2020) — taxonomía comprensiva de XAI.

Lo que sigue no es un catálogo de definiciones sino una construcción progresiva: cada concepto se apoya en el anterior y añade una exigencia adicional. Esta estructura no es solo pedagógica — refleja una relación de dependencia real entre las propiedades.

4.1.2 Transparencia

La **transparencia** es la propiedad más básica de la cadena. Un sistema es transparente cuando su estructura interna y su proceso de operación son accesibles para su inspección. Lipton (2018) distinguió tres niveles de transparencia: la transparencia del modelo completo (*simulability* — un humano puede simular mentalmente el proceso completo), la transparencia de los componentes (*decomposability* — cada componente tiene una interpretación inteligible), y la transparencia algorítmica (el algoritmo de entrenamiento produce modelos cuyo comportamiento es comprensible).

Un árbol de decisión con pocas ramas es transparente en los tres niveles. Un LLM no es transparente en ninguno: su estructura interna (miles de millones de parámetros) no es simulable, sus componentes (capas de atención, matrices de pesos) no tienen interpretación semántica individual, y el proceso de entrenamiento no produce modelos cuyo comportamiento sea comprensible a priori.

Lipton (2018): tres niveles de transparencia — simulability, decomposability, transparencia algorítmica.

La transparencia es condición necesaria pero no suficiente para la accountability. Un sistema puede ser transparente sin ser explicable: que los parámetros sean accesibles no significa que sean comprensibles. Y un sistema puede ser parcialmente explicable sin ser transparente:

las técnicas post-hoc generan explicaciones de sistemas opacos. Pero la accountability, como veremos, requiere una forma de transparencia que solo las representaciones explícitas pueden proporcionar.

4.1.3 Interpretabilidad

La **interpretabilidad** añade a la transparencia el requisito de comprensibilidad humana. Un sistema es interpretable cuando un humano puede comprender cómo produce sus decisiones. La distinción con la transparencia es sutil pero importante: la transparencia es una propiedad del sistema (está abierto a inspección), mientras que la interpretabilidad es una propiedad de la relación entre el sistema y un observador humano (el observador puede comprender lo que inspecciona).

Doshi-Velez y Kim (2017) propusieron evaluar la interpretabilidad en tres niveles: evaluación a nivel de aplicación (¿es útil la explicación para un usuario final en su tarea?), evaluación a nivel humano (¿es comprensible la explicación para un evaluador no experto?), y evaluación a nivel de función (¿produce la explicación resultados correctos según métricas formales?). Estos niveles son directamente relevantes para ACAI, porque el lenguaje de representación debe satisfacer la restricción de legibilidad (I1), que es esencialmente un requisito de interpretabilidad: un experto de dominio no técnico debe poder comprender las reglas formalizadas.

Molnar (2022), en su libro de referencia sobre interpretable machine learning, ofrece una distinción práctica que es útil para ACAI: los modelos *intrínsecamente* interpretables (modelos lineales, árboles de decisión, sistemas de reglas) versus los modelos que requieren técnicas de interpretabilidad *post-hoc* (redes neuronales profundas, LLMs). La arquitectura neuro-simbólica de ACAI combina ambos: el motor simbólico opera sobre reglas que son intrínsecamente interpretables; el LLM es opaco pero sus decisiones no son las que deben ser accountable — solo proporciona capacidad lingüística.

📖 Doshi-Velez & Kim (2017) — niveles de evaluación de la interpretabilidad.

4.1.4 Explicabilidad

La **explicabilidad** añade a la interpretabilidad el requisito de articulación causal: no solo se puede comprender qué hizo el sistema, sino *por qué* lo hizo. Arrieta et al. (2020) definen la explicabilidad como la capacidad de producir detalles o razones que hagan claro el funcionamiento del modelo a un observador humano, incluyendo las relaciones causales entre las entradas, los estados internos, y las salidas.

La distinción con la interpretabilidad es que un sistema puede ser interpretable (puedo comprender las reglas) sin ser explicable (no puedo

reconstruir por qué, en este caso concreto, se aplicó esta regla y no otra). La explicabilidad requiere que el sistema pueda articular la cadena causal que conecta la entrada con la salida, pasando por las decisiones intermedias y las reglas aplicadas.

Para un sistema conversacional en un sector regulado, la explicabilidad tiene un destinatario concreto que no es el usuario final sino el auditor: un profesional técnico o regulatorio que necesita verificar que el sistema operó correctamente en un caso específico. El auditor no pregunta «¿qué hizo el sistema?» (interpretabilidad) sino «¿por qué hizo lo que hizo, y era correcto hacerlo según las normas aplicables?» (explicabilidad). Esta pregunta requiere acceso a la cadena causal completa: qué input recibió, cómo lo interpretó, qué reglas aplicó, qué información consultó, qué precondiciones verificó, y por qué produjo esta respuesta y no otra.

4.1.5 Accountability

Accountability

La **accountability** es la propiedad de un sistema cuyas decisiones son explicables (se puede articular por qué se tomaron), verificables (se puede comprobar que cumplen una especificación formal), y atribuibles (se puede identificar qué componente, regla o agente es responsable de cada decisión). La accountability presupone transparencia, interpretabilidad y explicabilidad, y añade los requisitos de verificabilidad formal y atribución de responsabilidad.

La accountability es explicabilidad + verificabilidad + atribución de responsabilidad. Es el concepto más exigente de la cadena.

La verificabilidad distingue a la accountability de la mera explicabilidad. Un sistema puede ser explicable sin ser verificable: puede producir una explicación plausible que, sin embargo, no se puede contrastar contra una especificación formal. Un LLM que genera una explicación de su propia decisión produce un artefacto lingüístico que puede ser coherente y comprensible, pero que no se puede verificar contra reglas formales porque esas reglas no existen en el sistema. La accountability exige que exista una especificación — las reglas formalizadas en el modelo de conocimiento — y que la traza de decisión pueda contrastarse contra ella.

La atribución de responsabilidad distingue a la accountability de la verificabilidad abstracta. En un sistema complejo, no basta con verificar que el comportamiento cumple la especificación; es necesario poder determinar, en caso de fallo, *qué* falló: ¿fue la regla (está mal diseñada)? ¿Fue la interpretación del LLM (malinterpretó al usuario)? ¿Fue la información (el documento recuperado estaba desactualizado)?

¿Fue la acción (el servicio externo respondió incorrectamente)? Sin esta capacidad de atribución, la corrección de errores es ciega.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 12**. Las métricas operativas de accountability — consistencia, trazabilidad, verificabilidad, gobernabilidad — y las metodologías para medirlas se desarrollan en profundidad en la unidad de evaluación.

4.1.6 Relaciones entre los conceptos

La construcción progresiva revela una cadena de dependencias que puede sintetizarse así. La transparencia (acceso a la estructura interna) es condición necesaria para la interpretabilidad (comprensión de la estructura). La interpretabilidad es condición necesaria para la explicabilidad (articulación causal). La explicabilidad es condición necesaria para la accountability, que además requiere verificabilidad (contrastación con especificación) y atribución de responsabilidad.

Las propiedades como niveles de exigencia

Cada nivel de la cadena impone requisitos técnicos adicionales sobre el sistema. La transparencia requiere que la estructura sea accesible. La interpretabilidad requiere que sea comprensible por un humano. La explicabilidad requiere que la cadena causal sea articulable. La accountability requiere además que exista una especificación formal contra la cual verificar, y un mecanismo de atribución de responsabilidad.

Un sistema basado en LLM puro no satisface ningún nivel de la cadena para las decisiones operativas (aunque el texto generado pueda ser comprensible, el proceso de decisión no lo es). Un sistema con guardrails satisface parcialmente la transparencia (las reglas del guardrail son inspeccionables) pero no la accountability (las decisiones del LLM siguen siendo opacas).

La arquitectura ACAI está diseñada para satisfacer la accountability completa para las decisiones operativas: el motor simbólico opera sobre reglas transparentes, interpretables, que producen trazas explicables, verificables contra la especificación formal, y atribuibles a reglas y componentes concretos.

Implicaciones de la taxonomía

La taxonomía establece que la accountability no es un sinónimo elegante de explicabilidad. Es un concepto estrictamente más exigente que requiere verificabilidad formal y atribución. Esto tiene

una consecuencia directa para ACAI: no basta con que el sistema pueda explicar lo que hace (lo que un LLM puede hacer, al menos superficialmente). Es necesario que las decisiones puedan verificarse contra una especificación formal y que los fallos puedan atribuirse a componentes concretos. Estas dos propiedades adicionales son las que exigen representaciones formales explícitas.

4.2 Explicabilidad post-hoc vs. constitutiva

Sede principal de este concepto.

4.2.1 El paradigma post-hoc

La gran mayoría del trabajo en IA explicable ha operado bajo un paradigma que puede denominarse *post-hoc*: dado un modelo opaco que ya ha tomado una decisión, generar una explicación de esa decisión. Las técnicas más influyentes de este paradigma son bien conocidas. LIME (Ribeiro, Singh y Guestrin, 2016) genera explicaciones locales perturbando la entrada y observando cómo cambia la salida, construyendo un modelo interpretable que aproxima el comportamiento del modelo opaco en el entorno de la instancia explicada. SHAP (Lundberg y Lee, 2017) usa valores de Shapley de la teoría de juegos cooperativos para asignar a cada *feature* de la entrada una contribución a la predicción. Los métodos basados en atención (Vaswani et al., 2017; Jain y Wallace, 2019) usan los pesos de atención del transformer como indicadores de qué partes de la entrada influyeron más en la salida.

Ribeiro et al. (2016) — LIME. Lundberg & Lee (2017) — SHAP.

Estas técnicas han producido avances reales en la comprensión de modelos de machine learning, y siguen siendo herramientas valiosas para el análisis y la depuración de modelos. Pero como fundamento para la accountability, tienen una limitación que no es técnica sino epistemológica: la explicación que producen no es una descripción del proceso de decisión real del modelo sino una *aproximación interpretable* de ese proceso.

Esta limitación ha sido documentada con rigor creciente. Rudin (2019), en un artículo que ha tenido enorme influencia bajo el título programático «Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead», argumentó que las explicaciones post-hoc son inherentemente no fiables para decisiones de alto riesgo. Su argumento central es que una explicación post-hoc no tiene por qué corresponder al proceso real de decisión: es una reconstrucción que se ajusta a los datos observables (entradas y

Una explicación post-hoc no describe el proceso real de decisión — construye una narrativa plausible sobre él.

salidas) pero que puede ser consistente con múltiples procesos internos diferentes. Dos explicaciones post-hoc diferentes pueden ser igualmente compatibles con el comportamiento observado, y no hay forma de determinar cuál corresponde al proceso real.

Jain y Wallace (2019) demostraron experimentalmente que los pesos de atención no son explicaciones fiables: modelos con distribuciones de atención muy diferentes pueden producir las mismas predicciones, lo que significa que la atención no es una ventana al proceso de decisión sino un artefacto del mecanismo computacional. Adebayo et al. (2018) mostraron que varios métodos de explicación basados en saliencia producen mapas visualmente similares para modelos entrenados y para modelos con pesos aleatorios, cuestionando profundamente su validez como explicaciones.

Rudin (2019) — el argumento contra las explicaciones post-hoc para decisiones de alto riesgo.

4.2.2 Las explicaciones de los LLMs: racionalizaciones, no explicaciones

La aparición de los LLMs ha añadido una nueva modalidad de explicación post-hoc: el modelo genera, en lenguaje natural, una explicación de su propia respuesta. Cuando se le pide a un LLM que explique por qué ha dado una respuesta concreta, produce un texto coherente, articulado, y aparentemente razonable. La técnica de *chain-of-thought prompting* (Wei et al., 2022) explota esta capacidad pidiendo al modelo que razone paso a paso antes de producir la respuesta final.

Sin embargo, una serie de trabajos recientes ha demostrado que estas explicaciones son *racionalizaciones*: narrativas plausibles construidas para justificar la respuesta, no descripciones del proceso que la produjo.

Turpin et al. (2024) demostraron que los modelos producen cadenas de razonamiento que son sistemáticamente infieles cuando se introducen sesgos en el input. Los modelos son influenciados por los sesgos (cambian su respuesta) pero sus cadenas de razonamiento no mencionan los sesgos — construyen justificaciones que parecen razonables pero que no reflejan el proceso real. Lanham et al. (2023), en un estudio complementario, mostraron que modificar o truncar las cadenas de razonamiento intermedias frecuentemente no afecta a la respuesta final, lo que indica que el modelo no "usa" realmente esas cadenas para razonar sino que las genera como un artefacto lingüístico paralelo.

Turpin et al. (2024): el chain-of-thought puede ser sistemáticamente infiel al proceso real.

Para la accountability, las consecuencias son severas. Un sistema conversacional que, ante una auditoría, produce una explicación generada por el LLM está proporcionando una racionalización, no una explicación. La racionalización puede ser completamente coherente y completamente incorrecta como descripción del proceso real. Si el auditor pregunta «¿por qué el sistema rechazó la solicitud de cancelación?», una explica-

Lanham et al. (2023) — infidelidad de las cadenas de razonamiento.

ción post-hoc generada por el LLM dirá algo como «porque la póliza tiene menos de 30 días de antigüedad». Pero esta explicación puede ser una reconstrucción plausible que no corresponde al proceso real: quizás el modelo rechazó la solicitud por una combinación opaca de patrones en sus datos de entrenamiento, y la regla de los 30 días es simplemente la justificación más plausible que el modelo puede construir.

4.2.3 El paradigma constitutivo

Explicabilidad constitutiva

Un sistema exhibe **explicabilidad constitutiva** cuando su proceso de decisión opera sobre representaciones explícitas cuya traza es inherentemente explicativa. La explicación no se genera *después* de la decisión como una reconstrucción; *es* la traza del proceso de decisión. La correspondencia entre la explicación y el proceso real está garantizada por construcción, no por aproximación.

La diferencia entre ambos paradigmas puede ilustrarse con un ejemplo concreto.

En la explicabilidad constitutiva, la traza del proceso *es* la explicación. La correspondencia está garantizada por construcción.

Explicabilidad post-hoc vs. constitutiva

Situación: El sistema ha rechazado la solicitud de cancelación de un usuario. Un auditor pide la explicación.

Explicación post-hoc (LLM): «La solicitud fue rechazada porque la póliza tiene una antigüedad inferior a 30 días, lo que según las condiciones generales impide la cancelación sin penalización. El usuario no aceptó la penalización propuesta.»

Esta explicación es coherente, comprensible, y plausible. Pero no se puede verificar que corresponda al proceso real. Quizás el modelo rechazó la solicitud por otra razón y construyó esta justificación porque es la más coherente con los datos visibles.

Explicación constitutiva (ACAI): Traza de ejecución:

1. Input: «Quiero cancelar mi seguro» → NLU: acto `solicitud_cancelación`, producto `seguro_hogar_HG-2019-4421`.
2. Motor: regla `cancel_eligibility_01` → precondición: antigüedad $\geq$ 30 días → consulta: antigüedad = 18 días → precondición no satisfecha.
3. Motor: regla `cancel_penalty_01` → cancelación posible con penalización de 45 € → acción: informar y solicitar aceptación.
4. Input: «No, entonces prefiero esperar» → NLU: acto `rechazo_condición` → Motor: actualizar IS, solicitud can-

celada por el usuario. No ejecutar cancelación.

Esta traza no es una reconstrucción — es el registro literal de lo que ocurrió. Cada paso muestra la regla aplicada, los datos consultados, y la decisión tomada. La correspondencia con el proceso real es total porque la traza *es* el proceso.

La distinción entre ambos paradigmas no es de grado sino de naturaleza. La explicación post-hoc es una hipótesis sobre el proceso; la explicación constitutiva es un registro del proceso. La primera puede ser correcta o incorrecta; la segunda es correcta por construcción. La primera es útil para la comprensión; la segunda es necesaria para la accountability.

4.2.4 El argumento de Rudin y su extensión a la IA conversacional

El argumento de Rudin (2019) fue formulado para modelos de machine learning clásico — clasificadores, modelos de regresión, redes neuronales de clasificación — y proponía usar modelos intrínsecamente interpretables (modelos lineales dispersos, listas de reglas, modelos aditivos generalizados) en lugar de modelos opacos con explicaciones post-hoc. La extensión de este argumento a la IA conversacional requiere una adaptación que reconozca la especificidad del dominio.

En un sistema conversacional, no se puede simplemente reemplazar el LLM por un modelo interpretable: la variabilidad lingüística del lenguaje natural no es tratable con modelos lineales ni listas de reglas. El P1 de ACAI (separación de competencias) reconoce esta realidad: la capacidad lingüística requiere un LLM. Lo que el argumento de Rudin implica para ACAI es que las *decisiones operativas* — aquellas que tienen consecuencias de negocio, regulatorias o éticas — deben tomarse mediante un proceso intrínsecamente interpretable. Las decisiones lingüísticas (cómo interpretar, cómo generar) pueden delegarse al LLM; las decisiones operativas (qué hacer, qué reglas aplicar, qué información proporcionar) deben operar sobre representaciones explícitas con trazas constitutivas.

La explicabilidad constitutiva como requisito de diseño

La distinción post-hoc/constitutiva transforma la explicabilidad de un problema de análisis (dado un modelo opaco, generar una explicación) en un problema de diseño (diseñar el sistema de modo que las decisiones sean inherentemente explicables). Para ACAI, esto significa que la arquitectura no debe diseñarse primero y hacerse explicable después, sino que la explicabilidad debe ser

una propiedad constitutiva del diseño desde el origen. Esta es la razón por la que la tesis raíz dice que la accountability es un problema de *representación*: las representaciones formales son el mecanismo que hace posible la explicabilidad constitutiva.

4.3 El marco regulatorio como requisito de ingeniería

4.3.1 De la regulación a las especificaciones técnicas

El tratamiento del marco regulatorio en esta unidad no pretende ser un análisis jurídico sino una *traducción*: convertir los requisitos legales en especificaciones técnicas que el diseño de ACAI debe satisfacer. El investigador no necesita ser experto en derecho europeo, pero sí necesita entender qué le exige la regulación a un sistema conversacional de IA desplegado en un sector regulado.

Tres marcos regulatorios son especialmente relevantes para ACAI: el EU AI Act, que regula los sistemas de IA de forma general con requisitos específicos para sistemas de alto riesgo; DORA (*Digital Operational Resilience Act*), que impone requisitos de resiliencia digital al sector financiero; y el GDPR, que incluye disposiciones sobre decisiones automatizadas que afectan a individuos. Para cada uno, extraeremos los requisitos técnicos concretos.

La regulación no pide que los sistemas "intenten ser explicables" — exige trazabilidad, verificabilidad y supervisión humana como propiedades demostrables.

4.3.2 EU AI Act: requisitos para sistemas de alto riesgo

El EU AI Act, aprobado en 2024 y con entrada en vigor progresiva hasta 2027, clasifica los sistemas de IA en categorías de riesgo y establece requisitos proporcionales. Los sistemas de IA conversacional desplegados en sectores como seguros, banca, salud o administración pública pueden caer en la categoría de alto riesgo, dependiendo de su función y contexto de despliegue.

EU AI Act (2024) — Reglamento (UE) 2024/1689.

Para los sistemas de alto riesgo, los artículos 9 a 15 del Reglamento establecen requisitos que pueden traducirse en especificaciones técnicas concretas. Hacker et al. (2023), en su análisis de las implicaciones técnicas del AI Act, realizaron una primera traducción sistemática que resulta directamente útil para el diseño de ACAI.

Art. 9–15 del EU AI Act: requisitos para sistemas de alto riesgo.

El artículo 9 exige un **sistema de gestión de riesgos** que incluya la identificación, estimación y evaluación de los riesgos conocidos y previsibles. Para un sistema conversacional, esto se traduce en la necesidad de documentar formalmente qué puede salir mal (riesgos de interpretación incorrecta, de ejecución de acciones erróneas, de violación de políticas) y cómo se mitiga cada riesgo. Un sistema ACAI puede

satisfacer este requisito porque los modos de fallo están ligados a reglas formales identificables.

El artículo 11 exige **documentación técnica** que incluya una descripción del diseño, los datos de entrenamiento, el proceso de decisión, y los mecanismos de supervisión. El artículo 12 exige que el sistema tenga capacidades de **registro automático** (*logging*) que permitan la trazabilidad de su funcionamiento a lo largo del ciclo de vida. Para un sistema conversacional, esto se traduce en la necesidad de que cada turno del diálogo produzca una traza que incluya el input recibido, la interpretación, las reglas aplicadas, las acciones ejecutadas, y la respuesta generada. La arquitectura ACAI produce esta traza constitutivamente: no es un log añadido sino el registro natural del proceso de decisión del motor simbólico.

Hacker et al. (2023) — implementación técnica del AI Act.

El artículo 13 exige **transparencia** y la provisión de información suficiente para que los usuarios comprendan las capacidades y limitaciones del sistema. El artículo 14 exige **supervisión humana** efectiva, que incluye la capacidad de un humano de intervenir en el funcionamiento del sistema. Para un sistema conversacional, esto se traduce en interfaces de inspección del information state y mecanismos de intervención (escalación a humano) integrados en el flujo formal.

4.3.3 DORA: resiliencia operativa en el sector financiero

DORA (*Digital Operational Resilience Act*) impone requisitos adicionales al sector financiero que son relevantes cuando el sistema conversacional opera en el ámbito de banca o seguros. Los requisitos más pertinentes para ACAI son la obligación de realizar pruebas de resiliencia operativa digital, incluyendo pruebas de escenarios adversos; la exigencia de que los sistemas terceros (incluidos los servicios de IA) estén sujetos a gestión de riesgos de terceras partes; y la obligación de reportar incidentes relacionados con las TIC, incluyendo fallos de sistemas de IA que afecten al servicio.

DORA: Reglamento (UE) 2022/2554, en vigor desde enero de 2025.

Para un sistema conversacional basado en LLM, DORA plantea un desafío adicional: la naturaleza probabilística del LLM hace que las pruebas de resiliencia no puedan producir resultados deterministas. El mismo escenario de prueba puede producir resultados diferentes en ejecuciones diferentes. Un sistema ACAI mitiga este problema porque las decisiones operativas son deterministas (el motor simbólico produce el mismo resultado ante el mismo information state y las mismas reglas), y los componentes no deterministas (NLU, NLG) operan bajo supervisión formal.

4.3.4 GDPR: el derecho a explicación

El artículo 22 del GDPR, junto con los considerandos 63 y 71, establece que las personas tienen derecho a no ser objeto de decisiones basadas únicamente en el tratamiento automatizado que produzcan efectos jurídicos o significativos. Cuando tales decisiones se toman, el interesado tiene derecho a obtener una explicación de la lógica subyacente.

Veale y Zuiderveen Borgesius (2021) analizaron las ambigüedades de este derecho y concluyeron que, pese a los debates jurídicos sobre su alcance exacto, la tendencia regulatoria es inequívocamente hacia la exigencia de explicaciones significativas. Edwards y Veale (2017) señalaron que la mayoría de las técnicas de XAI no producen explicaciones que satisfagan el estándar legal, porque proporcionan información sobre el modelo en general pero no sobre la decisión específica que afecta al individuo.

Veale & Zuiderveen Borgesius (2021) — el derecho a explicación y sus ambigüedades.

Para un sistema conversacional ACAI, el derecho a explicación se satisface constitutivamente: la traza del motor simbólico para una conversación específica documenta exactamente qué reglas se aplicaron, qué datos se consultaron, y qué decisiones se tomaron en ese caso concreto. La explicación no es genérica («el modelo suele considerar la antigüedad de la póliza») sino específica («en su caso, la regla `cancel_penalty_01` se aplicó porque su póliza tiene 18 días de antigüedad, lo que requiere una penalización de 45 € según el artículo 23.2 de las Condiciones Generales»).

4.3.5 Síntesis: la tabla de requisitos técnicos

Requisitos técnicos derivados del marco regulatorio

El análisis de los tres marcos regulatorios converge en un conjunto de requisitos técnicos concretos que el sistema conversacional debe satisfacer:

Trazabilidad por turno: cada turno del diálogo debe producir una traza que incluya input, interpretación, reglas aplicadas, datos consultados, acciones ejecutadas, y respuesta generada (EU AI Act art. 12).

Verificabilidad: la traza debe poder contrastarse contra una especificación formal (las reglas del modelo de conocimiento) para determinar si el comportamiento fue correcto (EU AI Act art. 9).

Explicación específica: el sistema debe poder producir una explicación de una decisión concreta que afecta a un individuo concreto, no una explicación genérica del modelo (GDPR art. 22).

Supervisión humana: el sistema debe proporcionar mecanismos

para que un humano inspeccione el estado del diálogo e intervenga si es necesario (EU AI Act art. 14).
Determinismo operativo: las decisiones con consecuencias de negocio deben ser deterministas, de modo que las pruebas de resiliencia produzcan resultados reproducibles (DORA).
Documentación formal: el modelo de conocimiento, las reglas, y los procedimientos deben estar documentados de forma que un auditor pueda comprenderlos y evaluarlos (EU AI Act art. 11).
Estos seis requisitos son satisfechos constitutivamente por la arquitectura ACAI: no son capas añadidas sino consecuencias directas del diseño basado en representaciones formales con motor simbólico.

4.4 Gobernanza constitutiva vs. superpuesta

4.4.1 Dos estrategias de gobernanza

El análisis de los mecanismos de accountability actuales (guardrails, alineamiento, explicaciones post-hoc) y de los requisitos regulatorios (trazabilidad, verificabilidad, explicación específica) converge en una distinción que generaliza la oposición post-hoc/constitutiva a nivel arquitectónico completo.

Gobernanza superpuesta: controlar desde fuera. Gobernanza constitutiva: diseñar desde dentro.

La **gobernanza superpuesta** añade mecanismos de control y explicabilidad sobre un sistema que opera de forma autónoma. El sistema base (el LLM) toma sus decisiones; las capas superpuestas (guardrails, monitores, generadores de explicaciones) intentan controlar, observar y explicar esas decisiones. Esta estrategia es la dominante en la industria actual y produce resultados valiosos: reduce los comportamientos indeseados, proporciona mecanismos de seguridad, y genera explicaciones que satisfacen a muchos usuarios.

La **gobernanza constitutiva** diseña el sistema de modo que las propiedades deseadas — trazabilidad, verificabilidad, explicabilidad, determinismo — sean consecuencias de la estructura interna del sistema, no de las capas añadidas. Las decisiones se toman sobre representaciones formales; la traza de esas decisiones es inherentemente explicativa; la especificación formal permite la verificación; la estructura del motor permite la atribución.

La diferencia puede formalizarse en términos de garantías. La gobernanza superpuesta proporciona garantías *estadísticas*: reduce la probabilidad de comportamientos indeseados. La gobernanza constitutiva proporciona garantías *formales*: para las decisiones que operan sobre

el modelo formal, las propiedades de accountability se cumplen por construcción. Las garantías estadísticas son valiosas y tienen su lugar; las garantías formales son lo que los sectores regulados requieren.

4.4.2 La analogía estructural

La analogía más precisa para la diferencia entre ambas estrategias proviene de la ingeniería civil. Un edificio puede protegerse contra incendios de dos formas complementarias. La protección superpuesta consiste en instalar alarmas, rociadores, extintores y planes de evacuación. La protección constitutiva consiste en construir el edificio con materiales resistentes al fuego, diseñar las rutas de evacuación como parte de la estructura, y separar los compartimentos de forma que el fuego no pueda propagarse. Ambas estrategias son necesarias; pero solo la segunda ofrece la garantía de que un fuego en la cocina no colapsará el edificio. Los códigos de edificación exigen ambas, pero la constitutiva es el fundamento sobre el que la superpuesta opera.

ACAI adopta una posición análoga. La gobernanza constitutiva — el motor simbólico operando sobre representaciones formales — es el fundamento. La gobernanza superpuesta — guardrails adicionales, monitorización, alertas — puede añadirse como capa de seguridad adicional. Pero el diseño base debe garantizar las propiedades de accountability independientemente de las capas superpuestas, del mismo modo que la estructura del edificio debe resistir el fuego independientemente de que los rociadores funcionen.

4.5 Accountability y estructuras formales: la conexión con (AI)^2:TDG

4.5.1 La filosofía del proyecto marco

El proyecto (AI)^2:TDG (*Accountable and Interpretable AI through Topology and Discrete Geometry*) se funda en una tesis que puede formularse así: la explicabilidad robusta de los sistemas de IA requiere fundamentarse en *estructuras matemáticas formales* — topológicas, geométricas, algebraicas — no en aproximaciones numéricas ni en narrativas post-hoc. Las herramientas de la topología y la geometría discreta proporcionan invariantes, descomposiciones y relaciones estructurales que capturan propiedades de los sistemas de forma rigurosa y verificable.

Esta filosofía se manifiesta en las líneas de investigación del proyecto. RL1 explora cómo la topología algebraica y la geometría discreta pueden revelar estructura en datos de alta dimensionalidad y en los es-

pacios de representación de modelos de IA. RL2 investiga la integración neuro-simbólica y la representación del conocimiento como vehículos de accountability. RL3 aplica estas herramientas a dominios concretos, incluyendo los sistemas conversacionales verificables.

4.5.2 ACAI como instanciación de la filosofía

ACAI instancia la filosofía de (AI)^2:TDG en el dominio conversacional de una forma específica y coherente. El modelo de conocimiento formalizado es una estructura discreta — un grafo de reglas, estados, transiciones y restricciones — sobre el cual las propiedades de accountability son verificables formalmente. Las trazas conversacionales son trayectorias en un espacio de estados discreto cuyas propiedades — cobertura, convergencia, consistencia — son analizables con herramientas formales.

Las trazas conversacionales de ACAI como trayectorias en un espacio de estados discreto — un objeto de estudio natural para la topología.

La conexión es más profunda de lo que la analogía sugiere. Un modelo de conocimiento conversacional define un espacio de estados (el conjunto de information states posibles) y un conjunto de transiciones (las reglas que determinan cómo se transita entre estados). Las conversaciones son caminos en este espacio. Propiedades como «¿puede el sistema llegar a un estado de cancelación sin pasar por un estado de confirmación?» son propiedades de accesibilidad del grafo de estados. Propiedades como «¿hay conversaciones que no terminan?» son propiedades de convergencia. Propiedades como «¿las reglas son consistentes entre sí?» son propiedades de satisfacibilidad de la especificación. Todas son formalmente verificables sobre la estructura discreta del modelo.

Las herramientas topológicas de RL1 entran como extensión natural cuando el análisis requiere ir más allá de las propiedades discretas elementales. La topología del espacio conversacional — su forma, sus componentes conexas, sus "agujeros" — puede revelar patrones estructurales no capturados por las reglas individuales. Un "agujero" en el espacio conversacional (una región de information states que el sistema no puede alcanzar ni desde la cual puede salir) es un problema de diseño que el análisis topológico puede detectar. Esta conexión no se explota en el sprint ACAI pero fundamenta la continuación de la investigación dentro de (AI)^2:TDG.

> **ACAI como puente entre accountability práctica e investigación fundamental**
>
> ACAI opera en dos planos simultáneamente. En el plano práctico, produce un sistema conversacional con propiedades de accountability verificables. En el plano de investigación, genera las estruc-

turas formales — modelos de conocimiento, espacios de estados, trazas — sobre las cuales las herramientas de (AI)^2:TDG pueden operar. La Phase 1 construye el plano práctico; la Phase 2 y la investigación posterior explotan el plano fundamental.

Conexiones con otras unidades

→ Prerequisito: UD 1 (Mapa conceptual) para la tesis raíz y los principios fundacionales. **UD 2** (La conversación como fenómeno) para el diagnóstico de la brecha de representación.

⇄ Complementa: UD 3 (Ecosistema operativo) analiza los componentes tecnológicos y sus límites de gobernanza. UD4 proporciona el marco conceptual y regulatorio que explica *por qué* esos límites son inaceptables. Juntas, UD3 y UD4 cierran el diagnóstico completo del Eje A.

Aplicación: UD 12 (Evaluación) toma directamente la taxonomía de accountability de esta unidad y la traduce en métricas operativas. Las propiedades definidas aquí — trazabilidad, verificabilidad, explicabilidad, atribución — se convierten en dimensiones medibles en UD12.

Aplicación: UD 9 (Formalismos) evalúa los formalismos candidatos considerando, entre otros criterios, su capacidad para satisfacer los requisitos regulatorios derivados aquí: ¿permite el formalismo producir trazas verificables? ¿Son las reglas legibles para un auditor?

⇄ Complementa: UD 5 (Fundamentos neuro-simbólicos) proporciona el marco teórico que hace posible la explicabilidad constitutiva: la integración neuro-simbólica es el paradigma que permite tener simultáneamente capacidad lingüística y proceso de decisión formal.

Ejercicios

1. [Taxonomía aplicada] Seleccione un sistema conversacional comercial (chatbot de banca, asistente de atención al cliente, o similar) y evalúe qué niveles de la cadena transparencia → interpretabilidad → explicabilidad → accountability satisface. Para cada nivel, proporcione evidencia concreta de si se satisface o no. Si el sistema proporciona algún tipo de explicación, clasifíquela como post-hoc o constitutiva y justifique su clasificación.

2. [Análisis regulatorio] Seleccione tres artículos del EU AI Act (puede usar los artículos 9, 12 y 14 discutidos en esta unidad o elegir otros) y, para cada uno, formule: (a) el requisito legal en lenguaje simplificado, (b) la traducción a especificación técnica concreta para un sistema conversacional, y (c) cómo la arquitectura ACAI satisface ese requisito y cómo fallaría un sistema basado exclusivamente en LLM.

3. [Diseño de traza] Para el escenario de la aseguradora, diseñe el formato de una traza de accountability para un turno conversacional completo. La traza debe incluir toda la información necesaria para que un auditor pueda verificar que la decisión fue correcta, explicar por qué se tomó, y atribuir la responsabilidad en caso de error. Especifique qué campos incluiría, qué tipo de dato contiene cada campo, y quién (qué componente) produce cada dato.

4. [Reflexión crítica] El argumento de Rudin (2019) contra las explicaciones post-hoc ha sido influyente pero también criticado. Algunos autores argumentan que en muchos dominios prácticos, las explicaciones post-hoc son «suficientemente buenas» y que la insistencia en modelos intrínsecamente interpretables limita innecesariamente la capacidad del sistema. Evalúe este contraargumento en el contexto específico de ACAI: ¿hay situaciones en un sistema conversacional regulado donde la explicabilidad post-hoc sería suficiente? ¿Dónde estaría la frontera?

5. [Aplicación a ACAI] Considere el siguiente escenario: un regulador financiero realiza una auditoría del sistema conversacional de una aseguradora y solicita evidencia de que el sistema cumple con el artículo 12 del EU AI Act (capacidades de registro). Compare la evidencia que podría proporcionar: (a) un sistema basado en LLM con guardrails y logging de inputs/outputs, (b) un sistema basado en LLM con RAG y un sistema de explicaciones post-hoc, y (c) un sistema ACAI con motor simbólico y trazas constitutivas. Para cada caso, evalúe si la evidencia satisface el requisito legal y argumente su evaluación.

Referencias y lecturas

Lectura de entrada

Rudin, C. (2019). Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. *Nature Machine Intelligence*, 1(5), 206–215. El artículo más

influyente sobre la insuficiencia de las explicaciones post-hoc para decisiones de alto riesgo. Argumentación clara, accesible, y directamente relevante para la distinción post-hoc/constitutiva que fundamenta ACAI. Lectura de entrada imprescindible.

Lecturas esenciales

- Arrieta, A. B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R. & Herrera, F. (2020). Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58, 82–115. — El survey de referencia para la taxonomía conceptual. Extenso pero con una estructura clara que permite lectura selectiva.
- Lipton, Z. C. (2018). The Mythos of Model Interpretability. *Queue*, 16(3), 31–57. — El análisis que desambiguó el concepto de interpretabilidad. Breve, incisivo, y fundacional para la taxonomía de esta unidad.
- Hacker, P., Engel, A. & Mauer, M. (2023). Regulating ChatGPT and other Large Generative AI Models. *FAccT 2023*. — La traducción más rigurosa de los requisitos del AI Act a especificaciones técnicas para modelos generativos. Directamente útil para derivar requisitos de ACAI.
- Turpin, M., Michael, J., Perez, E. & Bowman, S. R. (2024). Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. *NeurIPS 2023*. — La demostración empírica de que las cadenas de razonamiento de los LLMs son infieles. Evidencia central para el argumento contra las explicaciones post-hoc generadas por LLMs.

Lecturas complementarias

- Doshi-Velez, F. & Kim, B. (2017). Towards A Rigorous Science of Interpretable Machine Learning. *ArXiv preprint arXiv:1702.08608*. — La propuesta de formalización de la interpretabilidad con niveles de evaluación.
- Molnar, C. (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable* (2.ª ed.). Leanpub. — Libro de referencia sobre técnicas de interpretabilidad. Disponible online gratuitamente. Útil como referencia para las técnicas post-hoc.
- Ribeiro, M. T., Singh, S. & Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *KDD 2016*. — LIME: la técnica de explicación local más influyente.
- Lundberg, S. M. & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS 2017*. — SHAP: valores de Shapley para explicación de modelos.
- Jain, S. & Wallace, B. C. (2019). Attention is not Explanation. *NAACL 2019*. — La demostración de que los pesos de atención no son explicaciones fiables.
- Adebayo, J., Gilmer, J., Muelly, M., Goodfellow, I., Hardt, M. & Kim, B. (2018). Sanity Checks for Saliency Maps. *NeurIPS 2018*. — Cuestionamiento de la validez de explicaciones basadas en saliencia.

- Lanham, T., Chen, A., Radhakrishnan, A., Steiner, B., Denison, C., Hernandez, D., Li, D., Durmus, E., Hubinger, E., Kernion, J., Lukošiūtė, K., Nguyen, K., Cheng, N., Joseph, N., Schiefer, N., Rausch, O., Larson, R., McCandlish, S., Kundu, S., Kadavath, S., Yang, S., Henighan, T., Maxwell, T., Telleen-Lawton, T., Hume, T., Hatfield-Dodds, Z., Kaplan, J., Brauner, J., Bowman, S. R. & Perez, E. (2023). Measuring Faithfulness in Chain-of-Thought Reasoning. *ArXiv preprint arXiv:2307.13702*. — Evidencia complementaria sobre la infidelidad de las cadenas de razonamiento.
- Schwalbe, G. & Finzel, B. (2023). A Comprehensive Taxonomy for Explainable Artificial Intelligence: A Systematic Survey of Surveys on Methods and Concepts. *Data Mining and Knowledge Discovery*, 38, 3183–3233. — Taxonomía actualizada de XAI con cobertura de LLMs.
- Holzinger, A., Saranti, A., Molnar, C., Biecek, P. & Samek, W. (2022). Explainable AI Methods — A Brief Overview. *xxAI — Beyond Explainable AI*, Lecture Notes in Computer Science 13200, 13–38. — Panorámica concisa y actualizada de métodos de XAI.
- Veale, M. & Zuiderveen Borgesius, F. (2021). Demystifying the Draft EU Artificial Intelligence Act. *Computer Law & Security Review*, 41, 105581. — Análisis accesible del AI Act con perspectiva técnica.
- Edwards, L. & Veale, M. (2017). Slave to the Algorithm? Why a "Right to an Explanation" Is Probably Not the Remedy You Are Looking For. *Duke Law & Technology Review*, 16(1), 18–84. — Análisis crítico del derecho a explicación en el GDPR.
- EU AI Act (2024). Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo por el que se establecen normas armonizadas en materia de inteligencia artificial. — Texto legal oficial.
- DORA (2022). Reglamento (UE) 2022/2554 sobre la resiliencia operativa digital del sector financiero. — Texto legal oficial.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS 2022*. — El artículo que popularizó chain-of-thought. Relevante como contexto para la discusión sobre racionalización.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«Symbols without grounding are fragile.
Learning without structure is blind.
The future belongs to their integration.»

Eje B — El paradigma neuro-simbólico

Unidad 5 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje B — El paradigma neuro-simbólico

Prerequisito: UD1 · Eje B — El paradigma neuro-simbólico

UNIDAD DIDÁCTICA 5

Fundamentos del paradigma neuro-simbólico

Contenidos de la unidad

5.1.1 Dos intuiciones sobre la inteligencia 3
5.1.2 Lo que cada paradigma captura y lo que pierde 5
Las fortalezas del razonamiento simbólico 5
Las debilidades del razonamiento simbólico 6
Las fortalezas del procesamiento neuronal 6
Las debilidades del procesamiento neuronal 7
5.1.3 La complementariedad como argumento para la integración 7
5.2.1 El programa de Garcez y Lamb 8
5.2.2 El estado de la tercera ola 9
5.3.1 El marco de referencia 10
5.3.2 Nivel 1: Symbolic Neuro Symbolic 10
5.3.3 Nivel 2: Symbolic[Neuro] 11
5.3.4 Nivel 3: Neuro : Symbolic ⇌ Neuro 12
5.3.5 Niveles 4 y 5: horizontes de investigación 13
5.3.6 El espectro como mapa de ruta 14
5.4.1 Coordenadas del posicionamiento 15
5.4.2 Las especificidades de ACAI 16
5.4.3 Lo que ACAI toma y lo que aporta 16

¿Por qué necesito esto para ACAI?

ACAI propone una arquitectura neuro-simbólica: un motor simbólico que gobierna las decisiones operativas, un LLM que maneja la variabilidad lingüística, y una integración progresiva entre ambos. Pero esta propuesta no surge del vacío. Se inscribe en un programa de investigación con décadas de historia, un marco teórico articulado, y una taxonomía de niveles de integración que estructura directamente las fases del sprint. Esta unidad proporciona ese fundamento. Al terminar, el investigador sabrá de dónde viene el paradigma neuro-simbólico, qué ofrece, cómo se clasifica la integración, y dónde se sitúa ACAI en ese paisaje. Sin esta comprensión, las decisiones arquitectónicas del sprint parecerán arbitrarias; con ella, se revelan como consecuencias de un análisis riguroso.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Caracterizar las fortalezas y debilidades de los paradigmas simbólico y conexionista con precisión técnica, no como caricaturas.
2. Explicar el programa de las tres olas de la IA neuro-simbólica y situar ACAI en la tercera ola.
3. Describir los cinco niveles de la taxonomía de Kautz, identificar qué capacidades ofrece cada nivel, y justificar por qué ACAI progresa del nivel 1–2 al 2–3.
4. Argumentar por qué la integración neuro-simbólica es una necesidad técnica — no una preferencia estética — para un sistema conversacional accountable.
5. Identificar el posicionamiento de ACAI en el paisaje neuro-simbólico y su contribución específica al programa de investigación.

5.1 La tensión fundacional de la inteligencia artificial

Sede principal de este concepto.

5.1.1 Dos intuiciones sobre la inteligencia

La inteligencia artificial nació, en la segunda mitad del siglo XX, con dos intuiciones simultáneas e incompatibles sobre cómo producir comportamiento inteligente en máquinas. Ambas intuiciones captaban

algo real sobre la naturaleza de la cognición; ambas perdían algo esencial. La tensión entre ellas ha definido el campo durante siete décadas y sigue siendo, en una forma transformada, la tensión que ACAI propone resolver en el dominio conversacional.

La primera intuición — la **intuición simbólica** — fue articulada con máxima precisión por Newell y Simon (1976) en su hipótesis del sistema de símbolos físicos: «Un sistema de símbolos físicos tiene los medios necesarios y suficientes para la acción inteligente general.» La inteligencia, según esta hipótesis, consiste en la capacidad de manipular representaciones simbólicas según reglas formales. El conocimiento se codifica en estructuras discretas con semántica definida — proposiciones, reglas, marcos, redes semánticas — y el razonamiento es un proceso de transformación formal de esas estructuras. La tradición que emergió de esta intuición produjo los sistemas expertos, la lógica computacional, los planificadores automáticos, las ontologías, y los lenguajes de representación del conocimiento que siguen siendo herramientas fundamentales de la ingeniería del conocimiento.

Dos intuiciones fundacionales: la inteligencia como manipulación de símbolos vs. la inteligencia como aprendizaje estadístico.

La segunda intuición — la **intuición conexionista** — tiene raíces en los trabajos de McCulloch y Pitts (1943) sobre neuronas artificiales y de Rosenblatt (1958) sobre el perceptrón, pero alcanzó madurez computacional con el redescubrimiento de la retropropagación por Rumelhart, Hinton y Williams (1986). La inteligencia, según esta intuición, no reside en reglas explícitas sino que emerge de la interacción entre grandes números de unidades simples conectadas, cuyos pesos se ajustan mediante aprendizaje a partir de datos. El conocimiento no se codifica explícitamente sino que se distribuye en los patrones de activación de la red. El razonamiento no es transformación formal de símbolos sino propagación de activación a través de capas de conexiones ponderadas.

Newell & Simon (1976) — la hipótesis del sistema de símbolos físicos.

Es importante resistir la tentación de tratar estas intuiciones como posiciones extremas que nadie sostiene realmente. En su forma más sofisticada, ambas tradiciones han producido resultados extraordinarios. La tradición simbólica produjo, entre otros logros, los sistemas de verificación formal que garantizan la corrección de software crítico, las bases de conocimiento que sustentan la web semántica, y los planificadores que controlan misiones espaciales. La tradición conexionista produjo, en su forma contemporánea, los modelos de lenguaje que comprenden y generan texto con fluidez sin precedentes, los sistemas de visión que superan la capacidad humana en tareas de clasificación, y los modelos generativos que producen imágenes, música y código. Ninguna de estas tradiciones es un error — cada una captura un aspecto genuino de lo que necesitamos para construir sistemas inteligentes.

Rumelhart, Hinton & Williams (1986): la retropropagación como fundamento del conexionismo moderno.

5.1.2 Lo que cada paradigma captura y lo que pierde

La tensión entre ambos paradigmas no es una cuestión de preferencia sino de *propiedades complementarias*. Cada paradigma ofrece capacidades que el otro no puede proporcionar, y cada uno tiene limitaciones que el otro no sufre. Caracterizar estas complementariedades con precisión es fundamental para entender por qué la integración neuro-simbólica es una necesidad técnica, no una preferencia estética.

Las fortalezas del razonamiento simbólico

El razonamiento simbólico posee cuatro propiedades que son directamente relevantes para la accountability y que el procesamiento neuronal no puede proporcionar.

Lo simbólico aporta estructura, composicionalidad, verificabilidad, y eficiencia en conocimiento — exactamente lo que ACAI necesita para la accountability.

La primera es la **composicionalidad**. Los sistemas simbólicos operan sobre representaciones que se componen de partes con significado propio, y el significado del todo se deriva del significado de las partes y de las reglas de composición. Una regla como «si el cliente solicita cancelación Y la antigüedad es menor a 30 días, ENTONCES aplicar penalización» se compone de condiciones y acciones que tienen significado independiente, y el significado de la regla completa se deriva de la semántica de la conjunción y del condicional. Esta propiedad permite construir sistemas cuyo comportamiento es analizable en términos de sus partes constituyentes. Fodor y Pylyshyn (1988), en su crítica clásica del conexionismo, argumentaron que la composicionalidad es una propiedad irrenunciable para cualquier sistema que aspire a la sistematicidad del pensamiento.

Fodor & Pylyshyn (1988) — composicionalidad y sistematicidad como propiedades irrenunciables.

La segunda es la **verificabilidad**. Un sistema que opera sobre reglas formales puede ser verificado contra una especificación: se puede comprobar que las reglas son consistentes entre sí, que cubren todos los casos requeridos, que no producen estados no deseados. Las técnicas de verificación formal — model checking, theorem proving, análisis estático — llevan décadas siendo aplicadas con éxito a sistemas simbólicos. Esta capacidad de verificación es exactamente la propiedad que la **UD 4** identificó como componente esencial de la accountability y que los requisitos regulatorios exigen.

La tercera es la **interpretabilidad intrínseca**. Las representaciones simbólicas tienen semántica definida: cada símbolo denota algo, cada regla tiene una lectura interpretable. La traza de un razonamiento simbólico es una secuencia de pasos con significado, no una secuencia de multiplicaciones de matrices. Esta propiedad es lo que permite la explicabilidad constitutiva introducida en la **UD 4**: la traza del proceso de decisión *es* la explicación.

La cuarta es la **eficiencia en conocimiento**. Un sistema simbólico

puede incorporar conocimiento de dominio directamente, sin necesidad de aprender de miles de ejemplos. Una regla de negocio se codifica una vez y se aplica correctamente en todos los casos. Esta eficiencia es crucial en dominios donde los datos etiquetados son escasos pero el conocimiento experto es abundante — exactamente la situación de muchos dominios regulados.

Las debilidades del razonamiento simbólico

Pero el razonamiento simbólico tiene debilidades igualmente fundamentales que han limitado históricamente su aplicabilidad.

La más conocida es la **fragilidad** ante la variabilidad del mundo real. Un sistema de reglas que espera la entrada «cancelar póliza» fallará ante «me gustaría dejar de tener el seguro» o «¿cómo hago para darme de baja?». La variabilidad lingüística, perceptual y situacional del mundo real es intratable con reglas escritas a mano. Es exactamente lo que la **UD 2** documentó como la limitación fatal de los sistemas de diálogo basados en reglas.

La segunda debilidad es el **cuello de botella de la ingeniería del conocimiento**. Codificar todo el conocimiento necesario para un dominio complejo en representaciones formales es un proceso costoso, lento, y propenso a errores y omisiones. Este problema, identificado ya por Feigenbaum (1977) como la «limitación fundamental» de los sistemas expertos, nunca fue resuelto satisfactoriamente por la tradición simbólica. ACAI lo aborda directamente con I2 (LLM como meta-herramienta para la formalización asistida), que es precisamente un intento de romper este cuello de botella usando capacidades generativas.

Feigenbaum (1977): el cuello de botella de la adquisición de conocimiento como limitación fundamental de la IA simbólica.

La tercera debilidad es la **dificultad para manejar la incertidumbre y la gradualidad**. El mundo real no es nítido: los conceptos tienen fronteras difusas, la información es incompleta, las categorías se solapan. Los sistemas simbólicos clásicos operan en lógica bivalente (verdadero o falso) y manejan la incertidumbre mediante extensiones (lógica difusa, redes bayesianas, factores de certeza) que añaden complejidad sin resolver el problema de fondo: la representación discreta tiene dificultades intrínsecas para capturar la continuidad del mundo.

Las fortalezas del procesamiento neuronal

Las fortalezas del procesamiento neuronal son, en buena medida, la imagen especular de las debilidades del simbólico.

La **robustez ante la variabilidad** es quizás la fortaleza más relevante para ACAI. Un modelo entrenado sobre datos lingüísticos reconoce formulaciones nunca vistas, maneja sinónimos, comprende elipsis, y tolera errores gramaticales. Esta capacidad de generalización es lo que los

LLMs aportan a la arquitectura ACAI: manejar la variabilidad lingüística que las reglas formales no pueden cubrir.

La **capacidad de aprendizaje desde datos** permite adquirir patrones complejos sin codificación manual. Un LLM aprende las convenciones del lenguaje natural — pragmática, implicatura, cortesía, registro — a partir de su exposición a cantidades masivas de texto, sin que nadie tenga que formular reglas explícitas para cada convención.

La **representación distribuida** permite capturar similitudes y relaciones graduales entre conceptos. En el espacio de representación de un modelo neuronal, «cancelar», «rescindir», «dar de baja» y «anular» están próximos; «renovar» y «ampliar» están próximos entre sí pero lejos de los anteriores. Esta geometría semántica emerge del entrenamiento sin necesidad de codificación manual.

Las debilidades del procesamiento neuronal

Las debilidades del procesamiento neuronal son, simétricamente, la imagen especular de las fortalezas del simbólico. La **opacidad**: las decisiones son el resultado de operaciones numéricas distribuidas que carecen de interpretación semántica accesible. La **inconsistencia**: el mismo input procesado dos veces puede producir resultados diferentes debido a la naturaleza estocástica de la generación. La **imposibilidad de verificación formal**: no hay especificación contra la cual verificar ni proceso cuya conformidad se pueda comprobar. Y el **hambre de datos**: los modelos requieren cantidades masivas de ejemplos para aprender patrones que una sola regla simbólica podría capturar.

Marcus (2020): catálogo sistemático de las limitaciones del deep learning — razonamiento, composicionalidad, robustez.

Marcus (2020), en su análisis de las limitaciones del aprendizaje profundo, documentó estas debilidades con extensos ejemplos y argumentó que son consecuencias estructurales de la arquitectura, no defectos que versiones futuras corregirán. Bengio (2019), desde dentro de la tradición conexionista, reconoció la necesidad de incorporar composicionalidad y razonamiento sistemático en los modelos neuronales, en lo que denominó una agenda para la "consciencia computacional" (*System 2 deep learning*). Lake et al. (2017) propusieron que la capacidad de construir modelos causales composicionales es lo que distingue la cognición humana de las capacidades actuales del aprendizaje profundo.

Lake et al. (2017) — *Building Machines That Learn and Think Like People*.

5.1.3 La complementariedad como argumento para la integración

La simetría entre fortalezas y debilidades de ambos paradigmas no es accidental. Refleja una complementariedad profunda que puede formularse así: **lo que cada paradigma necesita es exactamente lo que el otro ofrece**.

El razonamiento simbólico necesita un mecanismo para manejar la variabilidad del mundo real — el procesamiento neuronal lo proporciona. El procesamiento neuronal necesita estructura, composicionalidad y verificabilidad — el razonamiento simbólico los proporciona. El simbólico necesita reducir el coste de la adquisición de conocimiento — los modelos generativos pueden asistir en la formalización. El neuronal necesita que sus decisiones sean auditables — las representaciones formales proporcionan el marco de auditoría.

La integración como necesidad técnica

Para ACAI, la integración neuro-simbólica no es una opción de diseño entre varias posibles. Es una *consecuencia directa* del análisis de requisitos. La accountability exige propiedades que solo el razonamiento simbólico puede proporcionar (verificabilidad, trazabilidad, determinismo). La operación en lenguaje natural exige capacidades que solo el procesamiento neuronal puede proporcionar (comprensión de formulaciones variables, generación de texto natural). Ningún paradigma por separado satisface ambos conjuntos de requisitos. La integración es la única solución que los satisface simultáneamente.

5.2 Las tres olas de la IA neuro-simbólica

5.2.1 El programa de Garcez y Lamb

Garcez y Lamb (2023), en su artículo programático *Neurosymbolic AI: The 3rd Wave*, propusieron una narrativa del campo de la inteligencia artificial organizada en tres olas que corresponden a tres respuestas diferentes a la tensión fundacional. Esta narrativa no es una periodización histórica estricta — las tres olas coexisten temporalmente — sino un marco conceptual que identifica tres *paradigmas* con propiedades y limitaciones diferentes.

Garcez & Lamb (2023) — *Neurosymbolic AI: The 3rd Wave.*

La **primera ola** corresponde a la IA simbólica en sus diversas manifestaciones: sistemas expertos, planificación automática, representación del conocimiento, razonamiento lógico. El conocimiento se representa explícitamente y el comportamiento se deriva formalmente de esas representaciones. Las fortalezas de la primera ola son la interpretabilidad, la verificabilidad, y la eficiencia en dominios bien definidos. Su limitación fundamental es la fragilidad: la incapacidad de manejar la variabilidad y la incertidumbre del mundo real, y el coste prohibitivo de la adquisición manual de conocimiento.

Primera ola: representación sin aprendizaje. Segunda ola: aprendizaje sin representación. Tercera ola: la síntesis.

La **segunda ola** corresponde al aprendizaje profundo en sus diversas

manifestaciones: redes convolucionales, modelos de secuencia, transformers, modelos generativos. El conocimiento se adquiere automáticamente desde los datos y se almacena en los parámetros del modelo. Las fortalezas de la segunda ola son la capacidad de aprendizaje, la robustez ante la variabilidad, y el rendimiento en tareas perceptuales y lingüísticas. Su limitación fundamental es la opacidad: la incapacidad de proporcionar garantías formales sobre el comportamiento, y la dificultad para incorporar conocimiento de dominio de forma controlada.

La **tercera ola** es el programa de integración neuro-simbólica: la construcción de sistemas que combinen las fortalezas de ambos paradigmas, usando representaciones simbólicas para la estructura y el razonamiento, y redes neuronales para la percepción, el aprendizaje y la generación. Garcez y Lamb argumentan que esta tercera ola no es simplemente la suma de las dos anteriores sino un paradigma genuinamente nuevo que requiere nuevas arquitecturas, nuevos formalismos, y nuevas metodologías de evaluación.

5.2.2 El estado de la tercera ola

Es necesario ser honesto sobre el estado de la tercera ola. Como señalan los propios Garcez y Lamb, la integración neuro-simbólica es más un programa de investigación en desarrollo que un paradigma consolidado. No existe una arquitectura neuro-simbólica canónica del mismo modo que existe la arquitectura transformer para el procesamiento neuronal del lenguaje. Lo que existe es un conjunto creciente de resultados que demuestran que la integración es posible y productiva en dominios específicos, y un consenso emergente sobre los principios que deben guiar esa integración.

Hitzler et al. (2022): volumen colectivo que documenta el estado del arte neuro-simbólico con 25 contribuciones.

Hitzler et al. (2022), en su volumen colectivo que reúne 25 contribuciones de los principales grupos de investigación, documentan avances concretos en la integración de lógica y redes neuronales, en el aprendizaje de representaciones simbólicas a partir de datos, en el razonamiento neuronal sobre grafos de conocimiento, y en la generación de explicaciones combinando análisis simbólico y neuronal. Estos avances son genuinos pero parciales: cada uno resuelve un aspecto de la integración en un dominio específico, y la síntesis general sigue siendo un objetivo de investigación.

ACAI se sitúa en este paisaje como una contribución a la tercera ola en un dominio específico (IA conversacional) con un objetivo específico (accountability). No pretende resolver el problema general de la integración neuro-simbólica sino demostrar que, en el dominio conversacional, una integración cuidadosamente diseñada produce propiedades de accountability que ningún paradigma por separado puede proporcionar.

Esta modestia de alcance es una fortaleza: permite diseños concretos donde el programa general requiere abstracciones.

> **La tercera ola como programa, no como logro**
>
> El investigador debe entender que la IA neuro-simbólica de tercera ola es un programa de investigación activo, no una tecnología madura con soluciones establecidas. ACAI contribuye a este programa; no lo hereda como un kit de herramientas listo para usar. Las decisiones de diseño del sprint — qué nivel de integración intentar, qué formalismos usar, cómo evaluar la integración — son decisiones de investigación genuinas, no aplicaciones de recetas conocidas.

5.3 La taxonomía de Kautz: cinco niveles de integración

Sede principal de este concepto.

5.3.1 El marco de referencia

Kautz (2022) — AAAI Robert S. Engelmore Memorial Lecture. Taxonomía fundacional.

Kautz (2022), en su AAAI Robert S. Engelmore Memorial Lecture titulada *The Third AI Summer*, propuso una taxonomía de cinco niveles de integración neuro-simbólica que ha sido adoptada ampliamente como marco de referencia. La taxonomía describe un espectro que va desde la composición secuencial más simple hasta la fusión completa de representaciones neuronales y simbólicas. Cada nivel ofrece capacidades que el anterior no tiene, pero exige cimientos que el anterior no requiere.

Esta taxonomía es directamente relevante para ACAI porque la propuesta ejecutiva define explícitamente las fases del sprint en sus términos: la Phase 1 opera en los niveles 1–2, y la Phase 2 avanza hacia los niveles 2–3. Entender qué significa cada nivel, qué capacidades ofrece, y qué exige, es imprescindible para comprender las decisiones arquitectónicas del sprint.

5.3.2 Nivel 1: Symbolic Neuro Symbolic

> **Nivel 1 — Symbolic Neuro Symbolic**
>
> En el nivel 1, un sistema simbólico usa un módulo neuronal como componente encapsulado, tratándolo como una caja negra con una interfaz definida. El sistema simbólico invoca al módulo neuronal para una tarea específica (por ejemplo, clasificación o generación

de texto), usa el resultado, y continúa su razonamiento simbólico. La dirección de control va del componente simbólico al neuronal.

Nivel 1: el componente simbólico orquesta; el neuronal es un servicio.

El ejemplo más directo de nivel 1 en el contexto de ACAI es el pipeline básico de la Phase 1: el motor simbólico invoca al LLM como servicio de NLU («interpreta este input del usuario») y como servicio de NLG («genera una respuesta que comunique esta decisión»). Entre ambas invocaciones, el motor opera de forma puramente simbólica: consulta las reglas, evalúa las condiciones, actualiza el information state. El LLM no tiene conocimiento del razonamiento simbólico; es un servicio lingüístico invocado con una interfaz definida.

Las ventajas del nivel 1 son la simplicidad y la claridad de la separación de competencias. Las decisiones simbólicas son completamente trazables porque el componente neuronal no interviene en ellas. La desventaja principal es la rigidez de la interfaz: el motor simbólico no puede condicionar cómo el LLM procesa su tarea (qué buscar en la interpretación, qué matices capturar), y el LLM no puede informar al motor sobre incertidumbres o matices que no encajan en la interfaz predefinida.

La versión dual — Neuro Symbolic Neuro, donde un sistema neuronal usa un componente simbólico — también existe y es de hecho el patrón más común en la industria actual: un LLM que invoca herramientas (calculadoras, bases de datos, APIs) tratándolas como servicios. Pero esta versión dual no produce accountability porque las decisiones de cuándo y cómo invocar el componente simbólico las toma el LLM de forma opaca. ACAI elige deliberadamente la dirección de control simbólico → neuronal.

5.3.3 Nivel 2: Symbolic[Neuro]

Nivel 2 — Symbolic[Neuro]

En el nivel 2, el componente neuronal opera *dentro* del marco definido por el componente simbólico. El sistema simbólico no solo invoca al neuronal como servicio sino que le proporciona contexto estructurado que condiciona su operación: restricciones sobre la salida esperada, vocabulario permitido, esquema de la respuesta, o distribución de probabilidad condicionada por el estado simbólico.

Nivel 2: el componente simbólico no solo invoca al neuronal — lo condiciona.

La diferencia entre los niveles 1 y 2 es sutil pero importante. En el nivel 1, el motor simbólico pide al LLM «interpreta este input» y recibe una respuesta. En el nivel 2, el motor pide «interpreta este input sabiendo que

estamos en un diálogo de cancelación, que el usuario ya ha proporcionado su número de póliza, y que los únicos actos comunicativos posibles en este punto son `confirmar`, `rechazar`, `preguntar_condiciones` o `proporcionar_dato`». Esta información estructurada reduce el espacio de interpretación del LLM y aumenta la fiabilidad de su respuesta.

Phase 1
Opera en los niveles 1–2.

La Phase 1 de ACAI incluye elementos de nivel 2 cuando el motor simbólico proporciona al LLM información del information state como contexto para la interpretación y la generación. Las técnicas de *structured prompting* — donde el prompt del LLM incluye el esquema esperado de la respuesta, las entidades relevantes del dominio, y las restricciones del estado del diálogo — son implementaciones prácticas del nivel 2.

Las ventajas del nivel 2 son una mayor fiabilidad de la interfaz neuro-simbólica (el LLM comete menos errores cuando está condicionado por contexto estructurado) y una integración más rica (el estado simbólico influye en el procesamiento neuronal). La desventaja es que el condicionamiento opera a nivel de prompt, no de computación: el LLM recibe instrucciones más precisas pero sigue interpretándolas probabilísticamente.

5.3.4 Nivel 3: Neuro : Symbolic ⇌ Neuro

Nivel 3 — Integración bidireccional

En el nivel 3, la interacción entre los componentes neuronal y simbólico es bidireccional y opera a un nivel más profundo que el intercambio de información por interfaz. El estado simbólico condiciona la computación neuronal (no solo su entrada sino su proceso), y los resultados de la computación neuronal alimentan la evolución del conocimiento simbólico. Ambos componentes se influyen mutuamente durante la operación del sistema.

El nivel 3 introduce una diferencia cualitativa respecto a los anteriores. En los niveles 1 y 2, la frontera entre lo neuronal y lo simbólico es una *interfaz*: los componentes se comunican mediante representaciones intermedias pero operan internamente de forma independiente. En el nivel 3, la frontera se convierte en una *zona de interacción* donde los paradigmas se influyen mutuamente.

Phase 2
Avanza hacia los niveles 2–3.

La Phase 2 de ACAI explora tres mecanismos de nivel 3 que fueron presentados como frentes de innovación I3a–c en la **UD 1**. El primero es el **condicionamiento simbólico de la computación neuronal** (I3a): el estado del diálogo modifica no solo el prompt sino los mecanismos de atención o las restricciones de decodificación del LLM. Técnicas como el *constrained decoding* — donde la generación del LLM se fuerza a producir

tokens que satisfacen una gramática formal — son implementaciones concretas de este mecanismo.

El segundo es el **aprendizaje simbólico desde trazas** (I3b): las trazas conversacionales de la Phase 1 se analizan para detectar patrones no cubiertos por las reglas formales y proponer extensiones del modelo. El conocimiento simbólico evoluciona a partir de evidencia empírica, cerrando parcialmente el cuello de botella de la ingeniería del conocimiento.

El tercero es el **razonamiento abductivo compartido** (I3c): ante inputs ambiguos, el sistema genera hipótesis interpretativas combinando la distribución de probabilidad del LLM con las restricciones del estado simbólico, ordenadas por plausibilidad formal. La interpretación no es ni puramente neuronal (probabilística) ni puramente simbólica (determinista) sino una integración de ambas.

Nivel 2 vs. nivel 3 en la interpretación

Input del usuario: «Sí, pero espere un momento.»
Nivel 2: El motor informa al LLM que los actos posibles son `confirmar`, `rechazar`, `pedir_pausa`. El LLM interpreta: probablemente `confirmar` + `pedir_pausa`. El motor procesa ambos actos secuencialmente.
Nivel 3: El motor no solo informa los actos posibles sino que condiciona las probabilidades de decodificación según el estado del diálogo. Si el turno anterior fue una solicitud de confirmación de cancelación (operación irreversible), el sistema aumenta la probabilidad de interpretaciones que incluyan señales de vacilación o solicitud de tiempo. Además, el patrón «sí, pero» en contexto de decisión irreversible se detecta como señal de *incertidumbre pragmática*, lo que activa un protocolo de verificación reforzada en el motor simbólico. La interpretación integra evidencia neuronal (la distribución de probabilidad) con conocimiento simbólico (el patrón pragmático y el contexto de la operación).

5.3.5 Niveles 4 y 5: horizontes de investigación

Nivel 4 — Neuro:Symbolic

En el nivel 4, las representaciones neuronales y simbólicas se mapean entre sí de forma sistemática. Los conceptos simbólicos tienen correspondencias directas en el espacio de representación neuronal, y viceversa. Esto permite, por ejemplo, que el aprendizaje neuronal produzca directamente nuevos conceptos simbólicos, o que la estructura simbólica organice el espacio de representación neuronal.

Nivel 5 — Neuro≡Symbolic

En el nivel 5, las representaciones neuronales y simbólicas se fusionan en un sistema unificado donde la distinción entre ambos paradigmas pierde sentido. El sistema opera sobre representaciones que son simultáneamente distribuidas (como las neuronales) y estructuradas (como las simbólicas).

Los niveles 4 y 5 representan horizontes de investigación que van más allá del alcance del sprint ACAI. Trabajos como Logic Tensor Networks (Badreddine et al., 2022), que representan fórmulas lógicas como operaciones sobre tensores, y DeepProbLog (Manhaeve et al., 2021), que integra redes neuronales con programación probabilística lógica, son aproximaciones al nivel 4 en dominios específicos. La fusión completa del nivel 5 sigue siendo un objetivo de investigación fundamental.

Logic Tensor Networks (Badreddine et al., 2022), DeepProbLog (Manhaeve et al., 2021): aproximaciones al nivel 4.

Para ACAI, los niveles 4 y 5 son relevantes no como objetivo del sprint sino como horizonte que muestra la dirección del programa de investigación a largo plazo. Las representaciones formales que ACAI produce en la Phase 1 y los mecanismos de integración que explora en la Phase 2 sientan las bases para exploraciones más profundas que podrían, en proyectos futuros, acercarse a los niveles 4 y 5.

5.3.6 El espectro como mapa de ruta

La taxonomía de Kautz justifica directamente el P3 de ACAI (la integración como espectro) y la estructura en dos fases del sprint. Cada nivel de la taxonomía ofrece capacidades que el anterior no tiene, pero exige cimientos que el anterior no requiere.

P3: la integración es un espectro. Avanzar deliberadamente es más productivo que intentar la fusión sin los cimientos resueltos.

Para operar en el nivel 1, se necesita un motor simbólico funcional y un LLM accesible como servicio. Para el nivel 2, se necesita además un mecanismo de condicionamiento del LLM basado en el estado simbólico. Para el nivel 3, se necesitan además técnicas de constrained decoding, mecanismos de aprendizaje desde trazas, y un marco para el razonamiento abductivo. Intentar el nivel 3 sin haber consolidado las representaciones formales del nivel 1 es como intentar integrar dos sistemas que no están individualmente definidos.

La Phase 1 consolida los niveles 1–2: diseña el lenguaje de representación, construye el motor simbólico, establece las interfaces con el LLM, y demuestra que el pipeline gobernado produce propiedades de accountability medibles. La Phase 2 avanza hacia el nivel 3 sobre esos cimientos, explorando los tres mecanismos de integración bidireccional. La **métrica central** de la Phase 2 es comparativa: debe demostrar que la integración de nivel 2–3 mejora la de nivel 1–2 en dimensiones concretas

(robustez, manejo de ambigüedad, adaptación a cambios de dominio).

> **Progresión deliberada vs. ambición prematura**
>
> La progresión por niveles no es cautela sino rigor. Cada nivel añade complejidad arquitectónica y reduce la trazabilidad del sistema. En el nivel 1, las decisiones simbólicas son completamente independientes del procesamiento neuronal; su trazabilidad es total. En el nivel 2, las decisiones simbólicas están condicionadas por información del contexto neuronal; la trazabilidad se mantiene pero incluye la dependencia del condicionamiento. En el nivel 3, la interacción bidireccional hace que las decisiones de cada componente dependan del otro; la trazabilidad requiere registrar la interacción completa.
> Para ACAI, donde la trazabilidad es un objetivo central, esta progresión implica que cada avance en integración debe ir acompañado de una extensión del modelo de trazabilidad que lo documente. La evaluación comparativa entre fases es el mecanismo que garantiza que la integración más profunda produce beneficios que justifican la complejidad añadida.

5.4 Posicionamiento de ACAI en el paisaje neuro-simbólico

5.4.1 Coordenadas del posicionamiento

Los tres marcos presentados — la tensión fundacional (sección 5.1), las tres olas de Garcez y Lamb (sección 5.2), y la taxonomía de Kautz (sección 5.3) — permiten situar ACAI con precisión en el paisaje neuro-simbólico.

ACAI es un sistema de **tercera ola**: combina representación simbólica explícita con procesamiento neuronal, buscando la síntesis de las fortalezas de ambos paradigmas. Opera en los **niveles 1–3 de Kautz**: comienza con un pipeline gobernado (niveles 1–2) y explora la integración bidireccional (nivel 3). Explota la **complementariedad** de forma específica: asigna la variabilidad lingüística al LLM y la gobernabilidad al motor simbólico, con interfaces de complejidad creciente entre ambos.

Pero ACAI no es simplemente una instanciación genérica del programa neuro-simbólico. Tiene al menos tres especificidades que lo distinguen de otros sistemas neuro-simbólicos y que constituyen sus contribuciones propias al programa.

5.4.2 Las especificidades de ACAI

ACAI aporta al programa neuro-simbólico tres especificidades: la triple restricción, el LLM como meta-herramienta, y la accountability como métrica central.

La primera especificidad es la **triple restricción** (I1): el lenguaje de representación debe ser simultáneamente legible por un humano, procesable por un motor, y generable por un LLM. La tercera restricción — la generabilidad — es completamente nueva en el panorama neuro-simbólico. Los formalismos clásicos de representación del conocimiento nunca se diseñaron para ser producidos por un modelo generativo; los lenguajes formales del paradigma neuro-simbólico asumen que las representaciones se codifican manualmente o se aprenden por el propio sistema. Que un LLM pueda generar fiablemente representaciones formales correctas es un requisito que la era de los modelos generativos hace posible plantear y que ACAI coloca en el centro de su diseño.

🔗 **Referencia contextual.** Este concepto se desarrolla en profundidad en **UD 9** y **UD 11**. La evaluación de formalismos contra la triple restricción y el análisis de sus tensiones se desarrollan en profundidad.

La segunda especificidad es el uso del **LLM como meta-herramienta** (I2): el LLM no solo participa en la operación del sistema conversacional sino que construye las representaciones formales que luego lo gobiernan. Este meta-uso — el modelo genera la maquinaria que lo restringe — no tiene precedente directo en la literatura neuro-simbólica. Los sistemas neuro-simbólicos clásicos asumen que las representaciones simbólicas se proporcionan externamente; ACAI propone que el propio LLM las genere, bajo supervisión humana, a partir de descripciones en lenguaje natural.

🔗 **Referencia contextual.** Este concepto se desarrolla en profundidad en **UD 10**. La formalización asistida por LLM y los experimentos de generabilidad se desarrollan como unidad con entregable directo para la Fase 0.

La tercera especificidad es que la **accountability** es la métrica central de evaluación, no una propiedad secundaria. En la mayoría de los sistemas neuro-simbólicos, la métrica principal es el rendimiento en una tarea (accuracy, F1, BLEU). En ACAI, la métrica principal es la calidad de la accountability: ¿son las decisiones trazables? ¿Son verificables? ¿Son consistentes? El rendimiento en la tarea conversacional es importante pero subordinado: un sistema que conversa bien pero no es accountable fracasa en el objetivo de ACAI, mientras que un sistema que es plenamente accountable pero menos fluido cumple el objetivo y puede mejorarse iterativamente.

5.4.3 Lo que ACAI toma y lo que aporta

Es importante distinguir con claridad qué toma ACAI del programa neuro-simbólico y qué aporta a él. ACAI toma el marco conceptual ge-

neral (la complementariedad de paradigmas), la taxonomía de niveles (como guía de progresión), y un conjunto de técnicas concretas (constrained decoding, structured output, semantic parsing con restricciones) que la **UD 7** desarrollará en detalle.

Lo que ACAI aporta es la demostración de que, en el dominio específico de la IA conversacional, una integración neuro-simbólica diseñada para la accountability puede producir propiedades — trazabilidad, verificabilidad, consistencia — que ningún paradigma por separado puede ofrecer. Si esta demostración tiene éxito, contribuye al programa neuro-simbólico con un caso empírico detallado, con un lenguaje de representación nuevo (diseñado bajo la triple restricción), y con una metodología de evaluación centrada en la accountability que puede adaptarse a otros dominios.

ACAI como contribución al programa neuro-simbólico

ACAI no es un consumidor pasivo del paradigma neuro-simbólico sino un contribuyente activo. Si el sprint produce sus entregables, el campo neuro-simbólico tendrá: (a) un caso empírico de integración en niveles 1–3 de Kautz con evaluación comparativa entre niveles, (b) un lenguaje de representación diseñado bajo una restricción inédita (generabilidad por LLM), (c) un pipeline de formalización asistida que aborda el cuello de botella de la ingeniería del conocimiento con herramientas generativas, y (d) una metodología de evaluación centrada en accountability.

Conexiones con otras unidades

Prerequisito: UD 1 (Mapa conceptual ACAI) para los principios P1, P3 y los frentes de innovación I1–I3. La UD5 puede estudiarse en paralelo con el Eje A (UD2–UD4), pero las conexiones con el diagnóstico del problema son más ricas si se ha completado al menos la UD2.

Complementa: UD 2 (La conversación como fenómeno) proporcionó el diagnóstico de la brecha de representación desde la perspectiva del fenómeno conversacional. UD5 proporciona el paradigma que fundamenta la solución a esa brecha.

Complementa: UD 4 (Accountability y marco regulatorio) definió las propiedades que el sistema debe satisfacer. UD5 muestra que solo la integración neuro-simbólica puede satisfacerlas simultáneamente con la capacidad lingüística necesaria.

Aplicación: UD 6 (Arquitecturas y patrones) toma la taxonomía

de Kautz y la concreta en arquitecturas implementables. UD5 proporciona el marco; UD6 proporciona las piezas de construcción.

Aplicación: UD 7 (Técnicas neuro-simbólicas para NLP) toma las capacidades de nivel 2 y 3 y las materializa en técnicas concretas: constrained decoding, structured output, semantic parsing con restricciones. UD5 fundamenta la necesidad; UD7 proporciona las herramientas.

Ejercicios

1. [Análisis] Para el escenario de la aseguradora (cancelación de póliza con consulta de bonificación), describa cómo manejaría el diálogo completo un sistema puramente simbólico (primera ola) y un sistema puramente neuronal (segunda ola). Para cada caso, identifique con precisión: (a) qué hace bien, (b) dónde falla, y (c) qué propiedad del paradigma causa el fallo. Conecte su análisis con la tabla de fortalezas y debilidades de la sección 5.1.2.

2. [Clasificación] Para cada uno de los siguientes sistemas, determine en qué nivel de la taxonomía de Kautz opera y justifique su clasificación: (a) un chatbot basado en RASA con un modelo BERT para NLU y un gestor de diálogo basado en reglas, (b) un LLM con function calling que invoca una base de datos SQL, (c) un sistema que usa constrained decoding para forzar al LLM a generar respuestas en un formato JSON válido según un esquema de dominio, (d) un sistema donde las trazas conversacionales se usan para refinar las reglas del motor simbólico mediante aprendizaje supervisado.

3. [Diseño] Proponga un mecanismo concreto de nivel 3 (integración bidireccional) para el siguiente escenario: el usuario dice algo ambiguo («No estoy seguro de lo que me conviene») en un punto del diálogo donde hay múltiples interpretaciones posibles (¿pide información? ¿quiere cancelar el trámite? ¿solicita tiempo para pensar?). Describa cómo combinaría la distribución de probabilidad del LLM con las restricciones del estado simbólico para producir una interpretación ordenada por plausibilidad. Especifique qué información aporta cada componente y cómo se integran.

4. [Reflexión crítica] Marcus (2020) argumenta que el deep learning tiene limitaciones estructurales que no se resolverán con más datos ni con modelos más grandes. LeCun, Bengio y Hinton, en su respuesta, argumentan que las técnicas de aprendizaje profundo

seguirán avanzando y eventualmente resolverán los problemas que Marcus identifica. ¿Cómo afecta la posición que se adopte en este debate al diseño de ACAI? Si Marcus tiene razón, ¿qué implicaciones tiene para la integración neuro-simbólica? Si los defensores del deep learning tienen razón, ¿pierde ACAI su razón de ser?

5. [Aplicación a ACAI] La Phase 2 de ACAI debe demostrar que la integración de nivel 2–3 mejora respecto a la Phase 1 (nivel 1–2). Diseñe un experimento comparativo que permita medir esta mejora. Especifique: (a) qué dimensión de rendimiento o accountability mediría, (b) qué escenarios de prueba usaría, (c) qué resultado esperaría del sistema de nivel 1–2 y qué del sistema de nivel 2–3, y (d) qué resultado invalidaría la hipótesis de que la integración profunda aporta valor.

Referencias y lecturas

Lectura de entrada

Garcez, A. d'Avila & Lamb, L. C. (2023). Neurosymbolic AI: The 3rd Wave. *Artificial Intelligence Review*, 56, 12387–12406. El artículo programático que define las tres olas y establece la agenda de la IA neuro-simbólica. Accesible, bien argumentado, y directamente relevante para entender la posición de ACAI en el campo. Lectura de entrada imprescindible.

Lecturas esenciales

- Kautz, H. (2022). The Third AI Summer: AAAI Robert S. Engelmore Memorial Lecture. *AI Magazine*, 43(1), 93–104. — La taxonomía de cinco niveles que estructura las fases de ACAI. Breve, claro, y fundacional para esta unidad.
- Hitzler, P., Sarker, M. K. & Eberhart, A. (Eds.) (2022). *Neuro-Symbolic Artificial Intelligence: The State of the Art*. IOS Press. — El volumen colectivo de referencia. 25 contribuciones que cubren el paisaje completo. No es lectura de entrada sino de referencia: consultar las contribuciones específicas según necesidad.
- Lake, B. M., Ullman, T. D., Tenenbaum, J. B. & Gershman, S. J. (2017). Building Machines That Learn and Think Like People. *Behavioral and Brain Sciences*, 40, e253. — El argumento sobre la necesidad de composicionalidad y modelos causales. Relevante para entender las limitaciones del paradigma conexionista puro.
- Marcus, G. (2020). The Next Decade in AI: Four Steps Towards Robust Artificial Intelligence. *ArXiv preprint arXiv:2002.06177*. — El análisis más

articulado de las limitaciones del deep learning y la necesidad de integración con representaciones simbólicas.

Lecturas complementarias

- Newell, A. & Simon, H. A. (1976). Computer Science as Empirical Inquiry: Symbols and Search. *Communications of the ACM*, 19(3), 113–126. — La hipótesis del sistema de símbolos físicos. Lectura fundacional de valor histórico.
- Rumelhart, D. E., Hinton, G. E. & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536. — Retropropagación: el fundamento técnico del conexionismo moderno. Breve y fundacional.
- Fodor, J. A. & Pylyshyn, Z. W. (1988). Connectionism and cognitive architecture: A critical analysis. *Cognition*, 28(1–2), 3–71. — La crítica clásica del conexionismo desde la perspectiva de la composicionalidad. Relevante para P1.
- Bengio, Y. (2019). From System 1 Deep Learning to System 2 Deep Learning (conferencia invitada, NeurIPS 2019). — El reconocimiento desde dentro del conexionismo de la necesidad de composicionalidad y razonamiento. Relevante como complemento a Marcus (2020).
- Badreddine, S., Garcez, A. d'Avila, Serafini, L. & Spranger, M. (2022). Logic Tensor Networks. *Artificial Intelligence*, 303, 103649. — Representación de fórmulas lógicas como operaciones tensoriales. Ejemplo de integración en niveles 3–4.
- Manhaeve, R., Dumančić, S., Kimmig, A., Demeester, T. & De Raedt, L. (2021). Neural Probabilistic Logic Programming in DeepProbLog. *Artificial Intelligence*, 298, 103504. — Integración de redes neuronales con programación probabilística lógica. Ejemplo de integración en niveles 3–4.
- Hamilton, K., Nayak, A., Božić, B. & Longo, L. (2022). Is Neuro-Symbolic AI Meeting its Promises in Natural Language Processing? A Structured Review. *Semantic Web*, 13(6), 1047–1074. — Revisión del estado de la integración neuro-simbólica en NLP. Puente directo hacia la UD7.
- McCulloch, W. S. & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5, 115–133. — Neuronas artificiales: el origen histórico del conexionismo. Valor arqueológico.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408. — El perceptrón. Valor histórico.
- Feigenbaum, E. A. (1977). The Art of Artificial Intelligence: Themes and Case Studies of Knowledge Engineering. *IJCAI-77*. — El cuello de botella de la adquisición de conocimiento. Relevante para I2.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«Architecture is the allocation of decisions to the components best equipped to make them.»

Eje B — El paradigma neuro-simbólico

Unidad 6 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje B — El paradigma neuro-simbólico

Prerequisitos: UD1, UD5 · Eje B — El paradigma neuro-simbólico

UNIDAD DIDÁCTICA 6

Arquitecturas y patrones de integración neuro-simbólica

Contenidos de la unidad

6.1.1 Los tres ejes de decisión 3
Dirección de control 4
Granularidad de la interfaz 4
Timing de la integración 5
6.2.1 El pipeline gobernado 6
6.2.2 Neural con guardrails simbólicos 7
6.2.3 Symbolic con fallback neuronal 8
6.2.4 Structured prompting 9
6.3.1 Constrained decoding 10
6.3.2 El reasoning loop neuro-simbólico 12
6.3.3 Aprendizaje simbólico desde trazas 13
6.4.1 Una arquitectura, múltiples patrones 14
6.4.2 El information state como pilar integrador 15
6.5.1 Trazabilidad vs. flexibilidad 16
6.5.2 Latencia vs. profundidad de integración 16
6.5.3 Complejidad vs. verificabilidad 17

¿Por qué necesito esto para ACAI?

La UD 5 estableció el mapa teórico del paradigma neuro-simbólico: la tensión fundacional, las tres olas, la taxonomía de Kautz. Pero un mapa teórico no se construye: se necesitan piezas. Esta unidad proporciona las piezas de construcción — patrones arquitectónicos concretos que implementan la integración neuro-simbólica a diferentes niveles de profundidad. Al terminar, el investigador dispondrá de un repertorio de patrones evaluados contra las propiedades de accountability, entenderá cómo la arquitectura ACAI se compone a partir de esos patrones, y podrá analizar los trade-offs que toda decisión arquitectónica implica.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Identificar los tres ejes de decisión arquitectónica — dirección de control, granularidad de interfaz, timing de integración — y justificar las elecciones de ACAI en cada eje.
2. Describir los patrones arquitectónicos de nivel 1–2 (pipeline gobernado, neural con guardrails simbólicos, symbolic con fallback neuronal, structured prompting) y evaluar sus propiedades de accountability.
3. Describir los patrones de nivel 3 (constrained decoding, reasoning loop neuro-simbólico, aprendizaje simbólico desde trazas) y analizar qué ganan y qué pierden respecto a los patrones de nivel 1–2.
4. Explicar cómo la arquitectura ACAI se compone a partir de múltiples patrones que operan en diferentes momentos y niveles.
5. Evaluar los trade-offs de trazabilidad vs. flexibilidad, latencia vs. profundidad, y complejidad vs. verificabilidad en el diseño de sistemas neuro-simbólicos.

6.1 Principios de diseño para la integración

Sede principal de este concepto.

6.1.1 Los tres ejes de decisión

El diseño de una arquitectura neuro-simbólica no es un problema con una solución única. Es un espacio de decisiones donde diferentes

Toda arquitectura neuro-simbólica es el resultado de tres decisiones: quién orquesta, qué se comunican los componentes, y cuándo interactúan.

elecciones producen sistemas con propiedades radicalmente diferentes. Antes de presentar patrones concretos, es necesario mapear este espacio identificando los ejes de decisión que lo organizan. Garcez et al. (2019), en su trabajo sobre integración neuronal-simbólica, propusieron una taxonomía de dimensiones de diseño que puede refinarse, para los propósitos de ACAI, en tres ejes fundamentales.

Garcez et al. (2019) — dimensiones de diseño para la integración neuro-simbólica.

Dirección de control

El primer eje es la **dirección de control**: ¿qué componente orquesta el flujo de procesamiento? Esta decisión es la más consequencial para las propiedades del sistema porque determina dónde residen las decisiones operativas.

En un patrón *symbolic-in-the-loop*, el componente simbólico orquesta y el neuronal es un servicio invocado para tareas específicas. Las decisiones operativas las toma el motor simbólico; el LLM proporciona capacidad lingüística bajo demanda. La trazabilidad de las decisiones es directa porque el proceso de decisión es simbólico y, por tanto, inspectable.

En un patrón *neural-in-the-loop*, el componente neuronal orquesta y el simbólico es una herramienta invocada cuando se necesita. Es el patrón de los frameworks agénticos actuales: el LLM decide cuándo consultar una base de datos, cuándo aplicar una regla, cuándo invocar un servicio. Como analizó la **UD 3**, este patrón produce flexibilidad a costa de la gobernabilidad.

En un patrón de *co-orquestación*, ambos componentes tienen capacidad de iniciativa y se coordinan mediante un protocolo de interacción. Es el patrón más rico pero también el más complejo, y la trazabilidad requiere registrar las decisiones de coordinación.

ACAI elige deliberadamente *symbolic-in-the-loop* para las decisiones operativas.

ACAI elige *symbolic-in-the-loop* para las decisiones operativas como consecuencia directa del análisis de accountability. Si el objetivo es que las decisiones sean verificables contra una especificación formal, el componente que las toma debe operar sobre representaciones formales. Esto no significa que el LLM no tenga autonomía alguna: la tiene para la interpretación lingüística y la generación de texto. Lo que no tiene es autonomía para las decisiones de negocio — qué información proporcionar, qué acción ejecutar, qué política aplicar.

Granularidad de la interfaz

El segundo eje es la **granularidad de la interfaz**: ¿qué riqueza tienen las representaciones que fluyen entre los componentes? Este eje opera en un continuo que va desde interfaces mínimas hasta interfaces ricas.

Una *interfaz mínima* transmite strings: el motor envía un texto al LLM y recibe un texto como respuesta. Es la interfaz de nivel 1 en la taxonomía de Kautz. Su simplicidad es una ventaja (fácil de implementar, fácil de depurar) pero su pobreza es una limitación (no transmite incertidumbre, no especifica restricciones, no proporciona contexto estructurado).

Una *interfaz estructurada* transmite objetos tipados: el motor envía al LLM no solo un texto sino un objeto que incluye el texto del usuario, el estado relevante del diálogo (en formato estructurado), las restricciones sobre la respuesta esperada (esquema JSON, actos comunicativos válidos, entidades del dominio), y metadatos (nivel de confianza requerido, política de generación). El LLM responde con un objeto que incluye la interpretación estructurada, señales de confianza, y candidatos alternativos. Es la interfaz de nivel 2, y la que ACAI implementa en la Phase 1.

Una *interfaz computacional* permite que un componente intervenga en el proceso de cómputo del otro: no solo envía datos sino que modifica restricciones sobre cómo se procesan. El constrained decoding, donde el motor simbólico modifica las probabilidades de generación del LLM token a token, es una interfaz computacional. Es la interfaz de nivel 3.

La lección de la **UD 2** sobre los pipelines NLU–DM–NLG es directamente relevante aquí: aquellos sistemas fallaron porque la interfaz entre módulos — el par (intent, slots) — era demasiado pobre para transmitir la riqueza de la comprensión lingüística. ACAI debe diseñar interfaces suficientemente ricas para preservar la información necesaria sin perder la separación de competencias.

Timing de la integración

El tercer eje es el **timing de la integración**: ¿en qué momento del ciclo de vida del sistema interactúan los componentes?

La integración en *design time* ocurre durante la construcción del sistema, no durante su operación. El caso paradigmático en ACAI es la formalización asistida (I2): el LLM ayuda a construir las representaciones formales a partir de descripciones en lenguaje natural, pero una vez compiladas, las representaciones son puramente simbólicas y el LLM no participa en su ejecución. La ventaja es que las representaciones resultantes pueden verificarse antes del despliegue; la desventaja es que la adaptación a situaciones no previstas requiere un nuevo ciclo de design time.

La integración en *runtime* ocurre durante la operación del sistema, en cada turno de la conversación. El pipeline gobernado de ACAI opera en runtime: el LLM interpreta, el motor decide, el LLM genera. La ventaja es la adaptación en tiempo real; la desventaja es que la calidad depende

ACAI integra en design time (formalización asistida) y en runtime (pipeline gobernado), con diferentes propiedades.

de la fiabilidad del LLM en cada turno.

La integración en *learning time* ocurre cuando las trazas de operación se analizan para mejorar el sistema. Las trazas conversacionales de la Phase 1 pueden analizarse para detectar patrones no cubiertos y proponer extensiones del modelo formal. Es una integración que cierra el ciclo: la operación informa al diseño.

ACAI opera en los tres momentos, y esta es una de sus particularidades arquitectónicas: la mayoría de los sistemas neuro-simbólicos operan solo en uno o dos.

6.2 Patrones arquitectónicos de nivel 1–2

Sede principal de este concepto.

Los patrones de nivel 1–2 en la taxonomía de Kautz comparten una característica común: la frontera entre los componentes neuronal y simbólico es una *interfaz* bien definida. Los componentes se comunican mediante representaciones intermedias pero operan internamente de forma independiente. Esto produce una propiedad valiosa: las decisiones de cada componente pueden analizarse por separado. En contrapartida, la rigidez de la interfaz limita la riqueza de la integración.

6.2.1 El pipeline gobernado

Pipeline gobernado

El **pipeline gobernado** es un patrón de nivel 1–2 en el que el componente simbólico orquesta un flujo secuencial: interpretación (neuronal) → decisión (simbólico) → generación (neuronal). El motor simbólico determina qué se interpreta, cómo se decide, y qué se genera; el LLM ejecuta las tareas lingüísticas bajo las restricciones del motor.

Phase 1
Patrón central de la arquitectura.

El pipeline gobernado es el patrón central de la Phase 1 de ACAI y opera en dos modos complementarios.

El *modo de compilación* (design time) transforma conocimiento conversacional expresado en lenguaje natural en representaciones formales. Un experto de dominio describe, por ejemplo, las condiciones de cancelación de una póliza en prosa. El LLM, guiado por un prompt estructurado que incluye el esquema del formalismo ACAI, genera una representación formal de esas condiciones. Un validador simbólico verifica la corrección formal de la representación. Un experto humano revisa y aprueba. El resultado es un modelo formal verificado que no depende

del LLM para su ejecución. Este modo implementa I2 (LLM como meta-herramienta) y opera en nivel 2: el LLM no genera libremente sino que está condicionado por el esquema del formalismo.

El *modo de ejecución* (runtime) procesa cada turno de la conversación. El flujo puede describirse con precisión. Primero, el LLM recibe el input del usuario junto con el estado relevante del diálogo (interfaz estructurada de nivel 2) y produce una interpretación en el formato esperado por el motor: acto comunicativo, entidades, señales de *grounding*. Segundo, el motor simbólico recibe la interpretación, consulta el modelo de conocimiento, evalúa las reglas aplicables, actualiza el information state, y determina la acción del sistema: qué información proporcionar, qué preguntar, qué acción ejecutar, qué confirmar. Tercero, el LLM recibe la acción determinada por el motor junto con el contexto conversacional y las políticas de generación, y produce la respuesta en lenguaje natural. La traza registra cada paso con la regla aplicada, los datos consultados, y el information state resultante.

Las propiedades de accountability del pipeline gobernado son directas. La *trazabilidad* es completa para las decisiones simbólicas: cada decisión se rastrea hasta la regla que la produjo y el information state sobre el que operó. La *verificabilidad* es posible: las reglas pueden comprobarse formalmente antes del despliegue. La *consistencia* es garantizada para las decisiones operativas: el mismo information state y las mismas reglas producen la misma decisión.

Pan et al. (2023) — integración de LLMs con bases de conocimiento como pipeline gobernado.

Las limitaciones son igualmente claras. La calidad del sistema depende críticamente de la calidad de la interpretación del LLM en el paso 1: si el LLM malinterpreta un input, la decisión simbólica será correcta respecto a la interpretación errónea pero incorrecta respecto a la intención real del usuario. La interfaz entre NLU y motor es un cuello de botella: debe ser suficientemente rica para transmitir las dimensiones relevantes del acto comunicativo pero suficientemente estructurada para ser procesable por el motor. Y la rigidez del pipeline hace que las situaciones no previstas por las reglas (los *unknown unknowns*) produzcan fallos predecibles (no hay regla que aplicar) en lugar de degradaciones graduales.

6.2.2 Neural con guardrails simbólicos

Neural con guardrails simbólicos

En el patrón **neural con guardrails simbólicos**, un LLM opera como sistema principal y un componente simbólico filtra, valida o modifica su output según reglas formales. La dirección de control

es neural-in-the-loop: el LLM genera y lo simbólico restringe.

Este patrón es el más extendido en la industria actual y merece un análisis cuidadoso porque es la alternativa más visible a la arquitectura ACAI. NeMo Guardrails (Rebedea et al., 2023) implementa este patrón con un lenguaje de especificación (Colang) que define flujos conversacionales y restricciones que se aplican sobre la generación del LLM. Llama Guard (Inan et al., 2023) implementa una variante donde el filtro es él mismo un modelo neuronal entrenado sobre una taxonomía de categorías de seguridad. Guardrails AI implementa una versión programática donde las restricciones son funciones Python que validan el output.

Rebedea et al. (2023): NeMo Guardrails y Colang como ejemplo industrial del patrón.

El patrón tiene virtudes reales. Es fácil de desplegar: se añade sobre un sistema existente sin rediseñar la arquitectura. Es modular: los guardrails pueden añadirse, modificarse o eliminarse independientemente. Es efectivo para restricciones simples: filtrar contenido prohibido, validar formatos, detectar temas fuera de alcance.

Pero para la accountability, sus limitaciones son fundamentales, y se concentran en la dirección de control. El LLM toma las decisiones y el guardrail las filtra. Esto significa que el guardrail solo puede actuar sobre decisiones que el LLM ya ha tomado: puede rechazar una respuesta pero no puede determinar cuál sería la respuesta correcta. Puede detectar una violación de política pero no puede garantizar que la respuesta que no viola la política sea consistente con las decisiones anteriores. El guardrail opera sobre la *superficie* (el texto generado), no sobre el *proceso* (las razones que produjeron el texto). Y la traza que produce es una traza de filtrado (qué se bloqueó y por qué), no una traza de decisión (por qué se decidió lo que se decidió).

La diferencia con el pipeline gobernado de ACAI es arquitectónica, no de grado. En ACAI, el motor simbólico determina qué responder *antes* de que el LLM genere. En el patrón de guardrails, el LLM genera y el simbólico filtra *después*. El primero produce accountability constitutiva; el segundo produce control superficial.

6.2.3 Symbolic con fallback neuronal

Symbolic con fallback neuronal

En el patrón **symbolic con fallback neuronal**, un sistema de reglas opera como componente principal y recurre al LLM cuando ninguna regla cubre la situación actual. La dirección de control es symbolic-in-the-loop cuando las reglas aplican, y se pierde cuando el sistema recurre al fallback.

Este patrón es común en sistemas de diálogo híbridos como RASA con fallback a LLM, y en chatbots corporativos que combinan flujos predefinidos con una capacidad generativa para preguntas no anticipadas. Su atractivo es pragmático: permite construir un sistema que maneja los casos conocidos con reglas y los desconocidos con el LLM.

La limitación principal es que la transición al fallback es un punto de pérdida de gobernabilidad. Cuando el sistema activa el fallback, pasa de decisiones trazables a decisiones opacas, y la traza registra «fallback activado» sin más detalle. Para un auditor, esto es un vacío: sabe que el sistema tomó una decisión pero no puede verificar si fue correcta. Además, la decisión de *cuándo* activar el fallback — cuándo las reglas "no cubren" la situación — es en sí misma una decisión que puede ser incorrecta: el sistema puede activar el fallback cuando una regla aplicaba (pérdida de gobernabilidad innecesaria) o no activarlo cuando debería (respuesta incorrecta basada en reglas inadecuadas).

ACAI se distingue de este patrón en un aspecto importante: no tiene un "modo fallback" donde el LLM opere sin gobernanza. Cuando el motor simbólico no tiene reglas para una situación, puede escalar a un operador humano (una acción gobernada con traza), solicitar información adicional al usuario (una acción gobernada con traza), o informar de sus limitaciones (una acción gobernada con traza). Lo que no hace es delegar la decisión al LLM de forma no gobernada.

6.2.4 Structured prompting

Structured prompting

El **structured prompting** es un patrón de nivel 2 en el que el estado simbólico se inyecta en el prompt del LLM como contexto estructurado que condiciona su operación. El motor no solo invoca al LLM sino que configura su contexto de procesamiento con información estructurada: esquema de la respuesta esperada, entidades del dominio relevantes, restricciones del estado del diálogo, y políticas de generación.

El structured prompting ha evolucionado significativamente con la maduración de las APIs de LLMs. Las técnicas de *structured output* — donde la API del LLM garantiza que la respuesta se ajusta a un esquema JSON predefinido — representan una implementación industrializada del nivel 2. OpenAI (2024) introdujo Structured Outputs como una feature que garantiza conformidad con un *JSON Schema* proporcionado. Anthropic y Google ofrecen mecanismos análogos. Estas implementaciones resuelven un problema práctico del nivel 1: la necesidad de parsear

OpenAI (2024) — Structured Outputs: JSON mode garantizado por esquema.

texto libre para extraer información estructurada, que era una fuente frecuente de errores.

Para ACAI, el structured prompting es el mecanismo principal de la interfaz motor → LLM en la Phase 1. Cuando el motor invoca al LLM para NLU, le proporciona el esquema del acto comunicativo esperado, las entidades relevantes del dominio, el estado del diálogo, y las restricciones de interpretación. Cuando lo invoca para NLG, le proporciona la acción determinada, el contexto conversacional, la política de generación aplicable, y el formato de respuesta. En ambos casos, el LLM opera dentro de un espacio condicionado por el estado simbólico.

Structured Outputs: el LLM garantiza que la respuesta se ajusta a un esquema. Nivel 2 industrializado.

La ventaja del structured prompting respecto a la interfaz de strings del nivel 1 es sustancial: reduce significativamente los errores de interpretación (el LLM sabe qué buscar) y de generación (el LLM sabe qué formato producir). La limitación es que el condicionamiento opera a nivel de prompt, no de computación: el LLM recibe instrucciones más precisas pero sigue interpretándolas probabilísticamente. Un esquema JSON garantiza el formato pero no la corrección semántica del contenido.

6.3 Patrones arquitectónicos de nivel 3

Sede principal de este concepto.

Los patrones de nivel 3 se distinguen de los anteriores en un aspecto fundamental: la frontera entre componentes deja de ser una interfaz y se convierte en una zona de interacción. Los componentes no solo intercambian datos sino que se influyen mutuamente durante el procesamiento. Esto produce una integración más profunda con capacidades que los niveles anteriores no tienen, pero a costa de una complejidad significativamente mayor.

6.3.1 Constrained decoding

Constrained decoding

El **constrained decoding** es un patrón de nivel 3 en el que el componente simbólico interviene *durante* la generación del LLM, modificando las probabilidades de los tokens a generar para garantizar que la salida satisface restricciones formales. El componente simbólico no actúa antes (como en el structured prompting) ni después (como en los guardrails) sino *durante* la computación neuronal.

El mecanismo es técnicamente preciso. En cada paso de la generación autoregresiva, el LLM produce una distribución de probabilidad sobre el vocabulario completo de tokens. Antes de muestrear el siguiente token, una máscara derivada de una gramática formal o un conjunto de restricciones anula la probabilidad de los tokens que violarían las restricciones. El LLM solo puede generar tokens permitidos, pero dentro de esos tokens, la distribución de probabilidad refleja las preferencias aprendidas del modelo. El resultado es una salida que es simultáneamente natural (refleja los patrones del LLM) y formalmente correcta (satisface las restricciones simbólicas).

El constrained decoding es la implementación más directa de nivel 3: lo simbólico interviene durante la computación neuronal, no solo antes o después.

Willard y Louf (2023), en su trabajo sobre Outlines, demostraron que el constrained decoding con gramáticas formales es práctico y eficiente: el overhead computacional es moderado y la calidad de la generación se mantiene o incluso mejora (porque las restricciones eliminan salidas degeneradas). Su implementación permite definir restricciones como gramáticas context-free, expresiones regulares, o esquemas JSON, y las aplica durante la generación con un autómata que se ejecuta en paralelo con el modelo.

Willard & Louf (2023) — Outlines: constrained generation con gramáticas formales.

Zheng et al. (2023), con SGLang, propusieron un enfoque complementario que integra las restricciones de generación con el control del flujo del programa, permitiendo intercalar generación libre con generación restringida dentro de una misma secuencia. Esto es directamente relevante para ACAI: en un turno conversacional, algunas partes de la respuesta pueden estar fuertemente restringidas (datos factuales, compromisos formales) mientras que otras pueden ser más libres (expresiones de cortesía, transiciones discursivas).

Zheng et al. (2023): SGLang — intercalar generación libre y restringida en una misma secuencia.

Para la Phase 2 de ACAI, el constrained decoding ofrece una ganancia específica respecto al structured prompting de la Phase 1. En la Phase 1, el LLM genera una interpretación dentro de un esquema, pero la corrección semántica depende de que el LLM siga las instrucciones probabilísticamente. En la Phase 2 con constrained decoding, el LLM solo puede generar interpretaciones que satisfacen la gramática del formalismo ACAI. La corrección *sintáctica* está garantizada por construcción; la corrección *semántica* sigue dependiendo del LLM, pero el espacio de errores posibles se reduce drásticamente.

6.3.2 El reasoning loop neuro-simbólico

Reasoning loop neuro-simbólico

El **reasoning loop** es un patrón de nivel 3 en el que el LLM y el motor simbólico interactúan en múltiples ciclos dentro de un mismo turno conversacional. El LLM propone, el motor evalúa, y si la propuesta no satisface las restricciones, el motor proporciona feedback estructurado que condiciona la siguiente propuesta del LLM. El ciclo continúa hasta que se alcanza una propuesta válida o se agota un presupuesto de iteraciones.

El reasoning loop extiende el pipeline gobernado de un flujo lineal (NLU → decisión → NLG) a un flujo iterativo donde la interfaz neuronal-simbólica se cruza múltiples veces. La diferencia es análoga a la que existe entre un compilador de una pasada y un compilador con refinamiento iterativo: el segundo produce mejores resultados porque puede corregir errores y refinar aproximaciones.

Nye et al. (2021): *scratchpad reasoning* como proto-reasoning loop.

Nye et al. (2021), en su trabajo sobre *scratchpad reasoning*, demostraron que los modelos de lenguaje producen resultados significativamente mejores cuando pueden generar pasos intermedios que son evaluados y corregidos. Aunque su trabajo no usaba un motor simbólico explícito, la estructura es análoga: generación → evaluación → corrección → regeneración.

En el contexto de ACAI, el reasoning loop puede implementarse en al menos tres situaciones donde el pipeline lineal de la Phase 1 es insuficiente.

La primera situación es la *interpretación de inputs ambiguos*. Cuando el LLM produce una interpretación que el motor evalúa como ambigua (el acto comunicativo no se determina unívocamente), el motor puede devolver al LLM las hipótesis alternativas con sus restricciones respectivas y solicitar una desambiguación informada. El LLM no genera una segunda interpretación desde cero sino condicionada por el análisis simbólico de la ambigüedad.

La segunda situación es la *generación de respuestas complejas*. Cuando la respuesta del sistema debe integrar múltiples componentes — información factual, compromisos, opciones, solicitudes de confirmación —, el motor puede guiar la generación mediante un ciclo: generar un borrador, evaluar su completitud y corrección contra los requisitos simbólicos, proporcionar feedback sobre lo que falta o es incorrecto, regenerar.

La tercera situación es la *formalización asistida* (design time). El pipeline de compilación de la Phase 1 puede ser un reasoning loop: el LLM genera una primera formalización, el validador simbólico detecta erro-

res formales, el LLM corrige condicionado por el feedback del validador, y el ciclo continúa hasta obtener una representación válida.

Reasoning loop para interpretación ambigua

Input: «Me gustaría cambiar lo del seguro.»
Ciclo 1: LLM interpreta → acto: `solicitar_cambio`, objeto: `seguro` (sin especificar producto ni tipo de cambio). Motor evalúa: interpretación infraespecificada, múltiples lecturas compatibles (cambiar cobertura, cambiar titular, cancelar, modificar forma de pago). Feedback al LLM: «El input es compatible con los actos `modificar_cobertura`, `cambiar_titular`, `solicitar_cancelación`, `modificar_pago`. El usuario tiene dos pólizas activas: hogar (HG-2019-4421) y auto (AU-2021-0087). ¿Hay evidencia lingüística que discrimine entre estas interpretaciones?»
Ciclo 2: LLM reanaliza condicionado por las opciones concretas → la formulación «cambiar lo del seguro» con el uso de «lo del» sugiere referencia a un tema previo o a una situación conocida, no una operación técnica específica → clasificación: `solicitud_inespecífica` con alta incertidumbre. Motor decide: solicitar clarificación al usuario con las opciones pertinentes.
Resultado: El sistema no adivina la intención del usuario ni elige una interpretación arbitraria. La traza registra la ambigüedad, las hipótesis evaluadas, y la decisión de solicitar clarificación.

6.3.3 Aprendizaje simbólico desde trazas

Aprendizaje simbólico desde trazas

El **aprendizaje simbólico desde trazas** es un patrón de nivel 3 en el que las trazas de ejecución del sistema se analizan (usando técnicas neuronales) para detectar patrones no cubiertos por las reglas formales y proponer extensiones del modelo simbólico. La dirección de integración es inversa a la del pipeline gobernado: aquí es el componente neuronal el que alimenta la evolución del simbólico.

Este patrón implementa lo que la **UD 5** describió como I3b: la evolución del conocimiento simbólico a partir de evidencia empírica. A diferencia de los dos patrones anteriores, que operan en runtime, este opera en learning time — entre sesiones, no durante la conversación.

La mecánica puede describirse en cuatro pasos. Primero, el sistema acumula trazas conversacionales de la operación normal, incluyendo los casos donde el motor no encontró reglas aplicables, donde la interpretación del LLM fue corregida por el usuario, o donde el usuario expresó insatisfacción. Segundo, un proceso de análisis — que puede

usar el LLM como herramienta de procesamiento — clasifica estos casos e identifica patrones recurrentes que sugieren lagunas en el modelo formal. Tercero, el sistema propone extensiones o modificaciones del modelo formal: nuevas reglas, nuevas entidades, nuevos flujos. Cuarto, un experto humano evalúa las propuestas, las aprueba, modifica o descarta, y las incorpora al modelo formal.

Este patrón es especialmente relevante para ACAI porque aborda simultáneamente dos problemas. Por un lado, cierra parcialmente el cuello de botella de la ingeniería del conocimiento (el modelo formal se enriquece a partir de la operación real, no solo del diseño manual). Por otro, proporciona un mecanismo de mejora continua que es compatible con la accountability: las extensiones propuestas son reglas formales que se verifican antes de incorporarse, no ajustes opacos de parámetros.

El aprendizaje desde trazas cierra parcialmente el cuello de botella de la ingeniería del conocimiento.

6.4 La arquitectura ACAI como composición de patrones

6.4.1 Una arquitectura, múltiples patrones

El análisis de patrones de las secciones anteriores podría sugerir que ACAI debe elegir uno. La realidad es más rica: ACAI compone múltiples patrones que operan en diferentes momentos del ciclo de vida y a diferentes niveles de integración, coordinados por el information state como estructura central.

ACAI no es un patrón arquitectónico único sino una composición deliberada de patrones que operan en diferentes momentos y niveles.

En *design time*, ACAI usa el pipeline de formalización asistida: un reasoning loop (nivel 3) donde el LLM genera representaciones formales, el validador simbólico las evalúa, y el ciclo itera hasta obtener un modelo formal correcto que un experto humano aprueba. El resultado es un modelo de conocimiento compilado que incluye reglas de diálogo, reglas de negocio, políticas de generación, y configuración del ecosistema operativo (E1–E4).

En *runtime Phase 1*, ACAI usa el pipeline gobernado (nivel 1–2) con structured prompting: el motor simbólico orquesta, invocando al LLM con interfaces estructuradas para NLU y NLG, tomando las decisiones operativas sobre el modelo formal, y actualizando el information state con cada turno. La gobernanza del ecosistema operativo (RAG gobernado, capacidad agéntica gobernada, alineamiento formalizado) opera dentro de este pipeline.

En *runtime Phase 2*, ACAI añade al pipeline gobernado los mecanismos de nivel 3: constrained decoding para garantizar que las interpretaciones y generaciones satisfacen restricciones formales, y reasoning loops para manejar ambigüedad y complejidad. La frontera rígida de

la Phase 1 se convierte en una zona de interacción más fluida, pero el motor simbólico retiene la autoridad sobre las decisiones operativas.

En *learning time*, ACAI usa el aprendizaje simbólico desde trazas para proponer extensiones del modelo formal a partir de la operación real, cerrando el ciclo design → runtime → learning → design.

6.4.2 El information state como pilar integrador

En esta composición de patrones, el **information state** — introducido en la **UD 3** — funciona como la estructura que conecta todos los componentes y todos los momentos. Es el objeto sobre el que el motor simbólico opera, lo que el LLM recibe como contexto estructurado, lo que las trazas registran, y lo que el aprendizaje desde trazas analiza. Cada patrón de la composición lee del information state, escribe en él, o lo usa como referencia.

Flujo completo en la arquitectura compuesta

Contexto: El usuario dice «Quiero cancelar mi seguro de hogar, pero no sé si me conviene» en el turno 3 de la conversación. La identidad está verificada desde el turno 1.

NLU (structured prompting, nivel 2): El motor envía al LLM el input del usuario junto con el information state (identidad verificada, pólizas activas: hogar HG-2019-4421 y auto AU-2021-0087) y el esquema de interpretación (actos posibles, entidades del dominio). El LLM produce: acto principal `solicitar_cancelación` condicionado a acto secundario `consultar_conveniencia`, producto `seguro_hogar_HG-2019-4421`.

Decisión (motor simbólico): El motor recibe la interpretación estructurada. Evalúa: (1) regla `cancel_requires_confirmation` → marcar solicitud como pendiente de confirmación; (2) regla `conditional_request_handling` → la solicitud está condicionada a una consulta, priorizar la resolución de la consulta; (3) regla `bonus_info_required` → el usuario pregunta por conveniencia en contexto de cancelación, activar consulta de condiciones de bonificación. Information state actualizado: subtarea `consulta_bonificación` abierta, solicitud de cancelación en estado `condicionada_a_consulta`.

Ecosistema (RAG gobernado + capacidad agéntica gobernada): El motor determina que se necesitan dos datos: condiciones de bonificación (fuente: Condiciones Generales v2025.2, sección Bonificaciones) y datos de siniestros del cliente (fuente: sistema de gestión, API `siniestros_cliente`). Ambas consultas se ejecutan con trazas que registran fuente, consulta, y resultado.

NLG (structured prompting, nivel 2): El motor envía al LLM la acción determinada (informar sobre bonificación, datos del cliente, política de comunicación empática por tratarse de una cancelación potencial) y el

> LLM genera la respuesta.
> **Traza:** Cada paso registra: regla aplicada, datos consultados, fuentes accedidas, information state antes y después.

6.5 Trade-offs arquitectónicos

6.5.1 Trazabilidad vs. flexibilidad

El primero y más fundamental de los trade-offs es la tensión entre la trazabilidad de las decisiones y la flexibilidad del sistema para manejar situaciones imprevistas. En un extremo, un sistema puramente simbólico con reglas exhaustivas tiene trazabilidad perfecta pero no puede manejar ninguna situación que las reglas no anticipen. En el otro extremo, un LLM puro puede manejar cualquier situación pero sin trazabilidad alguna.

Cada incremento de integración gana flexibilidad y pierde simplicidad de verificación. El arte está en encontrar el balance correcto para cada dominio.

El pipeline gobernado de la Phase 1 ocupa una posición específica en este continuo: alta trazabilidad (todas las decisiones operativas son simbólicas) con flexibilidad limitada (las situaciones no cubiertas por las reglas requieren escalación o extensión del modelo). La adición de mecanismos de nivel 3 en la Phase 2 desplaza el balance: la flexibilidad aumenta (el reasoning loop puede manejar situaciones ambiguas, el constrained decoding permite generación más rica) pero la trazabilidad se hace más compleja (las interacciones entre componentes añaden pasos que deben registrarse).

Para ACAI, el balance se resuelve con un principio: *las decisiones con consecuencias operativas son siempre simbólicas y trazables; la flexibilidad neuronal opera dentro de límites definidos por el motor simbólico*. La Phase 2 no introduce decisiones neuronales autónomas; introduce mecanismos que enriquecen la interacción neuronal-simbólica sin transferir la autoridad de decisión al componente neuronal.

6.5.2 Latencia vs. profundidad de integración

El segundo trade-off es práctico y tiene impacto directo en la experiencia del usuario. Cada interacción entre componentes añade latencia al procesamiento de un turno. En el pipeline gobernado de la Phase 1, un turno requiere al menos dos llamadas al LLM (NLU y NLG) más el procesamiento simbólico. En la Phase 2 con reasoning loop, un turno puede requerir múltiples ciclos de interacción LLM-motor, cada uno con su latencia.

En un sistema conversacional, la latencia percibida por el usuario es un factor crítico de la experiencia. Estudios de diseño de interacción han

establecido que respuestas por encima de 2–3 segundos degradan significativamente la percepción de naturalidad del diálogo. El constrained decoding, por su parte, añade overhead computacional en cada paso de generación, aunque las implementaciones recientes como Outlines han reducido este overhead significativamente.

El trade-off de latencia no invalida los patrones de nivel 3 pero exige disciplina en su aplicación.

La resolución de este trade-off en ACAI es selectiva: los mecanismos de nivel 3 no se aplican a todos los turnos sino solo cuando el procesamiento de nivel 2 es insuficiente. El reasoning loop se activa cuando la interpretación inicial es ambigua, no para todos los inputs. El constrained decoding se aplica a las partes de la respuesta que requieren corrección formal, no a toda la generación. Esta aplicación selectiva mantiene la latencia promedio cercana a la del pipeline de la Phase 1, con aumentos puntuales cuando la complejidad del turno lo requiere.

6.5.3 Complejidad vs. verificabilidad

El tercer trade-off afecta directamente a la accountability y es, por tanto, especialmente relevante para ACAI. Los patrones de nivel 3 producen sistemas más capaces pero más complejos de verificar. En el pipeline lineal de la Phase 1, la verificación del sistema puede descomponerse: se verifican las reglas simbólicas por un lado y se evalúa la calidad del LLM por otro. En los patrones de nivel 3, las interacciones entre componentes crean caminos de ejecución adicionales que deben verificarse: ¿qué pasa si el reasoning loop no converge? ¿Qué pasa si el constrained decoding elimina todos los tokens salvo uno degenerado? ¿Cómo se verifica la consistencia de un sistema donde las restricciones simbólicas y las distribuciones neuronales interactúan?

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 12**. Las métricas de evaluación deben adaptarse para capturar la complejidad adicional de los patrones de nivel 3, incluyendo métricas de convergencia del reasoning loop y de calidad de la generación restringida.

La resolución en ACAI sigue el principio de la progresión deliberada (P3): la Phase 2 debe demostrar que cada mecanismo de nivel 3 añade valor medible que justifica la complejidad adicional. Si el reasoning loop mejora la calidad de la interpretación en un 15 % pero hace la verificación cuatro veces más compleja, la decisión de adoptarlo depende del balance concreto en el dominio de despliegue. La métrica central de la Phase 2 es precisamente esta: comparación cuantitativa entre niveles con beneficios y costes explícitos.

Conexiones con otras unidades

→ Prerequisito: UD 5 (Fundamentos neuro-simbólicos) para la taxonomía de Kautz y las fortalezas complementarias. **UD 1** (Mapa conceptual) para P1, P3, E1–E4.

⇄ Complementa: UD 3 (Ecosistema operativo) presentó los componentes E1–E4 y sus límites de gobernanza. UD6 muestra cómo los patrones arquitectónicos implementan la gobernanza de esos componentes.

Aplicación: UD 7 (Técnicas neuro-simbólicas para NLP) concreta los patrones de nivel 3 en técnicas específicas: constrained decoding, semantic parsing, structured generation. UD6 proporciona la arquitectura; UD7 proporciona las técnicas.

Aplicación: UD 9 (Formalismos) necesita saber qué patrones arquitectónicos usará el sistema para evaluar los formalismos candidatos: ¿es el formalismo compatible con el constrained decoding? ¿Se puede usar en el reasoning loop?

Aplicación: UD 10 (Formalización asistida) implementa directamente el pipeline de compilación y el reasoning loop descritos aquí.

Ejercicios

1. [Clasificación] Para cada uno de los siguientes sistemas, identifique qué patrón arquitectónico usa (o qué combinación de patrones), en qué nivel de Kautz opera, y cuál es la dirección de control: (a) un chatbot empresarial basado en GPT-4 con un system prompt extenso y NeMo Guardrails; (b) un sistema RASA con un modelo BERT para NLU, un gestor de diálogo basado en reglas, y templates para NLG; (c) un sistema donde un LLM genera SQL a partir de preguntas en lenguaje natural y un validador verifica que el SQL es sintáctica y semánticamente correcto antes de ejecutarlo; (d) un sistema donde las reglas de un chatbot se revisan mensualmente a partir de un análisis de conversaciones fallidas realizado con LLM.

2. [Diseño de interfaz] Diseñe la interfaz estructurada (nivel 2) entre el motor simbólico y el LLM para la tarea de NLU en el escenario de la aseguradora. Especifique: (a) qué información envía el motor al LLM (qué campos, qué tipos, qué restricciones), (b) qué información devuelve el LLM al motor (qué estructura tiene la interpretación), y (c) cómo se señala la incertidumbre cuando la

interpretación no es unívoca. Justifique la granularidad elegida: ¿por qué estos campos y no otros?

3. [Análisis de trade-offs] Considere un escenario donde el sistema ACAI debe procesar 500 conversaciones simultáneas, cada una con una latencia máxima de 3 segundos por turno. Analice: (a) cuántas llamadas al LLM requiere un turno en el pipeline de Phase 1 (nivel 2), (b) cuántas llamadas requiere un turno con reasoning loop (nivel 3) en el peor caso, (c) qué estrategias de optimización podrían reducir la latencia sin sacrificar la profundidad de integración.

4. [Reflexión crítica] El patrón "neural con guardrails simbólicos" es significativamente más fácil de implementar que el pipeline gobernado de ACAI y produce resultados aceptables en muchos escenarios. ¿En qué condiciones concretas sería preferible el patrón de guardrails al pipeline gobernado? ¿Hay dominios o tipos de conversación donde la accountability plena no justifica la complejidad arquitectónica adicional? Argumente su posición.

5. [Aplicación a ACAI] Diseñe un reasoning loop para la formalización asistida (design time): un experto de dominio describe en lenguaje natural las condiciones de renovación automática de una póliza de hogar. Especifique: (a) el prompt inicial al LLM con el esquema del formalismo, (b) al menos dos tipos de error que el validador simbólico podría detectar, (c) cómo el feedback del validador se estructura para que el LLM corrija los errores, y (d) las condiciones de terminación del loop (cuándo se acepta el resultado, cuándo se pide intervención humana).

Referencias y lecturas

Lectura de entrada

Yu, D., Yang, B., Liu, C., Sun, F. & Gonzalez, J. E. (2023). Neuro-Symbolic Approaches for Text-Based Reasoning: A Survey. *ArXiv preprint*. Survey actualizado que cubre los patrones de integración neuro-simbólica en el contexto específico de NLP y razonamiento textual. Proporciona una visión panorámica de las arquitecturas disponibles con análisis de trade-offs. Lectura de entrada que complementa bien la perspectiva más general de Garcez y Lamb (2023).

Lecturas esenciales

- Garcez, A. d'Avila, Besold, T. R., De Raedt, L., Földiák, P., Hitzler, P., Icard, T., Kühnberger, K.-U., Lamb, L. C., Miikkulainen, R. & Silver, D. L. (2019). Neural-Symbolic Computing: An Effective Methodology for Principled Integration of Machine Learning and Reasoning. *Journal of Applied Logics*, 6(4), 611–632. — El marco de diseño para la integración neuro-simbólica. Define dimensiones de diseño que esta unidad refina para ACAI.
- Willard, B. T. & Louf, R. (2023). Efficient Guided Generation for Large Language Models. *ArXiv preprint arXiv:2307.09702*. — Outlines: constrained decoding con gramáticas formales. Referencia técnica esencial para el patrón de nivel 3 más directamente implementable.
- Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J. & Wu, X. (2023). Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering*. — Integración de LLMs con bases de conocimiento formales. Directamente relevante para el pipeline gobernado con RAG gobernado de ACAI.
- Larsson, S. & Traum, D. (2000). Information state and dialogue management in the TRINDI dialogue move engine toolkit. *Natural Language Engineering*, 6(3–4), 323–340. — Ya citado en UD3, pero aquí relevante como fundamento del information state como pilar integrador de la arquitectura.

Lecturas complementarias

- Zheng, L., Yin, L., Xie, Z., Huang, J., Sun, C., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C. & Sheng, Y. (2023). Efficiently Programming Large Language Models using SGLang. *ArXiv preprint arXiv:2312.07104*. — Control de flujo integrado con generación restringida. Complementa a Outlines con un enfoque de programación.
- Rebedea, T., Dinu, R., Sreedhar, M., Parisien, C. & Cohen, J. (2023). NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. *EMNLP 2023 (demo)*. — El ejemplo industrial principal del patrón "neural con guardrails simbólicos".
- Inan, H. et al. (2023). Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *ArXiv preprint*. — Variante del patrón de guardrails donde el filtro es neuronal con taxonomía simbólica.
- Nye, M., Andreassen, A. J., Gur-Ari, G., Michalewski, H., Austin, J., Biber, D., Dohan, D., Lewkowycz, A., Bosma, M., Luan, D., Sutton, C. & Odena, A. (2021). Show Your Work: Scratchpads for Intermediate Computation with Language Models. *ArXiv preprint arXiv:2112.00114*. — Razonamiento con pasos intermedios como precursor del reasoning loop.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*. — El patrón ReAct como referencia del patrón "neural-in-the-loop" (contrapunto a ACAI).
- OpenAI (2024). Structured Outputs. *OpenAI API Documentation*. — Implementación industrial del structured prompting de nivel 2.

- De Raedt, L., Dumančić, S., Manhaeve, R. & Marra, G. (2020). From Statistical Relational to Neuro-Symbolic Artificial Intelligence. *ArXiv preprint arXiv:2003.08316*. — Perspectiva desde la programación lógica probabilística. Relevante para la fundamentación teórica de la integración.
- Donadello, I., Serafini, L. & Garcez, A. d'Avila (2017). Logic Tensor Networks for Semantic Image Interpretation. *IJCAI 2017*. — Ejemplo de integración profunda (nivel 3–4) en el dominio de la visión. Relevante como referencia de lo posible más allá de ACAI.
- Poesia, G., Polozov, O., Le, V., Tiwari, A., Soares, G., Meek, C. & Gulwani, S. (2022). Synchromesh: Reliable Code Generation from Pre-trained Language Models. *ICLR 2022*. — Constrained decoding aplicado a generación de código. Análogo al uso de ACAI para generación de representaciones formales.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«The bridge between language and logic is not a single span but a spectrum of crossings, each trading flexibility for guarantee.»

Eje B — El paradigma neuro-simbólico

Unidad 7 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje B — El paradigma neuro-simbólico

Prerequisitos: UD5, UD6 · Eje B — El paradigma neuro-simbólico

UNIDAD DIDÁCTICA 7

Técnicas neuro-simbólicas para el procesamiento del lenguaje natural

Contenidos de la unidad

7.1.1 El espectro de restricción 4
7.2.1 Semantic parsing: la tradición y su transformación 5
7.2.2 Structured extraction con LLMs 6
7.2.3 El espectro de restricción en la interpretación 8
Restricción gramatical: constrained decoding para NLU 8
Verificación post-generación: el reasoning loop para NLU 9
7.2.4 Dialogue state tracking como tarea neuro-simbólica 10
7.3.1 El problema de la fidelidad 11
7.3.2 Técnicas de generación fiel 11
Prompting con especificación de fidelidad 11
Template-guided generation 12
Constrained decoding para NLG 12
Verificación post-generación 13
7.3.3 Políticas de generación como restricciones formalizadas 13
7.4.1 La compilación como text-to-code 14
7.4.2 El pipeline de compilación 14
7.4.3 La triple restricción como desafío técnico 16
7.5.1 Síntesis del espectro 17
7.5.2 Restricción dinámica 18
7.6.1 Mapeo de técnicas a componentes 19
7.6.2 La UD7 como bisagra 19

¿Por qué necesito esto para ACAI?

La UD 5 explicó por qué la integración neuro-simbólica es necesaria. La UD 6 mostró con qué patrones arquitectónicos se implementa. Esta unidad responde la pregunta restante: *con qué técnicas concretas*. El investigador necesita un repertorio preciso de herramientas para cruzar la frontera entre lenguaje natural y representaciones formales — en ambas direcciones y en los distintos momentos del ciclo de vida del sistema. Sin estas técnicas, la arquitectura ACAI es un esquema; con ellas, es un plan implementable. Esta es la unidad más extensa del monográfico porque es la más directamente operativa: lo que aquí se presenta es lo que el investigador usará durante el sprint.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Formular el problema central de ACAI como un problema de cruce entre lenguaje natural y representaciones formales, con cuatro flujos diferenciados.
2. Seleccionar y aplicar técnicas de semantic parsing y structured extraction adecuadas para la interpretación del input en el formalismo ACAI.
3. Aplicar el espectro de restricción — desde prompting hasta constrained decoding con verificación — evaluando el balance entre flexibilidad y garantía en cada punto.
4. Diseñar un pipeline de generación fiel que garantice correspondencia entre las decisiones del motor y el texto generado.
5. Diseñar un pipeline de compilación (design time) que transforme descripciones en lenguaje natural en representaciones formales verificadas.
6. Evaluar las técnicas contra la triple restricción (legibilidad, procesabilidad, generabilidad) y sus tensiones.

7.1 El problema del puente

Sede principal de este concepto.

Toda la maquinaria técnica de ACAI puede entenderse como un conjunto de soluciones a un problema único: cómo cruzar la frontera entre dos mundos con propiedades radicalmente diferentes. El lenguaje natural es inherentemente ambiguo — una misma frase puede tener múltiples interpretaciones legítimas. Es contextual — el significado

El problema técnico central de ACAI es cruzar la frontera entre lenguaje natural y representaciones formales — en ambas direcciones y en distintos momentos del ciclo de vida.

depende de la historia del diálogo, del conocimiento compartido, de las expectativas pragmáticas. Es gradual — los conceptos tienen fronteras difusas, las implicaturas son más o menos fuertes, la certeza del hablante es una cuestión de grado. Y es creativo — un hablante puede expresar la misma idea de un número impredecible de formas.

Las representaciones formales, por el contrario, son precisas — cada símbolo tiene una semántica definida que no depende de la interpretación. Son descontextualizadas — una regla significa lo mismo independientemente del turno en que se invoque. Son discretas — un estado se satisface o no, una condición es verdadera o falsa. Y son cerradas — solo se pueden expresar los conceptos previstos por el vocabulario y la gramática del formalismo.

Cruzar esta frontera implica una pérdida inevitable en ambas direcciones. De lenguaje natural a formal, se pierde la riqueza pragmática que el formalismo no captura: el matiz emocional, la fuerza ilocutiva exacta, las implicaturas conversacionales sutiles. De formal a lenguaje natural, se pierde la precisión: el texto generado es siempre una aproximación del significado formal, susceptible de malinterpretación. La calidad de un sistema neuro-simbólico se mide en buena medida por cuánto se pierde en cada cruce.

Para ACAI, este cruce se produce en cuatro flujos diferenciados que organizan toda esta unidad. El primero es la *interpretación* (NL → Formal en runtime): mapear lo que dice el usuario a términos del modelo formal. El segundo es la *generación* (Formal → NL en runtime): expresar las decisiones del motor en lenguaje natural fiel. El tercero es la *compilación* (NL → Formal en design time): construir el modelo formal a partir de descripciones en lenguaje natural. El cuarto es el *aprendizaje* (Formal → análisis → Formal en learning time): analizar las trazas de operación para proponer extensiones del modelo.

7.1.1 El espectro de restricción

Transversal a los cuatro flujos, un eje organiza las técnicas según el nivel de garantía formal que ofrecen. Este **espectro de restricción** va desde las técnicas más flexibles y menos garantizadas hasta las más rígidas y más garantizadas.

En un extremo, la *restricción blanda* (prompting): el LLM recibe instrucciones en lenguaje natural sobre qué producir y en qué formato. La calidad depende de que el LLM siga las instrucciones, lo que ocurre con alta probabilidad pero sin garantía.

En el segundo nivel, la *restricción estructural* (schemas): el LLM debe producir su salida en un formato estructurado que se ajusta a un esquema predefinido (JSON Schema, function calling). La corrección sin-

táctica del formato está garantizada por la API del modelo; la corrección semántica del contenido no.

En el tercer nivel, la *restricción gramatical* (constrained decoding): la generación del LLM se fuerza, token a token, a producir secuencias compatibles con una gramática formal. La corrección sintáctica respecto a la gramática está garantizada por construcción; la corrección semántica depende de que la gramática capture las restricciones relevantes.

En el cuarto nivel, la *verificación post-generación* (reasoning loop): la salida del LLM se evalúa contra el modelo formal completo. Si la verificación falla, el LLM regenera condicionado por el feedback del verificador. La corrección se busca iterativamente con garantía de detección de errores (aunque no de corrección en tiempo acotado).

El espectro de restricción no es un ranking: más restricción no es siempre mejor. Es un mapa de trade-offs.

Cada nivel del espectro ofrece un balance diferente entre flexibilidad (la capacidad del LLM para manejar situaciones imprevistas), garantía (la probabilidad de que el resultado sea formalmente correcto), latencia (el tiempo de procesamiento), y complejidad (el esfuerzo de implementación). La elección del nivel adecuado no es uniforme: depende de la consecuencia de un error en cada componente. Para ACAI, el motor simbólico puede determinar dinámicamente el nivel de restricción según el contexto del diálogo y la criticidad de la operación — una forma de gobernanza que conecta el espectro de restricción con el information state.

7.2 De lenguaje a estructura: el cruce NL→Formal en runtime

Sede principal de este concepto.

La interpretación del input del usuario en términos del modelo formal es la tarea más crítica del pipeline de ACAI. Si la interpretación es incorrecta, la decisión del motor será correcta respecto a una premisa falsa — un tipo de error particularmente difícil de detectar y particularmente costoso en términos de accountability. Las técnicas de esta sección abordan cómo minimizar el error de interpretación con garantías crecientes.

7.2.1 Semantic parsing: la tradición y su transformación

Kamath & Das (2019) — survey de semantic parsing, de CCG a modelos neuronales.

El **semantic parsing** — la tarea de mapear lenguaje natural a representaciones formales ejecutables — es la tarea de NLP más directamente relevante para ACAI. Su historia traza una evolución que recapitula, en miniatura, la tensión fundacional analizada en la **UD 5**.

Los primeros parsers semánticos operaban sobre gramáticas forma-

les diseñadas manualmente. Zettlemoyer y Collins (2005, 2007), en trabajos fundacionales, usaron gramáticas categóricas combinatorias (CCG) para aprender mapeos de lenguaje natural a expresiones en lambda calculus. El enfoque producía representaciones formalmente correctas por construcción — la gramática garantizaba la buena formación — pero era frágil ante la variabilidad lingüística y requería gramáticas diseñadas para cada dominio.

La siguiente generación adoptó modelos neuronales de secuencia a secuencia. Jia y Liang (2016) y Dong y Lapata (2016) demostraron que redes neuronales podían aprender el mapeo NL → Formal directamente de pares de entrenamiento, sin gramáticas explícitas. El enfoque ganaba robustez ante la variabilidad (la red generaliza a formulaciones no vistas) pero perdía la garantía de corrección formal: la red podía generar expresiones sintácticamente malformadas.

Zettlemoyer & Collins (2005): semantic parsing con CCG — corrección garantizada, fragilidad ante variabilidad.

La generación actual usa LLMs de propósito general. Shin et al. (2021), con PICARD, demostraron que un LLM condicionado con constrained decoding produce SQL a partir de lenguaje natural con calidad superior a los modelos especializados, y con la garantía de que el SQL generado es siempre sintácticamente válido. Rajkumar et al. (2022) extendieron este resultado a otros formalismos. La idea central de PICARD — usar un LLM potente y restringir su generación para que satisfaga la gramática del formalismo de destino — es directamente el patrón que ACAI necesita.

📖 Shin et al. (2021) — PICARD: LLM con constrained decoding para semantic parsing.

Para ACAI, el semantic parsing tiene una especificidad que lo distingue de los benchmarks académicos. El formalismo de destino no es SQL (cuya gramática es bien conocida y estable) ni lambda calculus (cuya semántica es canónica), sino el lenguaje de representación propio de ACAI, diseñado bajo la triple restricción. Esto significa que el LLM no puede apoyarse en el conocimiento extenso de SQL que posee de su entrenamiento sino que debe aprender el formalismo ACAI mediante in-context learning, few-shot examples, y fine-tuning selectivo. Las técnicas de adaptación a formalismos nuevos son, por tanto, especialmente relevantes.

7.2.2 Structured extraction con LLMs

La técnica más directamente aplicable en la Phase 1 de ACAI es la **structured extraction**: usar el LLM para extraer información estructurada del input del usuario en un formato predefinido. A diferencia del semantic parsing completo (que produce una representación formal ejecutable), la structured extraction produce una estructura intermedia que el motor simbólico procesa para derivar la representación formal completa.

La structured extraction de ACAI no es una extracción de intents y slots como la que la **UD 2** analizó y criticó. Opera sobre una estructura significativamente más rica que captura múltiples dimensiones del acto comunicativo.

La structured extraction de ACAI captura cinco componentes: actos comunicativos, entidades con mapeo ontológico, señales pragmáticas, relaciones entre actos, y nivel de certeza.

El primer componente es el *acto comunicativo*, que va más allá del intent simple. Un acto comunicativo tiene tipo (solicitar, informar, confirmar, rechazar, preguntar), objeto (la entidad sobre la que opera), condiciones (bajo qué circunstancias), y fuerza ilocutiva (directa, indirecta, tentativa). «Quiero cancelar mi seguro» es un acto directo de tipo solicitar con objeto cancelación. «No sé si me conviene seguir con el seguro» es un acto indirecto de tipo explorar-cancelación con fuerza tentativa. La diferencia tiene consecuencias para el motor: el primero activa el protocolo de cancelación; el segundo activa un protocolo de consulta.

El segundo componente es la *extracción de entidades con mapeo ontológico*. No basta con reconocer que «seguro de hogar» es una entidad de tipo producto; es necesario mapearla a la entidad formal correspondiente en el modelo de conocimiento: el producto específico `seguro_hogar_HG-2019-4421` del cliente identificado. Este mapeo requiere que el LLM tenga acceso al contexto del information state (qué pólizas tiene el cliente) como parte de su prompt estructurado.

El tercer componente son las *señales pragmáticas*: indicadores de grounding («entiendo», «vale»), señales de reparación («no, quiero decir...»), marcadores de incertidumbre («creo que...», «no estoy seguro»), y actos de gestión del diálogo («espere un momento», «¿puede repetir?»). Estas señales corresponden a las dimensiones 2–5 de la conversación analizadas en la **UD 2** y son las que los sistemas basados en intents ignoran sistemáticamente.

El cuarto componente son las *relaciones entre actos* cuando el input contiene múltiples actos. «Quiero cancelar, pero antes dígame si pierdo la bonificación» contiene dos actos (solicitar cancelación, solicitar información sobre bonificación) con una relación de precedencia (el segundo debe resolverse antes de ejecutar el primero). Capturar esta estructura requiere que la extracción produzca no una lista plana de actos sino un grafo con relaciones.

El quinto componente es el *nivel de certeza* de la extracción: una señal que indica al motor cuán confiable es la interpretación. Una extracción con alta certeza puede procesarse directamente; una con baja certeza puede activar el reasoning loop o una solicitud de clarificación.

La implementación práctica de la structured extraction usa las técnicas de in-context learning documentadas por Brown et al. (2020) y analizadas en profundidad por Min et al. (2022). El prompt del LLM incluye: el esquema de la estructura de salida (JSON Schema), ejemplos

anotados de inputs y sus extracciones (few-shot examples), el estado relevante del diálogo (information state parcial), y las entidades del dominio disponibles. Los trabajos recientes sobre la calidad del in-context learning (Wei et al., 2023; Dong et al., 2024) muestran que la selección de los ejemplos y su ordenación tienen impacto significativo en la calidad de la extracción.

📖 Min et al. (2022) — análisis de qué aprenden los LLMs del in-context learning.

7.2.3 El espectro de restricción en la interpretación

Las técnicas de structured extraction descritas operan en los dos primeros niveles del espectro de restricción (prompting y schemas). Para situaciones que requieren mayor garantía, ACAI dispone de técnicas en los niveles superiores del espectro.

Restricción gramatical: constrained decoding para NLU

El constrained decoding, descrito arquitectónicamente en la **UD 6**, se implementa técnicamente mediante un autómata que se ejecuta en paralelo con la generación del LLM. En cada paso de la generación autoregresiva, el autómata determina qué tokens son compatibles con la gramática del formalismo de destino dado el prefijo generado hasta ese punto. Los tokens incompatibles reciben probabilidad cero en la distribución del LLM, lo que garantiza que la secuencia completa satisface la gramática.

📖 Willard & Louf (2023) — Outlines: la referencia técnica para constrained generation con gramáticas.

Willard y Louf (2023), en su implementación Outlines, demostraron que este mecanismo es práctico y eficiente. Su enfoque convierte la gramática (que puede ser una gramática context-free, una expresión regular, o un JSON Schema) en un índice de tokens permitidos para cada estado del autómata, lo que reduce el overhead a una consulta en tabla por paso de generación. El impacto en la latencia es modesto (típicamente un 10–20 % de overhead) y en la calidad es frecuentemente positivo: la eliminación de tokens degenerados mejora la coherencia de la generación.

Para ACAI, el constrained decoding en NLU garantiza que la interpretación producida por el LLM es siempre una expresión sintácticamente válida del formalismo de actos comunicativos. Si el formalismo define que un acto tiene tipo, objeto, condiciones y fuerza, el constrained decoding garantiza que la salida del LLM contendrá exactamente esos campos con valores del vocabulario permitido. La corrección sintáctica queda garantizada por construcción; la corrección semántica (¿es este el acto correcto?) sigue dependiendo de la capacidad del LLM.

Scholak et al. (2021) aplicaron esta idea específicamente al semantic parsing con su sistema PICARD, que demostró ganancias significativas

en la tarea de text-to-SQL al combinar un LLM potente (T5) con un parser incremental que verifica la validez del SQL en cada paso de generación. La extensión de este enfoque al formalismo ACAI requiere definir la gramática del formalismo como un autómata compatible con las implementaciones disponibles — un requisito que condiciona el diseño del lenguaje de representación y que la **UD 9** analizará en detalle.

Scholak et al. (2021): PICARD — constrained decoding específico para semantic parsing.

Verificación post-generación: el reasoning loop para NLU

El cuarto nivel del espectro añade una verificación semántica que el constrained decoding no puede proporcionar. Una interpretación puede ser sintácticamente válida (es una expresión bien formada del formalismo) y semánticamente incorrecta (atribuye al usuario un acto que no realizó, o mapea una entidad incorrectamente).

La verificación semántica opera evaluando la interpretación contra el information state y las restricciones del modelo formal. Si el LLM interpreta que el usuario solicita cancelar una póliza que no existe en su cartera, la verificación detecta la inconsistencia. Si interpreta que el usuario confirma una operación que nunca se le propuso, la verificación detecta la incoherencia temporal. En cada caso, el feedback estructurado al LLM especifica la naturaleza del error y las restricciones que la nueva interpretación debe satisfacer.

Espectro de restricción en acción

Input del usuario: «A ver, lo que quiero es que no me cobren más pero sin perder lo que llevo acumulado, ¿se puede?»
Nivel 1 (prompting): El LLM interpreta libremente → acto: `solicitar_información` sobre opciones de reducción de coste. Plausible pero impreciso: no captura la condición sobre lo acumulado.
Nivel 2 (schema): El LLM produce un JSON con campos obligatorios → acto_principal: `solicitar_cambio`, condición: `preservar_bonificación_acumulada`, objeto: `coste_prima`. Mejor: la estructura fuerza a explicitar la condición.
Nivel 3 (constrained decoding): La gramática del formalismo fuerza a seleccionar entre actos y condiciones del vocabulario formal → acto: `explorar_alternativa`(`reducción_prima`), restricción: `mantener`(`bonificación_no_siniestralidad`), modalidad: `consulta_viabilidad`. Sintácticamente garantizado.
Nivel 4 (reasoning loop): El motor evalúa: el cliente tiene bonificación por 3 años sin siniestros. Las opciones compatibles son: suspensión temporal (mantiene bonificación), cambio de cobertura (mantiene bonificación), cancelación (pierde bonificación). El feedback al LLM incluye estas opciones concretas → el LLM refina: el usuario explora alternativas a la cancelación que preserven su historial, compatible con

```
suspensión_temporal o cambio_cobertura.
```

7.2.4 Dialogue state tracking como tarea neuro-simbólica

El **dialogue state tracking** (DST) — la tarea de mantener una representación actualizada del estado del diálogo — conecta la interpretación turno a turno con el mantenimiento del information state a lo largo de la conversación. En los sistemas de diálogo clásicos, el DST era una tarea separada con modelos especializados. En ACAI, es una función distribuida entre el LLM (que proporciona la interpretación de cada turno) y el motor simbólico (que actualiza el information state según las reglas formales).

La tradición de DST es extensa. Los DSTC challenges (Henderson et al., 2014) establecieron benchmarks que impulsaron el desarrollo de modelos progresivamente más sofisticados: de modelos estadísticos con features manuales a modelos neuronales especializados (Zhong et al., 2018; Wu et al., 2019), y más recientemente a LLMs con in-context learning (Feng et al., 2023; Hudecek y Dusek, 2023). La tendencia muestra una mejora constante en la capacidad de tracking pero una persistente dificultad con los cambios de estado complejos: correcciones del usuario, cambios de opinión, retracciones de compromisos.

Feng et al. (2023) — DST con LLMs: estado del arte y limitaciones.

Para ACAI, las lecciones del DST son directamente operativas. La primera es que la representación del estado debe ser suficientemente rica para capturar no solo los valores actuales de las variables del diálogo sino también su *historial de cambios*: un dato que fue proporcionado, luego corregido, tiene un estatus diferente al de un dato proporcionado una sola vez. El information state de ACAI incorpora esta dimensión temporal.

La segunda lección es que el DST con LLMs funciona significativamente mejor cuando el LLM recibe el estado anterior como parte de su contexto, en lugar de inferirlo de toda la historia del diálogo. Esto justifica la decisión arquitectónica de ACAI de mantener un information state explícito que se proporciona al LLM en cada turno como contexto estructurado.

La tercera lección es que los errores de DST se acumulan: un error en el turno 3 persiste y se amplifica en los turnos siguientes si no se corrige. En ACAI, el motor simbólico funciona como mecanismo de corrección: al evaluar la interpretación del LLM contra las reglas formales y el information state, puede detectar inconsistencias que un DST puramente neuronal propagaría.

7.3 De estructura a lenguaje: el cruce Formal→NL en runtime

Sede principal de este concepto.

7.3.1 El problema de la fidelidad

La generación de lenguaje natural a partir de representaciones formales en ACAI tiene un problema específico que no es el que los LLMs resuelven bien (la naturalidad) sino uno que no suelen abordar: la **fidelidad**. La fidelidad es la correspondencia estricta entre lo que el motor simbólico determinó y lo que el texto generado expresa. Una respuesta infiel puede ser perfectamente natural y completamente incorrecta desde la perspectiva de la accountability.

El problema de la generación en ACAI no es la naturalidad — es la fidelidad: que el texto exprese exactamente lo que el motor determinó, sin más y sin menos.

La infidelidad puede manifestarse de tres formas. La *adición*: el LLM añade información que el motor no autorizó. Si el motor determina informar sobre la bonificación y el LLM añade «y probablemente le convenga esperar a fin de año», ha producido un consejo financiero no autorizado. La *omisión*: el LLM omite información que el motor requiere comunicar. Si el motor determina informar sobre la penalización y la bonificación, y el LLM solo menciona la bonificación, ha omitido información legalmente relevante. La *distorsión*: el LLM modifica las condiciones formales. Si la bonificación es válida durante 24 meses y el LLM genera «la bonificación se mantiene durante un tiempo», ha introducido una imprecisión con consecuencias contractuales.

Maynez et al. (2020) propusieron una taxonomía de infidelidades en la generación abstractiva que es directamente aplicable: infidelidades intrínsecas (que contradicen la fuente) y extrínsecas (que añaden información no presente en la fuente). Honovich et al. (2022) desarrollaron métodos automáticos de detección de inconsistencias factuales usando modelos de *natural language inference* (NLI). Estos métodos son adaptables al contexto de ACAI, donde la "fuente" no es un documento sino la especificación formal del motor.

Maynez et al. (2020) — taxonomía de infidelidades en la generación de texto.

7.3.2 Técnicas de generación fiel

El espectro de restricción se aplica a la generación con las mismas cuatro gradaciones que a la interpretación, cada una con sus propiedades específicas.

Prompting con especificación de fidelidad

En el nivel más básico del espectro, el LLM recibe la decisión del motor como parte del prompt con instrucciones explícitas de fidelidad:

«Genera una respuesta que comunique exactamente la siguiente información al usuario: [especificación del motor]. No añadas información que no esté en la especificación. Incluye todos los datos numéricos tal como aparecen. Si la especificación incluye condiciones, comunícalas completamente.» Esta técnica funciona razonablemente bien en la mayoría de los casos pero sin garantía formal. Liu et al. (2023) documentaron que la adherencia a las instrucciones del prompt se degrada con la longitud del contexto y la complejidad de las instrucciones.

Template-guided generation

Una técnica intermedia que es especialmente prometedora para ACAI es la **template-guided generation**: el motor proporciona un template parcial donde los elementos críticos (datos numéricos, condiciones formales, compromisos) están fijados como texto literal, y el LLM completa los espacios intermedios con transiciones naturales, expresiones de cortesía, y adaptación al tono del diálogo.

La template-guided generation combina la fidelidad de los templates con la naturalidad de la generación libre.

Por ejemplo, el motor genera el template: «Su bonificación por ausencia de siniestros asciende a {monto} y se mantiene durante {período}. [TRANSICIÓN] Si cancela antes de {fecha}, se aplicará una penalización de {penalización}. [CIERRE]». Los valores entre llaves son literales del modelo formal; las etiquetas [TRANSICIÓN] y [CIERRE] son espacios que el LLM completa con texto natural adaptado al contexto.

Esta técnica ofrece fidelidad estricta para los elementos críticos (los datos literales no pueden ser distorsionados porque no los genera el LLM) con naturalidad para los elementos expresivos (el LLM aporta la fluidez que un template rígido no tendría). El trade-off es que requiere que el motor produzca templates, lo que añade complejidad al modelo formal.

Constrained decoding para NLG

El constrained decoding puede aplicarse a la generación para garantizar la inclusión de elementos obligatorios y la exclusión de patrones prohibidos. Los trabajos sobre *lexically constrained decoding* son directamente relevantes. Hokamp y Liu (2017) propusieron Grid Beam Search, que modifica la búsqueda beam para garantizar que ciertos tokens o secuencias aparecen en la salida. Post y Vilar (2018) propusieron Dynamic Beam Allocation para hacer este proceso más eficiente. Lu et al. (2022) extendieron estas técnicas con NeuroLogic Decoding, que permite especificar restricciones complejas (conjunciones, disyunciones, negaciones) sobre los tokens de la salida.

Para ACAI, las restricciones de generación pueden derivarse del motor simbólico: los tokens obligatorios (el número de la póliza, el monto

📖 Lu et al. (2022) — NeuroLogic Decoding: restricciones complejas sobre la generación.

de la bonificación, el período), los patrones prohibidos (asesoramiento financiero, promesas no autorizadas, datos personales de otros clientes), y las restricciones de estructura (la penalización debe mencionarse *después* de las opciones alternativas, no antes, según la política de retención).

Verificación post-generación

El nivel más alto del espectro añade una verificación de la respuesta generada contra la especificación del motor. Esta verificación puede usar técnicas de NLI (*Natural Language Inference*) para comprobar que cada proposición de la respuesta es consecuencia de la especificación (Laban et al., 2022), o puede usar un segundo LLM como verificador que evalúa la fidelidad con una rúbrica formal (Tang et al., 2024). En el contexto de ACAI, la verificación puede ser parcialmente simbólica: extraer los datos numéricos de la respuesta generada y compararlos con los datos del motor, verificar que las condiciones mencionadas corresponden a las condiciones formales, comprobar que no se han añadido compromisos no autorizados.

Tang et al. (2024): verificación de fidelidad con rúbricas formales — adaptable al contexto ACAI.

7.3.3 Políticas de generación como restricciones formalizadas

> **Políticas de generación en el modelo formal**
>
> Las políticas de generación — tono, nivel de detalle, estrategia comunicativa, formato — pueden formalizarse como parámetros que el motor simbólico determina y que condicionan la generación del LLM. Por ejemplo, el motor puede determinar que el turno actual requiere *tono empático* (porque el contexto es una cancelación), *nivel de detalle alto* (porque hay datos numéricos que comunicar), y *estrategia de retención* (porque la política comercial indica ofrecer alternativas antes de ejecutar la cancelación).
> Estos parámetros se inyectan en el prompt del LLM como parte de la especificación de generación, junto con la decisión operativa. La formalización de las políticas en el modelo de conocimiento permite que las decisiones de *cómo* comunicar sean tan auditables como las decisiones de *qué* comunicar: la traza registra no solo la información proporcionada sino la política de comunicación aplicada.

7.4 De lenguaje a formalismo: el cruce NL→Formal en design time

Sede principal de este concepto.

7.4.1 La compilación como text-to-code

La transformación de descripciones en lenguaje natural en representaciones formales — lo que ACAI denomina *compilación* — es estructuralmente análoga al problema de text-to-code que ha sido extensamente estudiado.

La compilación de ACAI es estructuralmente análoga al text-to-code, pero con un formalismo de destino nuevo y con requisitos de verificación específicos del dominio.

Chen et al. (2021) — Codex: LLM para generación de código.

Chen et al. (2021), con Codex, demostraron que un LLM entrenado sobre código puede generar programas correctos a partir de docstrings en lenguaje natural con una precisión notable: pass@1 del 28.8 % y pass@100 del 72.3 % en el benchmark HumanEval. Estos números tienen dos lecturas complementarias. Optimista: en el 72 % de los casos, al menos una de las muestras es correcta, lo que es útil en un pipeline con verificación. Pesimista: en el 71 % de los casos, la primera muestra no es correcta, lo que hace imprescindible un mecanismo de verificación.

Trabajos posteriores han mejorado significativamente estos números. Li et al. (2022) con AlphaCode y Rozière et al. (2024) con Code Llama demostraron que modelos especializados en código alcanzan pass@1 superiores al 50 % en benchmarks estándar. Más relevante para ACAI, los trabajos sobre generación de código en lenguajes de dominio específico (DSLs) — SQL, SPARQL, expresiones regulares — muestran que los LLMs pueden adaptarse a formalismos nuevos mediante few-shot learning, aunque la precisión depende de la complejidad y regularidad del formalismo.

La analogía text-to-code es precisa pero con un límite: el formalismo ACAI no está en los datos de entrenamiento del LLM.

La analogía tiene un límite importante: los LLMs conocen Python, SQL y JavaScript por su presencia masiva en los datos de entrenamiento. El formalismo ACAI es nuevo y no está en esos datos. Esto significa que la compilación de ACAI depende enteramente del in-context learning: la capacidad del LLM para aprender el formalismo a partir de la descripción del esquema y los ejemplos proporcionados en el prompt. Esta dependencia refuerza la importancia de la tercera restricción (generabilidad): el formalismo debe diseñarse para ser aprendible por el LLM a partir de pocos ejemplos.

7.4.2 El pipeline de compilación

Phase 0 / Phase 1
La compilación es la operación central de la formalización asistida.

El pipeline de compilación opera en cinco pasos, cada uno con técnicas específicas.

Paso 1: Elicitación estructurada. El LLM asiste al experto de dominio mediante un diálogo de elicitación que sigue un esquema predefinido. El esquema determina qué información es necesaria para producir una

formalización completa: descripción de la regla, condiciones de aplicación, excepciones, relaciones con otras reglas, datos requeridos, acciones resultantes. El LLM formula preguntas de seguimiento cuando la descripción del experto es incompleta o ambigua. La elicitación no es un formulario rígido sino un diálogo guiado donde el LLM usa su capacidad lingüística para interpretar descripciones informales y traducirlas a los campos del esquema.

Paso 2: Generación de primera formalización. El LLM genera una representación formal a partir de la descripción elicitada, condicionado por tres elementos: el esquema del formalismo ACAI (la gramática y las restricciones de buena formación), ejemplos de reglas existentes en el modelo (few-shot examples que muestran el estilo y las convenciones del formalismo), y el modelo de conocimiento actual (para garantizar compatibilidad con las reglas existentes). La generación puede usar structured output (nivel 2 del espectro) o constrained decoding (nivel 3) según la complejidad de la regla.

Paso 3: Validación simbólica. Un validador comprueba la representación generada en tres dimensiones. La *corrección sintáctica*: ¿es una expresión válida de la gramática del formalismo? La *consistencia con el modelo*: ¿contradice alguna regla existente? ¿Usa entidades y relaciones definidas en la ontología? La *completitud local*: ¿faltan condiciones que las reglas relacionadas tienen? ¿Están todos los campos obligatorios presentes?

Paso 4: Corrección iterativa. Si la validación detecta errores, el feedback se estructura y se devuelve al LLM para regeneración. El feedback es específico: no dice «la regla es incorrecta» sino «la condición `antigüedad > 30` referencia la entidad `antigüedad` que no está definida en la ontología; las entidades disponibles son `antigüedad_póliza`, `antigüedad_cliente`, `antigüedad_siniestro`». El LLM regenera condicionado por el error concreto y la corrección esperada. El ciclo continúa hasta que la validación no detecta errores o se alcanza un límite de iteraciones.

Paso 5: Revisión humana. La formalización validada se presenta al experto de dominio para su aprobación. La presentación incluye la representación formal y una *verbalización* generada por el LLM que traduce la regla formal de vuelta a lenguaje natural, permitiendo al experto verificar que la formalización captura su intención. Si el experto detecta discrepancias, el ciclo vuelve al paso 2 con las correcciones como input adicional.

Pipeline de compilación

Descripción del experto: «Cuando un cliente con más de dos años de antigüedad cancela, si no ha tenido siniestros, la bonificación se congela durante dos años por si quiere volver.»
Paso 1 (elicitación): LLM pregunta: ¿la congelación aplica a todos los productos o solo a hogar? ¿Qué pasa si el cliente vuelve después de los dos años? ¿La ausencia de siniestros se refiere a toda la vida de la póliza o a un período? Experto clarifica: solo hogar, después de dos años se pierde, se refiere a los últimos 3 años.
Paso 2 (generación): LLM genera la regla formal en el formalismo ACAI con las condiciones, restricciones de producto, período de referencia para siniestros, período de congelación, y acción resultante.
Paso 3 (validación): El validador detecta: la regla usa `producto = hogar` pero la ontología define `producto = seguro_hogar`. Error de vocabulario.
Paso 4 (corrección): Feedback al LLM: «Vocabulario incorrecto: `hogar` → `seguro_hogar`». LLM corrige. Segunda validación: correcta.
Paso 5 (revisión): Verbalización: «Cuando un cliente con más de 2 años de antigüedad cancela su seguro de hogar, y no ha tenido siniestros en los últimos 3 años, su bonificación por no-siniestralidad se congela durante 24 meses. Si contrata una nueva póliza de seguro de hogar dentro de ese período, recupera la bonificación acumulada. Pasados los 24 meses, la bonificación se pierde.» Experto aprueba.

7.4.3 La triple restricción como desafío técnico

La UD 5 introdujo la triple restricción (I1) como especificidad conceptual de ACAI. Aquí se concreta como un conjunto de requisitos técnicos que condicionan tanto el diseño del formalismo como las técnicas de compilación.

La **restricción de legibilidad** exige que un experto de dominio no técnico pueda leer y comprender las reglas formalizadas. Esto condiciona la sintaxis del formalismo: no puede usar notación lógica abstracta (el experto no lee $\forall x.P(x) \rightarrow Q(x)$) ni convenciones de programación opacas (el experto no lee `lambda x: x.policy.age > 30`). Los formalismos legibles tienden a ser verbosos, cercanos al lenguaje natural, con palabras clave inteligibles.

Las tres restricciones están en tensión: la legibilidad favorece formalismos verbosos, la procesabilidad favorece formalismos formales, y la generabilidad favorece formalismos regulares.

La **restricción de procesabilidad** exige que el motor simbólico pueda ejecutar las reglas de forma determinista y eficiente. Esto condiciona la semántica del formalismo: debe tener semántica operacional definida, sin ambigüedades de interpretación. Los formalismos procesables tienden a ser compactos, precisos, con una gramática formal estricta.

La **restricción de generabilidad** exige que el LLM pueda producir las reglas con fiabilidad a partir de descripciones en lenguaje natural.

Esto condiciona la regularidad del formalismo: las reglas deben seguir patrones predecibles que el LLM pueda aprender con pocos ejemplos. Los formalismos generables tienden a ser regulares, con estructuras repetitivas y un vocabulario controlado.

Las tres restricciones están en tensión porque favorecen propiedades diferentes del formalismo. Un formalismo óptimo para la legibilidad (verbose, cercano al NL) puede ser difícil de procesar (ambiguo) y de generar (irregular). Un formalismo óptimo para la procesabilidad (compacto, formal) puede ser difícil de leer (opaco) y de generar (el LLM no lo ha visto en sus datos de entrenamiento). Un formalismo óptimo para la generabilidad (regular, predecible) puede ser artificialmente rígido para el lector humano y poco expresivo para el motor.

Referencia contextual. Este concepto se desarrolla en profundidad en **UD 9** y **UD 11**. La evaluación sistemática de formalismos contra la triple restricción y el análisis detallado de sus tensiones y resoluciones se desarrollan en profundidad en el Eje C.

La resolución de estas tensiones no es un compromiso único sino un espacio de diseño que el sprint debe explorar. Los experimentos de la Phase 0 (directamente apoyados por la **UD 10**) consisten precisamente en probar diferentes puntos de este espacio: generar reglas con varios formalismos candidatos, evaluar la legibilidad con expertos, la procesabilidad con prototipos de motor, y la generabilidad con métricas de corrección del LLM.

> **La triple restricción como programa experimental**
>
> La triple restricción convierte el diseño del formalismo en un problema experimental, no solo teórico. No se puede determinar a priori qué formalismo satisface mejor las tres restricciones porque la generabilidad depende de propiedades empíricas del LLM que solo se conocen experimentando. Esto es una diferencia fundamental con el diseño clásico de lenguajes formales, donde los criterios de evaluación (expresividad, decidibilidad, complejidad) son formales. ACAI añade un criterio empírico — la generabilidad — que requiere experimentación.

7.5 El espectro de restricción como principio unificador

7.5.1 Síntesis del espectro

El espectro de restricción recorre toda esta unidad como eje transversal. Cada flujo (NLU, NLG, compilación) tiene sus cuatro niveles de restricción, y en cada flujo la elección del nivel produce un balance

diferente entre flexibilidad y garantía. La síntesis muestra que todas las técnicas presentadas son instancias de un principio común: *el grado de control simbólico sobre la computación neuronal es un parámetro de diseño continuo, no una decisión binaria*.

Esta formulación generaliza la taxonomía de Kautz dentro de cada nivel. Un sistema de nivel 2 puede operar con restricciones blandas (prompting simple) o con restricciones estructurales (schemas), y la diferencia en propiedades de accountability es significativa. Un sistema de nivel 3 puede operar con constrained decoding solo o con constrained decoding más reasoning loop, y de nuevo la diferencia importa. El espectro de restricción es un eje de variación *dentro* de cada nivel de la taxonomía de Kautz.

7.5.2 Restricción dinámica

Restricción dinámica

La **restricción dinámica** es el principio según el cual el nivel de restricción aplicado en cada interacción neuronal-simbólica se determina en función del contexto del diálogo y la criticidad de la operación, en lugar de ser fijo para todo el sistema.

La restricción dinámica es una forma de gobernanza sofisticada donde el motor simbólico no solo determina *qué* debe hacer el LLM sino *con cuánta libertad* puede hacerlo. En un turno donde el usuario pide información general sobre sus pólizas, el motor puede aplicar restricción blanda (prompting): la consecuencia de un error menor es baja. En un turno donde el sistema va a ejecutar una cancelación irreversible, el motor aplica restricción máxima (constrained decoding + reasoning loop + verificación post-generación): la consecuencia de un error es alta.

La restricción dinámica conecta el espectro de restricción con el information state y las políticas de riesgo del dominio.

Esta idea conecta tres elementos de la arquitectura ACAI: el espectro de restricción (las técnicas disponibles), el information state (el contexto que determina la criticidad), y las políticas del dominio (las reglas que definen qué nivel de error es aceptable en cada situación). La restricción dinámica es un ejemplo de cómo la gobernanza simbólica se extiende no solo a las decisiones operativas sino al *modo* en que se ejecutan las competencias delegadas al LLM.

7.6 Síntesis: el repertorio técnico de ACAI

7.6.1 Mapeo de técnicas a componentes

Las técnicas presentadas se mapean a los componentes de la arquitectura ACAI y a las fases del sprint de la forma siguiente.

Para la **interpretación** (NLU), la Phase 1 usa structured extraction con schemas (nivel 2) como mecanismo principal, con el information state como contexto estructurado. La Phase 2 añade constrained decoding para garantizar corrección sintáctica y reasoning loops para manejar ambigüedad, con restricción dinámica según la criticidad del turno.

Para la **generación** (NLG), la Phase 1 usa prompting con especificación de fidelidad y template-guided generation para los elementos críticos (nivel 2). La Phase 2 añade lexically constrained decoding para garantizar la inclusión de elementos obligatorios y verificación post-generación para los turnos de alta consecuencia.

Para la **compilación** (design time), ambas fases usan el pipeline de cinco pasos con reasoning loop iterativo. La Phase 1 opera con schemas (nivel 2) para la generación de formalizaciones. La Phase 2 explora constrained decoding (nivel 3) para reducir los errores de formalización y acelerar la convergencia del reasoning loop.

Para el **aprendizaje** (learning time), la Phase 2 introduce el análisis de trazas con LLM para detectar lagunas en el modelo formal, usando las técnicas de structured extraction sobre los logs conversacionales en lugar de sobre los inputs del usuario.

7.6.2 La UD7 como bisagra

> **Conexión Eje B → Eje C**
>
> Esta unidad cierra el Eje B (el paradigma neuro-simbólico) y abre el camino al Eje C (representación y formalización). Las técnicas aquí presentadas condicionan directamente qué formalismos son viables: un formalismo cuya gramática no se puede expresar como autómata para constrained decoding es incompatible con las técnicas de nivel 3; un formalismo demasiado irregular para el in-context learning del LLM no satisface la restricción de generabilidad; un formalismo demasiado opaco para el experto de dominio no satisface la restricción de legibilidad. La **UD 9** evaluará los formalismos candidatos contra estos criterios técnicos; la **UD 10** implementará el pipeline de compilación; la **UD 11** analizará las tensiones de la triple restricción. Las técnicas de la UD7 son el puente que conecta el paradigma con la representación.

Conexiones con otras unidades

→ **Prerequisito: UD 5** (Fundamentos neuro-simbólicos) para la taxonomía de Kautz y las fortalezas complementarias. **UD 6** (Arquitecturas y patrones) para los patrones que aquí se llenan con técnicas concretas.

⇄ **Complementa: UD 2** (La conversación como fenómeno) definió las cinco dimensiones conversacionales que la structured extraction de esta unidad debe capturar. UD7 proporciona las técnicas para implementar esa captura.

⇄ **Complementa: UD 3** (Ecosistema operativo) definió el concepto de RAG gobernado, capacidad agéntica gobernada, y alineamiento formalizado. Las técnicas de esta unidad (structured extraction, constrained decoding, template-guided generation) son los mecanismos concretos que implementan esa gobernanza.

Aplicación: UD 9 (Formalismos candidatos) evalúa los formalismos contra criterios que incluyen la compatibilidad con constrained decoding, la generabilidad por LLM con in-context learning, y la legibilidad.

Aplicación: UD 10 (Formalización asistida) implementa directamente el pipeline de compilación descrito en la sección 7.4.

Aplicación: UD 11 (La triple restricción) analiza en profundidad las tensiones entre las tres restricciones que la sección 7.4.3 introduce técnicamente.

Ejercicios

1. [Implementación] Diseñe el prompt completo (system prompt + few-shot examples) para la structured extraction de ACAI en el escenario de la aseguradora. El prompt debe incluir: (a) la descripción del esquema de extracción (acto comunicativo, entidades, señales pragmáticas, relaciones, certeza), (b) al menos tres ejemplos anotados de inputs con sus extracciones, y (c) las instrucciones de formato. Pruebe el prompt con un LLM disponible sobre los siguientes inputs: «Quiero cancelar mi seguro de hogar», «Bueno, no sé, ¿qué me recomienda?», y «Sí pero espere, antes de cancelar, ¿qué pasa con la bonificación?». Evalúe la calidad de las extracciones.

2. [Espectro de restricción] Para el input «Me gustaría cambiar lo del seguro», produzca cuatro interpretaciones correspondientes a los cuatro niveles del espectro de restricción. Para cada nivel,

especifique: (a) qué técnica se usa, (b) qué garantías ofrece, (c) qué errores son posibles, y (d) qué latencia añade. Evalúe cuál sería el nivel apropiado si el contexto es una primera consulta informativa vs. si el contexto es la confirmación de una operación irreversible.

3. [Generación fiel] Dado que el motor simbólico ha determinado la siguiente acción: «Informar al usuario: bonificación acumulada = 135 €, período de validez = 24 meses, condición = ausencia de siniestros en últimos 3 años. Política: tono empático, ofrecer alternativas a cancelación», diseñe: (a) un template para template-guided generation con espacios libres y elementos fijados, (b) las restricciones para un constrained decoding (tokens obligatorios, patrones prohibidos), y (c) una rúbrica de verificación post-generación con 5 criterios de fidelidad.

4. [Pipeline de compilación] Un experto de dominio describe la siguiente regla: «Si el cliente llama para quejarse de una subida de prima y lleva más de 5 años con nosotros, le ofrecemos mantener la prima anterior durante un año como gesto comercial, pero solo si no ha tenido más de un siniestro en los últimos dos años.» Ejecute los pasos 1–4 del pipeline de compilación: (a) formule al menos 3 preguntas de elicitación que clarificarían la regla, (b) genere una formalización en pseudocódigo estructurado, (c) identifique al menos 2 errores o ambigüedades que un validador detectaría, y (d) especifique el feedback estructurado para la corrección.

5. [Triple restricción] Considere dos formalismos candidatos para representar la regla del ejercicio anterior. El primero es un formalismo cercano al lenguaje natural: `WHEN client.tenure > 5 years AND client.claims(last 2 years) <= 1 AND call.reason = premium_increase THEN OFFER maintain_premium FOR 1 year`. El segundo es una notación lógica: `tenure(X,T), T>5, claims(X,2y,N), N=<1, reason(call,premium_up) => offer(X, maintain_premium, 12m)`. Evalúe ambos contra las tres restricciones: (a) legibilidad (¿lo entiende un experto no técnico?), (b) procesabilidad (¿se puede ejecutar determinísticamente?), y (c) generabilidad (¿puede un LLM producirlo fiablemente con few-shot learning?). Argumente cuál preferiría y por qué.

Referencias y lecturas

Lectura de entrada

Shin, R., Leblond, C. H., Thomson, S. & Song, D. (2021). Constrained Language Models Yield Few-Shot Semantic Parsers. *EMNLP 2021*. PICARD: la demostración más influyente de que un LLM con constrained decoding supera a los modelos especializados en semantic parsing. Directamente relevante para la estrategia de NLU de ACAI. Lectura de entrada que integra semantic parsing, constrained decoding, y LLMs en una sola técnica.

Lecturas esenciales

- Willard, B. T. & Louf, R. (2023). Efficient Guided Generation for Large Language Models. *ArXiv preprint arXiv:2307.09702*. — Outlines: la referencia técnica para constrained generation con gramáticas. Ya citado en UD6; aquí esencial como implementación.
- Kamath, A. & Das, R. (2019). A Survey on Semantic Parsing. *AKBC 2019*. — Survey comprensivo de semantic parsing, desde gramáticas categóricas hasta modelos neuronales. Proporciona el contexto histórico.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H., Kaplan, J. et al. (2021). Evaluating Large Language Models Trained on Code. *ArXiv preprint arXiv:2107.03374*. — Codex. Referencia central para la compilación text-to-code.
- Maynez, J., Narayan, S., Bohnet, B. & McDonald, R. (2020). On Faithfulness and Factuality in Abstractive Summarization. *ACL 2020*. — Taxonomía de infidelidades en generación. Base para el análisis de fidelidad en NLG.

Lecturas complementarias

- Zettlemoyer, L. S. & Collins, M. (2005). Learning to Map Sentences to Logical Form: Structured Classification with Probabilistic Categorial Grammars. *UAI 2005*. — Semantic parsing con CCG. Fundacional.
- Zettlemoyer, L. S. & Collins, M. (2007). Online Learning of Relaxed CCG Grammars for Parsing to Logical Form. *EMNLP-CoNLL 2007*. — Extensión del trabajo anterior. Fundacional.
- Jia, R. & Liang, P. (2016). Data Recombination for Neural Semantic Parsing. *ACL 2016*. — Semantic parsing neuronal. Transición de gramáticas a redes.
- Dong, L. & Lapata, M. (2016). Language to Logical Form with Neural Attention. *ACL 2016*. — Modelos de atención para semantic parsing.
- Rajkumar, N., Li, R. & Baber, D. (2022). Evaluating the Text-to-SQL Capabilities of Large Language Models. *ArXiv preprint*. — LLMs para semantic parsing: estado del arte.
- Scholak, T., Schucher, N. & Bahdanau, D. (2021). PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. *EMNLP 2021*. — Implementación técnica de constrained decoding para semantic parsing.
- Brown, T. B. et al. (2020). Language Models are Few-Shot Learners. *NeurIPS 2020*. — GPT-3 y few-shot learning. Base para la structured extraction con

in-context learning.

- Min, S., Lyu, X., Holtzman, A., Arber, M., Lewis, M., Hajishirzi, H. & Zettlemoyer, L. (2022). Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? *EMNLP 2022*. — Análisis de qué aprenden los LLMs del in-context learning.
- Hokamp, C. & Liu, Q. (2017). Lexically Constrained Decoding for Sequence Generation Using Grid Beam Search. *ACL 2017*. — Constrained decoding léxico. Fundacional para NLG restringida.
- Post, M. & Vilar, D. (2018). Fast Lexically Constrained Decoding with Dynamic Beam Allocation. *NAACL 2018*. — Mejora de eficiencia sobre Grid Beam Search.
- Lu, X., West, P., Zellers, R., Le Bras, R., Bhagavatula, C. & Choi, Y. (2022). NeuroLogic A*esque Decoding: Constrained Text Generation with Lookahead Heuristics. *NAACL 2022*. — Restricciones complejas sobre la generación. Extensión sofisticada.
- Honovich, O., Aharoni, R., Herzig, J., Taitelbaum, H., Kuber, D., Choi, V., Szpektor, I. & Levy, O. (2022). TRUE: Re-evaluating Factual Consistency Evaluation. *NAACL 2022*. — Evaluación de consistencia factual. Adaptable a verificación de fidelidad.
- Laban, P., Schnabel, T., Bennett, P. N. & Hearst, M. A. (2022). SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization. *TACL*, 10, 163–177. — NLI para detección de inconsistencias. Adaptable a verificación post-generación.
- Tang, L., Sun, Z., Patel, N., Obeid, J. & Jiang, X. (2024). Evaluating Large Language Models on Medical Evidence Summarization. *NPJ Digital Medicine*, 6, 158. — Verificación de fidelidad con rúbricas en dominio médico. Paradigma transferible.
- Henderson, M., Thomson, B. & Williams, J. D. (2014). The Second Dialog State Tracking Challenge. *SIGDIAL 2014*. — DSTC2: benchmark fundacional de dialogue state tracking.
- Feng, Y., Lipani, A., Ye, F., Zhang, Q. & Yilmaz, E. (2023). A Survey on Dialogue State Tracking: Models, Datasets, and Challenges. *ACM Computing Surveys*, 55(7). — Survey actualizado de DST.
- Hudecek, V. & Dusek, O. (2023). Are LLMs All You Need for Task-Oriented Dialogue? *SIGDIAL 2023*. — Evaluación de LLMs para DST y gestión de diálogo.
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R. et al. (2022). Competition-Level Code Generation with AlphaCode. *Science*, 378, 1092–1097. — AlphaCode.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. et al. (2024). Code Llama: Open Foundation Models for Code. *ArXiv preprint arXiv:2308.12950*. — Code Llama.
- Liu, N. F. et al. (2023). Lost in the Middle: How Language Models Use

Long Contexts. *Transactions of the ACL*. — Degradación del seguimiento de instrucciones en contextos largos.

- Mernik, M., Heering, J. & Sloane, A. M. (2005). When and How to Develop Domain-Specific Languages. *ACM Computing Surveys*, 37(4), 316–344. — Diseño de DSLs. Relevante para la triple restricción.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«Before asking how to represent,
one must ask what to represent.
The answer is never as obvious as it seems.»

Eje C — Representación y formalización

Unidad 8 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje C — Representación y formalización

Prerequisitos: UD2, UD3, UD7 · Eje C — Representación y formalización

UNIDAD DIDÁCTICA 8

Dimensiones del conocimiento conversacional

Contenidos de la unidad

8.1.1 La ilusión de homogeneidad 3
8.1.2 El error de aplanamiento 4
8.2.1 Dimensión 1: Conocimiento de dominio 5
8.2.2 Dimensión 2: Conocimiento procedimental de diálogo 6
8.2.3 Dimensión 3: Conocimiento regulatorio y de políticas 8
8.2.4 Dimensión 4: Conocimiento del estado conversacional 9
8.2.5 Dimensión 5: Conocimiento pragmático y comunicativo 10
8.3.1 Matriz de propiedades 12
Temporalidad 12
Modalidad 12
Determinismo 13
Composicionalidad 13
Cardinalidad 13
8.4.1 Interacciones entre dominio y procedimiento 14
8.4.2 Interacciones entre regulatorio y procedimiento 14
8.4.3 Interacciones entre estado y todas las demás dimensiones 14
8.4.4 Interacciones entre pragmática y generación 15
8.5.1 Metodología 15
8.5.2 Test de necesidad 16
8.5.3 Test de suficiencia 16
8.6.1 Requisitos por dimensión 17
R1: Capacidad ontológica 17
R2: Capacidad procedimental 17
R3: Capacidad normativa 18
R4: Capacidad de modelado de estado dinámico 18
R5: Capacidad pragmática 18
R6: Capacidad de interacción entre dimensiones 18
R7: Trazabilidad constitutiva 18
8.6.2 La triple restricción como meta-requisito 18

¿Por qué necesito esto para ACAI?

Los Ejes A y B han establecido el problema (la accountability exige representación formal) y el paradigma (la integración neuro-simbólica). La UD 7 cerró el Eje B con un repertorio de técnicas que cruzan la frontera entre lenguaje natural y representaciones formales. Pero esas técnicas requieren un formalismo de destino: ¿qué debe ser capaz de representar ese formalismo? Esta pregunta — anterior y más fundamental que la pregunta sobre qué formalismo elegir — es la que esta unidad responde. Al terminar, el investigador dispondrá de un inventario estructurado de las dimensiones del conocimiento conversacional, con las propiedades formales de cada dimensión y los requisitos que imponen al formalismo. Este inventario es el input directo de la UD 9, donde los formalismos candidatos se evaluarán contra estos requisitos.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Argumentar por qué el conocimiento conversacional es multidimensional y por qué tratar todas las dimensiones como homogéneas produce formalismos inadecuados.
2. Describir las cinco dimensiones del conocimiento conversacional (dominio, procedimental, regulatorio, estado conversacional, pragmático) con sus propiedades formales.
3. Analizar las interacciones entre dimensiones y explicar por qué el formalismo debe capturarlas.
4. Derivar los requisitos de representación que un formalismo candidato debe satisfacer, organizados por dimensión y por propiedad formal.
5. Verificar la completitud del inventario de dimensiones mediante su aplicación al escenario de referencia.

8.1 Por qué no basta con «representar el diálogo»

Sede principal de este concepto.

8.1.1 La ilusión de homogeneidad

Es tentador pensar que un sistema conversacional necesita representar «el diálogo» y que, por tanto, el formalismo debe ser un formalismo de diálogo. Esta intuición es comprensible pero profundamente inadecuada. Debajo de la superficie de cada turno conversacional opera un

El conocimiento que un sistema conversacional accountable necesita no es «el diálogo» — es un sistema multidimensional donde la conversación es solo la superficie visible.

sistema de conocimiento que incluye, como mínimo, una ontología del dominio sobre el que se conversa, los protocolos que determinan cómo avanza la conversación, las restricciones normativas que definen qué se puede y qué no se puede hacer, un modelo dinámico del estado de la interacción, y un conjunto de reglas pragmáticas que determinan cómo se comunica.

Cada uno de estos tipos de conocimiento tiene propiedades formales diferentes. El conocimiento de dominio es estático y declarativo: describe qué cosas existen y qué relaciones tienen. El conocimiento procedimental es dinámico e imperativo: describe qué hacer en cada situación. El conocimiento regulatorio es deontológico: describe obligaciones y prohibiciones con prioridades. El estado conversacional es efímero y acumulativo: existe solo durante la conversación y crece con cada turno. El conocimiento pragmático es situacional y gradual: depende del contexto emocional y opera en escalas continuas, no en valores discretos.

Tratar estos tipos de conocimiento como si fueran homogéneos — como si una sola representación sirviera igual para «una póliza de hogar cubre daños por agua» y para «si el usuario expresa frustración, adoptar tono empático» — produce formalismos que son adecuados para alguna dimensión e inadecuados para las demás. Los sistemas expertos clásicos representaban bien el dominio pero mal los protocolos de diálogo. Los gestores de diálogo basados en máquinas de estados representaban bien los protocolos pero mal el conocimiento de dominio. Los guardrails representan bien las prohibiciones pero mal las obligaciones con prioridades.

8.1.2 El error de aplanamiento

El aplanamiento del conocimiento es la causa técnica más frecuente de inadecuación en sistemas conversacionales.

El **error de aplanamiento** consiste en representar conocimiento multidimensional en una estructura plana que no distingue entre tipos. En los sistemas de diálogo actuales, este error se manifiesta de varias formas.

La forma más común es codificar todo el conocimiento en el system prompt del LLM: las reglas de dominio, los protocolos de diálogo, las políticas de comunicación, las restricciones normativas — todo como texto plano en un prompt. El LLM interpreta este texto como instrucciones probabilísticas, sin distinguir entre una restricción regulatoria (que tiene fuerza legal) y una sugerencia de tono (que es orientativa). Como analizó la **UD 4**, esta falta de distinción es incompatible con la accountability.

Otra forma es representar todo el conocimiento como reglas de producción del mismo tipo: `IF condición THEN acción`. Esto captura

bien el conocimiento procedimental pero pierde las propiedades específicas de las otras dimensiones: las reglas de dominio no son procedimientos (son hechos), las restricciones normativas tienen prioridades y excepciones que las reglas de producción simples no modelan, y el estado conversacional necesita operaciones de actualización incremental, no reglas de disparo.

ACAI necesita un formalismo — o una familia coherente de formalismos — que respete las propiedades específicas de cada dimensión del conocimiento. Esta unidad identifica esas dimensiones y sus propiedades. La **UD 9** evaluará qué formalismos satisfacen esos requisitos.

8.2 Las cinco dimensiones del conocimiento conversacional

Sede principal de este concepto.

El análisis de las unidades anteriores — el fenómeno conversacional (**UD 2**), el ecosistema operativo (**UD 3**), los requisitos de accountability (**UD 4**) y las técnicas neuro-simbólicas (**UD 7**) — converge en la identificación de cinco dimensiones del conocimiento que un sistema conversacional accountable debe representar formalmente. Estas cinco dimensiones son *necesarias* (omitir cualquiera produce un sistema incapaz de operar con accountability) y su unión es *suficiente* (cubren todo el conocimiento necesario para la operación conversacional gobernada). La sección 8.5 verificará estas afirmaciones contra el escenario de referencia.

8.2.1 Dimensión 1: Conocimiento de dominio

Conocimiento de dominio

El **conocimiento de dominio** es la representación de las entidades, relaciones, atributos y restricciones de integridad del dominio sobre el que el sistema conversa. Responde a la pregunta: ¿sobre qué se conversa?

En el escenario de la aseguradora, el conocimiento de dominio incluye: los tipos de producto (seguro de hogar, seguro de auto, seguro de vida), sus coberturas y exclusiones, las relaciones entre productos y clientes (una póliza vincula un cliente con un producto), los atributos de cada entidad (la prima de una póliza, la antigüedad de un cliente, el monto de una bonificación), y las restricciones de integridad (un cliente no puede tener dos pólizas activas del mismo tipo para el mismo bien asegurado).

Las propiedades formales del conocimiento de dominio son las siguientes.

El conocimiento de dominio es estático, declarativo, compartido, y tiene estructura ontológica. El formalismo necesita capacidad ontológica.

Es *estático* en el sentido de que no cambia durante una conversación individual: las coberturas de un producto no se modifican porque un cliente llame. Cambia con baja frecuencia en ciclos de negocio (actualización anual de condiciones, lanzamiento de nuevos productos).

Es *declarativo*: describe qué es, no qué hacer. «Una póliza de hogar incluye cobertura por daños de agua» es una proposición, no una instrucción. Las reglas de derivación del dominio («la bonificación se calcula como el 5 % de la prima por cada año sin siniestros, hasta un máximo del 30 %») son declarativas en el sentido de que definen relaciones entre conceptos, aunque son computables.

Es *compartido*: el mismo conocimiento de dominio aplica a todas las conversaciones y a todos los clientes. Un cliente específico tiene una póliza específica, pero las reglas que definen qué es una póliza y qué coberturas tiene son las mismas para todos.

Tiene *estructura ontológica*: las entidades se organizan en jerarquías (un seguro de hogar es un tipo de seguro), tienen relaciones tipadas (una póliza cubre a un cliente), y los atributos tienen dominios definidos (la prima es un valor monetario positivo).

El formalismo de ACAI necesita, por tanto, *capacidad ontológica*: la representación de jerarquías de conceptos, relaciones tipadas entre entidades, atributos con dominios, restricciones de integridad, y reglas de derivación. La tradición de ontologías formales (Guarino et al., 2009; Gruber, 1993) y de knowledge graphs (Hogan et al., 2021) proporciona los modelos de referencia para esta capacidad.

Guarino et al. (2009) — ontología aplicada: principios de modelado ontológico.

La dimensión de dominio tiene una relación específica con el ecosistema operativo de la **UD 3**: el RAG gobernado necesita que las consultas al conocimiento documental se formulen en términos de la ontología de dominio, y la capacidad agéntica gobernada necesita que las acciones se definan sobre las entidades y relaciones del dominio. Sin una ontología de dominio formalizada, la gobernanza del ecosistema carece de vocabulario.

Gruber (1993): definición clásica de ontología como especificación explícita de una conceptualización.

8.2.2 Dimensión 2: Conocimiento procedimental de diálogo

Conocimiento procedimental de diálogo

El **conocimiento procedimental de diálogo** es la representación de los protocolos conversacionales: qué pasos seguir para cada tipo de interacción, en qué orden, con qué condiciones de avance, retroceso y ramificación. Responde a la pregunta: ¿cómo se avanza

en la conversación?

En el escenario de la aseguradora, el conocimiento procedimental incluye: el protocolo de cancelación (verificar identidad → confirmar póliza → informar condiciones → solicitar confirmación → ejecutar cancelación → confirmar ejecución), las condiciones de avance entre pasos (la confirmación requiere que el usuario produzca un acto de confirmación explícito), los puntos de ramificación (si el cliente pregunta por la bonificación, abrir un subdiálogo de consulta antes de continuar), y los protocolos de reparación (si el usuario cambia de opinión en cualquier punto, retroceder al estado anterior).

Las propiedades formales del conocimiento procedimental son las siguientes.

El conocimiento procedimental es dinámico, imperativo, contextual, y tiene estructura de flujo con anidamiento. El formalismo necesita capacidad procedimental.

Es *dinámico*: evoluciona durante la conversación. El protocolo no es una secuencia fija sino un espacio de posibilidades que se va concretando con cada turno. En el turno 1, múltiples protocolos son posibles; en el turno 5, el diálogo está en un punto específico de un protocolo específico.

Es *imperativo*: describe qué hacer, no qué es. «Si el cliente confirma la cancelación, ejecutar la cancelación» es una instrucción condicional, no una proposición.

Es *contextual*: el mismo acto del usuario puede desencadenar acciones diferentes según el punto del protocolo en que se encuentre el diálogo. Un «sí» en respuesta a «¿desea cancelar?» avanza la cancelación; un «sí» en respuesta a «¿desea que le informe sobre la bonificación?» abre un subdiálogo informativo.

Tiene *estructura de flujo con anidamiento*: los protocolos no son secuencias lineales sino grafos con ramificaciones, bucles, y subdiscursos que se anidan dentro de discursos principales. La tradición de Grosz y Sidner (1986), analizada en la **UD 2**, mostró que la estructura del discurso es inherentemente jerárquica, con segmentos que se abren, se cierran, y se anidan.

El formalismo necesita, por tanto, *capacidad procedimental*: la representación de secuencias de acciones, condiciones de transición, puntos de decisión, subdiscursos anidados, mecanismos de retroceso, y protocolos de reparación. La tradición de gestión de diálogo proporciona modelos de referencia: máquinas de estados finitos (limitadas por su planitud), redes de Petri (que modelan concurrencia y sincronización), planes parciales (Perrault y Allen, 1980), y el enfoque de information state update (Larsson y Traum, 2000) que la **UD 3** introdujo.

Grosz & Sidner (1986): la estructura del discurso es jerárquica, no lineal.

8.2.3 Dimensión 3: Conocimiento regulatorio y de políticas

Conocimiento regulatorio y de políticas

El **conocimiento regulatorio** es la representación de las restricciones normativas que el sistema debe cumplir: requisitos legales, políticas de negocio, y políticas de comunicación. Responde a la pregunta: ¿qué se debe y qué no se debe hacer?

En el escenario de la aseguradora, el conocimiento regulatorio incluye: requisitos del EU AI Act (trazabilidad de las decisiones, transparencia al usuario), requisitos del GDPR (informar sobre el tratamiento de datos personales antes de acceder a la información del cliente), requisitos de DORA (resiliencia operativa, gestión de incidentes), políticas de negocio (ofrecer alternativas antes de ejecutar una cancelación cuando el cliente tiene bonificación acumulada), y políticas de comunicación (no proporcionar asesoramiento financiero, informar siempre de las condiciones contractuales antes de una operación irreversible).

El conocimiento regulatorio es deontológico, priorizado, con ámbitos, y con excepciones. El formalismo necesita capacidad normativa.

Las propiedades formales del conocimiento regulatorio son las siguientes.

Es *deontológico*: describe obligaciones («el sistema debe informar de las condiciones»), prohibiciones («el sistema no debe proporcionar asesoramiento financiero»), y permisos («el sistema puede ofrecer alternativas a la cancelación»). Las modalidades deontológicas no se reducen a condiciones verdadero/falso: una obligación que no se cumple no es un simple error sino una violación normativa con consecuencias jurídicas diferentes a las de una recomendación no seguida.

Tiene *prioridades* entre normas: una restricción regulatoria prevalece sobre una política comercial; una política de seguridad prevalece sobre una política de experiencia de usuario. Cuando dos normas entran en conflicto (la política comercial dice «retener al cliente» pero la regulación dice «ejecutar la solicitud del cliente sin dilación»), las prioridades determinan cuál prevalece.

Governatori et al. (2009) — representación formal de normas con lógica *defeasible*.

Tiene *ámbitos de aplicación*: cada norma aplica a situaciones específicas. La obligación de informar sobre la bonificación aplica cuando existe una bonificación; no aplica cuando no existe. La prohibición de asesoramiento financiero aplica a todas las conversaciones. Los ámbitos pueden ser condicionales («si el cliente tiene más de 65 años, informar sobre las condiciones de renovación automática»).

Tiene *excepciones* y *derrotabilidad*: las normas pueden tener excepciones que las anulan en circunstancias específicas. La obligación de ofrecer alternativas a la cancelación puede tener una excepción cuando el cliente ha expresado su intención de forma inequívoca tres veces.

La representación formal de normas con excepciones requiere mecanismos de *defeasible reasoning* (Governatori et al., 2009): la capacidad de representar conclusiones que son válidas salvo que una excepción las derrote.

El formalismo necesita, por tanto, *capacidad normativa*: la representación de obligaciones, prohibiciones, permisos, prioridades entre normas, ámbitos de aplicación, y excepciones. Esta capacidad es la que conecta directamente con los requisitos regulatorios traducidos a especificaciones técnicas en la **UD 4**.

8.2.4 Dimensión 4: Conocimiento del estado conversacional

Conocimiento del estado conversacional

El **conocimiento del estado conversacional** — el information state — es la representación del estado del diálogo en cada punto: qué información se ha intercambiado, qué compromisos se han adquirido, qué queda pendiente, y con qué nivel de verificación se ha establecido cada dato. Responde a la pregunta: ¿dónde estamos en la conversación?

En el escenario de la aseguradora, el estado conversacional en el turno 5 podría incluir: identidad del usuario verificada (nivel: autenticación completa), póliza objeto de la consulta identificada (HG-2019-4421, nivel: confirmada por el usuario), solicitud de cancelación registrada (nivel: expresada pero no confirmada), consulta sobre bonificación resuelta (nivel: información proporcionada y acuse de recibo del usuario), subtarea de confirmación de cancelación abierta (nivel: pendiente).

El estado conversacional es efímero, acumulativo, estructurado, y con niveles de confirmación. El formalismo necesita capacidad de modelado de estado dinámico.

Las propiedades formales del estado conversacional son las siguientes.

Es *efímero*: existe solo durante la conversación y se destruye (o archiva como traza) cuando la conversación termina. No es conocimiento persistente del dominio sino un modelo transitorio de la interacción en curso.

Es *acumulativo*: cada turno añade información al estado. El estado del turno 5 contiene toda la información del turno 4 más la información nueva del turno 5. Pero la acumulación no es simplemente aditiva: un turno puede *modificar* información anterior (el usuario corrige un dato), puede *retractar* un compromiso (el usuario cambia de opinión), o puede *elevar* el nivel de confirmación de un dato (el usuario confirma explícitamente lo que antes solo había mencionado).

Es *estructurado*: no es un log plano de turnos sino un modelo con componentes diferenciados. La tradición de information state update

(Larsson y Traum, 2000) distingue componentes como la agenda (qué queda por hacer), los compromisos compartidos (qué se ha acordado), las preguntas abiertas (*questions under discussion*, QUD, Ginzburg, 2012), y las creencias privadas del sistema (lo que el sistema sabe pero no ha comunicado).

Tiene *niveles de confirmación*: un dato no es simplemente «conocido» o «desconocido» sino que tiene un nivel de verificación que determina qué acciones se pueden tomar sobre él. Un número de póliza mencionado por el usuario pero no verificado contra la base de datos tiene un nivel diferente al mismo número verificado y confirmado. El sistema no debería ejecutar una cancelación sobre un dato con nivel «mencionado»; necesita el nivel «verificado y confirmado». Esta gradualidad del nivel de confirmación conecta con la noción de *grounding* de Clark (1996), analizada en la **UD 2**.

Ginzburg (2012) — *Questions under Discussion*: modelo formal del estado informativo del diálogo.

El formalismo necesita, por tanto, *capacidad de modelado de estado dinámico*: la representación de estados parciales con componentes diferenciados, operaciones de actualización incremental (adición, modificación, retractación, elevación de nivel), y niveles de confirmación. La tradición de TrindiKit (Larsson y Traum, 2000) y TTR — Type Theory with Records (Ginzburg, 2012) — proporciona los modelos de referencia.

8.2.5 Dimensión 5: Conocimiento pragmático y comunicativo

Conocimiento pragmático y comunicativo

El **conocimiento pragmático** es la representación de las reglas que gobiernan *cómo* se comunica: cuándo adoptar un tono empático, cuándo ser conciso, cómo manejar la incertidumbre del usuario, cómo adaptar el registro al contexto emocional. Responde a la pregunta: ¿cómo se debe comunicar?

En el escenario de la aseguradora, el conocimiento pragmático incluye: reglas de tono («en contexto de cancelación, tono empático; en contexto informativo, tono neutro»), reglas de nivel de detalle («si hay datos numéricos relevantes, ser explícito con cantidades y fechas; si la consulta es general, ser conciso»), reglas de estrategia comunicativa («si el cliente muestra vacilación, ofrecer tiempo para decidir; no presionar»), y reglas de reparación («si el cliente expresa frustración con el sistema, reconocer la frustración antes de continuar con el protocolo»).

Las propiedades formales del conocimiento pragmático son las siguientes.

El conocimiento pragmático es situacional, gradual, derivado, y modula las otras dimensiones. El formalismo necesita capacidad de representación de condiciones contextuales y parámetros graduales.

Es *situacional*: depende del contexto emocional y pragmático de la conversación en un momento dado. El tono apropiado en el turno 2

puede no ser el apropiado en el turno 8 del mismo diálogo, porque la situación emocional ha cambiado.

Es *gradual*: opera en escalas continuas, no en valores discretos. La empatía no es un booleano (empático / no empático) sino un grado que se ajusta al contexto. La urgencia del tono varía según la situación. El nivel de detalle se modula, no se enciende o apaga.

Es *derivado*: se computa a partir de las otras dimensiones, no se establece independientemente. El tono se deriva del estado conversacional (¿qué ha expresado el usuario?), del conocimiento procedimental (¿en qué fase del protocolo estamos?), y del conocimiento regulatorio (¿hay restricciones sobre cómo comunicar esta información?). Las reglas pragmáticas son, en este sentido, reglas de segundo orden: operan sobre el resultado de las otras dimensiones.

Es *modulador*: no determina *qué* se comunica (eso lo determinan las dimensiones 2 y 3) sino *cómo* se comunica. La decisión de informar sobre la bonificación es procedimental y regulatoria; la decisión de hacerlo con tono empático y ofreciendo tiempo para decidir es pragmática. Esta distinción es importante para la arquitectura porque la dimensión pragmática opera en la interfaz entre el motor simbólico y el LLM: el motor determina qué comunicar y con qué parámetros pragmáticos, y el LLM materializa esos parámetros en texto natural.

La dimensión pragmática conecta con las dimensiones 4 y 5 de la conversación de la **UD 2**: gestión de la relación y reparación.

El formalismo necesita, por tanto, *capacidad pragmática*: la representación de condiciones situacionales (derivadas del estado conversacional), parámetros graduales (que se transmiten al LLM como parte de la especificación de generación), y reglas de derivación que computan el estilo comunicativo a partir del contexto. Esta capacidad es la menos desarrollada en la literatura de representación del conocimiento y constituye una contribución específica de ACAI.

La taxonomía ISO 24617-2 de funciones comunicativas (Bunt et al., 2012) proporciona un marco de referencia para las funciones pragmáticas, aunque fue diseñada como taxonomía de anotación, no como formalismo de representación. La adaptación de esta taxonomía a un formato procesable por un motor simbólico es una de las tareas de diseño del formalismo ACAI.

Bunt et al. (2012) — ISO 24617-2: taxonomía de funciones comunicativas de diálogo.

8.3 Propiedades formales de las dimensiones

Sede principal de este concepto.

8.3.1 Matriz de propiedades

Las cinco dimensiones difieren en cinco propiedades formales que condicionan qué tipo de formalismo es adecuado para cada una. El análisis sistemático de estas propiedades es lo que convierte el inventario de dimensiones en input técnico para la evaluación de formalismos de la **UD 9**.

Las cinco dimensiones difieren en temporalidad, modalidad, determinismo, composicionalidad y cardinalidad. Un formalismo único debe acomodar estas diferencias.

Temporalidad

La **temporalidad** describe cómo cambia el conocimiento de cada dimensión. El conocimiento de dominio es *estático*: cambia con baja frecuencia, fuera del ciclo de la conversación. El conocimiento procedimental es *dinámico de instanciación*: los protocolos se definen estáticamente pero se instancian y recorren dinámicamente durante la conversación. El conocimiento regulatorio es *estático con vigencia*: las normas cambian con baja frecuencia pero tienen períodos de vigencia y transición. El estado conversacional es *efímero*: se crea, evoluciona y destruye dentro de una conversación. El conocimiento pragmático es *derivado en tiempo real*: se computa en cada turno a partir del estado actual.

La implicación para el formalismo es que debe soportar tanto representaciones estáticas (compiladas una vez y ejecutadas muchas veces) como representaciones dinámicas (creadas y modificadas durante la ejecución). El modelo de conocimiento de ACAI tiene, por tanto, dos componentes temporales: el *modelo compilado* (dimensiones 1, 2, 3 y las reglas de la 5) y el *modelo instanciado* (dimensión 4 y los valores computados de la 5).

Modalidad

La **modalidad** describe el carácter lógico del conocimiento. El dominio es *declarativo*: proposiciones sobre lo que es. El procedimental es *imperativo*: instrucciones sobre lo que se debe hacer. El regulatorio es *deontológico*: normas sobre lo que se debe, se puede y no se puede. El estado es *factual-temporal*: hechos sobre lo que ha ocurrido en la conversación. El pragmático es *evaluativo*: juicios sobre qué es apropiado dado el contexto.

La coexistencia de modalidades diferentes en un mismo sistema de conocimiento es un desafío clásico de la representación del conocimiento. Las lógicas modales (Chellas, 1980) proporcionan el marco teórico para combinar modalidades, pero las implementaciones prácticas suelen recurrir a estratificación: capas diferentes del formalismo para cada modalidad, con mecanismos de interacción entre capas.

Determinismo

El **determinismo** describe si las inferencias sobre el conocimiento producen resultados únicos. El dominio es *determinista*: dados los hechos y las reglas de derivación, la bonificación tiene un valor único. El procedimental es *determinista condicionado*: dado un estado y un input, el protocolo determina una acción, pero el input es probabilístico (depende de la interpretación del LLM). El regulatorio es *determinista con prioridades*: las normas individuales son deterministas, pero los conflictos entre normas se resuelven mediante prioridades que deben estar formalizadas. El estado es *determinista*: la actualización del estado dado un input interpretado es una función determinista. El pragmático es *gradual*: el tono empático no es verdadero o falso sino un grado que depende de la situación.

La gradualidad de la dimensión pragmática es la que más tensiona un formalismo simbólico, que opera naturalmente en dominios discretos. La resolución en ACAI es delegar la materialización de la gradualidad al LLM: el formalismo representa la dirección («más empático»), no el grado exacto, y el LLM traduce esa dirección a texto con la gradualidad apropiada.

Composicionalidad

La **composicionalidad** describe cómo interactúan las unidades de conocimiento dentro de cada dimensión. Las reglas de dominio se componen por herencia y especialización (un seguro de hogar hereda las propiedades generales de un seguro y añade propiedades específicas). Los protocolos de diálogo se componen por anidamiento (un subdiálogo de consulta se anida dentro del diálogo principal de cancelación). Las normas se componen por prioridad y ámbito (una norma más específica prevalece sobre una más general). El estado se compone por acumulación (cada turno se añade a los anteriores). Las reglas pragmáticas se componen por combinación de factores (el tono resultante depende del estado emocional, la fase del protocolo, y las restricciones normativas).

Cardinalidad

La **cardinalidad** indica el volumen típico de conocimiento en cada dimensión. El conocimiento de dominio suele tener cardinalidad media-alta (decenas a cientos de conceptos, relaciones y reglas de derivación). El procedimental tiene cardinalidad media (decenas de protocolos con sus variantes). El regulatorio tiene cardinalidad media-baja (decenas de normas con sus excepciones). El estado tiene cardinalidad baja pero dinámica (crece durante la conversación, típicamente hasta decenas de componentes). El pragmático tiene cardinalidad baja (decenas de reglas

de estilo).

La cardinalidad importa para la generabilidad (tercera restricción): un formalismo con miles de reglas de dominio necesita que el LLM pueda generar esas reglas con fiabilidad, lo que es un desafío diferente al de generar unas pocas reglas pragmáticas.

8.4 Interacciones entre dimensiones

Las cinco dimensiones no operan en compartimentos estancos. Interactúan de formas que el formalismo debe representar explícitamente, porque es en las interacciones donde se producen las decisiones operativas complejas.

Las dimensiones no son independientes — interactúan de formas específicas que el formalismo debe capturar. Las interacciones son tan importantes como las dimensiones mismas.

8.4.1 Interacciones entre dominio y procedimiento

El conocimiento procedimental opera *sobre* el conocimiento de dominio. Un protocolo de cancelación no es abstracto: cancela una póliza específica de un producto específico. Las condiciones de transición del protocolo referencian entidades y atributos del dominio: «si la póliza tiene bonificación acumulada, abrir subdiálogo de información». La interacción dominio-procedimiento exige que el formalismo permita que las condiciones procedimentales contengan consultas al modelo de dominio.

8.4.2 Interacciones entre regulatorio y procedimiento

Las normas regulatorias *restringen* los protocolos de diálogo. Una norma que obliga a informar sobre las condiciones antes de ejecutar una cancelación impone un orden en el protocolo que el diseñador no puede alterar. Una norma que prohíbe el asesoramiento financiero elimina ciertos tipos de respuesta del repertorio del sistema. La interacción regulatorio-procedimiento exige que el formalismo permita expresar restricciones normativas sobre los flujos procedimentales, y que el motor pueda verificar que los protocolos satisfacen las restricciones.

8.4.3 Interacciones entre estado y todas las demás dimensiones

El estado conversacional es el *medio* a través del cual todas las demás dimensiones se materializan en el diálogo concreto. El motor simbólico consulta el dominio, evalúa el procedimiento, aplica la regulación, y computa la pragmática, todo *en función del estado actual*. El estado es el objeto central que el motor lee y escribe en cada turno. La interacción estado-todo exige que el formalismo defina operaciones de lectura y

escritura del estado que sean compatibles con las representaciones de las demás dimensiones.

8.4.4 Interacciones entre pragmática y generación

El conocimiento pragmático opera en la *frontera* entre el motor simbólico y el LLM. El motor determina los parámetros pragmáticos a partir del estado y las reglas; esos parámetros se transmiten al LLM como parte de la especificación de generación. La interacción pragmática-generación exige que el formalismo produzca representaciones pragmáticas que sean interpretables por el LLM como instrucciones de generación — una forma específica de la restricción de legibilidad que aquí se aplica no al humano sino al LLM como lector.

Interacciones en acción — turno 4 del diálogo de cancelación

Estado (dimensión 4): Identidad verificada. Póliza HG-2019-4421 identificada. Solicitud de cancelación registrada (no confirmada). Consulta sobre bonificación abierta.
Dominio (dimensión 1): HG-2019-4421 tiene bonificación acumulada de 135 € por 3 años sin siniestros. Período de congelación: 24 meses.
Procedimiento (dimensión 2): El protocolo de cancelación está en el paso «informar condiciones». La consulta sobre bonificación abierta tiene prioridad (protocolo de consulta anidado).
Regulatorio (dimensión 3): Obligación de informar sobre bonificación antes de solicitar confirmación (regulación). Obligación de informar sobre período de congelación (política de negocio, prioridad menor). Prohibición de asesoramiento financiero (regulación, prioridad máxima).
Pragmática (dimensión 5): Tono empático (derivado: el usuario contempla una pérdida económica). Nivel de detalle alto (derivado: hay datos numéricos relevantes). Estrategia de retención pasiva (derivado: ofrecer información, no presionar).
Decisión integrada: Informar sobre bonificación (135 €, 3 años, congelación 24 meses) con tono empático, detalle numérico completo, y sin asesoramiento. No solicitar confirmación aún (la consulta debe resolverse primero).
Cada dimensión contribuye un aspecto de la decisión. La decisión integrada no puede derivarse de ninguna dimensión por separado.

8.5 Verificación de completitud

8.5.1 Metodología

La afirmación de que las cinco dimensiones son necesarias y suficientes requiere verificación. La verificación opera en dos direcciones: la *necesidad* se demuestra mostrando que omitir cualquier dimensión

produce un sistema incapaz de operar con accountability en al menos un turno del escenario; la *suficiencia* se demuestra mostrando que todo el conocimiento necesario para cada turno está cubierto por alguna dimensión.

8.5.2 Test de necesidad

Para cada dimensión, se identifica al menos un turno donde su omisión produce una falla de accountability.

Sin dominio (dimensión 1): El sistema no puede verificar que la póliza HG-2019-4421 existe, que pertenece al cliente, o que tiene una bonificación de 135 €. Cualquier dato proporcionado sería una invención del LLM, no una consulta verificable. La trazabilidad de los datos es imposible.

Sin procedimiento (dimensión 2): El sistema no tiene protocolo para la cancelación. No sabe qué verificar primero, cuándo informar, cuándo pedir confirmación. El orden de las acciones depende de la improvisación del LLM. La consistencia del flujo es imposible.

Sin regulatorio (dimensión 3): El sistema no sabe que debe informar sobre la bonificación antes de cancelar, ni que no puede dar asesoramiento financiero. Puede ejecutar una cancelación sin informar al usuario de lo que pierde, violando la regulación. La conformidad normativa es imposible.

Sin estado (dimensión 4): El sistema no recuerda que la identidad ya fue verificada, que el usuario ya preguntó por la bonificación, o que la solicitud de cancelación está pendiente de confirmación. Cada turno opera sin contexto. La coherencia conversacional es imposible.

Sin pragmática (dimensión 5): El sistema informa sobre la penalización de cancelación con el mismo tono que usaría para dar una bienvenida. Comunica datos correctos de forma inapropiada. La calidad comunicativa se degrada, y con ella la confianza del usuario.

La necesidad de cada dimensión se verifica por la falla de accountability que produce su ausencia.

8.5.3 Test de suficiencia

Para el test de suficiencia, se recorre el diálogo de cancelación turno a turno (8 turnos), y para cada decisión del sistema se verifica que todo el conocimiento necesario procede de alguna de las cinco dimensiones.

Los turnos del diálogo cubren: saludo y verificación de identidad (dimensiones 2, 4), identificación de la solicitud (dimensiones 1, 2, 4), consulta de datos del cliente (dimensiones 1, 4), información sobre bonificación (dimensiones 1, 3, 5), gestión de la pausa del usuario (dimensiones 2, 4, 5), solicitud de confirmación (dimensiones 2, 3, 4), ejecución de

la cancelación (dimensiones 1, 2, 3, 4), y confirmación y cierre (dimensiones 2, 4, 5).

> **Suficiencia relativa al dominio**
>
> La suficiencia de las cinco dimensiones está verificada para el dominio de los servicios conversacionales regulados. Otros dominios conversacionales — por ejemplo, la conversación social, la negociación multipartita, o el diálogo pedagógico — podrían requerir dimensiones adicionales. La taxonomía de ACAI no pretende universalidad sino adecuación al dominio del sprint.

8.6 De las dimensiones a los requisitos del formalismo

Sede principal de este concepto.

8.6.1 Requisitos por dimensión

El análisis de las secciones anteriores se traduce en un conjunto de requisitos que todo formalismo candidato debe satisfacer. Estos requisitos son el input directo de la **UD 9**.

Siete capacidades del formalismo derivadas de las cinco dimensiones, más la triple restricción como meta-requisito transversal.

R1: Capacidad ontológica

Derivada de la dimensión 1 (dominio). El formalismo debe poder representar: jerarquías de conceptos (herencia de propiedades), relaciones tipadas entre entidades (con cardinalidad y restricciones), atributos con dominios (tipos de datos, rangos permitidos), restricciones de integridad (condiciones que toda instancia válida debe satisfacer), y reglas de derivación (que computan valores a partir de otros valores).

R2: Capacidad procedimental

Derivada de la dimensión 2 (procedimiento). El formalismo debe poder representar: secuencias de acciones con condiciones de transición, puntos de decisión (ramificación según el estado), subdiscursos anidados (un diálogo dentro de otro), mecanismos de retroceso (volver a un estado anterior), protocolos de reparación (manejar correcciones del usuario), y acciones atómicas del sistema (informar, preguntar, ejecutar, confirmar).

R3: Capacidad normativa

Derivada de la dimensión 3 (regulatorio). El formalismo debe poder representar: obligaciones (lo que el sistema debe hacer en ciertas condiciones), prohibiciones (lo que no puede hacer), permisos (lo que puede hacer opcionalmente), prioridades entre normas (qué prevalece cuando hay conflicto), ámbitos de aplicación (a qué situaciones aplica cada norma), y excepciones (condiciones que anulan una norma).

R4: Capacidad de modelado de estado dinámico

Derivada de la dimensión 4 (estado). El formalismo debe poder representar: componentes diferenciados del estado (agenda, compromisos, datos, nivel de confirmación), operaciones de actualización incremental (adición, modificación, retractación), niveles de confirmación graduales (mencionado, verificado, confirmado), y la persistencia del estado a lo largo de la conversación con acceso eficiente.

R5: Capacidad pragmática

Derivada de la dimensión 5 (pragmática). El formalismo debe poder representar: condiciones situacionales derivadas del estado, parámetros graduales (nivel de empatía, nivel de detalle), reglas de derivación que computan el estilo comunicativo a partir del contexto, y la interfaz con el LLM (los parámetros pragmáticos como parte de la especificación de generación).

R6: Capacidad de interacción entre dimensiones

Derivada de las interacciones (sección 8.4). El formalismo debe permitir que las condiciones procedimentales referencien el modelo de dominio, que las normas restrinjan los procedimientos, que el estado sea accesible desde todas las dimensiones, y que las reglas pragmáticas se computen a partir de las demás dimensiones.

R7: Trazabilidad constitutiva

Derivada del requisito de accountability (**UD 4**). Toda decisión del sistema debe producir una traza que identifique qué dimensiones contribuyeron, qué reglas se aplicaron, qué datos se consultaron, y qué estado resultó. El formalismo debe diseñarse de modo que la traza sea un subproducto natural de la ejecución, no una reconstrucción posterior.

8.6.2 La triple restricción como meta-requisito

Los siete requisitos anteriores deben satisfacerse simultáneamente con las tres restricciones transversales de I1. Todo formalismo que

satisfaga R1–R7 pero no sea legible por un experto humano, o no sea procesable determinísticamente, o no sea generable por un LLM, no es adecuado para ACAI.

El espacio de búsqueda del formalismo

La combinación de siete requisitos funcionales y tres restricciones transversales define un espacio de búsqueda exigente. Ningún formalismo existente satisface todos los requisitos de forma nativa: los lenguajes ontológicos (OWL, RDF) son fuertes en R1 pero débiles en R2 y R4; las máquinas de estado son fuertes en R2 pero débiles en R1 y R3; las lógicas deontológicas son fuertes en R3 pero no están diseñadas para R4 ni R5. La **UD 9** evaluará los formalismos candidatos contra este espacio de requisitos, y es probable que la solución sea un formalismo compuesto o un DSL nuevo — lo que converge con la propuesta de la Phase 0 del sprint.

Conexiones con otras unidades

Prerequisito: UD 2 (La conversación como fenómeno) para las dimensiones del acto comunicativo y la estructura del discurso que informan las dimensiones 2, 4 y 5. **UD 3** (Ecosistema operativo) para el information state y la gobernanza del ecosistema. **UD 7** (Técnicas neuro-simbólicas) para el espectro de restricción y las técnicas que el formalismo debe soportar.

Complementa: UD 4 (Accountability) definió las propiedades de accountability que R7 traduce a requisito del formalismo.

Aplicación: UD 9 (Formalismos candidatos) evalúa los formalismos contra los requisitos R1–R7 derivados aquí. Sin esta unidad, la UD9 no tiene criterios de evaluación.

Aplicación: UD 10 (Formalización asistida) necesita saber qué dimensiones del conocimiento debe capturar el pipeline de compilación.

Aplicación: UD 11 (La triple restricción) analiza las tensiones entre los requisitos funcionales (R1–R7) y las restricciones transversales (legibilidad, procesabilidad, generabilidad).

Ejercicios

1. [Análisis] Tome un dominio diferente al de la aseguradora — por ejemplo, atención médica ambulatoria o soporte técnico de tele-

comunicaciones — e identifique las cinco dimensiones del conocimiento conversacional para ese dominio. Para cada dimensión, proporcione al menos tres ejemplos concretos de conocimiento que pertenece a esa dimensión.

2. [Propiedades formales] Para el dominio elegido en el ejercicio anterior, analice las propiedades formales de cada dimensión (temporalidad, modalidad, determinismo, composicionalidad, cardinalidad). ¿Difieren significativamente de las propiedades del dominio asegurador? ¿Qué implicaciones tienen las diferencias para el diseño del formalismo?

3. [Interacciones] Diseñe un escenario de diálogo de 6 turnos en el dominio elegido. Para cada turno, identifique qué dimensiones intervienen y cómo interactúan. Verifique: ¿hay algún turno donde una dimensión interviene que no esté entre las cinco definidas? ¿Hay alguna dimensión que no interviene en ningún turno?

4. [Requisitos] Para el dominio elegido, evalúe si los siete requisitos (R1–R7) son suficientes o si algún aspecto del dominio requiere un requisito adicional. Si identifica un requisito adicional, formalícelo con la misma estructura: nombre, dimensión de la que se deriva, y capacidades específicas que exige del formalismo.

5. [Evaluación rápida de formalismos] Tome dos formalismos que conozca (por ejemplo, JSON Schema y Prolog) y evalúe informalmente su adecuación a los siete requisitos. Para cada requisito, asigne una calificación: satisface nativamente, satisface con extensiones, no satisface. ¿Cuál de los dos es más adecuado para ACAI? ¿Alguno satisface todos los requisitos? Anticipe las conclusiones que la **UD 9** desarrollará en profundidad.

Referencias y lecturas

Lectura de entrada

Brachman, R. J. & Levesque, H. J. (2004). *Knowledge Representation and Reasoning*. Morgan Kaufmann. El manual de referencia de representación del conocimiento. Cubre ontologías, lógica, acciones, planificación, y razonamiento con incertidumbre. No es específico del diálogo pero proporciona los fundamentos formales que las cinco dimensiones requieren. Lectura de referencia, no de entrada: consultar los capítulos relevantes según la dimensión de interés.

Lecturas esenciales

- Guarino, N., Oberle, D. & Staab, S. (2009). What Is an Ontology? En *Handbook on Ontologies* (2nd ed.), Springer, 1–17. — Fundamentos de ontología aplicada. Referencia para la dimensión 1.
- Larsson, S. & Traum, D. (2000). Information state and dialogue management in the TRINDI dialogue move engine toolkit. *Natural Language Engineering*, 6(3–4), 323–340. — Referencia central para la dimensión 4 (information state). Ya citado en UD3 y UD6; aquí esencial como modelo de estado conversacional.
- Ginzburg, J. (2012). *The Interactive Stance: Meaning for Conversation*. Oxford University Press. — TTR (Type Theory with Records) como formalismo para el estado conversacional. Referencia para la dimensión 4 y la interacción estado-pragmática.
- Bunt, H., Alexandersson, J., Choe, J.-W., Fang, A. C., Hasida, K., Petukhova, V., Popescu-Belis, A. & Traum, D. (2012). ISO 24617-2: A Semantically-Based Standard for Dialogue Act Annotation. *LREC 2012*. — Taxonomía ISO de funciones comunicativas. Referencia para la dimensión 5 (pragmática).
- Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J. E., Navigli, R., Neumaier, S., Ngomo, A.-C. N., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S. & Zimmermann, A. (2021). Knowledge Graphs. *ACM Computing Surveys*, 54(4), 71. — Survey comprensivo de knowledge graphs. Referencia para la dimensión 1 y la interacción dominio-procedimiento.

Lecturas complementarias

- Gruber, T. R. (1993). A Translation Approach to Portable Ontology Specifications. *Knowledge Acquisition*, 5(2), 199–220. — Definición clásica de ontología. Fundacional para la dimensión 1.
- Governatori, G., Rotolo, A., Sartor, G. & Palmirani, M. (2009). Variants of Temporal Defeasible Logic for Modelling Norms. *ICAIL 2009*. — Representación formal de normas con excepciones y temporalidad. Referencia para la dimensión 3.
- Grosz, B. J. & Sidner, C. L. (1986). Attention, Intentions, and the Structure of Discourse. *Computational Linguistics*, 12(3), 175–204. — Estructura jerárquica del discurso. Ya citado en UD2; aquí relevante para la dimensión 2.
- Clark, H. H. (1996). *Using Language*. Cambridge University Press. — Grounding y niveles de confirmación. Fundacional para la dimensión 4.
- Chellas, B. F. (1980). *Modal Logic: An Introduction*. Cambridge University Press. — Lógicas modales. Referencia teórica para la coexistencia de modalidades.
- Perrault, C. R. & Allen, J. F. (1980). A Plan-Based Analysis of Indirect Speech Acts. *Computational Linguistics*, 6(3–4), 167–182. — Planes como modelo de la acción conversacional. Relevante para la dimensión 2.
- Bunt, H. (2011). Multifunctionality in Dialogue. *Computer Speech and Language*, 25(2), 222–245. — Multifuncionalidad del acto comunicativo. Referencia

para la dimensión 5.

- Mernik, M., Heering, J. & Sloane, A. M. (2005). When and How to Develop Domain-Specific Languages. *ACM Computing Surveys*, 37(4), 316–344. — Diseño de DSLs. Relevante para la traducción de requisitos a diseño del formalismo.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«Expressiveness is easy.
Tractability is easy.
Having both at once is the real problem.»

— adaptado de Levesque & Brachman (1987)

Eje C — Representación y formalización

Unidad 9 de 12 ★ Entregable de Fase 0

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje C — Representación y formalización

Prerequisitos: UD7, UD8 · Eje C — Representación y formalización

UNIDAD DIDÁCTICA 9

Formalismos candidatos

Contenidos de la unidad

9.1.1 Criterios y escalas 3
9.1.2 Cobertura dimensional 4
9.1.3 El trade-off expresividad-tratabilidad 4
9.2.1 OWL y la pila de la web semántica 5
9.2.2 Knowledge graphs con esquemas tipados 6
9.3.1 Statecharts 7
9.3.2 BPMN y lenguajes de workflow 8
9.3.3 Colang y lenguajes de diálogo específicos 9
9.4.1 Prolog y programación lógica 9
9.4.2 Lógicas deontológicas 11
9.4.3 TTR y formalismos de information state 11
9.5.1 La opción DSL 12
9.5.2 Formatos intermedios: YAML/JSON con esquemas de dominio 13
9.6.1 La tabla de evaluación 14
9.6.2 Dos caminos posibles 14
9.7.1 Principios de diseño 15
9.7.2 Esbozo de representación por dimensiones 16
9.7.3 Serialización y constrained decoding 17

¿Por qué necesito esto para ACAI?

La UD 8 estableció qué debe representar el formalismo de ACAI: cinco dimensiones del conocimiento conversacional con siete requisitos funcionales. La UD 7 estableció con qué técnicas se cruzará la frontera entre lenguaje natural y representaciones formales. Esta unidad responde la pregunta que ambas dejan abierta: *con qué formalismo*. La respuesta no es obvia porque ningún formalismo existente satisface simultáneamente los siete requisitos funcionales y las tres restricciones transversales. Esta unidad evalúa sistemáticamente los candidatos, documenta sus fortalezas y carencias, y argumenta hacia la solución que ACAI adoptará. Es una de las dos unidades que producen un entregable directo de la Fase 0 del sprint: la evaluación comparada de formalismos con recomendación fundamentada.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Aplicar el marco de evaluación (R1–R7 + triple restricción) para analizar cualquier formalismo candidato con rigor y consistencia.
2. Evaluar formalismos del polo ontológico (OWL/RDF, knowledge graphs), del polo procedimental (statecharts, BPMN, Colang), del polo lógico (Prolog, lógicas deontológicas, TTR), y del polo DSL.
3. Interpretar la evaluación cruzada y argumentar por qué ningún formalismo existente cubre la totalidad de los requisitos.
4. Justificar la decisión de diseñar un DSL conversacional para ACAI con argumentos técnicos derivados de la evaluación.
5. Formular los principios de diseño del DSL ACAI y esbozar cómo representaría cada dimensión del conocimiento.

9.1 El marco de evaluación

Sede principal de este concepto.

9.1.1 Criterios y escalas

Antes de evaluar candidatos es necesario fijar el marco que garantice consistencia y comparabilidad. Los criterios son los establecidos por las unidades anteriores: los siete requisitos funcionales derivados en la UD 8 (R1 capacidad ontológica, R2 capacidad procedimental, R3 capacidad

Diez criterios de evaluación: siete requisitos funcionales (R1–R7) y tres restricciones transversales (legibilidad, procesabilidad, generabilidad).

normativa, R4 modelado de estado dinámico, R5 capacidad pragmática, R6 interacción entre dimensiones, R7 trazabilidad constitutiva) y las tres restricciones transversales de I1 (legibilidad, procesabilidad, generabilidad).

Para cada criterio, se emplea una escala de cuatro niveles. *Satisface nativamente*: el formalismo cubre el requisito con sus mecanismos estándar, sin extensiones. *Satisface con extensiones menores*: el formalismo necesita adaptaciones modestas (convenciones de uso, vocabulario controlado, capas adicionales ligeras) que no alteran su naturaleza. *Satisface con extensiones significativas*: el formalismo necesita mecanismos nuevos que van más allá de su diseño original y cuya integración es no trivial. *No satisface*: el requisito excede las capacidades del formalismo incluso con extensiones.

9.1.2 Cobertura dimensional

Un concepto auxiliar útil es la **cobertura dimensional**: qué dimensiones del conocimiento conversacional (de las cinco de la **UD 8**) cubre cada formalismo nativamente. Un formalismo con cobertura dimensional alta cubre más dimensiones con menos pegamento; uno con cobertura baja requiere combinar múltiples formalismos o extender el formalismo significativamente.

9.1.3 El trade-off expresividad-tratabilidad

La evaluación debe enmarcarse en un principio clásico de la representación del conocimiento que Levesque y Brachman (1987) articularon con precisión: existe una tensión fundamental entre la expresividad de un formalismo (cuánto puede representar) y su tratabilidad (cuán eficiente es razonar sobre las representaciones). Un formalismo muy expresivo — la lógica de primer orden, por ejemplo — puede representar casi cualquier cosa pero la inferencia es indecidible o intratable. Un formalismo muy tratable — las expresiones regulares, por ejemplo — tiene inferencia eficiente pero su capacidad de representación es muy limitada.

Levesque & Brachman (1987) — el trade-off fundacional entre expresividad y tratabilidad.

ACAI añade a este trade-off clásico una dimensión nueva: la *generabilidad*. No basta con que el formalismo sea expresivo y tratable; debe ser generable por un LLM con fiabilidad. Esta tercera dimensión transforma un trade-off bidimensional en un espacio tridimensional donde la solución óptima no es la misma que la del análisis clásico.

ACAI opera en un espacio tridimensional: expresividad, tratabilidad, generabilidad. La solución óptima difiere de la clásica.

Evaluación orientada a la decisión

Esta unidad no es un survey de formalismos. Es una evaluación orientada a una decisión de diseño: qué formalismo adoptar para ACAI. Los candidatos se evalúan con la profundidad necesaria para fundamentar la decisión, no con la exhaustividad de una revisión enciclopédica. Los formalismos que claramente no satisfacen los requisitos centrales se descartan con un análisis breve; los que son candidatos serios reciben un análisis detallado.

9.2 Candidatos del polo ontológico

Sede principal de este concepto.

Los formalismos del polo ontológico fueron diseñados para representar conocimiento de dominio: entidades, relaciones, jerarquías, restricciones. Son los candidatos naturales para la dimensión 1 de la **UD 8**.

9.2.1 OWL y la pila de la web semántica

OWL (Web Ontology Language), junto con RDF (Resource Description Framework) y RDFS, constituye la pila tecnológica de la web semántica. OWL es un lenguaje de ontologías basado en lógicas de descripción que permite representar clases, propiedades, individuos, restricciones de cardinalidad, y axiomas de equivalencia y disyunción. OWL 2 añade expresiones de propiedad más ricas, datatypes, y perfiles con diferentes balances de expresividad y tratabilidad (OWL 2 EL, OWL 2 QL, OWL 2 RL).

Hitzler et al. (2012) — OWL 2 Web Ontology Language Primer.

La evaluación contra los requisitos de ACAI revela un perfil predecible.

R1 (ontología): satisface nativamente. OWL fue diseñado exactamente para este propósito. Jerarquías de conceptos, relaciones tipadas, restricciones de integridad, y inferencia sobre la ontología son sus capacidades centrales. Para el dominio asegurador, la representación de productos, coberturas, pólizas, clientes, y sus relaciones es directa.

R2 (procedimiento): no satisface. OWL es un formalismo declarativo sin mecanismos nativos para representar flujos de acciones, condiciones de transición, o anidamiento de procesos. Extensiones como SWRL (Semantic Web Rule Language) añaden reglas de producción, pero no flujos procedimentales con estado.

R3 (normativo): satisface con extensiones significativas. Las obligaciones y prohibiciones pueden modelarse como anotaciones o como clases de restricciones, pero OWL no tiene modalidades deontológicas

nativas ni mecanismos de prioridad entre normas. Trabajos como los de Palmirani et al. (2011) sobre representación de normas legales en OWL muestran que es posible pero requiere una capa ontológica adicional para las normas sobre las normas.

R4 (estado dinámico): no satisface. OWL es un formalismo para conocimiento estático. No tiene mecanismos de actualización incremental, ni niveles de confirmación, ni el concepto de estado transitorio. El information state de ACAI no puede representarse en OWL sin desnaturalizarlo.

R5 (pragmática): no satisface. Los parámetros graduales y las condiciones situacionales de la dimensión pragmática no tienen representación natural en OWL.

R6 (interacciones): satisface parcialmente. SPARQL permite consultas que combinan información de diferentes partes de la ontología, pero la interacción entre dimensiones heterogéneas (dominio + procedimiento + normas) excede las capacidades de la pila semántica.

R7 (trazabilidad): no satisface nativamente. OWL no tiene un modelo de ejecución que produzca trazas; es un formalismo de representación, no de operación.

Triple restricción. *Legibilidad*: baja. La serialización RDF/XML es ilegible para no técnicos; la notación funcional de OWL (Manchester Syntax) mejora pero sigue siendo técnica. *Procesabilidad*: alta. Los razonadores (HermiT, Pellet, ELK) son maduros y eficientes para los perfiles tratables. *Generabilidad*: media. Los LLMs conocen RDF/OWL por su presencia en los datos de entrenamiento, pero la generación fiable de ontologías con axiomas complejos es irregular.

OWL: excelente para el dominio (R1), inadecuado para procedimiento (R2), estado (R4) y pragmática (R5).

Cobertura dimensional: 1 de 5 (dimensión de dominio).

Veredicto: OWL es la referencia para R1 pero su cobertura dimensional es demasiado estrecha para ACAI como formalismo único. Sus principios de modelado ontológico son, sin embargo, relevantes para el diseño de la capacidad ontológica del formalismo ACAI.

9.2.2 Knowledge graphs con esquemas tipados

Los knowledge graphs (KGs) representan conocimiento como tripletas (sujeto, predicado, objeto) organizadas en un grafo dirigido etiquetado. Los KGs con esquemas tipados añaden una capa de restricciones sobre los tipos de nodos y aristas permitidos. Wikidata, con su sistema de propiedades tipadas y restricciones, es el ejemplo más accesible.

La evaluación es similar a la de OWL pero con matices. Los KGs son más flexibles (aceptan datos heterogéneos sin una ontología cerrada) pero menos formales (las restricciones de integridad son convencionales, no axiomáticas). Para R1, satisfacen con extensiones menores (necesitan

Hogan et al. (2021) — survey comprensivo de knowledge graphs.

un esquema tipado que no siempre existe). Para R2–R5, comparten las limitaciones de OWL. La generabilidad es ligeramente mejor: las tripletas son un formato regular que los LLMs generan con razonable fiabilidad, especialmente en formato JSON-LD.

Veredicto: los KGs son una opción interesante para la capa de dominio de ACAI, pero no resuelven las dimensiones procedimentales, normativas, ni de estado. Su formato de tripletas puede influir en el diseño de la capa de dominio del DSL ACAI.

9.3 Candidatos del polo procedimental

Sede principal de este concepto.

Los formalismos del polo procedimental fueron diseñados para representar flujos de acciones: procesos, protocolos, workflows. Son los candidatos naturales para la dimensión 2 de la **UD 8**.

9.3.1 Statecharts

Las máquinas de estados finitos (FSM) son el modelo más elemental de flujo procedimental y el más usado en sistemas de diálogo clásicos. Sus limitaciones — planitud (sin jerarquía), explosión de estados (sin composición), ausencia de memoria — fueron abordadas por Harel (1987) con los **statecharts**, una extensión que añade jerarquía (estados compuestos), concurrencia (regiones ortogonales), y variables de contexto (acciones asociadas a transiciones).

Harel (1987) — statecharts: extensión jerárquica de máquinas de estados.

Los statecharts tienen una semántica operacional precisa: dado un estado actual y un evento, el siguiente estado y las acciones a ejecutar están determinados. Esto los hace atractivos para ACAI porque la procesabilidad determinista es un requisito central.

R1 (ontología): no satisface. Los statecharts no tienen mecanismos para representar jerarquías de conceptos, relaciones tipadas, ni restricciones de integridad. El dominio es externo al formalismo.

R2 (procedimiento): satisface nativamente. La jerarquía de estados, las transiciones condicionales, los estados compuestos para subdiscursos, y las variables de contexto cubren la dimensión procedimental de forma directa. Un protocolo de cancelación con subdiscurso de consulta se modela como un estado compuesto con subestados.

R3 (normativo): no satisface. Los statecharts no distinguen entre acciones obligatorias y opcionales, no tienen prioridades entre restricciones, y no modelan excepciones. Las normas se codifican implícitamente en la estructura del autómata, pero no son inspeccionables como normas.

Statecharts: excelentes para procedimiento (R2), incapaces para dominio (R1) y normas (R3).

R4 (estado dinámico): satisface parcialmente. Las variables de con-

texto de los statecharts pueden modelar el information state, pero sin los niveles de confirmación, las operaciones de retractación, ni la estructura de componentes diferenciados que ACAI requiere.

R5 (pragmática): no satisface. Los parámetros graduales y las reglas de derivación pragmática no tienen representación en los statecharts.

R6 (interacciones): satisface con extensiones significativas. Las guardas de transición pueden referenciar variables de contexto que contengan datos de dominio, pero la integración es ad hoc, no formal.

R7 (trazabilidad): satisface nativamente. La ejecución de un statechart produce una secuencia de estados y transiciones que es directamente una traza.

Triple restricción. *Legibilidad*: alta para diagramas, media-baja para serialización textual (SCXML, JSON). La representación visual de los statecharts es intuitiva para no técnicos; la serialización no lo es. *Procesabilidad*: alta (semántica operacional definida, motores maduros). *Generabilidad*: media. Los LLMs pueden generar FSMs simples razonablemente, pero los statecharts con jerarquía y concurrencia producen errores frecuentes.

Cobertura dimensional: 1.5 de 5 (procedimiento + estado parcial).

Veredicto: los statecharts son la referencia para R2 y R7, y los principios de jerarquía y semántica operacional son directamente relevantes para el diseño del formalismo ACAI. Su incapacidad para R1, R3 y R5 los descarta como formalismo único.

9.3.2 BPMN y lenguajes de workflow

BPMN (Business Process Model and Notation) es un estándar ISO para el modelado de procesos de negocio. Ofrece compuertas de decisión, eventos, subprocesos, carriles de responsabilidad, y una semántica de ejecución definida. Su uso en el sector financiero y asegurador lo hace relevante para ACAI.

van der Aalst (2016) — *Process Mining*: fundamentos de modelado de procesos.

La evaluación es similar a la de los statecharts con algunos matices. BPMN es más expresivo para flujos complejos (compuertas paralelas, eventos intermedios, señales), pero menos formal en la semántica de las condiciones: las guardas de las compuertas se expresan frecuentemente en lenguaje natural o en pseudocódigo, no en un lenguaje formal. Para R2 satisface nativamente; para R3, BPMN tiene extensiones para reglas de negocio (DMN, Decision Model and Notation) que cubren parcialmente la dimensión normativa. La legibilidad visual es alta; la serialización XML es ilegible. La generabilidad es baja: BPMN/XML es demasiado verboso y estructuralmente complejo para los LLMs.

Veredicto: BPMN aporta ideas relevantes (subprocesos, compuertas decisionales, la separación entre flujo y reglas de decisión) pero su

formato de serialización es inadecuado para la triple restricción.

9.3.3 Colang y lenguajes de diálogo específicos

Colang: diseñado para diálogo con LLMs, pero con dirección de control invertida respecto a ACAI.

Colang, el lenguaje de NeMo Guardrails (Rebedea et al., 2023), merece mención específica porque fue diseñado para un problema cercano al de ACAI: definir comportamientos conversacionales de forma controlada sobre un LLM. Colang permite definir flujos de diálogo con una sintaxis legible, y su diseño como guardrail lo orienta hacia las restricciones sobre el comportamiento del LLM.

Sin embargo, Colang opera en el patrón "neural con guardrails simbólicos" analizado en la **UD 6**: el LLM orquesta y Colang restringe. Su modelo de ejecución no incluye un motor simbólico que tome decisiones operativas independientes del LLM. Para R1 no tiene capacidad ontológica; para R3 tiene capacidad de prohibición pero no de obligación con prioridades; para R4 no tiene un information state formal. Su legibilidad es alta (fue diseñado para ser legible), su generabilidad es potencialmente buena (sintaxis regular), pero su modelo de ejecución es incompatible con la arquitectura ACAI.

Veredicto: la sintaxis legible de Colang y su enfoque de lenguaje diseñado para no técnicos son relevantes como inspiración. Su modelo de ejecución no lo es.

9.4 Candidatos del polo lógico

Sede principal de este concepto.

Los formalismos del polo lógico usan la lógica formal como base de representación y razonamiento. Son los candidatos más poderosos en expresividad pero frecuentemente los más difíciles en legibilidad y generabilidad.

9.4.1 Prolog y programación lógica

Sterling & Shapiro (1994) — *The Art of Prolog*: referencia de programación lógica.

Prolog es el representante más maduro de la programación lógica: un formalismo donde el conocimiento se expresa como cláusulas de Horn (hechos y reglas) y la inferencia se realiza por resolución SLD. Su modelo de ejecución es determinista (para programas sin cortes y con terminación garantizada) y su capacidad de representación es significativa: hechos estructurados, reglas con variables, recursión, y la capacidad de metaprogramación (programas que manipulan programas).

R1 (ontología): satisface con extensiones menores. Los hechos Prolog pueden representar entidades y relaciones; las reglas pueden co-

dificar restricciones de integridad y derivaciones. Falta la jerarquía de conceptos nativa, pero puede codificarse con convenciones.

R2 (procedimiento): satisface con extensiones menores. Los protocolos de diálogo pueden representarse como cláusulas que definen las transiciones, con el estado del diálogo como argumento acumulativo. La recursión cubre el anidamiento de subdiscursos. Pero la representación es indirecta: el flujo procedimental se codifica como relaciones lógicas, no como flujo explícito.

R3 (normativo): satisface con extensiones significativas. Prolog estándar no distingue modalidades deontológicas. Extensiones como la programación lógica defeasible (Nute, 1994) y los frameworks de argumentación (Dung, 1995) añaden la capacidad de manejar excepciones y prioridades, pero requieren implementaciones no estándar.

R4 (estado dinámico): satisface con extensiones significativas. El estado mutable no es natural en Prolog (que es declarativo). Puede simularse con assert/retract o con acumuladores, pero el resultado es frágil y difícil de mantener. Los frameworks como TrindiKit usaron Prolog como lenguaje de implementación del information state, pero añadiendo una capa de gestión de estado sobre el lenguaje base.

Prolog: expresivo y procesable, pero legibilidad y generabilidad son problemas serios para ACAI.

R5 (pragmática): satisface con extensiones menores. Las reglas pragmáticas pueden expresarse como cláusulas Prolog que computan parámetros a partir del estado. Es una de las representaciones más naturales para reglas de derivación condicionales.

R6 (interacciones): satisface nativamente. Las cláusulas Prolog pueden referenciar cualquier hecho o regla del programa, lo que permite la interacción entre dimensiones sin mecanismos adicionales.

R7 (trazabilidad): satisface con extensiones menores. La ejecución Prolog produce un árbol de resolución que puede registrarse como traza. Las extensiones de metaprogramación permiten instrumentar la ejecución para producir trazas detalladas.

Triple restricción. *Legibilidad*: baja. La notación de cláusulas de Horn con variables lógicas, unificación, y convenciones Prolog (listas, operadores, cortes) es inaccesible para un experto de dominio no técnico. "`bonificacion(X,B) :- poliza(X,P), antiguedad(P,A), A > 2, siniestros(P,3,0), B is A * 0.05 * prima(P).`" es procesable y correcto pero ilegible para un directivo de seguros. *Procesabilidad*: alta (resolución SLD, implementaciones maduras). *Generabilidad*: media-baja. Los LLMs generan Prolog razonablemente para cláusulas simples pero cometen errores frecuentes en programas con recursión, listas, y efectos secundarios. La regularidad de la sintaxis es buena, pero la semántica de la unificación y la resolución no es intuitiva para el LLM.

Cobertura dimensional: 3.5 de 5 (dominio parcial, procedimiento

parcial, normativo con extensiones, pragmática parcial, interacciones).

Veredicto: Prolog tiene la cobertura dimensional más alta de los candidatos evaluados, pero su legibilidad es un problema fundamental para ACAI. Un directivo de seguros no puede revisar ni aprobar cláusulas Prolog. Sin embargo, los *principios* de la programación lógica (reglas declarativas con variables, inferencia por unificación, composición por conjunción) son directamente relevantes para el diseño del DSL ACAI.

9.4.2 Lógicas deontológicas

Las lógicas deontológicas extienden la lógica proposicional o de predicados con operadores modales para obligación (O), prohibición (F), y permiso (P). Las variantes defeasible (Governatori et al., 2009) añaden la capacidad de representar excepciones y prioridades entre normas: una conclusión normativa es válida salvo que una excepción más específica la derrote.

Governatori et al. (2009) — representación formal de normas con lógica defeasible.

Para la dimensión 3 de ACAI (conocimiento regulatorio), las lógicas deontológicas son el candidato más adecuado: satisfacen R3 nativamente con obligaciones, prohibiciones, permisos, prioridades, y excepciones. Pero su cobertura dimensional es estrecha: R1, R2, R4 y R5 quedan fuera de su alcance. La legibilidad es baja (notación lógica formal) y la generabilidad es baja (los LLMs no tienen exposición significativa a lógicas deontológicas).

Veredicto: referencia para R3. Los *conceptos* (obligación, prohibición, permiso, prioridad, excepción) deben estar presentes en el DSL ACAI; la *notación* no.

9.4.3 TTR y formalismos de information state

TTR (Type Theory with Records), desarrollado por Ginzburg (2012), es el formalismo más sofisticado para la representación del estado conversacional. Un record type en TTR es una estructura tipada con campos que representan los componentes del information state: el contexto compartido, las preguntas abiertas (QUD), los compromisos, y las relaciones entre ellos. Las actualizaciones del estado se modelan como funciones que transforman un record type en otro, y el anclaje al diálogo se realiza mediante reglas de update que mapean actos comunicativos a transformaciones del estado.

Ginzburg (2012) — TTR: teoría de tipos con registros para el diálogo.

Para R4 (estado dinámico), TTR satisface nativamente y con una sofisticación que ningún otro candidato iguala: los niveles de confirmación, las retractaciones, y la estructura de componentes diferenciados son parte del diseño del formalismo. Para R6 (interacciones), TTR permite que las condiciones de update referencien información de cualquier

campo del record, lo que facilita la interacción entre dimensiones. Para R2 (procedimiento), TTR tiene mecanismos de reglas de update que modelan transiciones, aunque el flujo procedimental no es su foco.

Las limitaciones son la legibilidad (la notación de TTR es altamente técnica: tipos dependientes, funciones de actualización, juicios de tipo) y la generabilidad (los LLMs no han visto TTR en sus datos de entrenamiento). Para R1 y R3, TTR no tiene mecanismos específicos.

Veredicto: la referencia teórica para R4 y el modelo conceptual de information state más riguroso disponible. Los *conceptos* de TTR (record types como estados, funciones de update, QUD) deben informar el diseño del DSL ACAI; la *notación* es inadecuada para la legibilidad y la generabilidad.

9.5 Candidatos del polo DSL

Sede principal de este concepto.

9.5.1 La opción DSL

Los formalismos evaluados hasta ahora fueron diseñados para propósitos diferentes al de ACAI. Cada uno es excelente para su propósito original pero parcial para el nuestro. La alternativa es diseñar un **Domain-Specific Language** (DSL) — un lenguaje formal de propósito específico — diseñado desde el inicio para satisfacer los diez criterios de evaluación.

Un DSL permite diseñar para la triple restricción desde el inicio, en lugar de verificarla post-hoc sobre un formalismo existente.

Mernik et al. (2005), en su survey clásico, identifican las condiciones que justifican la creación de un DSL: cuando el dominio es bien definido, cuando las soluciones de propósito general son inadecuadas, cuando los usuarios del lenguaje no son programadores, y cuando la productividad y fiabilidad justifican la inversión. ACAI satisface las cuatro condiciones: el dominio es el conocimiento conversacional accountable (bien definido), los formalismos existentes son parciales (inadecuados), los usuarios son expertos de dominio (no programadores), y la fiabilidad es el objetivo central (justifica la inversión).

Mernik et al. (2005) — cuándo y cómo diseñar un DSL.

La ventaja central de un DSL es que la triple restricción se convierte en criterio de diseño, no en propiedad a verificar. La legibilidad se logra eligiendo una sintaxis cercana al lenguaje natural controlado. La procesabilidad se garantiza definiendo una semántica operacional formal. La generabilidad se optimiza diseñando una gramática regular con patrones predecibles. Estas decisiones de diseño son imposibles con un formalismo existente cuya sintaxis y semántica están fijadas.

La desventaja es igualmente clara: un DSL nuevo no tiene ecosiste-

ma. No hay motores de ejecución, no hay herramientas de verificación, no hay comunidad de usuarios, y el LLM no lo ha visto en sus datos de entrenamiento. Todo debe construirse. Pero esta desventaja es parcialmente mitigable: el motor de ejecución forma parte de la Phase 1 del sprint, la verificación puede apoyarse en técnicas existentes (model checking sobre el grafo de estados), y la generabilidad se basa en in-context learning (que no requiere que el formalismo esté en los datos de entrenamiento, como argumentó la **UD 7**).

9.5.2 Formatos intermedios: YAML/JSON con esquemas de dominio

Antes de llegar al DSL propiamente dicho, merece consideración una opción intermedia: usar YAML o JSON con esquemas de dominio diseñados para ACAI. La idea es usar un formato de serialización estándar (que los LLMs conocen bien) con un esquema específico (que codifica las cinco dimensiones del conocimiento).

La evaluación de esta opción es mixta. La generabilidad es alta: los LLMs generan JSON y YAML con gran fiabilidad, y los esquemas (JSON Schema) permiten constrained decoding eficiente. La procesabilidad es alta: parsear JSON/YAML es trivial. Pero la legibilidad depende enteramente del diseño del esquema: un JSON con claves técnicas y anidamiento profundo es tan ilegible como cualquier formalismo; un YAML con claves en lenguaje natural y estructura plana puede ser razonablemente legible.

JSON/YAML: generabilidad excelente, legibilidad dependiente del diseño del esquema.

La limitación principal es la expresividad: JSON/YAML son formatos de datos, no lenguajes con semántica. Las reglas con condiciones, las normas con prioridades, y los flujos con transiciones no tienen una representación natural en la estructura clave-valor. Pueden codificarse (una regla como un objeto JSON con campos "condición", "acción", "prioridad"), pero la codificación es una convención interpretada por el motor, no una semántica intrínseca del formato.

Veredicto: JSON/YAML como formato de serialización del DSL ACAI es una opción seria. La gramática del DSL se define como un JSON Schema estricto, lo que habilita constrained decoding y facilita la generabilidad. La semántica la aporta el motor, no el formato. Esta combinación — DSL con semántica propia y serialización en JSON/YAML — puede ofrecer lo mejor de ambos mundos.

9.6 Evaluación cruzada

Sede principal de este concepto.

9.6.1 La tabla de evaluación

La evaluación de las secciones anteriores se sintetiza en una comparación directa que revela el paisaje completo. Para cada candidato, se indica el nivel de satisfacción de cada requisito usando la escala definida: nativo (**N**), extensiones menores (**m**), extensiones significativas (**S**), no satisface (**–**).

Los candidatos del polo ontológico (OWL, KGs) satisfacen R1 nativamente pero R2–R5 quedan fuera de alcance. Los del polo procedimental (statecharts, BPMN) satisfacen R2 nativamente pero R1, R3, R5 quedan fuera. Los del polo lógico tienen mayor cobertura — Prolog alcanza R1(m), R2(m), R3(S), R5(m), R6(N), R7(m) — pero fallan en legibilidad y generabilidad. TTR satisface R4(N) como ningún otro pero sus demás capacidades son limitadas y la triple restricción es problemática. Los candidatos DSL (JSON/YAML) satisfacen la generabilidad y la legibilidad pero carecen de semántica propia.

Ningún formalismo existente satisface más de 4 de los 7 requisitos funcionales nativamente o con extensiones menores.

El resultado es claro: **ningún formalismo existente satisface más de cuatro de los siete requisitos funcionales sin extensiones significativas**. Y ninguno satisface simultáneamente los requisitos funcionales y las tres restricciones transversales. Este resultado no es sorprendente — la **UD 8** anticipó que la combinación de requisitos era exigente — pero documentarlo con precisión es el aporte de esta evaluación.

9.6.2 Dos caminos posibles

Ante este resultado, dos caminos son posibles.

El primer camino es el **formalismo compuesto**: usar OWL para la dimensión de dominio, statecharts para la procedimental, lógica defeasible para la normativa, una estructura ad hoc para el information state, y reglas de producción para la pragmática, con un motor que integre los cinco formalismos. La ventaja es que cada dimensión se representa con el formalismo más adecuado. La desventaja es triple: la interacción entre dimensiones (R6) requiere un pegamento complejo entre formalismos heterogéneos, la trazabilidad (R7) requiere unificar las trazas de cinco motores diferentes, y la triple restricción se multiplica por cinco (cada formalismo debe ser legible, procesable y generable, y el pegamento también).

El segundo camino es el **DSL unificado**: diseñar un lenguaje que integre las capacidades de las cinco dimensiones en una sintaxis y semántica coherentes. La ventaja es que R6 y R7 se resuelven por diseño (un solo formalismo, un solo motor, una sola traza), y la triple restricción se aplica una vez. La desventaja es que el DSL debe cubrir capacidades para las que cada formalismo especializado tiene décadas de desarrollo.

El argumento a favor del DSL unificado

Para ACAI, el DSL unificado es el camino preferido, por tres razones.

Primera: la *triple restricción favorece la unidad*. Un DSL diseñado desde el inicio para la legibilidad, la procesabilidad y la generabilidad tiene ventaja sobre un compuesto donde cada componente tiene su propio perfil de triple restricción. El LLM no necesita aprender cinco formalismos sino uno.

Segunda: las *interacciones entre dimensiones* son centrales para ACAI. Un formalismo compuesto relega las interacciones al pegamento; un DSL unificado las hace parte del lenguaje.

Tercera: la *trazabilidad constitutiva* requiere que una sola ejecución produzca una traza coherente. Con un formalismo compuesto, la traza es un mosaico de fragmentos de cinco motores diferentes.

La desventaja — que el DSL debe cubrir capacidades heterogéneas — se mitiga con un diseño modular: el DSL tiene sublenguajes para cada dimensión que comparten sintaxis y semántica base pero extienden el vocabulario y las construcciones según las necesidades de cada dimensión.

9.7 Hacia el formalismo ACAI

Sede principal de este concepto.

9.7.1 Principios de diseño

La evaluación de las secciones anteriores, combinada con el análisis de la triple restricción de la UD 7, permite formular cinco principios de diseño para el DSL ACAI.

Cinco principios de diseño del DSL ACAI: cercanía al lenguaje natural controlado, semántica operacional definida, regularidad para generabilidad, modularidad dimensional, y trazabilidad por construcción.

Principio D1: Cercanía al lenguaje natural controlado (para legibilidad). La sintaxis del DSL debe usar palabras clave en lenguaje natural, con una estructura que un experto de dominio pueda leer como prosa estructurada. Las palabras clave (WHEN, THEN, MUST, MAY, IF, UNLESS) deben ser transparentes. Los identificadores de entidades deben ser descriptivos (`seguro_hogar`, no `SH01`). Las condiciones deben leerse como oraciones («`client.tenure > 2 years`», no «`gt(tenure(X),2)`»).

Principio D2: Semántica operacional definida (para procesabilidad). Cada construcción del DSL debe tener una semántica operacional precisa: dado un estado y una expresión del DSL, el resultado de la evaluación está determinado. Esto es lo que permite la trazabilidad

constitutiva: la traza *es* la secuencia de evaluaciones operacionales.

Principio D3: Regularidad para generabilidad. La gramática del DSL debe ser regular en el sentido de predecible: las construcciones siguen patrones repetitivos, los bloques tienen estructura uniforme, y las variaciones se limitan a los contenidos (condiciones, acciones, entidades) mientras que la estructura se mantiene constante. Esta regularidad es lo que permite al LLM aprender el formalismo con pocos ejemplos de in-context learning.

Principio D4: Modularidad dimensional. El DSL debe tener sublenguajes reconocibles para cada dimensión — ontología, procedimiento, normas, estado, pragmática — que comparten sintaxis base pero extienden el vocabulario según las necesidades. Esto permite que el motor procese cada dimensión con los mecanismos adecuados (inferencia ontológica para el dominio, ejecución de autómata para los procedimientos, evaluación defeasible para las normas) mientras mantiene una representación unificada.

Principio D5: Trazabilidad por construcción. El DSL debe diseñarse de modo que cada regla evaluada, cada condición comprobada, y cada acción ejecutada produzcan un registro de traza como subproducto natural de la ejecución. La traza no se añade después; se genera durante.

9.7.2 Esbozo de representación por dimensiones

Sin pretender definir el DSL completo — esa tarea corresponde a la Phase 1 del sprint —, es posible esbozar cómo cada dimensión se representaría según los principios de diseño.

Para la *dimensión de dominio*, el DSL adoptaría una notación de entidades con atributos tipados y relaciones, inspirada en los knowledge graphs pero con sintaxis legible. Las reglas de derivación se expresarían como reglas declarativas con palabras clave transparentes.

Para la *dimensión procedimental*, el DSL adoptaría una notación de flujos con bloques anidados, inspirada en los statecharts pero con serialización textual legible. Las transiciones se expresarían como condiciones sobre el estado con acciones asociadas.

Para la *dimensión normativa*, el DSL adoptaría las modalidades deontológicas (MUST, MUST NOT, MAY) con ámbitos de aplicación, prioridades explícitas, y cláusulas de excepción (UNLESS), inspiradas en la lógica defeasible pero con sintaxis de lenguaje natural controlado.

Para la *dimensión de estado*, el DSL adoptaría una estructura de record con componentes tipados y niveles de confirmación, inspirada en TTR pero con notación de clave-valor legible.

Para la *dimensión pragmática*, el DSL adoptaría reglas de derivación condicionales que computan parámetros de generación a partir del

estado.

La regla de bonificación en cuatro formalismos

OWL (Manchester Syntax):

```
Class: BonificacionNoSiniestralidad
  EquivalentTo: Bonificacion and (tienePoliza some
    (Poliza and (tieneSiniestros value 0)))
```

Prolog:

```
bonificacion(Cliente, Monto) :-
  poliza(Cliente, Poliza), activa(Poliza),
  antiguedad(Poliza, A), A > 2,
  siniestros_recientes(Poliza, 3, 0),
  prima(Poliza, P), Monto is min(A * 0.05 * P, 0.3 *
P).
```

Statechart (pseudonotación textual):

```
state: consulta_bonificacion
  on_entry: query(poliza.bonificacion)
  transition: [datos_obtenidos] ->
informar_bonificacion
```

Esbozo DSL ACAI:

```
RULE bonificacion_no_siniestralidad
  WHEN client.policy IS ACTIVE
    AND client.policy.tenure > 2 years
    AND client.policy.claims(last 3 years) = 0
  THEN bonus = min(tenure * 5% * premium, 30% *
premium)
  NOTE "Bonificación acumulada por ausencia de
siniestros"
```

El esbozo del DSL ACAI satisface simultáneamente: legibilidad (un experto de seguros lo lee como prosa estructurada), procesabilidad (cada construcción tiene semántica operacional definida), y regularidad (el patrón RULE / WHEN / THEN / NOTE es uniforme para todas las reglas de dominio).

9.7.3 Serialización y constrained decoding

Phase 0

La evaluación de formalismos y el esbozo de principios de diseño del DSL son entregables directos.

Un aspecto práctico crucial del diseño del DSL es su *serialización*: cómo se almacena y transmite. La evaluación sugiere que la serialización en JSON (o YAML) con un JSON Schema estricto ofrece la mejor combinación de generabilidad (los LLMs generan JSON fiablemente), compatibilidad con constrained decoding (Outlines y SGLang soportan JSON Schema nativamente), y interoperabilidad (JSON es el formato de intercambio universal).

La estructura sería: la representación "canónica" del DSL es la nota-

ción legible tipo la del esbozo (para humanos), y la serialización "operacional" es un JSON equivalente (para el motor y para el LLM). La compilación de la **UD 10** produce la serialización JSON; la verbalización (para revisión humana) produce la notación legible. Ambas son transformaciones deterministas entre sí.

Doble representación

La doble representación — notación legible para humanos, JSON para máquinas — no es redundancia sino diseño deliberado. La restricción de legibilidad opera sobre la notación legible; la restricción de procesabilidad opera sobre el JSON; la restricción de generabilidad opera sobre ambas (el LLM debe poder generar el JSON, y la verbalización debe poder generarse a partir del JSON). Las tres restricciones se satisfacen en representaciones diferentes del mismo conocimiento, con transformaciones deterministas entre ellas.

Conexiones con otras unidades

Prerequisito: UD 8 (Dimensiones del conocimiento) para los requisitos R1–R7 que constituyen el marco de evaluación. **UD 7** (Técnicas neuro-simbólicas) para las técnicas de constrained decoding y structured output que condicionan la generabilidad.

Complementa: UD 5 (Fundamentos neuro-simbólicos) estableció la taxonomía de Kautz que informa la integración del formalismo con los componentes neuronales: el DSL debe ser compatible con las técnicas de nivel 2 y 3.

Aplicación: UD 10 (Formalización asistida) implementa el pipeline de compilación que genera representaciones en el DSL ACAI. Los principios de diseño del DSL condicionan directamente la implementabilidad del pipeline.

Aplicación: UD 11 (La triple restricción) analiza en profundidad las tensiones entre los principios D1, D2 y D3, que esta unidad formula pero cuyo análisis detallado corresponde a la UD11.

Aplicación: UD 12 (Métricas) necesita que el formalismo esté definido para poder diseñar las métricas de cobertura, consistencia y completitud que evaluarán el modelo formal.

Ejercicios

1. [Evaluación] Elija un formalismo no evaluado en esta unidad —

por ejemplo, Answer Set Programming (ASP), Datalog, o un lenguaje de reglas de negocio como DMN — y evalúelo contra los diez criterios del marco de evaluación. Presente sus resultados en el formato de la evaluación de esta unidad: un párrafo por requisito con la calificación, más la evaluación de la triple restricción y la cobertura dimensional. ¿Cambia el resultado de la evaluación cruzada?

2. [Formalismo compuesto] Diseñe un formalismo compuesto para ACAI usando: OWL para la dimensión de dominio, statecharts para la procedimental, reglas defeasible para la normativa, y JSON para el estado. Especifique: (a) cómo interactúan los componentes (¿cómo una transición del statechart consulta la ontología OWL?), (b) cómo se produce una traza unificada, y (c) cómo se satisface la triple restricción para el compuesto (no solo para cada componente). Evalúe la complejidad de su propuesta frente a un DSL unificado.

3. [Diseño DSL] Extienda el esbozo del DSL ACAI para representar una norma regulatoria: «El sistema debe informar al cliente sobre su bonificación acumulada antes de solicitar confirmación de la cancelación, salvo que el cliente haya indicado expresamente que no desea esta información.» Formule la norma en la notación del DSL (usando MUST, BEFORE, UNLESS). Verifique que su formulación satisface: legibilidad (¿un directivo la entiende?), procesabilidad (¿un motor puede evaluar si se cumple?), y generabilidad (¿la estructura es regular y predecible?).

4. [Generabilidad] Tome la regla de bonificación del escenario de esta unidad en formato de esbozo DSL ACAI. Diseñe un prompt de in-context learning (system prompt + 2 ejemplos + la tarea) para que un LLM genere la regla a partir de la descripción en lenguaje natural. Pruebe el prompt con un LLM disponible. Evalúe: (a) ¿el LLM genera una regla sintácticamente correcta?, (b) ¿captura todas las condiciones?, (c) ¿cuántos intentos necesita para producir una versión correcta?

5. [Reflexión crítica] El argumento a favor del DSL unificado asume que la triple restricción se satisface mejor con un solo formalismo que con cinco. ¿Es esto siempre cierto? Construya un contraargumento: identifique al menos un escenario donde un formalismo compuesto sería preferible al DSL unificado. ¿Bajo qué condiciones cambiaría la recomendación de esta unidad?

Referencias y lecturas

Lectura de entrada

Levesque, H. J. & Brachman, R. J. (1987). Expressiveness and Tractability in Knowledge Representation and Reasoning. *Computational Intelligence*, 3(1), 78–93. El artículo que formuló el trade-off fundacional de la representación del conocimiento. Breve, elegante, y directamente relevante para entender por qué no existe un formalismo perfecto. Lectura de entrada que enmarca toda la evaluación de esta unidad.

Lecturas esenciales

- Mernik, M., Heering, J. & Sloane, A. M. (2005). When and How to Develop Domain-Specific Languages. *ACM Computing Surveys*, 37(4), 316–344. — El marco de referencia para la decisión de crear un DSL. Ya citado en UD7 y UD8; aquí central.
- Harel, D. (1987). Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*, 8(3), 231–274. — Statecharts: la referencia para la dimensión procedimental.
- Hitzler, P., Krötzsch, M., Parsia, B., Patel-Schneider, P. F. & Rudolph, S. (2012). OWL 2 Web Ontology Language Primer (Second Edition). *W3C Recommendation*. — La referencia técnica para OWL. Necesaria para evaluar el polo ontológico.
- Ginzburg, J. (2012). *The Interactive Stance*. Oxford University Press. — TTR y el information state formal. Ya citado en UD8; aquí evaluado como candidato.

Lecturas complementarias

- Hogan, A. et al. (2021). Knowledge Graphs. *ACM Computing Surveys*, 54(4), 71. — Ya citado en UD8. Referencia para la evaluación de KGs.
- Governatori, G. et al. (2009). Variants of Temporal Defeasible Logic for Modelling Norms. *ICAIL 2009*. — Ya citado en UD8. Referencia para la evaluación de lógicas deontológicas.
- Sterling, L. & Shapiro, E. (1994). *The Art of Prolog* (2nd ed.). MIT Press. — La referencia de programación lógica.
- van der Aalst, W. M. P. (2016). *Process Mining: Data Science in Action* (2nd ed.). Springer. — BPMN y modelado de procesos. Referencia para el polo procedimental.
- Nute, D. (1994). Defeasible Logic. En D. M. Gabbay, C. J. Hogger & J. A. Robinson (Eds.), *Handbook of Logic in Artificial Intelligence and Logic Programming*, Vol. 3. Oxford University Press. — Lógica defeasible: fundamentos.

- Dung, P. M. (1995). On the Acceptability of Arguments and its Fundamental Role in Nonmonotonic Reasoning, Logic Programming and n-Person Games. *Artificial Intelligence*, 77(2), 321–357. — Frameworks de argumentación. Relevante para R3.
- Rebedea, T. et al. (2023). NeMo Guardrails. *EMNLP 2023 (demo)*. — Colang como ejemplo de DSL conversacional.
- Palmirani, M., Governatori, G., Rotolo, A., Tabet, S., Boley, H. & Paschke, A. (2011). LegalRuleML: XML-Based Rules and Norms. *RuleML 2011*. — Representación de normas legales en formato XML. Referencia para la interacción entre polo normativo y serialización.
- Fowler, M. (2010). *Domain-Specific Languages*. Addison-Wesley. — Guía práctica de diseño de DSLs. Complementa a Mernik et al. con perspectiva de ingeniería de software.
- Larsson, S. & Traum, D. (2000). Information state and dialogue management in the TRINDI dialogue move engine toolkit. *Natural Language Engineering*, 6(3–4), 323–340. — Ya citado múltiples veces. Aquí relevante como ejemplo de formalismo de information state implementado.
- Scholak, T., Schucher, N. & Bahdanau, D. (2021). PICARD. *EMNLP 2021*. — Ya citado en UD7. Aquí relevante para la viabilidad de constrained decoding sobre la gramática del DSL.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«The formalization bottleneck is not solved by removing the human from the loop, but by giving the human a better loop.»

Eje C — Representación y formalización

Unidad 10 de 12 ★ Entregable de Fase 0

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje C — Representación y formalización

Prerequisitos: UD7, UD8, UD9 · Eje C — Representación y formalización

UNIDAD DIDÁCTICA 10

Formalización asistida por LLM

Contenidos de la unidad

10.1.1 El problema histórico 4
10.1.2 La transformación del cuello de botella 4
10.2.1 Los cinco pasos 5
10.2.2 El pipeline por dimensiones 5
10.3.1 El diálogo de elicitación 7
10.3.2 Estrategias de elicitación 7
10.4.1 El prompt de generación 8
10.4.2 Técnicas de generación por nivel de restricción 9
Nivel 2: Structured output con JSON Schema 9
Nivel 3: Constrained decoding con gramática del DSL 10
Nivel 4: Reasoning loop con validación semántica 10
10.5.1 Las tres dimensiones de la validación 11
10.5.2 El feedback estructurado 11
10.5.3 Convergencia del reasoning loop 12
10.6.1 La verbalización como puente 12
10.6.2 Protocolos de revisión 13
10.6.3 El ciclo completo 13
10.7.1 Objetivos experimentales 14
10.7.2 Diseño de los experimentos 14
Experimento 1: Generabilidad sintáctica 14
Experimento 2: Convergencia semántica 15
Experimento 3: Eficiencia de revisión 15
10.7.3 Criterios de viabilidad 16
10.8.1 Taxonomía de fallos 16
10.8.2 El principio de detección temprana 17

¿Por qué necesito esto para ACAI?

La UD 9 concluyó que ACAI necesita un DSL con principios de diseño específicos. Pero un DSL sin contenido es un lenguaje vacío: su valor depende de que se pueble con el conocimiento del dominio — las reglas de negocio, los protocolos conversacionales, las normas regulatorias, las políticas comunicativas. Este poblamiento es la *formalización*, y es históricamente el cuello de botella de todo sistema basado en conocimiento. La UD 5 lo identificó como el *knowledge engineering bottleneck* de Feigenbaum (1977): la adquisición y codificación del conocimiento es el paso más costoso, más lento, y más propenso a error.

ACAI propone una solución que es simultáneamente su innovación más distintiva y su contribución más arriesgada: usar el LLM como *asistente de formalización*, capaz de transformar descripciones en lenguaje natural en representaciones formales del DSL. Esta unidad describe el pipeline de formalización asistida con la precisión necesaria para implementarlo. Es la segunda unidad que produce un entregable directo de la Fase 0: el diseño detallado del pipeline y los experimentos de viabilidad.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Diagnosticar el knowledge engineering bottleneck en el contexto de ACAI y explicar por qué la formalización asistida es la respuesta adecuada.
2. Diseñar e implementar cada paso del pipeline de formalización asistida: elicitación, generación, validación, corrección iterativa, y revisión humana.
3. Diseñar los prompts de elicitación y generación con las técnicas de in-context learning, structured output, y constrained decoding de la UD 7.
4. Diseñar experimentos de viabilidad que midan la generabilidad del formalismo ACAI y la calidad de la formalización asistida.
5. Identificar los modos de fallo del pipeline y los mecanismos de mitigación para cada uno.

10.1 El cuello de botella de la formalización

Sede principal de este concepto.

10.1.1 El problema histórico

Feigenbaum (1977) identificó el problema hace casi cinco décadas: la adquisición de conocimiento es el principal obstáculo para la construcción de sistemas expertos. El conocimiento existe en la mente de los expertos de dominio, pero transferirlo a una representación formal requiere un ingeniero del conocimiento que entienda tanto el dominio como el formalismo — un perfil escaso y un proceso lento. Los proyectos de sistemas expertos de los años 80 documentaron tasas de formalización de 2 a 5 reglas por hora de trabajo conjunto entre experto e ingeniero del conocimiento, con tasas de error significativas que requerían ciclos de verificación adicionales.

La formalización manual es el cuello de botella histórico de los sistemas basados en conocimiento. La formalización asistida por LLM no lo elimina sino que lo transforma: de *escribir* conocimiento formal a *revisar y aprobar* conocimiento formal.

El problema persiste en los sistemas conversacionales. Los sistemas basados en intents y slots (analizados en la **UD 2**) simplificaron el formalismo — reduciendo el conocimiento a pares intent-respuesta — pero a costa de empobrecer la representación. Los sistemas más sofisticados (TrindiKit, information state update) ofrecen representaciones más ricas pero heredan el cuello de botella: cada regla de actualización del information state debe diseñarse manualmente.

Feigenbaum (1977) — el knowledge engineering bottleneck: diagnóstico original.

Para ACAI, el cuello de botella se multiplica porque las cinco dimensiones del conocimiento (**UD 8**) requieren formalización de tipos diferentes: ontología de dominio, protocolos conversacionales, normas regulatorias, estructura del information state, y reglas pragmáticas. La formalización manual de todo este conocimiento para un dominio como el asegurador — decenas de productos, cientos de reglas de negocio, decenas de restricciones regulatorias — haría inviable el sprint.

10.1.2 La transformación del cuello de botella

La formalización asistida por LLM no elimina el cuello de botella sino que lo transforma cualitativamente. El experto de dominio deja de ser la fuente de la que el ingeniero extrae conocimiento para codificarlo; se convierte en el *revisor y aprobador* de formalizaciones generadas por el LLM. El cambio es profundo.

En el modelo tradicional, el experto describe, el ingeniero codifica, y el experto no puede verificar la codificación porque no entiende el formalismo. La verificación requiere un segundo ingeniero o un ciclo de pruebas. En el modelo ACAI, el experto describe, el LLM codifica, y el experto verifica directamente gracias a la restricción de legibilidad (D1 de la **UD 9**): el formalismo está diseñado para que el experto lo lea. La verificación se produce en el punto donde el conocimiento es más fresco — inmediatamente después de la descripción — en lugar de semanas después en un ciclo de validación.

Es fundamental subrayar lo que la formalización asistida *no es*: no es una generación autónoma de conocimiento. El LLM no inventa reglas de negocio ni deduce normas regulatorias. Transforma descripciones humanas en representaciones formales, y cada representación requiere aprobación humana. La innovación I2 de ACAI — el LLM como meta-herramienta que genera las representaciones que luego lo gobiernan — se concreta aquí, pero con el humano como autoridad final en cada paso.

El LLM es asistente de formalización, no sustituto del experto. La autoridad sobre el conocimiento sigue siendo humana.

10.2 Arquitectura del pipeline de formalización

Sede principal de este concepto.

10.2.1 Los cinco pasos

La UD 7 describió el pipeline de compilación en cinco pasos a nivel conceptual. Esta sección lo concreta a nivel de implementación. El pipeline opera como un ciclo interactivo entre tres agentes: el *experto de dominio* (que posee el conocimiento), el *LLM* (que transforma descripciones en representaciones formales), y el *validador simbólico* (que verifica la corrección formal de las representaciones generadas).

Paso 1: Elicitación estructurada. El LLM asiste al experto mediante un diálogo guiado que sigue un esquema predefinido para cada dimensión del conocimiento.

Paso 2: Generación condicionada. El LLM genera una representación formal en el DSL ACAI, condicionado por el esquema del formalismo, ejemplos existentes, y el modelo de conocimiento actual.

Paso 3: Validación simbólica. Un validador comprueba corrección sintáctica, consistencia con el modelo existente, y completitud local.

Paso 4: Corrección iterativa. Si la validación detecta errores, el feedback estructurado se devuelve al LLM para regeneración condicionada.

Paso 5: Revisión humana. La formalización validada se presenta al experto con su verbalización para aprobación, modificación, o rechazo.

El pipeline tiene cinco pasos: elicitación estructurada, generación condicionada, validación simbólica, corrección iterativa, y revisión humana. Cada paso tiene técnicas específicas y modos de fallo identificados.

Los cinco pasos no son siempre secuenciales: el paso 4 puede iterar múltiples veces entre los pasos 2 y 3, el paso 5 puede devolver el control al paso 1 si el experto detecta una omisión fundamental, y los pasos 1–2 pueden fusionarse cuando el experto proporciona una descripción suficientemente precisa. Pero la secuencia canónica 1→2→3→4→5 es el flujo principal.

10.2.2 El pipeline por dimensiones

Cada dimensión del conocimiento (UD 8) requiere una instanciación específica del pipeline, porque el tipo de conocimiento condiciona los

esquemas de elicitación, los patrones de generación, y los criterios de validación.

Para la *dimensión de dominio*, la elicitación se centra en entidades, atributos, relaciones, y reglas de derivación. El esquema pregunta: ¿qué entidades existen? ¿Qué atributos tiene cada una? ¿Cómo se relacionan? ¿Qué restricciones aplican? La generación produce definiciones ontológicas en el DSL. La validación comprueba que las entidades referenciadas están definidas, que los tipos de atributos son consistentes, y que las restricciones no se contradicen.

Para la *dimensión procedimental*, la elicitación se centra en flujos de interacción. El esquema pregunta: ¿cuál es el objetivo de la interacción? ¿Qué pasos sigue? ¿Qué condiciones determinan cada paso? ¿Qué ocurre si el usuario se desvía? La generación produce definiciones de flujo en el DSL (bloques FLOW con transiciones). La validación comprueba que todos los estados son alcanzables, que no hay estados sin salida (deadlocks), y que las acciones referenciadas están definidas.

Para la *dimensión normativa*, la elicitación se centra en obligaciones, prohibiciones, y sus condiciones. El esquema pregunta: ¿qué debe hacer el sistema en esta situación? ¿Qué no puede hacer? ¿Hay excepciones? ¿Cuál es la fuente de la norma (regulación, política, práctica)? La generación produce reglas normativas con modalidades deontológicas (MUST, MUST NOT, MAY) y cláusulas de excepción (UNLESS). La validación comprueba que las normas no se contradicen sin resolución de prioridad, que los ámbitos están bien definidos, y que las excepciones referencian condiciones evaluables.

Para la *dimensión de estado*, la elicitación se centra en qué información necesita el sistema para operar. El esquema pregunta: ¿qué datos se recogen durante la conversación? ¿Qué niveles de confirmación se requieren? ¿Qué compromisos adquiere el sistema? La generación produce la estructura del information state como un esquema tipado. La validación comprueba que todos los campos referenciados por las reglas procedimentales y normativas están presentes en el esquema de estado.

Para la *dimensión pragmática*, la elicitación se centra en políticas de comunicación. El esquema pregunta: ¿qué tono es apropiado en cada situación? ¿Cuánto detalle se proporciona? ¿Cómo se maneja la incertidumbre del usuario? La generación produce reglas de derivación pragmática que computan parámetros de generación. La validación comprueba que las condiciones referencian campos del information state y que los parámetros producidos son compatibles con la interfaz LLM.

10.3 Elicitación estructurada

Sede principal de este concepto.

10.3.1 El diálogo de elicitación

La elicitación no es un formulario rígido sino un *diálogo guiado* donde el LLM usa su capacidad lingüística para interpretar las descripciones del experto, detectar ambigüedades, y formular preguntas de seguimiento. El objetivo es obtener una descripción suficientemente precisa y completa para que el paso 2 (generación) produzca una formalización correcta.

El LLM de elicitación opera con un prompt que incluye tres componentes. El *esquema de elicitación* define qué información es necesaria para cada tipo de conocimiento, con campos obligatorios y opcionales. El *modelo de conocimiento actual* proporciona contexto: las entidades ya definidas, las reglas existentes, los protocolos formalizados. Esto permite al LLM formular preguntas contextuales («Ha mencionado la bonificación; en el modelo actual, la bonificación se calcula como el 5 % por año. ¿Esta nueva regla modifica ese cálculo o lo complementa?»). La *historia del diálogo de elicitación* permite que las preguntas de seguimiento sean coherentes con lo ya discutido.

La calidad de la formalización depende directamente de la calidad de la elicitación. Un prompt de elicitación mal diseñado produce formalizaciones incompletas.

10.3.2 Estrategias de elicitación

La investigación en adquisición del conocimiento identifica varias estrategias de elicitación que el LLM puede implementar.

La **elicitación por descomposición** parte de una descripción general y la descompone en componentes. «Describa el protocolo de cancelación» se descompone en: ¿cuáles son los pasos? ¿qué ocurre en cada paso? ¿qué condiciones determinan el avance? Esta estrategia es adecuada para el conocimiento procedimental.

Hoffman et al. (1995) — técnicas de elicitación de conocimiento experto: taxonomía y evaluación.

La **elicitación por casos** parte de ejemplos concretos y generaliza. «Descríbame un caso típico de cancelación» → «¿Qué pasaría si el cliente tuviera bonificación?» → «¿Y si no la tuviera?» → generalización de las condiciones. Esta estrategia es adecuada para el conocimiento de dominio y normativo.

La **elicitación por contraste** identifica diferencias entre situaciones. «¿En qué se diferencia una cancelación con bonificación de una sin bonificación?» → «¿Y una cancelación de hogar de una de auto?» Esta estrategia es especialmente útil para identificar condiciones y excepciones que el experto no menciona espontáneamente porque las da por supuestas.

La **elicitación por revisión** parte de una formalización tentativa (generada a partir de documentación o de una descripción previa) y

pide al experto que la corrija. «El modelo actual dice que la bonificación se congela 24 meses. ¿Es correcto? ¿Aplica a todos los productos?» Esta estrategia es especialmente eficiente en iteraciones posteriores del pipeline.

Diálogo de elicitación — protocolo de cancelación

LLM: Para formalizar el protocolo de cancelación, necesito entender los pasos principales. ¿Podría describir qué ocurre desde que el cliente expresa su intención de cancelar hasta que la cancelación se ejecuta?
Experto: Primero verificamos quién es, luego confirmamos qué póliza quiere cancelar, le informamos de las condiciones, y si confirma, se ejecuta.
LLM: Entendido. Cuatro pasos principales: verificación de identidad, identificación de póliza, información de condiciones, y ejecución. Algunas preguntas de seguimiento. Sobre las condiciones: ¿qué información se proporciona exactamente al cliente antes de pedir confirmación?
Experto: Depende. Si tiene bonificación acumulada, se lo decimos. Si hay penalización por cancelación anticipada, también. Y siempre le decimos cuándo se haría efectiva la cancelación.
LLM: Bien, tres tipos de información condicional: bonificación (si existe), penalización (si aplica), y fecha de efecto (siempre). ¿El sistema debe ofrecer alternativas a la cancelación, o solo informar?
Experto: Si tiene bonificación acumulada, sí, le ofrecemos alternativas. Si no, solo informamos y pedimos confirmación.
LLM: Entendido: la oferta de alternativas es condicional a la existencia de bonificación. ¿Qué alternativas se ofrecen típicamente?
Experto: Suspensión temporal o cambio de cobertura. Ambas mantienen la bonificación.

10.4 Generación condicionada

Sede principal de este concepto.

10.4.1 El prompt de generación

Una vez completada la elicitación, el LLM recibe un prompt de generación que contiene cuatro componentes, cada uno con una función específica.

La generación de la formalización no es generación libre sino generación fuertemente condicionada por el esquema del DSL, ejemplos del modelo existente, y el resultado de la elicitación.

El primer componente es la *especificación del DSL*: la gramática del formalismo, las construcciones disponibles, y las convenciones de estilo. Esta especificación es lo que el LLM usa para aprender el formalismo por in-context learning. La calidad de la especificación — su claridad, su completitud, sus ejemplos — determina directamente la calidad de la generación. Los trabajos de Min et al. (2022) sobre la naturaleza del

in-context learning sugieren que los ejemplos son más importantes que las instrucciones abstractas: el LLM aprende el patrón de los ejemplos más que las reglas verbales.

El segundo componente son los *ejemplos del modelo existente*: reglas ya formalizadas y aprobadas que muestran el estilo, las convenciones, y el nivel de detalle esperado. Los ejemplos deben seleccionarse por relevancia: para formalizar una regla de bonificación, los ejemplos más útiles son otras reglas de dominio (no flujos procedimentales). La selección de ejemplos puede automatizarse mediante similitud semántica con la descripción a formalizar.

El tercer componente es el *resultado de la elicitación*: la descripción estructurada del conocimiento a formalizar, con todos los campos del esquema completados.

El cuarto componente es el *modelo de conocimiento actual*: las entidades, relaciones, y reglas ya definidas, necesarias para que la nueva formalización sea consistente con el modelo existente. El LLM necesita saber que la entidad `seguro_hogar` ya existe y tiene ciertos atributos para referenciarla correctamente en la nueva regla.

La selección de ejemplos por relevancia semántica mejora significativamente la calidad de la generación.

10.4.2 Técnicas de generación por nivel de restricción

El espectro de restricción de la **UD 7** se aplica directamente a la generación de formalizaciones. Cada nivel ofrece un balance diferente entre flexibilidad y garantía.

Nivel 2: Structured output con JSON Schema

En la Phase 1, la técnica principal es la generación con structured output: el LLM genera la formalización como un objeto JSON que debe conformar a un JSON Schema predefinido. Las APIs de los principales proveedores (Structured Outputs de OpenAI, tool use de Anthropic) garantizan la conformidad sintáctica del JSON al schema. Esto elimina los errores de formato (campos faltantes, tipos incorrectos, estructura malformada) que son frecuentes en la generación libre.

La limitación es que el JSON Schema captura la estructura pero no todas las restricciones semánticas. El schema puede especificar que el campo `condiciones` es un array de objetos con campos `variable`, `operador`, y `valor`, pero no puede especificar que la variable debe ser una entidad definida en la ontología o que el operador debe ser compatible con el tipo de la variable.

Nivel 3: Constrained decoding con gramática del DSL

En la Phase 2, la generación puede usar constrained decoding con la gramática completa del DSL ACAI (no solo el JSON Schema). Willard y Louf (2023) con Outlines demostraron que esto es práctico: la gramática del DSL se convierte en un autómata que filtra los tokens del LLM en cada paso, garantizando que la salida es una expresión válida del DSL.

La ventaja sobre el JSON Schema es que las restricciones de la gramática son más expresivas: pueden capturar relaciones entre campos (si el tipo de acto es `solicitar`, el campo `objeto` es obligatorio), vocabularios controlados (el valor de `producto` debe ser uno de los tipos definidos), y estructuras de anidamiento (un bloque UNLESS solo puede aparecer dentro de un bloque MUST o MUST NOT).

Nivel 4: Reasoning loop con validación semántica

El nivel más alto añade la iteración entre generación y validación. El LLM genera una formalización (con nivel 2 o 3); el validador simbólico la evalúa contra el modelo existente; si detecta errores semánticos, el feedback estructurado se inyecta en un nuevo prompt de generación. El ciclo continúa hasta que la validación es exitosa o se alcanza un límite de iteraciones.

> **Generación con reasoning loop**
>
> **Iteración 1:** El LLM genera la regla de congelación de bonificación. El validador detecta: la regla referencia `producto = hogar` pero la ontología define `producto = seguro_hogar`. Feedback: «`hogar` no es un valor válido para `producto`; valores disponibles: `seguro_hogar`, `seguro_auto`, `seguro_vida`.»
> **Iteración 2:** El LLM corrige el vocabulario. El validador detecta: la regla define congelación de 24 meses pero no especifica la condición de reactivación. Feedback: «La regla define el período de congelación pero no la acción cuando el cliente contrata una nueva póliza dentro del período. Reglas relacionadas: `reactivacion_bonificacion` define condiciones de reactivación.»
> **Iteración 3:** El LLM añade la referencia a la regla de reactivación. Validación exitosa: la regla es sintácticamente correcta, consistente con la ontología, y completa respecto a las reglas relacionadas.

10.5 Validación simbólica

Sede principal de este concepto.

10.5.1 Las tres dimensiones de la validación

La validación opera en tres dimensiones: corrección sintáctica, consistencia con el modelo, y completitud local. Cada dimensión detecta tipos de errores diferentes.

El validador simbólico es el componente que cierra el bucle entre generación neuronal y corrección simbólica. Su función es detectar errores que el LLM no puede detectar por sí mismo porque requieren evaluación contra un modelo formal externo.

La *corrección sintáctica* verifica que la representación generada es una expresión válida de la gramática del DSL. Si se usa constrained decoding (nivel 3), esta verificación es redundante: la corrección sintáctica está garantizada por construcción. Si se usa structured output (nivel 2), la corrección del formato JSON está garantizada pero la corrección respecto a la gramática del DSL (que es más restrictiva que el JSON Schema) necesita verificación adicional.

La *consistencia con el modelo* verifica que la representación no contradice el conocimiento ya formalizado. Las verificaciones incluyen: las entidades referenciadas existen en la ontología, los atributos usados son atributos definidos para esas entidades, los tipos de los valores son compatibles con los tipos de los atributos, las normas no contradicen normas existentes de mayor prioridad sin una excepción explícita, y las transiciones de los flujos referencian estados definidos.

La *completitud local* verifica que la representación no omite elementos que el contexto sugiere que deberían estar presentes. Si una regla define condiciones de cancelación para `seguro_hogar`, y el modelo ya tiene reglas equivalentes para `seguro_auto` y `seguro_vida`, la completitud local señala que podría faltar una generalización o una diferenciación explícita. La completitud local no es un error sino una advertencia que el experto evalúa.

10.5.2 El feedback estructurado

El feedback estructurado al LLM es específico, accionable, y contextual — no genérico.

La calidad del reasoning loop depende de la calidad del feedback que el validador proporciona al LLM. El feedback debe ser *específico* (indicar exactamente qué es incorrecto), *accionable* (indicar qué se espera en su lugar), y *contextual* (proporcionar la información necesaria para la corrección).

Un feedback genérico — «la regla es incorrecta» — no ayuda al LLM. Un feedback específico — «la condición `client.age > 65` referencia el atributo `age` que no está definido para la entidad `client`; los atributos disponibles son: `tenure`, `risk_profile`, `claims_history`, `birth_date`; nota: la edad puede derivarse de `birth_date`» — permite al LLM corregir con alta probabilidad.

El diseño del formato de feedback es, por tanto, una tarea de ingeniería de prompts. Los trabajos sobre self-refinement (Madaan et al., 2023) y

sobre corrección iterativa con feedback externo (Chen et al., 2024) muestran que la especificidad del feedback es el factor más determinante de la calidad de la corrección.

Madaan et al. (2023) — Self-Refine: refinamiento iterativo con autofeedback.

10.5.3 Convergencia del reasoning loop

Una pregunta práctica es si el reasoning loop converge: ¿el LLM produce una formalización válida en un número acotado de iteraciones? La respuesta depende de la complejidad de la formalización y de la calidad del feedback.

> **Convergencia empírica**
>
> La experiencia con text-to-code sugiere que los reasoning loops con feedback específico convergen típicamente en 2–4 iteraciones para tareas de complejidad moderada (Chen et al., 2021, pass@k analysis). Para formalizaciones complejas (reglas con múltiples condiciones, excepciones anidadas, referencias cruzadas), la convergencia puede requerir más iteraciones o puede no producirse, en cuyo caso el pipeline escala a intervención humana.
>
> La Phase 0 del sprint incluye experimentos específicos para medir la tasa de convergencia del reasoning loop sobre el dominio asegurador: ¿en cuántas iteraciones converge? ¿Qué tipos de errores persisten? ¿Cuál es la tasa de éxito por tipo de formalización?

10.6 Revisión humana y verbalización

Sede principal de este concepto.

10.6.1 La verbalización como puente

El paso 5 del pipeline — la revisión humana — es donde la restricción de legibilidad se pone a prueba. El experto debe poder verificar que la formalización captura su conocimiento correctamente. Hay dos mecanismos complementarios para esto.

La verbalización traduce la representación formal de vuelta a lenguaje natural, permitiendo al experto verificar la fidelidad de la formalización sin necesidad de leer el formalismo directamente.

El primer mecanismo es la *lectura directa*: gracias al principio D1 (cercanía al lenguaje natural controlado), el experto debería poder leer las reglas del DSL y entenderlas. Una regla como «`WHEN client.policy IS ACTIVE AND client.policy.tenure > 2 years THEN ...`» es razonablemente legible para un profesional del sector.

El segundo mecanismo es la *verbalización*: el LLM genera una descripción en lenguaje natural de la formalización, que el experto compara

con su intención original. La verbalización es un cruce Formal→NL (como los descritos en la sección 3 de la **UD 7**) que produce texto como: «Cuando un cliente con una póliza activa y más de 2 años de antigüedad cancela, si no ha tenido siniestros en los últimos 3 años, su bonificación acumulada se congela durante 24 meses.»

La verbalización ofrece una ventaja que la lectura directa no: permite al experto comparar la formalización con su descripción *original* y detectar discrepancias semánticas que la corrección sintáctica y la consistencia del validador no pueden capturar. Si el experto dijo «más de dos años» y la formalización dice «`>= 2`», la verbalización genera «dos años o más» y el experto detecta la discrepancia (¿incluye exactamente 2 años o no?).

10.6.2 Protocolos de revisión

La revisión humana produce tres resultados posibles. La *aprobación*: la formalización es correcta y se incorpora al modelo. La *modificación*: el experto indica correcciones que se inyectan como feedback adicional en el pipeline (retorno al paso 2 con las correcciones del experto). El *rechazo*: la formalización es fundamentalmente incorrecta y requiere repetir la elicitación.

La revisión humana es el punto de control de calidad donde la autoridad del conocimiento vuelve al experto.

Para que la revisión sea eficiente, el protocolo debe presentar al experto: la regla formalizada en la notación del DSL (lectura directa), su verbalización en lenguaje natural (comprensión rápida), las reglas relacionadas en el modelo existente (contexto), y las advertencias de completitud local del validador (señales de posibles omisiones). La presentación debe ser sucinta: el experto no debe tener que leer todo el modelo para evaluar una regla.

10.6.3 El ciclo completo

Ciclo completo de formalización — regla de penalización

Paso 1 (elicitación):
LLM: ¿Existe penalización por cancelación anticipada? ¿En qué condiciones?
Experto: Sí. Si la póliza tiene menos de un año, se cobra el 15 % de la prima anual. Entre uno y dos años, el 8 %. Más de dos años, sin penalización.
LLM: ¿Aplica a todos los productos? ¿Hay excepciones?
Experto: Solo hogar y auto. Vida no tiene penalización. Y si la cancelación es por cambio de compañía aseguradora tras un siniestro mal gestionado por nosotros, se exime de penalización independientemente del plazo.
Paso 2 (generación): LLM genera tres reglas: penalización para < 1 año, para 1–2 años, y la excepción por siniestro mal gestionado. Formato DSL

ACAI con condiciones, porcentajes, y cláusula UNLESS.
Paso 3 (validación): Sintaxis correcta. Consistencia: las entidades `seguro_hogar` y `seguro_auto` existen. Completitud: el validador señala que la regla no especifica qué ocurre si la cancelación se produce exactamente al cumplir 1 año (¿15 % o 8 %?).
Paso 4 (corrección): Feedback al LLM sobre el caso frontera. LLM regenera con condición «`tenure < 1 year`» (estricto) para el 15 % y «`tenure >= 1 year AND tenure < 2 years`» para el 8 %.
Paso 5 (revisión): Verbalización presentada al experto. Experto aprueba las tres reglas pero añade: «Ah, y la exención por siniestro mal gestionado requiere que la reclamación haya sido aceptada por el departamento de calidad.» Retorno al paso 2 con esta condición adicional. LLM regenera la excepción con la condición añadida. Validación exitosa. Experto aprueba.

10.7 Diseño experimental para la Fase 0

10.7.1 Objetivos experimentales

Phase 0
Los experimentos de viabilidad son un entregable directo de esta unidad.

La Phase 0 del sprint necesita responder una pregunta empírica que ningún análisis teórico puede resolver: ¿es viable la formalización asistida por LLM para el dominio y el formalismo de ACAI? La respuesta requiere experimentos controlados que midan las variables relevantes.

La primera pregunta es la *generabilidad sintáctica*: ¿qué proporción de las formalizaciones generadas son sintácticamente correctas en el primer intento, con structured output, y con constrained decoding? Esta pregunta mide la restricción de generabilidad del formalismo.

La segunda pregunta es la *convergencia semántica*: ¿el reasoning loop produce formalizaciones semánticamente correctas? ¿En cuántas iteraciones? ¿Para qué tipos de formalización converge y para cuáles no? Esta pregunta mide la viabilidad del pipeline completo.

La tercera pregunta es la *eficiencia de revisión*: ¿el experto puede verificar las formalizaciones en tiempo razonable? ¿La verbalización ayuda? ¿Cuál es la tasa de errores no detectados por la validación simbólica que el experto detecta? Esta pregunta mide la viabilidad del human-in-the-loop.

Tres preguntas experimentales: ¿el LLM genera formalizaciones sintácticamente correctas? ¿La corrección semántica mejora con el reasoning loop? ¿El experto puede verificar las formalizaciones eficientemente?

10.7.2 Diseño de los experimentos

Experimento 1: Generabilidad sintáctica

Material: un conjunto de 20–30 reglas del dominio asegurador, representativas de las cinco dimensiones (reglas de dominio, pasos procedimentales, normas, componentes de estado, reglas pragmáticas).

Condiciones: generación libre (prompting), structured output (JSON

Schema), constrained decoding (gramática del DSL). Tres LLMs si es posible (al menos dos para comparar).

Métricas: tasa de corrección sintáctica (proporción de generaciones que son expresiones válidas del DSL), distribución de errores sintácticos por tipo, latencia de generación por condición.

Análisis: la comparación entre condiciones mide el valor añadido de cada nivel de restricción. Si la tasa de corrección con prompting es >90 %, el constrained decoding añade poco valor para la sintaxis. Si es <70 %, el constrained decoding es necesario.

Experimento 2: Convergencia semántica

Material: las mismas 20–30 reglas, con una referencia gold-standard formalizada manualmente por el investigador y validada por el experto.

Condiciones: generación directa (sin reasoning loop), reasoning loop con feedback genérico, reasoning loop con feedback específico.

Métricas: tasa de corrección semántica (proporción de formalizaciones que coinciden con el gold standard en todas las condiciones, acciones, y excepciones), número de iteraciones hasta convergencia, distribución de errores semánticos por tipo (entidad incorrecta, condición omitida, excepción faltante, tipo de acto erróneo).

Análisis: la comparación mide el valor añadido del reasoning loop y de la especificidad del feedback. La distribución de errores informa sobre qué aspectos del diseño del DSL y del prompt necesitan refinamiento.

Experimento 3: Eficiencia de revisión

Material: formalizaciones generadas por el pipeline (no gold standard), con errores deliberados insertados en una proporción controlada.

Condiciones: revisión con solo la formalización (lectura directa), revisión con formalización + verbalización, revisión con formalización + verbalización + advertencias del validador.

Métricas: tiempo de revisión por regla, tasa de detección de errores insertados, tasa de falsos positivos (el experto rechaza formalizaciones correctas).

Análisis: mide si la triple presentación (formalización + verbalización + advertencias) es significativamente más eficiente que la lectura directa sola.

10.7.3 Criterios de viabilidad

Umbrales de viabilidad

Los experimentos producen métricas que deben evaluarse contra umbrales de viabilidad definidos a priori. Los umbrales propuestos son:
Generabilidad: tasa de corrección sintáctica $\geq 95\,\%$ con structured output y $\geq 99\,\%$ con constrained decoding. Si no se alcanzan, el diseño del DSL requiere simplificación.
Convergencia: tasa de corrección semántica $\geq 80\,\%$ con reasoning loop en ≤ 4 iteraciones. Si no se alcanza, el pipeline necesita mejoras en el feedback o en los ejemplos de in-context learning.
Eficiencia: tiempo de revisión ≤ 5 minutos por regla con tasa de detección de errores $\geq 90\,\%$. Si no se alcanza, la presentación de revisión necesita rediseño.
Estos umbrales son orientativos y se ajustarán con los primeros resultados. Su función es proporcionar criterios de decisión: ¿avanzar a la Phase 1 con el diseño actual, o iterar sobre el diseño del DSL y el pipeline?

10.8 Modos de fallo y mitigación

10.8.1 Taxonomía de fallos

Seis modos de fallo del pipeline, cada uno con mecanismos de detección y mitigación específicos.

El pipeline de formalización asistida puede fallar de seis formas principales.

El *fallo de elicitación incompleta* ocurre cuando el LLM no formula las preguntas necesarias para obtener una descripción completa. Detección: la validación de completitud local detecta omisiones. Mitigación: checklists de elicitación por dimensión que el LLM debe recorrer.

El *fallo de interpretación* ocurre cuando el LLM malinterpreta la descripción del experto — por ejemplo, interpreta «más de dos años» como ≥ 2 en lugar de > 2. Detección: la verbalización revela la discrepancia si el experto la lee con atención. Mitigación: la elicitación incluye preguntas de clarificación para condiciones frontera.

El *fallo de vocabulario* ocurre cuando el LLM usa identificadores que no están en la ontología. Detección: la validación de consistencia lo detecta directamente. Mitigación: constrained decoding con vocabulario controlado.

El *fallo de cobertura* ocurre cuando la formalización omite condiciones o excepciones que el experto mencionó. Detección: parcialmente por la

validación de completitud local y por la revisión humana. Mitigación: el prompt de generación incluye instrucciones explícitas de incluir todas las condiciones mencionadas en la elicitación.

El *fallo de omisión implícita* ocurre cuando el conocimiento necesario no se elicita porque el experto lo da por supuesto. El experto no menciona que la verificación de identidad es necesaria porque es obvia para él. Detección: difícil; parcialmente por la completitud local si el modelo tiene reglas relacionadas que incluyen la verificación. Mitigación: checklists de dominio que incluyen los elementos que los expertos suelen omitir.

El fallo más difícil de detectar es el de omisión implícita: lo que el experto no dice porque lo da por supuesto.

El *fallo de no-convergencia* ocurre cuando el reasoning loop no produce una formalización válida tras el número máximo de iteraciones. Detección: por definición, al alcanzar el límite de iteraciones. Mitigación: escalado a intervención humana directa (el investigador interviene en la formalización).

10.8.2 El principio de detección temprana

El diseño del pipeline sigue un principio de *detección temprana*: los fallos deben detectarse lo antes posible en el pipeline, porque el coste de corrección crece con la distancia al punto de fallo. Un error de elicitación detectado en el paso 1 se corrige con una pregunta adicional; el mismo error detectado en el paso 5 (por el experto) requiere repetir todo el ciclo.

Las defensas se estratifican: el esquema de elicitación previene las omisiones más comunes, el constrained decoding previene los errores sintácticos, la validación simbólica previene los errores de consistencia, el reasoning loop corrige los errores semánticos detectables, y la revisión humana es la red de seguridad final.

> **Conexiones con otras unidades**
>
> → **Prerequisito: UD 7** (Técnicas neuro-simbólicas) para el espectro de restricción, las técnicas de structured output, constrained decoding, y reasoning loop que el pipeline usa directamente. **UD 8** (Dimensiones) para la instanciación del pipeline por dimensión. **UD 9** (Formalismos) para los principios de diseño del DSL que condicionan la generabilidad.
>
> ⇄ **Complementa: UD 5** (Fundamentos) estableció que el LLM como meta-herramienta (I2) — generador de las representaciones que luego lo gobiernan — es una innovación central de ACAI. Esta unidad es la concreción operativa de I2.
>
> ⇄ **Complementa: UD 3** (Ecosistema) definió la formalización asis-

tida como una de las cuatro capacidades del ecosistema. Esta unidad la desarrolla como pipeline implementable.

Aplicación: UD 11 (La triple restricción) analiza las tensiones entre legibilidad, procesabilidad y generabilidad que los experimentos de esta unidad miden empíricamente.

Aplicación: UD 12 (Métricas) definirá las métricas de evaluación del modelo formal que incluyen las métricas de calidad de la formalización propuestas aquí como parte de su framework.

Ejercicios

1. [Diseño de elicitación] Diseñe el esquema de elicitación completo para la dimensión normativa del dominio asegurador. El esquema debe incluir: (a) las preguntas obligatorias (fuente de la norma, tipo de modalidad, condiciones de aplicación, excepciones, prioridad), (b) las preguntas de seguimiento condicionadas (si hay excepción, ¿bajo qué condiciones?), y (c) al menos un ejemplo completo de diálogo de elicitación para una norma regulatoria específica (por ejemplo, la obligación de informar sobre la bonificación antes de cancelar).

2. [Prompt de generación] Diseñe el prompt completo de generación para una regla de dominio del DSL ACAI. El prompt debe incluir: (a) la especificación del sublenguaje de dominio del DSL (construcciones disponibles, sintaxis), (b) tres ejemplos de reglas formalizadas, (c) el resultado de la elicitación para la regla a formalizar, y (d) un fragmento del modelo de conocimiento actual. Pruebe el prompt con un LLM y evalúe la calidad de la generación.

3. [Diseño de validador] Especifique las verificaciones que el validador simbólico debe realizar para una regla normativa (dimensión 3). Para cada verificación, especifique: (a) qué comprueba, (b) cómo detecta el error, (c) qué feedback genera para el LLM, y (d) un ejemplo de error y su feedback. Incluya al menos: verificación de vocabulario, verificación de prioridades, verificación de ámbito, y verificación de excepciones.

4. [Experimento piloto] Seleccione 5 reglas del dominio asegurador (una por dimensión) y ejecute un experimento piloto del pipeline completo usando un LLM disponible. Para cada regla: (a) ejecute la elicitación (usted como experto simulado), (b) genere la formalización, (c) evalúe la corrección sintáctica y semántica, (d) aplique corrección iterativa si es necesario. Documente: número de itera-

ciones, tipos de errores, y calidad final percibida.

5. [Análisis de fallos] Tome la regla de penalización del escenario de esta unidad e identifique los seis modos de fallo en ese caso concreto. Para cada modo: (a) describa cómo se manifestaría (ejemplo concreto de error), (b) en qué paso del pipeline se detectaría, y (c) cómo se mitigaría. ¿Hay algún modo de fallo que no se detectaría con los mecanismos descritos?

Referencias y lecturas

Lectura de entrada

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K. M., Welleck, S., Yazdanbakhsh, A. & Clark, P. (2023). Self-Refine: Iterative Refinement with Self-Feedback. *NeurIPS 2023*. Demuestra que los LLMs pueden mejorar iterativamente sus salidas cuando reciben feedback estructurado. Directamente relevante para el reasoning loop del pipeline de formalización. Lectura de entrada que fundamenta el paso 4.

Lecturas esenciales

- Chen, M. et al. (2021). Evaluating Large Language Models Trained on Code. *ArXiv preprint arXiv:2107.03374*. — Codex. Ya citado en UD7. Aquí relevante por los datos de pass@k que informan las expectativas de convergencia del reasoning loop.
- Willard, B. T. & Louf, R. (2023). Efficient Guided Generation for Large Language Models. *ArXiv preprint arXiv:2307.09702*. — Outlines. Ya citado en UD6 y UD7. Aquí esencial como implementación del constrained decoding en el paso 2.
- Min, S. et al. (2022). Rethinking the Role of Demonstrations. *EMNLP 2022*. — Ya citado en UD7. Aquí relevante para el diseño del prompt de generación.
- Feigenbaum, E. A. (1977). The Art of Artificial Intelligence: Themes and Case Studies of Knowledge Engineering. *IJCAI 1977*. — El diagnóstico original del knowledge engineering bottleneck. Fundacional para el argumento de esta unidad.

Lecturas complementarias

- Hoffman, R. R., Shadbolt, N. R., Burton, A. M. & Klein, G. (1995). Eliciting Knowledge from Experts: A Methodological Analysis. *Organizational Behavior and Human Decision Processes*, 62(2), 129–158. — Técnicas de elicitación de conocimiento: taxonomía.

- Chen, X., Lin, M., Schärli, N. & Zhou, D. (2024). Teaching Large Language Models to Self-Debug. *ICLR 2024*. — Corrección iterativa con feedback externo para generación de código. Directamente relevante para el reasoning loop.
- Jain, N., Vaidyanath, S., Iyer, A., Natarajan, N., Parthasarathy, S., Rajamani, S. & Sharma, R. (2022). Jigsaw: Large Language Models Meet Program Synthesis. *ICSE 2022*. — Síntesis de programas con LLMs y validación. Pipeline análogo al de ACAI.
- Poesia, G., Polozov, O., Le, V., Tiwari, A., Soares, G., Meek, C. & Gulwani, S. (2022). Synchromesh: Reliable Code Generation from Pre-trained Language Models. *ICLR 2022*. — Constrained decoding para generación de código. Ya citado en UD6; aquí relevante para la generación de formalizaciones.
- Wagner, G., Giese, M. & Lukichev, S. (2006). A Usable Knowledge Representation for Business Rules. *RuleML 2006*. — Diseño de lenguajes de reglas legibles para no técnicos. Relevante para D1.
- Musen, M. A. (2015). The Protégé Project: A Look Back and a Look Forward. *AI Matters*, 1(4). — Protégé como ejemplo de herramienta de formalización asistida (en ontologías). Relevante como precedente histórico.
- Larsson, S. & Traum, D. (2000). Information state and dialogue management in the TRINDI dialogue move engine toolkit. *Natural Language Engineering*, 6(3–4), 323–340. — Ya citado múltiples veces. Aquí relevante como ejemplo de formalización manual de information state cuyo coste motiva la formalización asistida.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«A formalism optimized for one reader is a formalism hostile to the other two. The art is in the negotiation.»

Eje C — Representación y formalización

Unidad 11 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje C — Representación y formalización

Prerequisitos: UD7, UD9, UD10 · Eje C — Representación y formalización

UNIDAD DIDÁCTICA 11

La triple restricción

Contenidos de la unidad

11.1.1 Tres lectores, un formalismo 3
11.1.2 Las tensiones como gradientes 4
11.2.1 La naturaleza de la tensión 5
11.2.2 Estrategias de resolución 5
Doble representación 5
Lenguaje natural controlado 6
11.3.1 La naturaleza de la tensión 6
11.3.2 Estrategias de resolución 7
Forma canónica con verbalización 7
Normalización post-generación 7
Diseño de gramáticas LLM-friendly 7
11.4.1 La naturaleza de la tensión 8
11.4.2 Estrategias de resolución 8
Gramática context-free + validación semántica post-generación 9
Inyección de contexto en el prompt 9
Vocabulario controlado con autocompletado 9
11.5.1 El triángulo de la triple restricción 9
11.5.2 Factores que condicionan la posición 10
11.6.1 El diseño en capas 11
11.6.2 Flujo de las capas 11
11.6.3 Evaluación de la resolución 12
11.7.1 Métricas por restricción 12
11.7.2 Experimentos de diseño comparativo 13

¿Por qué necesito esto para ACAI?

La triple restricción — legibilidad, procesabilidad, generabilidad — ha aparecido en múltiples unidades como meta-requisito transversal del formalismo ACAI. La UD 5 la introdujo como innovación conceptual (I1), la UD 7 la concretó como desafío técnico, la UD 9 la usó como criterio de evaluación de formalismos, y la UD 10 diseñó experimentos para medirla. Pero ninguna de esas unidades analizó en profundidad las *tensiones* entre las tres restricciones: los puntos donde optimizar para una degrada las otras, los mecanismos de resolución disponibles, y las implicaciones para el diseño del formalismo y del sprint. Esta unidad cierra el Eje C con ese análisis, proporcionando al investigador un mapa del espacio de diseño con sus compromisos explícitos.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Analizar las tensiones entre legibilidad, procesabilidad y generabilidad como un sistema de trade-offs interdependientes.
2. Identificar las tensiones bilaterales (L–P, L–G, P–G) y la tensión trilateral, con ejemplos concretos en el dominio ACAI.
3. Aplicar las estrategias de resolución disponibles: partición de audiencias, doble representación, diseño de gramáticas LLM-friendly, y restricción dinámica del espacio de diseño.
4. Evaluar decisiones de diseño del formalismo en términos de su impacto sobre las tres restricciones simultáneamente.
5. Formular un programa experimental para navegar el espacio de diseño de la triple restricción durante el sprint.

11.1 La triple restricción como sistema de tensiones

Sede principal de este concepto.

11.1.1 Tres lectores, un formalismo

La triple restricción puede reformularse con precisión como un problema de diseño para múltiples audiencias. El formalismo ACAI tiene tres «lectores» que lo procesan con capacidades y necesidades diferentes.

El *experto humano* lee el formalismo para verificar que captura su conocimiento correctamente. Necesita sintaxis transparente, palabras clave inteligibles, y una estructura que se corresponda con su modelo mental del dominio. Procesa el formalismo secuencialmente, con me-

La triple restricción exige que un mismo formalismo sea simultáneamente óptimo para tres lectores con capacidades radicalmente diferentes: el experto humano, el motor simbólico, y el LLM.

moria de trabajo limitada, y se apoya en redundancia y contexto para la comprensión. Un formalismo optimizado para este lector sería verbose, redundante, cercano al lenguaje natural, con nombres largos y descriptivos.

El *motor simbólico* lee el formalismo para ejecutarlo determinísticamente. Necesita una gramática formal sin ambigüedad, una semántica operacional definida, y una estructura que permita inferencia eficiente. Procesa el formalismo por parsing sintáctico y evaluación semántica, sin necesidad de redundancia (que es ruido) ni de cercanía al lenguaje natural (que es fuente de ambigüedad). Un formalismo optimizado para este lector sería compacto, preciso, con notación formal y sin redundancia.

El *LLM* lee el formalismo para aprender a generarlo (in-context learning) y lo genera token a token. Necesita patrones regulares y predecibles, un vocabulario que se alinee con su distribución de tokens, y una estructura que sea compatible con la generación autoregresiva. Procesa el formalismo como una secuencia de tokens con dependencias estadísticas, sin comprensión semántica profunda. Un formalismo optimizado para este lector sería regular, con tokens frecuentes en los datos de entrenamiento, y con dependencias locales (no de largo alcance).

Cada lector tiene necesidades que, llevadas al extremo, contradicen las de los otros dos.

El problema es que las necesidades de los tres lectores, llevadas al extremo, son incompatibles. La redundancia que el humano necesita es ruido para el motor y distractor para el LLM. La compacidad que el motor prefiere es opacidad para el humano e irregularidad para el LLM. La regularidad que el LLM necesita puede producir formalismos rígidos que limitan la expresividad para el motor y la naturalidad para el humano.

11.1.2 Las tensiones como gradientes

Las tensiones no son binarias sino gradientes: cada decisión de diseño del formalismo se sitúa en algún punto de un espacio tridimensional donde las tres restricciones ejercen fuerzas en direcciones parcialmente opuestas. El objetivo no es eliminar las tensiones — son inherentes al problema — sino navegar el espacio de diseño con consciencia explícita de los compromisos.

Las secciones siguientes analizan las tres tensiones bilaterales y la tensión trilateral, con ejemplos concretos del dominio ACAI y estrategias de resolución para cada una.

11.2 Tensión L–P: legibilidad versus procesabilidad

Sede principal de este concepto.

11.2.1 La naturaleza de la tensión

La tensión L–P es la más antigua en el diseño de formalismos: lo que el humano lee con facilidad es frecuentemente ambiguo para la máquina, y lo que la máquina procesa sin ambigüedad es frecuentemente opaco para el humano.

La tensión entre legibilidad y procesabilidad es clásica en ciencias de la computación. Los lenguajes de programación de alto nivel la ilustran: Python es más legible que C, que es más legible que ensamblador, y en cada paso la legibilidad se gana a costa de abstracciones que el compilador debe resolver (y que pueden introducir ambigüedades o ineficiencias). Los formalismos lógicos la ilustran aún más: $\forall x.P(x) \rightarrow Q(x)$ es procesable pero ilegible para un no lógico; «todo lo que es P también es Q» es legible pero ambiguo (¿«todo» incluye los casos vacuos?).

Para ACAI, la tensión se manifiesta en decisiones concretas de diseño del DSL.

Nombres de identificadores. La legibilidad favorece nombres largos y descriptivos: `bonificacion_por_ausencia_de_siniestros_en_los_ultimos_tres_anios`. La procesabilidad favorece identificadores compactos y normalizados: `bonus_no_claims_3y`. Los primeros son autoexplicativos pero difíciles de parsear y propensos a errores tipográficos en la generación. Los segundos son eficientes pero opacos sin un diccionario.

Estructura de las condiciones. La legibilidad favorece condiciones cercanas al lenguaje natural: «`WHEN the client has been insured for more than 2 years`». La procesabilidad favorece condiciones formales: «`WHEN client.tenure > 2y`». Las primeras son transparentes pero requieren un parser sofisticado para extraer la semántica; las segundas son directamente evaluables pero requieren conocer el esquema de atributos.

Manejo de excepciones. La legibilidad favorece la expresión de excepciones en contexto: «`MUST inform about bonus UNLESS client has explicitly declined this information`». La procesabilidad favorece la separación de la regla base y sus excepciones en definiciones independientes, lo que permite la evaluación modular y la verificación de consistencia. Las primeras son legibles pero dificultan el razonamiento defeasible automático; las segundas son verificables pero menos legibles porque la lógica de la excepción se separa de la regla que modifica.

11.2.2 Estrategias de resolución

Doble representación

La estrategia más directa, anticipada en la **UD 9**, es la **doble representación**: mantener una notación legible para el humano y una serialización procesable para el motor, con transformaciones deterministas entre ambas. El humano ve «`WHEN client tenure exceeds 2 years`»;

el motor procesa `{çondition": {"field": çlient.tenure", .ºp": »", "value": "2y"}}`. La equivalencia entre ambas se garantiza por construcción: la notación legible se genera a partir de la serialización (verbalización) y la serialización se genera a partir de la notación legible (compilación).

La doble representación tiene un coste: mantener la consistencia entre las dos formas requiere transformaciones verificadas, y cualquier extensión del formalismo debe extender ambas representaciones. Pero el coste es manejable si la transformación es una función del diseño del lenguaje, no una traducción ad hoc.

La doble representación no es redundancia — es separación de intereses: cada representación está optimizada para su lector.

Lenguaje natural controlado

Una estrategia intermedia es diseñar la notación primaria como un **lenguaje natural controlado** (CNL): un subconjunto del lenguaje natural con gramática formal definida. Los CNLs como Attempto Controlled English (Fuchs et al., 2008) demuestran que es posible diseñar lenguajes que son simultáneamente legibles por humanos y parseables por máquinas. La limitación es que el CNL debe ser suficientemente restrictivo para ser parseable (lo que limita la naturalidad) y suficientemente expresivo para capturar el conocimiento del dominio.

Para ACAI, un CNL parcial — donde las condiciones y acciones se expresan en una sintaxis controlada pero los nombres de entidades y atributos se toman del vocabulario del dominio — podría reducir la tensión L–P sin resolverla completamente.

Fuchs et al. (2008) — Attempto Controlled English: la demostración más madura de que un CNL puede ser simultáneamente legible y parseable.

11.3 Tensión L–G: legibilidad versus generabilidad

Sede principal de este concepto.

11.3.1 La naturaleza de la tensión

La tensión L–G es específica de ACAI y no tiene precedentes extensos en la literatura porque la generabilidad por LLM es un requisito nuevo. La tensión se manifiesta en la relación entre variabilidad y fiabilidad.

La legibilidad se beneficia de la variabilidad expresiva: un formalismo donde la misma idea puede expresarse de formas ligeramente diferentes es más natural para el lector humano. «`WHEN client.tenure > 2 years`» y «`WHEN the client has been a member for over 2 years`» son variantes que un humano procesa sin dificultad. Pero para el LLM, cada variante es una distribución de probabilidad diferente sobre los tokens de salida. Si el LLM ha visto la primera forma en los ejemplos de in-context learning, puede generar la segunda como va-

Lo que el humano lee con facilidad (expresiones variadas, vocabulario rico, estructura flexible) es lo que el LLM genera con menos fiabilidad. Lo que el LLM genera fiablemente (patrones repetitivos, vocabulario acotado) puede resultar rígido o artificial para el humano.

riante "natural" pero con riesgo de que la variación introduzca un error sutil («over 2 years» es lo mismo que «more than 2 years»? ¿Incluye exactamente 2 años?).

La generabilidad se beneficia de la regularidad estricta: un formalismo con una sola forma canónica para cada construcción reduce la incertidumbre del LLM. Si la forma canónica de una condición temporal es siempre `entity.attribute op value unit`, el LLM aprende este patrón y lo reproduce con alta fiabilidad. Pero la regularidad estricta puede producir formalizaciones que parecen rígidas o artificiales al lector humano.

11.3.2 Estrategias de resolución

Forma canónica con verbalización

La resolución más directa es la **forma canónica con verbalización**: el LLM genera siempre la forma canónica (regular, predecible) y la verbalización para revisión humana produce una forma más natural. El LLM genera «`WHEN client.policy.tenure > 2 YEARS AND client.policy.claims_count(LAST 3 YEARS) = 0`» (forma canónica); la verbalización produce «Cuando el cliente lleva más de 2 años asegurado y no ha tenido siniestros en los últimos 3 años» (forma legible). El experto revisa la verbalización y, si es correcta, la forma canónica se incorpora al modelo.

Normalización post-generación

Una estrategia alternativa permite cierta flexibilidad en la generación y la normaliza después. Si el LLM genera «`WHEN the client has no claims in 3 years`», un normalizador transforma esta expresión a la forma canónica «`WHEN client.policy.claims_count(LAST 3 YEARS) = 0`». La normalización es posible si la gramática del DSL define equivalencias entre formas y un parser puede reconocer las variantes. El riesgo es que la normalización sea incorrecta (la variante libre es ambigua y el normalizador elige una interpretación).

La normalización es la transformación de una expresión libre a su forma canónica, preservando la semántica.

Diseño de gramáticas LLM-friendly

Una estrategia proactiva consiste en diseñar la gramática del DSL teniendo en cuenta la distribución de tokens del LLM. Los tokens que son frecuentes en los datos de entrenamiento del LLM (WHEN, IF, THEN, AND, OR, NOT) se generan con más fiabilidad que tokens raros (operadores simbólicos, palabras clave inventadas). Un DSL cuyas palabras clave son tokens frecuentes del inglés (o del español) será más generable que uno con palabras clave inventadas.

Impacto de la elección de palabras clave

Palabras clave alineadas con tokens frecuentes:

```
RULE name WHEN condition THEN action NOTE
description
```

El LLM genera cada palabra clave como un token individual con alta probabilidad. Los errores de generación son raros.

Palabras clave inventadas:

```
DEF-REGLA name CONDICION condition EFECTO action
GLOSA description
```

El LLM debe generar secuencias de tokens menos frecuentes (DEF-REGLA puede requerir 3 tokens, CONDICION puede generar CONDICIÓN con tilde, GLOSA es un token raro). Los errores de generación son más frecuentes.

El primer diseño es LLM-friendly; el segundo no. Ambos pueden ser igualmente legibles y procesables, pero la generabilidad difiere significativamente.

11.4 Tensión P–G: procesabilidad versus generabilidad

Sede principal de este concepto.

11.4.1 La naturaleza de la tensión

La procesabilidad requiere que el formalismo capture restricciones semánticas que van más allá de la sintaxis: que las entidades referenciadas existan, que los tipos sean compatibles, que las condiciones sean evaluables, que las normas no se contradigan. Estas restricciones hacen la gramática *context-sensitive*: la validez de una expresión depende del contexto (el modelo de conocimiento actual).

La procesabilidad demanda una gramática expresiva con restricciones semánticas; la generabilidad demanda una gramática simple con patrones predecibles. Las restricciones semánticas que garantizan la corrección son las que el LLM tiene más dificultad para respetar.

La generabilidad se beneficia de gramáticas *context-free* o incluso regulares: la validez de una expresión depende solo de su estructura local, no del contexto global. El constrained decoding funciona eficientemente con gramáticas context-free (Willard y Louf, 2023); las gramáticas context-sensitive requieren verificación adicional que no puede hacerse token a token.

La tensión se concreta en que las restricciones que hacen el formalismo más correcto (referencias cruzadas, compatibilidad de tipos, consistencia con el modelo) son precisamente las que el LLM no puede respetar durante la generación porque requieren información global que no está disponible en la generación autoregresiva local.

11.4.2 Estrategias de resolución

Gramática context-free + validación semántica post-generación

La resolución estándar, ya descrita en la **UD 10**, es dividir la verificación en dos fases: el constrained decoding garantiza la corrección context-free (sintáctica) durante la generación, y la validación simbólica verifica las restricciones context-sensitive (semánticas) después de la generación. Esta división es limpia pero implica que la generación puede producir expresiones sintácticamente válidas pero semánticamente incorrectas, que el reasoning loop debe corregir.

Inyección de contexto en el prompt

Una estrategia complementaria es inyectar las restricciones context-sensitive como parte del prompt del LLM: las entidades disponibles, los atributos de cada entidad, las normas existentes. El LLM no garantiza que respetará estas restricciones, pero la inyección de contexto reduce significativamente la tasa de errores semánticos. Los trabajos sobre retrieval-augmented generation (Lewis et al., 2020) muestran que la inyección de conocimiento relevante en el prompt mejora la factualidad de la generación.

Inyectar contexto en el prompt reduce los errores semánticos pero no los elimina. La validación simbólica sigue siendo necesaria.

Vocabulario controlado con autocompletado

Una estrategia más agresiva es restringir el vocabulario del DSL a un conjunto cerrado que se proporciona al LLM como parte de las restricciones de generación. Si los nombres de entidades son un vocabulario finito, el constrained decoding puede restringir la generación a ese vocabulario, eliminando los errores de referencia a entidades inexistentes. Zheng et al. (2023) con SGLang demostraron que es posible alternar generación libre y generación restringida dentro de la misma secuencia, lo que permite que el LLM genere la estructura libremente pero los valores referenciados se seleccionen de un vocabulario cerrado.

11.5 La tensión trilateral y el espacio de diseño

11.5.1 El triángulo de la triple restricción

Las tres tensiones bilaterales se combinan en una tensión trilateral que define un espacio de diseño tridimensional. Cada punto del espacio corresponde a un diseño de formalismo con un perfil específico en las tres restricciones. Los vértices del triángulo representan los extremos:

El vértice de *máxima legibilidad* es el lenguaje natural puro: perfectamente legible, pero no procesable determinísticamente y generado con demasiada variabilidad para ser fiable. El vértice de *máxima procesabili-*

Tres extremos del espacio de diseño: el lenguaje natural (máxima legibilidad, mínima procesabilidad), la lógica formal (máxima procesabilidad, mínima legibilidad), y el JSON Schema (máxima generabilidad, mínima expresividad). ACAI necesita

dad es la lógica formal: procesable con precisión pero ilegible para no lógicos y difícilmente generable por LLMs. El vértice de *máxima generabilidad* es un formato regular minimalista (JSON con esquema estricto): generable con fiabilidad pero ni legible ni expresivo.

ACAI necesita un punto interior del triángulo — un diseño que sacrifica algo de cada extremo para obtener un balance viable de las tres propiedades. La pregunta no es «cuál es el punto óptimo» (no existe un óptimo universal) sino «cuál es el punto adecuado para el dominio, las técnicas, y los objetivos de ACAI».

11.5.2 Factores que condicionan la posición

El punto adecuado depende de factores específicos del proyecto.

El *perfil del experto* condiciona cuánta legibilidad es necesaria. Un experto de dominio sin formación técnica necesita máxima legibilidad; un experto con formación en lógica de negocio puede tolerar más formalidad. Para ACAI, el experto es un profesional del sector asegurador — legibilidad alta es necesaria pero no extrema: las notaciones tipo «`WHEN ... THEN`» son accesibles.

La *complejidad del dominio* condiciona cuánta procesabilidad es necesaria. Un dominio con reglas simples puede tolerar un formalismo menos formal; un dominio con reglas complejas, excepciones anidadas, y prioridades entre normas necesita un formalismo más formal para garantizar la corrección. Para ACAI, el dominio asegurador tiene complejidad media-alta: procesabilidad alta es necesaria.

La *capacidad del LLM* condiciona cuánta generabilidad es alcanzable. Los LLMs más potentes toleran formalismos más irregulares; los menos potentes necesitan formalismos más regulares. Las mejoras futuras en los LLMs relajarán la restricción de generabilidad, lo que sugiere diseñar el formalismo con la procesabilidad y la legibilidad como restricciones estables y la generabilidad como restricción que se relajará con el tiempo.

> **La generabilidad como restricción temporal**
>
> La legibilidad y la procesabilidad son propiedades del formalismo que no cambian con el tiempo: un formalismo legible hoy será legible en 5 años, y uno procesable hoy será procesable en 5 años. La generabilidad, en cambio, es una propiedad de la *interacción* entre el formalismo y el LLM, y los LLMs mejoran rápidamente. Un formalismo que hoy requiere constrained decoding para ser generable puede ser generable por prompting simple con un LLM de la próxima generación.
>
> Esto sugiere una estrategia de diseño: invertir en legibilidad y pro-

cesabilidad (que son duraderas) y aceptar que la generabilidad se resuelve parcialmente con técnicas de restricción (constrained decoding, reasoning loop) que compensan las limitaciones actuales del LLM y que serán menos necesarias en el futuro.

11.6 La resolución integrada de ACAI

Sede principal de este concepto.

11.6.1 El diseño en capas

Las estrategias de resolución de las secciones anteriores se integran en un diseño que aborda las tres tensiones simultáneamente. La resolución de ACAI se basa en un principio de *separación por capas*: cada capa del formalismo está optimizada para un lector, y las transformaciones entre capas son deterministas.

La *capa canónica* es la representación interna del modelo de conocimiento, optimizada para la procesabilidad. Tiene semántica operacional definida, gramática formal estricta, y estructura que permite inferencia eficiente. Es lo que el motor simbólico ejecuta. No necesita ser legible ni generable directamente.

Tres capas del formalismo ACAI: capa canónica (optimizada para procesabilidad), capa de serialización (optimizada para generabilidad), y capa de verbalización (optimizada para legibilidad).

La *capa de serialización* es la representación que el LLM genera y consume, optimizada para la generabilidad. Es un JSON Schema estricto con patrones regulares, vocabulario controlado, y estructura que es compatible con constrained decoding. La transformación de serialización a representación canónica es un parser determinista.

La *capa de verbalización* es la representación que el experto humano lee, optimizada para la legibilidad. Es una notación de lenguaje natural controlado generada a partir de la capa canónica por un verbalizador (que puede ser el propio LLM o un generador basado en templates). La transformación de verbalización a representación canónica es el pipeline de compilación de la **UD 10**.

Las tres capas no son tres formalismos diferentes sino tres vistas del mismo conocimiento, con transformaciones deterministas entre ellas.

11.6.2 Flujo de las capas

El flujo del conocimiento a través de las tres capas depende del momento del ciclo de vida.

En *design time* (formalización asistida), el flujo es: experto describe en NL → LLM genera en capa de serialización → parser transforma a capa canónica → validador verifica → verbalizador genera capa de verbalización → experto revisa.

En *runtime* (operación del sistema), el flujo es: el motor ejecuta sobre la capa canónica → las decisiones se especifican en capa canónica → el LLM recibe la especificación en capa de serialización (como parte de su prompt) → el LLM genera la respuesta en NL condicionado por la especificación.

En *learning time* (mejora continua), el flujo es: las trazas se almacenan en capa canónica → el análisis se presenta al investigador en capa de verbalización → las propuestas de extensión siguen el flujo de design time.

11.6.3 Evaluación de la resolución

El diseño en capas resuelve las tensiones pero introduce una complejidad nueva: el mantenimiento de tres representaciones consistentes. La consistencia se garantiza por construcción si las transformaciones son funciones deterministas (no traducciones heurísticas). Esto requiere que la capa canónica sea suficientemente expresiva para ser la fuente de verdad de la que las otras dos capas se derivan.

> **⚠ La consistencia entre capas es un invariante del sistema**
>
> Si las transformaciones entre capas introducen pérdida de información o ambigüedad, el sistema pierde la propiedad de trazabilidad constitutiva: la traza (en capa canónica) no correspondería exactamente a lo que el experto aprobó (en capa de verbalización) ni a lo que el LLM generó (en capa de serialización). Garantizar la equivalencia entre capas es un requisito no negociable del diseño.

11.7 Programa experimental para la triple restricción

11.7.1 Métricas por restricción

Fase 0–1

Los experimentos de la triple restricción son transversales al sprint. Cada restricción se mide con métricas específicas que permiten evaluar cuantitativamente las decisiones de diseño del formalismo.

La *legibilidad* se mide empíricamente con expertos del dominio. Las métricas incluyen: tasa de comprensión correcta (proporción de reglas que el experto interpreta correctamente), tiempo de comprensión (segundos por regla), tasa de detección de errores (proporción de errores insertados que el experto detecta), y preferencia subjetiva (escala Likert de dificultad percibida). El benchmark es la comprensión de la verbalización: si el experto comprende la verbalización significativamente mejor que la notación directa, la legibilidad de la notación es insuficiente.

La *procesabilidad* se mide formalmente. Las métricas incluyen: decidibilidad de la gramática (la pertenencia de una expresión al lenguaje es decidible en tiempo polinomial), determinismo de la semántica (da-

da una expresión y un estado, el resultado es único), completitud de la verificación (el validador detecta todas las inconsistencias del tipo especificado), y eficiencia de la ejecución (tiempo de evaluación de una regla dado un information state).

La *generabilidad* se mide con los experimentos de la **UD 10**: tasa de corrección sintáctica, tasa de corrección semántica, número de iteraciones del reasoning loop, y robustez entre LLMs diferentes. Una métrica adicional específica de la triple restricción es la *sensibilidad al diseño*: cómo cambian las métricas de generabilidad cuando se modifica un aspecto del diseño del formalismo (cambiar una palabra clave, añadir una construcción, modificar la estructura de una regla).

11.7.2 Experimentos de diseño comparativo

El programa experimental incluye experimentos que comparan variantes de diseño del formalismo para medir su impacto en las tres restricciones simultáneamente.

Experimento A: Verbosidad versus compacidad. Dos variantes del mismo conocimiento: una versión verbosa (nombres descriptivos, palabras clave expandidas) y una versión compacta (nombres cortos, operadores simbólicos). Se mide legibilidad (con expertos), procesabilidad (ambas deben ser equivalentes si el diseño es correcto), y generabilidad (con LLMs). El resultado informa sobre el punto óptimo del eje L–P.

Experimento B: Regulación versus flexibilidad. Dos variantes de la gramática: una estricta (una sola forma para cada construcción) y una flexible (formas alternativas permitidas). Se mide legibilidad (la flexible debería ser ligeramente mejor), procesabilidad (ambas si el parser acepta las variantes), y generabilidad (la estricta debería ser mejor). El resultado informa sobre el punto óptimo del eje L–G.

Experimento C: Restricción semántica. Dos variantes del pipeline de generación: una con constrained decoding sintáctico solo y otra con vocabulario controlado (restricción de las referencias a entidades existentes). Se mide la tasa de corrección semántica. El resultado informa sobre el valor añadido de las restricciones context-sensitive.

> **El programa experimental como brújula**
>
> Los experimentos no producen un diseño óptimo por sí mismos, sino una *brújula*: indicaciones sobre en qué dirección modificar el diseño para mejorar el balance de las tres restricciones. El diseño del formalismo ACAI es un proceso iterativo donde cada ronda experimental informa la siguiente decisión de diseño. La Fase 0 produce la primera ronda; la Fase 1 produce las siguientes.

Conexiones con otras unidades

→ Prerequisito: UD 7 (Técnicas) para el espectro de restricción y las técnicas de constrained decoding y reasoning loop. **UD 9** (Formalismos) para los principios de diseño D1–D5 y la decisión del DSL. **UD 10** (Formalización) para el pipeline y los experimentos de viabilidad.

⇄ Complementa: UD 5 (Fundamentos) introdujo la triple restricción como innovación I1. Esta unidad es el análisis en profundidad de esa innovación.

Aplicación: UD 12 (Métricas) incorpora las métricas de la triple restricción en el framework general de evaluación de accountability.

Ejercicios

1. [Tensión L–P] Tome la regla de penalización por cancelación anticipada de la **UD 10** y formúlela en tres puntos del eje L–P: (a) máxima legibilidad (cercana al lenguaje natural), (b) máxima procesabilidad (notación formal compacta), y (c) un punto intermedio que equilibre ambas. Para cada formulación, evalúe: ¿la entiende un experto no técnico? ¿Es parseable sin ambigüedad? Argumente cuál de las tres elegiría para el DSL ACAI.

2. [Tensión L–G] Diseñe dos variantes de una construcción normativa del DSL: una con palabras clave en inglés (`MUST`, `UNLESS`) y otra con palabras clave en español (`DEBE`, `SALVO QUE`). Argumente qué impacto tendría cada elección en la legibilidad (para un experto hispanohablante) y en la generabilidad (dado que los LLMs tienen más datos en inglés). ¿Cuál elegiría y por qué?

3. [Diseño en capas] Para la regla de bonificación del escenario de la **UD 9**, produzca las tres capas de representación: (a) capa canónica (estructura formal procesable), (b) capa de serialización (JSON Schema compatible), y (c) capa de verbalización (lenguaje natural controlado para revisión humana). Verifique que las transformaciones entre capas son deterministas y preservan la semántica.

4. [Programa experimental] Diseñe un experimento que mida la sensibilidad de la generabilidad a la elección de palabras clave del DSL. Especifique: (a) las variantes a comparar (al menos 3), (b) el material (qué reglas se generan), (c) las métricas, y (d) el análisis estadístico. ¿Cuántas generaciones por variante necesita para

detectar una diferencia del 10 % en tasa de corrección?

5. [Reflexión crítica] El argumento de la «generabilidad como restricción temporal» sugiere que las mejoras futuras en LLMs relajarán la restricción de generabilidad. ¿Qué implicaciones tendría esto para el diseño actual del formalismo? ¿Debería ACAI diseñar para los LLMs de hoy (más regular, más restrictivo) o para los de mañana (más expresivo, confiando en que los LLMs mejorarán)? Argumente ambas posiciones.

Referencias y lecturas

Lectura de entrada

Fuchs, N. E., Kaljurand, K. & Kuhn, T. (2008). Attempto Controlled English for Knowledge Representation. En C. Baroglio, P. A. Bonatti, J. Maluszynski, M. Marchiori, A. Polleres & S. Schaffert (Eds.), *Reasoning Web* (LNCS 5224), Springer, 104–124. La referencia más madura de un lenguaje natural controlado con gramática formal. Demuestra que la tensión L–P es resoluble para un subconjunto del lenguaje natural. Lectura de entrada que inspira el diseño del DSL ACAI.

Lecturas esenciales

- Levesque, H. J. & Brachman, R. J. (1987). Expressiveness and Tractability in Knowledge Representation and Reasoning. *Computational Intelligence*, 3(1), 78–93. — Ya citado en UD9. El trade-off fundacional que la triple restricción extiende.
- Willard, B. T. & Louf, R. (2023). Efficient Guided Generation for Large Language Models. *ArXiv preprint arXiv:2307.09702*. — Ya citado múltiples veces. Aquí relevante por los límites del constrained decoding con gramáticas context-free.
- Mernik, M., Heering, J. & Sloane, A. M. (2005). When and How to Develop Domain-Specific Languages. *ACM Computing Surveys*, 37(4), 316–344. — Ya citado. Aquí relevante para las decisiones de diseño del DSL.

Lecturas complementarias

- Kuhn, T. (2014). A Survey and Classification of Controlled Natural Languages. *Computational Linguistics*, 40(1), 121–170. — Survey comprensivo de CNLs. Referencia para la estrategia de resolución L–P.
- Schwitter, R. (2010). Controlled Natural Languages for Knowledge Representation. *COLING 2010*. — CNLs para representación del conocimiento. Complementa a Fuchs et al.

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*. — RAG original. Relevante para la inyección de contexto como estrategia de resolución P–G.
- Zheng, L. et al. (2023). SGLang: Efficient Execution of Structured Language Model Programs. *ArXiv preprint*. — Ya citado en UD6. Aquí relevante por la alternancia de generación libre y restringida.
- Barwise, J. & Etchemendy, J. (2002). *Language, Proof and Logic*. CSLI Publications. — Referencia sobre la relación entre lenguaje natural y lógica formal. Contexto teórico para L–P.

MONOGRÁFICO FORMATIVO

Fundamentos de IA Conversacional Neuro-Simbólica

«If you can't measure it,
you can't demonstrate it.
And if you can't demonstrate it,
accountability is just a promise.»

Eje D — Evaluación

Unidad 12 de 12

Research Sprint ACAI · Proyecto $(AI)^2$:TDG · 2026

Eje D — Evaluación

Prerequisitos: UD4, UD8, UD10, UD11 · Eje D — Evaluación

UNIDAD DIDÁCTICA 12

Métricas y evaluación de accountability

Contenidos de la unidad

12.1.1 Lo que las métricas de diálogo miden 3
12.1.2 Las cuatro dimensiones de accountability como ejes de evaluación 4
12.2.1 Qué significa medir la trazabilidad 4
12.3.1 Qué significa medir la explicabilidad 6
12.4.1 Qué significa medir la conformidad 7
12.5.1 Qué significa medir la gobernanza 8
12.6.1 Evaluar el formalismo, no solo el sistema 9
12.8.1 Organización del framework 10
12.8.2 Evaluación comparativa entre fases 11
12.8.3 El corpus de evaluación 12
12.8.4 Niveles de evaluación 12
12.8.5 De las métricas a las decisiones 12
12.9.1 Lo que el monográfico ha construido 13
12.9.2 Lo que queda por construir 14

¿Por qué necesito esto para ACAI?

Las once unidades anteriores han construido un edificio conceptual, arquitectónico y técnico. Pero un edificio sin instrumentos de medición es un edificio que no puede demostrar que funciona. La accountability, el objetivo central de ACAI, no es una propiedad que se declara: es una propiedad que se demuestra con evidencia. Esta unidad cierra el monográfico definiendo el framework de evaluación: qué se mide, cómo se mide, con qué métricas, y cómo las métricas se organizan en un sistema coherente que permite evaluar las decisiones de diseño del sprint. Sin esta unidad, el sprint produce artefactos pero no puede demostrar que son accountable.

Objetivos de aprendizaje

Al completar esta unidad, el investigador será capaz de:

1. Argumentar por qué la evaluación de accountability requiere métricas específicas que las métricas estándar de sistemas de diálogo no cubren.
2. Diseñar métricas operativas para las cuatro dimensiones de accountability: trazabilidad, explicabilidad, conformidad normativa, y gobernanza.
3. Diseñar métricas de calidad del formalismo: cobertura, consistencia, completitud, y la triple restricción.
4. Diseñar métricas de calidad del pipeline de formalización asistida: generabilidad, convergencia, y eficiencia de revisión.
5. Organizar las métricas en un framework de evaluación que permita la comparación entre fases del sprint y la validación progresiva de las hipótesis de ACAI.

12.1 La insuficiencia de las métricas estándar

Sede principal de este concepto.

12.1.1 Lo que las métricas de diálogo miden

La evaluación de sistemas de diálogo tiene una tradición consolidada. Las métricas se organizan típicamente en tres categorías. Las métricas de *tarea* miden si el sistema cumple el objetivo del usuario: tasa de éxito de la tarea (task completion rate), número de turnos necesarios, y eficiencia. Las métricas de *calidad lingüística* miden la naturalidad del

Las métricas estándar de sistemas de diálogo miden la calidad de la conversación como experiencia de usuario; no miden si las decisiones del sistema son trazables, explicables, y conformes con la normativa.

lenguaje generado: fluidez, coherencia, relevancia, y diversidad léxica. Las métricas de *satisfacción* miden la experiencia subjetiva del usuario: escalas Likert, Net Promoter Score, y métricas de engagement.

Estas métricas son necesarias pero insuficientes para ACAI. Un sistema puede completar la tarea con éxito (cancelar la póliza), con lenguaje fluido y natural, y con alta satisfacción del usuario, pero haber tomado decisiones no trazables (no se puede reconstruir por qué se ofreció cierta alternativa), no explicables (no se puede justificar por qué se informó de ciertas condiciones y no de otras), y no conformes (se omitió una obligación regulatoria que el usuario no echó en falta pero que la auditoría detectaría).

Deriu et al. (2021) — survey de métricas de evaluación de sistemas de diálogo.

La accountability no es una propiedad de la experiencia de usuario sino de la gobernanza del sistema. Requiere métricas que operen en un nivel diferente: no «¿el usuario está satisfecho?» sino «¿se puede demostrar que cada decisión fue correcta, justificable, y conforme?».

12.1.2 Las cuatro dimensiones de accountability como ejes de evaluación

La **UD 4** estableció las cuatro dimensiones de la accountability: trazabilidad (¿se puede reconstruir qué ocurrió y por qué?), explicabilidad (¿se puede justificar cada decisión en términos comprensibles?), conformidad normativa (¿el sistema cumple las restricciones regulatorias y de políticas?), y gobernanza (¿los mecanismos de control operan correctamente?).

Estas cuatro dimensiones se convierten en los cuatro ejes del framework de evaluación de ACAI. Cada eje tiene métricas específicas, métodos de medición, y umbrales de aceptabilidad. Los ejes se complementan con dos ejes adicionales que evalúan la infraestructura sobre la que opera la accountability: la calidad del formalismo y la calidad del pipeline de formalización.

12.2 Métricas de trazabilidad

Sede principal de este concepto.

12.2.1 Qué significa medir la trazabilidad

La trazabilidad constitutiva — que la **UD 4** distinguió de la trazabilidad por registro — implica que cada decisión del sistema produce una traza como subproducto natural de la ejecución. La calidad de esa traza determina la calidad de la trazabilidad.

La trazabilidad se mide por la completitud, la fidelidad, y la granularidad de las trazas que el sistema produce como subproducto de su operación.

M1: Completitud de la traza. Proporción de decisiones del sistema que producen un registro de traza. Una traza es completa si para cada acción del sistema (informar, preguntar, ejecutar, confirmar) existe un registro que identifica: qué regla se aplicó, sobre qué datos, en qué estado del diálogo, y con qué resultado. La métrica es la proporción de acciones con traza completa sobre el total de acciones.

Método de medición: para un corpus de n diálogos, inspeccionar cada acción del sistema y verificar que el registro de traza contiene los cuatro componentes. La métrica es $M1 = \frac{\text{acciones con traza completa}}{\text{total de acciones}}$.

Umbral: $M1 \geq 0{,}99$ para operación en producción. Cada acción sin traza es un punto ciego de accountability.

M2: Fidelidad de la traza. Proporción de trazas que reflejan correctamente lo que ocurrió. Una traza es fiel si la regla registrada es la que efectivamente se evaluó, los datos registrados son los que efectivamente se consultaron, y el estado registrado es el que efectivamente existía. La infidelidad puede producirse si la traza se genera por un mecanismo diferente al de la ejecución (por ejemplo, una reconstrucción post-hoc que infiere la traza a partir del resultado).

La trazabilidad constitutiva garantiza fidelidad por construcción: la traza *es* la ejecución.

Método de medición: para un subconjunto de diálogos, reproducir la ejecución paso a paso con el motor simbólico en modo de depuración y comparar la traza de depuración con la traza producida en operación normal. La fidelidad constitutiva implica que ambas son idénticas; cualquier discrepancia indica un defecto en el mecanismo de trazado.

Umbral: $M2 = 1{,}0$. La fidelidad es binaria: la traza es fiel o no lo es.

M3: Granularidad de la traza. Nivel de detalle de la traza. Una traza puede registrar solo la decisión final («se informó sobre la bonificación») o la cadena completa de evaluaciones que condujo a esa decisión («se evaluó la regla R17, que consultó el atributo `bonificacion` de la entidad `poliza_HG-2019-4421`, cuyo valor es 135 €, lo que activó la obligación O3 de informar»). La granularidad se mide como el número de niveles de detalle disponibles en la traza.

Método de medición: definir niveles de granularidad (acción, regla, condición, dato) y verificar cuántos niveles captura la traza.

Umbral: mínimo 3 niveles (acción, regla aplicada, datos consultados) para operación; 4 niveles (incluyendo condiciones evaluadas) para auditoría.

12.3 Métricas de explicabilidad

Sede principal de este concepto.

12.3.1 Qué significa medir la explicabilidad

La explicabilidad no es una propiedad del sistema sino de la relación entre el sistema y su audiencia: una explicación es explicable si la audiencia la comprende. Esto hace que las métricas de explicabilidad tengan un componente empírico ineludible: requieren evaluación con audiencias reales.

La explicabilidad se mide por la capacidad del sistema de generar justificaciones comprensibles y correctas para sus decisiones, evaluada con las audiencias relevantes.

M4: Generabilidad de explicaciones. Proporción de decisiones del sistema para las que se puede generar una explicación en lenguaje natural a partir de la traza. No todas las trazas son automáticamente explicables: una traza que registra «se evaluó la regla R17» necesita un paso adicional de verbalización para producir «Le informo sobre su bonificación porque la normativa requiere que conozca este dato antes de confirmar la cancelación».

Método de medición: para un corpus de decisiones, intentar generar una explicación a partir de la traza y verificar que la explicación es gramaticalmente correcta, factualmente fiel a la traza, y suficientemente informativa.

M5: Comprensibilidad de explicaciones. Proporción de explicaciones generadas que la audiencia comprende correctamente. Se mide empíricamente con dos audiencias: usuarios finales (que necesitan entender por qué el sistema actúa de cierta forma) y auditores (que necesitan verificar la conformidad).

Hoffman et al. (2018) — métricas de explicabilidad en sistemas de IA: framework de evaluación.

Método de medición: presentar explicaciones a evaluadores y pedir que (a) parafraseen la explicación (para verificar comprensión), (b) identifiquen la decisión que se explica, y (c) evalúen si la explicación es suficiente para juzgar la corrección de la decisión. La métrica es la proporción de explicaciones correctamente comprendidas.

Umbral: $M5 \geq 0{,}90$ para usuarios finales, $M5 \geq 0{,}95$ para auditores.

M6: Fidelidad de explicaciones. Proporción de explicaciones que son fieles a la decisión real del sistema: la explicación no atribuye la decisión a razones diferentes de las que efectivamente la determinaron. La infidelidad de las explicaciones es un riesgo conocido en XAI (Rudin, 2019): un sistema puede generar explicaciones plausibles pero incorrectas.

Rudin (2019): las explicaciones post-hoc pueden ser infieles. La explicabilidad constitutiva mitiga este riesgo.

Método de medición: comparar la explicación generada con la traza del motor simbólico. Si la explicación menciona la regla R17 y la traza registra que se evaluó R17, la explicación es fiel para ese componente. La fidelidad se mide como la proporción de componentes de la explicación que corresponden a componentes de la traza.

Umbral: $M6 \geq 0{,}95$. Las explicaciones infieles son peores que la ausencia de explicación porque generan confianza injustificada.

12.4 Métricas de conformidad normativa

Sede principal de este concepto.

12.4.1 Qué significa medir la conformidad

La conformidad se mide por la proporción de interacciones donde el sistema cumple todas las obligaciones, respeta todas las prohibiciones, y opera dentro de todos los permisos definidos en el modelo normativo.

La conformidad normativa conecta directamente con el conocimiento regulatorio (dimensión 3 de la **UD 8**) y con los requisitos de la **UD 4**. Las métricas de conformidad operan sobre la relación entre el comportamiento del sistema y el modelo normativo formalizado.

M7: Tasa de cumplimiento de obligaciones. Proporción de situaciones donde una obligación era aplicable y el sistema la cumplió. Si la obligación O3 (informar sobre la bonificación antes de solicitar confirmación) era aplicable en 100 diálogos de cancelación con bonificación, y el sistema informó en 98, la tasa es 0.98.

Método de medición: para cada obligación del modelo normativo, identificar las situaciones donde es aplicable (por las condiciones de ámbito de la norma) y verificar en la traza que la acción obligada se ejecutó antes del plazo requerido.

Umbral: $M7 \geq 0{,}99$ para obligaciones regulatorias; $M7 \geq 0{,}95$ para obligaciones de política de negocio. La distinción de umbrales refleja la diferencia de consecuencias: una violación regulatoria tiene consecuencias jurídicas; una violación de política tiene consecuencias operativas.

M8: Tasa de respeto de prohibiciones. Proporción de situaciones donde una prohibición era relevante y el sistema la respetó. Si la prohibición P1 (no proporcionar asesoramiento financiero) era relevante en 500 turnos y el sistema no generó asesoramiento en ninguno, la tasa es 1.0.

Método de medición: más complejo que M7 porque requiere verificar la *ausencia* de una acción. Para prohibiciones sobre el contenido generado (como P1), se requiere evaluación del texto generado, que puede hacerse con clasificadores de contenido o con evaluación humana.

Umbral: $M8 \geq 0{,}99$ para todas las prohibiciones. Una sola violación de una prohibición regulatoria puede tener consecuencias desproporcionadas.

Medir la ausencia de violaciones es más difícil que medir la presencia de cumplimiento. Las prohibiciones requieren vigilancia continua.

M9: Cobertura normativa. Proporción del modelo normativo que es verificable automáticamente. No todas las normas son verificables por el motor: algunas requieren juicio humano («el tono debe ser apropiado»). La cobertura normativa mide qué fracción de las normas puede ser evaluada automáticamente por el motor simbólico.

Método de medición: para cada norma del modelo, determinar si la traza contiene información suficiente para evaluar su cumplimiento automáticamente. La métrica es $M9 = \frac{\text{normas evaluables}}{\text{total de normas}}$.

Umbral: $M9 \geq 0{,}80$ para la Phase 1; idealmente creciente con el refinamiento del formalismo.

12.5 Métricas de gobernanza

Sede principal de este concepto.

12.5.1 Qué significa medir la gobernanza

La gobernanza mide si los mecanismos de control del sistema operan correctamente: si el motor simbólico efectivamente gobierna al LLM, si las restricciones se aplican, y si el sistema se comporta de forma predecible.

La gobernanza se mide por la efectividad de los mecanismos de control: ¿el motor simbólico gobierna efectivamente al LLM?

M10: Tasa de adherencia a las especificaciones del motor. Proporción de turnos donde el texto generado por el LLM es consistente con la especificación del motor simbólico. Si el motor especifica «informar sobre la bonificación de 135 € con tono empático», el texto generado debe mencionar la bonificación, incluir la cantidad correcta, y emplear un tono empático. La adherencia se mide por la presencia de los elementos obligatorios y la ausencia de elementos prohibidos.

Método de medición: para cada turno, extraer la especificación del motor (qué informar, con qué datos, con qué tono) y evaluar el texto generado contra esa especificación. La evaluación puede ser parcialmente automática (presencia de datos numéricos correctos) y parcialmente humana (adecuación del tono).

Umbral: $M10 \geq 0{,}95$ para elementos factuales (datos, nombres, cantidades); $M10 \geq 0{,}85$ para elementos pragmáticos (tono, nivel de detalle).

M11: Tasa de efectividad del espectro de restricción. Proporción de turnos de alta consecuencia donde los mecanismos de restricción de nivel alto (constrained decoding, verificación post-generación) se activaron correctamente. La restricción dinámica del espectro (**UD 7**) determina que los turnos críticos reciban máxima restricción; M11 mide si esto ocurre.

M12: Consistencia entre sesiones. Proporción de escenarios equivalentes que producen decisiones equivalentes en sesiones diferentes. Un sistema accountable debe producir la misma decisión ante la misma situación independientemente de cuándo se ejecute (controlando por la versión del modelo de conocimiento). La inconsistencia indica que la variabilidad del LLM está dominando las decisiones del motor simbólico.

Método de medición: ejecutar el mismo escenario de diálogo (secuencia de inputs del usuario) en k sesiones independientes y medir

la varianza en las decisiones del sistema. La consistencia es $M12 = 1 - \frac{\text{decisiones divergentes}}{\text{total de decisiones} \times k}$.

Umbral: $M12 \geq 0{,}95$. La variabilidad aceptable se limita a aspectos pragmáticos (formulación del texto), no a aspectos sustantivos (qué se informa, qué se ejecuta).

12.6 Métricas de calidad del formalismo

Sede principal de este concepto.

12.6.1 Evaluar el formalismo, no solo el sistema

Las métricas anteriores evalúan el sistema en operación. Pero la accountability de ACAI depende de la calidad del modelo de conocimiento formalizado: un modelo incompleto produce accountability incompleta, y un modelo inconsistente produce accountability contradictoria.

El formalismo es la infraestructura de la accountability. Si el formalismo es incompleto o inconsistente, la accountability tiene lagunas estructurales que ningún mecanismo de ejecución puede compensar.

M13: Cobertura del modelo. Proporción de las decisiones operativas del sistema que están cubiertas por al menos una regla del modelo formal. Una decisión no cubierta es una decisión que el LLM toma sin gobernanza simbólica — un punto ciego de accountability.

Método de medición: para un corpus representativo de diálogos, identificar cada punto de decisión del sistema y verificar que existe una regla en el modelo que aplica a ese punto. La métrica es $M13 = \frac{\text{decisiones cubiertas}}{\text{total de decisiones}}$.

Umbral: $M13 \geq 0{,}90$ para la Phase 1; creciente con la madurez del modelo.

M14: Consistencia del modelo. Ausencia de contradicciones en el modelo formal. Dos reglas son contradictorias si, para algún estado posible, una prescribe una acción y la otra prescribe la acción opuesta sin resolución de prioridad. La consistencia es verificable automáticamente por el motor simbólico si el formalismo tiene semántica operacional definida.

La consistencia del modelo es verificable automáticamente — una de las ventajas de la formalización explícita sobre el conocimiento implícito en prompts.

Método de medición: verificación exhaustiva (si el espacio de estados es finito) o verificación por muestreo (si es infinito) de pares de reglas con ámbitos solapantes.

Umbral: $M14 = 1{,}0$ — cero contradicciones no resueltas.

M15: Completitud dimensional. Proporción de las cinco dimensiones del conocimiento (UD 8) que están formalizadas con al menos un nivel mínimo de cobertura. Un modelo que formaliza el dominio y los procedimientos pero no las normas tiene completitud dimensional 0.4 (2/5).

Umbral: $M15 = 1{,}0$ para la Phase 1 (todas las dimensiones represen-

tadas); la profundidad de cada dimensión se mide con M13.

M16: Métricas de la triple restricción. Las tres restricciones del formalismo (UD 11) se evalúan con las métricas específicas diseñadas en esa unidad: tasa de comprensión correcta por expertos (legibilidad), decidibilidad y eficiencia de evaluación (procesabilidad), y tasa de corrección sintáctica y semántica por LLMs (generabilidad). M16 es un compuesto que resume las tres.

12.7 Métricas del pipeline de formalización

Sede principal de este concepto.

Las métricas del pipeline de formalización asistida (UD 10) evalúan la calidad del proceso de construcción del modelo de conocimiento. Son métricas de *proceso*, no de *producto*: miden cuán eficiente y fiable es la formalización asistida.

Las métricas del pipeline miden la viabilidad de la innovación central de ACAI: el LLM como asistente de formalización.

M17: Tasa de corrección al primer intento. Proporción de formalizaciones generadas por el LLM que pasan la validación simbólica sin corrección iterativa. Esta métrica mide la calidad de la generación directa y es el indicador más directo de la generabilidad del formalismo.

M18: Tasa de convergencia del reasoning loop. Proporción de formalizaciones que alcanzan la validación exitosa dentro del límite de iteraciones (típicamente 4). Las formalizaciones que no convergen requieren intervención humana directa.

M19: Velocidad de formalización. Tiempo total desde el inicio de la elicitación hasta la aprobación por el experto, medido en minutos por regla. Comparar con la velocidad de formalización manual (2–5 reglas por hora en la literatura de ingeniería del conocimiento) para evaluar la ganancia de productividad.

M20: Tasa de errores no detectados. Proporción de errores en la formalización que pasan la validación simbólica y son detectados solo por el experto en la revisión humana. Esta métrica mide la calidad del validador: si es alta, el validador necesita mejoras; si es baja, la revisión humana puede aligerarse.

12.8 El framework integrado de evaluación

Sede principal de este concepto.

12.8.1 Organización del framework

Veinte métricas organizadas en seis ejes: trazabilidad (M1–M3), explicabilidad (M4–M6), conformidad (M7–M9), gobernanza (M10–M12), formalismo (M13–M16), y pipeline (M17–M20).

Las veinte métricas se organizan en un framework con seis ejes que cubren la totalidad del sistema ACAI. Los cuatro primeros ejes (trazabilidad, explicabilidad, conformidad, gobernanza) evalúan las propiedades de accountability del sistema en operación. El quinto eje (formalismo) evalúa la infraestructura representacional. El sexto eje (pipeline) evalúa el proceso de construcción.

La distinción es importante: un sistema puede tener buenas métricas de accountability en operación (ejes 1–4) pero malas métricas de formalismo (eje 5) o de pipeline (eje 6). Esto indicaría que la accountability se logra a pesar del formalismo, no gracias a él — una situación frágil que no escala. Inversamente, un formalismo excelente (eje 5) con un pipeline eficiente (eje 6) pero malas métricas de accountability en operación (ejes 1–4) indicaría un fallo en la integración entre el modelo de conocimiento y el motor de ejecución.

12.8.2 Evaluación comparativa entre fases

Phase 0 → 1 → 2
El framework permite la comparación cuantitativa entre fases del sprint.

El framework está diseñado para permitir la evaluación comparativa entre las fases del sprint. Al final de cada fase, se miden las veinte métricas sobre un corpus de evaluación fijo (el mismo conjunto de escenarios de diálogo). La comparación entre fases revela: qué dimensiones de accountability mejoran con los avances técnicos de cada fase, qué dimensiones se estancan o retroceden, y dónde invertir esfuerzo en la fase siguiente.

La Phase 0 establece la *línea base*: las métricas del sistema sin formalismo (solo LLM con prompt engineering). Esta línea base es el punto de comparación para evaluar el valor añadido de la integración neuro-simbólica. Si la Phase 1 (con formalismo y motor simbólico) no mejora significativamente las métricas de accountability respecto a la Phase 0, la hipótesis central de ACAI — que la formalización mejora la accountability — se ve comprometida.

> **La línea base como test de la hipótesis**
>
> La línea base es más que un punto de referencia: es un test de la hipótesis central del sprint. Si un LLM bien configurado (con un system prompt detallado, ejemplos, y guardrails básicos) alcanza $M7 \geq 0{,}95$, $M10 \geq 0{,}90$, y $M12 \geq 0{,}90$, el argumento a favor de la formalización simbólica se debilita — los costes de la formalización deben justificarse con mejoras medibles. La línea base obliga al sprint a demostrar su valor empíricamente, no solo argumentativamente.

12.8.3 El corpus de evaluación

El framework requiere un *corpus de evaluación* fijo que se use consistentemente en todas las mediciones. El corpus consiste en un conjunto de escenarios de diálogo que cubren:

Escenarios *nominales*: diálogos que siguen el protocolo estándar sin desviaciones (cancelación directa, consulta simple, modificación de póliza). Diseñados para medir M1–M3, M7, M10.

Escenarios *de frontera*: diálogos que tocan los límites del modelo normativo (cliente que pide asesoramiento financiero, solicitud ambigua que podría ser cancelación o suspensión, caso no cubierto por las reglas existentes). Diseñados para medir M8, M9, M13.

Escenarios *de estrés*: diálogos donde el usuario se desvía del protocolo (cambia de opinión a mitad del diálogo, proporciona información contradictoria, expresa frustración). Diseñados para medir M11, M12.

Escenarios *adversariales*: diálogos diseñados para provocar fallos de accountability (intentos de extraer información prohibida, de eludir la verificación de identidad, de forzar una acción sin confirmación). Diseñados para medir M8, M10.

12.8.4 Niveles de evaluación

Tres niveles de evaluación: automática (métricas computables), semi-automática (métricas con componente automático y humano), y humana (métricas que requieren juicio experto).

No todas las métricas pueden evaluarse con el mismo método. El framework distingue tres niveles.

La evaluación *automática* aplica a las métricas computables directamente a partir de las trazas y el modelo: M1, M2, M3, M9, M12, M13, M14, M15, M17, M18. Estas métricas se miden con cada ejecución del corpus de evaluación sin intervención humana.

La evaluación *semi-automática* aplica a las métricas que combinan verificación automática y humana: M4, M7, M8, M10, M11, M16, M19, M20. La parte automática filtra los casos que claramente cumplen o no cumplen; la parte humana evalúa los casos ambiguos.

La evaluación *humana* aplica a las métricas que requieren juicio experto: M5, M6. Estas métricas se miden con evaluadores (usuarios finales y auditores) sobre una muestra del corpus.

12.8.5 De las métricas a las decisiones

> **El framework como instrumento de decisión**
>
> El framework no es un ejercicio académico de medición sino un instrumento de decisión para el sprint. Las métricas informan tres tipos de decisiones.

Decisiones de avance: si las métricas de la Phase 1 alcanzan los umbrales definidos, se avanza a la Phase 2. Si no, se itera sobre la Phase 1.
Decisiones de diseño: si M17 (corrección al primer intento) es baja, el formalismo necesita simplificarse para mejorar la generabilidad. Si M10 (adherencia a especificaciones) es baja, la interfaz entre motor y LLM necesita mejorar. Las métricas diagnostican *dónde* está el problema.
Decisiones de priorización: si M7 (cumplimiento de obligaciones) es alta pero M5 (comprensibilidad de explicaciones) es baja, el sprint debe priorizar la explicabilidad sobre la conformidad en la siguiente fase.
Cada sesión de evaluación produce un perfil de veinte métricas que es el instrumento de navegación del sprint: indica dónde estamos, hacia dónde vamos, y qué debemos ajustar.

12.9 Síntesis: del mapa al territorio

Esta unidad cierra el monográfico formativo. Es apropiado cerrar no solo la unidad sino el recorrido completo con una reflexión sobre lo que el monográfico ha construido y lo que queda por construir.

12.9.1 Lo que el monográfico ha construido

El monográfico ha construido un edificio de doce unidades organizadas en cuatro ejes. El **Eje A** (unidades 2–4) estableció el problema: la conversación es un fenómeno multidimensional cuya complejidad los sistemas actuales simplifican; el ecosistema operativo necesita gobernanza formal; y la accountability exige representación explícita del conocimiento. El **Eje B** (unidades 5–7) estableció el paradigma: la integración neuro-simbólica ofrece el marco para combinar las capacidades del LLM con la gobernanza formal, con técnicas específicas para cruzar la frontera entre lenguaje natural y representaciones formales. El **Eje C** (unidades 8–11) estableció la representación: las cinco dimensiones del conocimiento conversacional, los requisitos del formalismo, la evaluación de candidatos, el diseño del DSL, el pipeline de formalización asistida, y las tensiones de la triple restricción. El **Eje D** (esta unidad) estableció la evaluación: las métricas que permiten demostrar — no solo afirmar — que el sistema es accountable.

La unidad 1, como mapa conceptual, tejió todos estos hilos en una estructura coherente antes de desarrollar cada uno. Las conexiones entre

unidades — los prerequisitos, las complementariedades, las aplicaciones — forman una red donde cada concepto está anclado en los demás.

12.9.2 Lo que queda por construir

El monográfico es el mapa. El sprint es el territorio.

El monográfico es formativo: prepara al investigador para el sprint, no sustituye al sprint. Lo que queda por construir es el territorio que el mapa describe.

La **Phase 0** debe producir: la especificación detallada del DSL ACAI (principios de diseño de la UD 9 convertidos en gramática formal), los experimentos de viabilidad de la formalización asistida (UD 10), la primera ronda experimental de la triple restricción (UD 11), y la línea base de accountability (esta unidad). La Phase 0 valida las hipótesis del sprint antes de invertir en implementación.

La **Phase 1** debe producir: el motor simbólico que ejecuta el DSL, el pipeline de formalización asistida implementado, el modelo de conocimiento del dominio asegurador formalizado, y la primera evaluación completa con el framework de esta unidad. La Phase 1 es la prueba de concepto.

La **Phase 2** debe producir: las extensiones del espectro de restricción (constrained decoding, restricción dinámica), el análisis de trazas para mejora continua, y la evaluación comparativa entre fases que demuestra el valor añadido de la integración neuro-simbólica.

El monográfico ha proporcionado los fundamentos teóricos, las referencias bibliográficas, los marcos de análisis, y los criterios de evaluación. El sprint proporcionará la implementación, los datos, los experimentos, y la evidencia. Juntos, producirán la contribución de ACAI: la demostración de que la accountability conversacional es formalmente alcanzable.

> **Conexiones con otras unidades**
>
> → **Prerequisito:** UD 4 (Accountability) para las cuatro dimensiones de accountability que definen los ejes de evaluación. UD 8 (Dimensiones) para las cinco dimensiones del conocimiento que M13–M15 evalúan. UD 10 (Formalización) para el pipeline que M17–M20 evalúan. UD 11 (Triple restricción) para las métricas específicas que M16 integra.
>
> ⇄ **Complementa:** Todas las unidades anteriores: el framework de evaluación mide las propiedades que todo el monográfico ha definido. Cada métrica tiene su raíz en una unidad específica.

Ejercicios

1. [Línea base] Diseñe el experimento de línea base para la Phase 0. Especifique: (a) la configuración del LLM sin formalismo (system prompt, ejemplos, guardrails básicos), (b) el corpus de evaluación (5 escenarios nominales, 3 de frontera, 2 de estrés), y (c) las métricas que medirá (seleccione las 10 más relevantes de las 20 definidas). Para cada métrica, especifique cómo la medirá sin motor simbólico (algunas métricas no son aplicables sin formalismo — identifique cuáles).

2. [Aplicación del framework] Tome el escenario de cancelación del monográfico y ejecute una evaluación simulada. Para cada turno del diálogo, identifique qué métricas son relevantes y qué valor tendrían si el sistema operara correctamente. Identifique al menos un turno donde una métrica tendría valor bajo y explique qué fallo de accountability indica.

3. [Métricas de conformidad] La métrica M8 (respeto de prohibiciones) es especialmente difícil de medir porque requiere verificar la ausencia de una acción. Diseñe un protocolo de medición para M8 que combine: (a) un clasificador automático de contenido prohibido (entrenado o basado en reglas), (b) una muestra de evaluación humana, y (c) escenarios adversariales específicamente diseñados para provocar violaciones. ¿Cómo estima la tasa de falsos negativos del clasificador?

4. [Corpus de evaluación] Diseñe el corpus de evaluación para el dominio asegurador. Para cada tipo de escenario (nominal, frontera, estrés, adversarial), especifique al menos 3 escenarios concretos con los inputs del usuario turno a turno. Para cada escenario, identifique qué métricas está diseñado para evaluar y qué comportamiento del sistema sería una violación de accountability.

5. [Reflexión sobre la línea base] El argumento de la «línea base como test de la hipótesis» sugiere que si un LLM bien configurado alcanza métricas altas de accountability sin formalismo, el valor de ACAI se ve comprometido. ¿Es esto realmente cierto? ¿Hay aspectos de la accountability que la línea base *no puede* capturar por definición (porque requieren formalismo)? Identifique al menos tres métricas del framework que son *inmedibles* sin formalismo y argumente que la línea base es necesariamente incompleta como test de la hipótesis.

Referencias y lecturas

Lectura de entrada

Deriu, J., Rodrigo, A., Otegi, A., Echegoyen, G., Rosset, S., Agirre, E. & Cieliebak, M. (2021). Survey on Evaluation Methods for Dialogue Systems. *Artificial Intelligence Review*, 54, 755–810. Survey comprensivo de métricas de evaluación de sistemas de diálogo. Útil para entender qué miden las métricas estándar — y, por contraste, qué no miden y qué ACAI necesita añadir. Lectura de entrada.

Lecturas esenciales

- Rudin, C. (2019). Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. *Nature Machine Intelligence*, 1(5), 206–215. — El argumento más influyente a favor de la interpretabilidad constitutiva sobre las explicaciones post-hoc. Fundacional para M6 (fidelidad de explicaciones).
- Hoffman, R. R., Mueller, S. T., Klein, G. & Litman, J. (2018). Metrics for Explainability: A Framework for Quantitative Evaluation of XAI Systems. *ArXiv preprint arXiv:1812.04608*. — Framework de métricas de explicabilidad. Referencia para M4–M6.
- Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kiber, R. & Horvitz, E. (2019). Guidelines for Human-AI Interaction. *CHI 2019*. — Directrices de interacción humano-IA que informan las métricas de experiencia.

Lecturas complementarias

- Mehri, S. & Eskenazi, M. (2020). Unsupervised Evaluation of Interactive Dialog with DialoGPT. *SIGDIAL 2020*. — Métricas automáticas de diálogo. Contexto para las métricas estándar que ACAI complementa.
- Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D. & Barnes, P. (2020). Closing the AI Accountability Gap. *FAT* 2020*. — El gap de accountability en IA. Referencia para el argumento de que la accountability requiere evidencia medible.
- EU AI Act (2024). Regulation (EU) 2024/1689 of the European Parliament and of the Council. — Ya citado en UD4. Aquí relevante por los requisitos de evaluación que el framework debe satisfacer.
- Shneiderman, B. (2020). Bridging the Gap Between Ethics and Practice: Guidelines for Reliable, Safe, and Trustworthy Human-centered AI Systems. *ACM Transactions on Interactive Intelligent Systems*, 10(4). — Principios de IA confiable que informan el diseño del framework.
- Doshi-Velez, F. & Kim, B. (2017). Towards a Rigorous Science of Interpretable Machine Learning. *ArXiv preprint arXiv:1702.08608*. — Hacia una ciencia rigurosa de la interpretabilidad. Marco teórico para las métricas de explicabilidad.

- Papenmeier, A., Englebienne, G. & Seifert, C. (2019). How Model Accuracy and Explanation Fidelity Influence User Trust. *ArXiv preprint arXiv:1907.12652*. — La relación entre fidelidad de explicaciones y confianza. Relevante para M6.